



Programming & Compiler Mental Notes

A dependency-ordered computer-science curriculum from mathematical reasoning to CPUs, memory, operating systems, language runtimes, networks, browsers, graphics, embedded systems, and reverse engineering—with Rust libraries used as executable models of the underlying ideas.

format [Markdown](#) primary language [Rust](#) scope [compilers](#) | [systems](#) | [networking](#) status [living document](#)



ElevenLabs editions: use the [chaptered EPUB](#) for Studio, or the [plain-text fallback](#). The spoken editions retain the lessons but omit exact code and visual diagrams; return to this Markdown edition when you want the implementation.



How to Use These Notes

These notes are a **dependency-ordered computer-science curriculum** first and a reference second. On a first read, move from top to bottom. Each later chapter assumes the representations, invariants, and machine boundaries developed earlier.

The document uses Rust libraries as **executable models of computer-science ideas**:

Library or project	Use it to understand
Pest	tokenization, PEG recognition, ordered choice, parse-tree construction
hosted-languages and Lua interpreters	values, environments, call frames, bytecode, garbage collection
Inkwell / LLVM	IR, control-flow graphs, SSA, ABI lowering, verification
Rust collections and patterns	ownership, representation, invariants, error models
Actix	actors, capabilities, mailboxes, backpressure, supervision
smoltcp	layered protocols, explicit buffers, timers, TCP state machines
Quinn	encrypted transport, multiplexing, loss recovery, stream independence
winit / SDL	event loops, OS windows, input, timing, presentation
wgpu	GPU queues, resources, validation, pipelines, asynchronous completion
wasm-bindgen	FFI, host-owned objects, identity, lifetime across runtimes
Diesel	relational modeling, typed queries, transactions, schema boundaries
Bevy	ECS, data-oriented storage, scheduling, change detection
Tauri	process boundaries, capabilities, native/web IPC



Library rule: learn the model before the convenience API. For every example, ask what state exists, how it is represented, which transition the call requests, what invariant the library checks, and what failure or backpressure reaches the caller.

Fast Concept-Reading Loop

Read each concept in six passes:

1. **Definition** — say what the concept is in one precise sentence.
2. **Problem** — identify the constraint that makes the concept necessary.
3. **Representation** — find the bytes, types, nodes, buffers, or tables that store it.
4. **Mechanism** — trace one input through each state transition to an output.
5. **Invariant and failure** — state what must remain true and how it can break.
6. **Transfer** — find the same model in a compiler, OS, network, browser, database, or embedded device.

Code is evidence for the mechanism, not the lesson by itself. Comments explain why a field or transition exists, which assumption it encodes, and what observable behavior results.

Recommended Top-to-Bottom Reading Order

Phase	Build understanding in this order	Dependency it establishes
1 · Reasoning foundations	concept map → sets/logic/proofs/probability/graphs/linear algebra	vocabulary for state, relations, change, and proof
2 · The physical machine	CPU → instructions → memory → translation levels	how abstract programs become finite machine state
3 · Runtime substrate	memory systems → OS → executables/ABI → embedded → concurrency	how code receives storage, devices, scheduling, and isolation
4 · Program construction	Rust ownership/types/errors → patterns → functional models → algorithms → ML	how to encode and analyze reliable transformations
5 · Language implementation	grammar → parsing → AST → recursion → types → bytecode → runtime → native code	how source becomes meaning and execution
6 · Connected systems	service architecture → databases/caches → networks → cryptography/security → TCP/IP → QUIC	how independent state machines communicate under failure
7 · Interactive systems	graphics/media/GPU → Chrome → GUI/TUI → WebAssembly/DOM → SDL → applications	how events become state, layout, commands, and pixels
8 · Read systems backward	x64 → memory evidence → defensive instrumentation	how to infer hidden structure from observable behavior
9 · Practice and recall	terminal/tooling → automation → workflow → progression → glossary → Anki	how to retain and apply the models

 **Prerequisite rule:** when a later chapter names an unfamiliar earlier concept, follow the backward link, learn the model, then return. The repetition is deliberate: the same ideas—**representation, state machine, ownership, graph, queue, invariant, capability, and feedback loop**—reappear at larger scales.

Jump to a Goal

Goal	Start here	Then continue to
 Build a parser	Grammar	Building a Parser
 Design a PEG grammar	Pest in Practice	Execution Pipeline
 Build an interpreter	Execution Pipeline	Bytecode & VMs
 Build a hosted runtime	Hosted Runtime Design	Minimal Lua Interpreter
 Add static types	Types & Type Inference	LLVM IR
 Test LLVM codegen	Inkwell Test Patterns	Compiler Debugging
 Understand memory	Memory Layout	Low-Level Memory
 Design a memory manager	Allocation Mechanics	Garbage Collection
 Debug memory problems	Memory Accounting	Memory Debugging
 Understand the OS	Processes & the Kernel	Executables & Linking
 Reverse x64 Windows	x64 Windows Reversing	Calling Conventions
 Reverse a binary or VM	Reverse Engineering	Reversing a Binary
 Write safer Rust tools	Idiomatic Rust	Networking
 Translate Python to Rust	Rust/Python Idioms	Functional Rust
λ Learn functional Rust	Functional Rust	Algorithms
 Practice algorithms	Two Pointers	Compiler Applications
 Learn linked lists	Linked-List Ownership	Low-Level Memory
 Solve repeated states	Backtracking & Memoization	Dynamic Programming
 Learn CS mathematics	Mathematical Foundations	Algorithms
 Understand ML	Machine Learning	System Design
 Learn media & GPU math	Math, 3D, Audio & Video	SDL3
 Design an actor system	Actors	System Design
 Design a service	System Design in Rust	Networking
 Choose data & caching	Data, APIs & Caches	HTTP Caching
 Learn identity/security	Network Services & Security	Rust Web Boundaries

 Write low-level Rust	Executables & Linking	Bare Metal & Unsafe
 Learn firmware & VMs	Bare Metal & Unsafe	Hypervisors and Hyper-V
 Design Rust APIs	Idiomatic Rust	Rust Patterns & Recipes
 Embed a TCP/IP stack	smoltcp	QUIC with Quinn
 Learn QUIC	QUIC with Quinn	Browser Engines
 Build Rust web code	Rust Web Boundaries	Applied Rust
 Control the browser DOM	DOM from Rust	Wasm Ownership
 Understand a browser	Browser Engines	Rust Web Boundaries
 Build a GUI	GUI Architecture	SDL3
 Build a TUI	TUI Architecture	Ratatui Application
 Build a native loop	SDL3	Bevy
 Ship a Rust service	Zero to Production	Actix Web
 Build applied Rust apps	Applied Rust	Practical Workflow
 Study memory tooling	Low-Level Memory	Authorized Memory Labs
 Study defensive scanners	Authorized Reversing	Pattern Scanning
 Master developer tools	Developer Tooling	Practical Workflow
 Automate repetitive work	Practical Automation in Rust	Practical Workflow
 Look up a low-level term	Cross-Linked Glossary	Follow the term's Learn more link

Reading Conventions

-  **Definition** introduces a concept.
-  **Mental model** gives a useful approximation, not a formal guarantee.
-  **Invariant** is a condition that must remain true for an implementation to be correct.
-  **Boundary** is where data changes representation or trust level.
-  **Failure mode** names what can go wrong and why.
-  **Rust examples** favor clarity and explicit checks over cleverness.

 **Source rule:** these notes distill the linked material into transferable mental models. For version-sensitive APIs, command flags, ABIs, and hardware details, verify the current primary source before depending on an example.

Table of Contents

► Click to expand all 48 curriculum chapters



Computer Science as Layers of State and Transformation

The archived [Carl Cheo computer-science overview](#) groups beginner concepts into algorithms/data structures, artificial intelligence, architecture, concurrency, security, and development methodology. The analogies are useful entry points; the tables below tighten them into contracts that transfer to language implementation and reverse engineering.

1. The Major Areas Reinforce Each Other

Area	Central question	Language/reversing connection
Algorithms and data structures	How is information represented and transformed?	ASTs, symbol tables, graphs, worklists
Artificial intelligence	How can a system search, learn, or choose?	optimizers, heuristics, autotuning
Computer architecture	What physically executes an operation?	instruction sets, caches, ABI, memory layout
Concurrency	What can overlap, and how is shared state protected?	parallel passes, runtimes, event loops
Security	What does an input or caller have permission to do?	parser limits, sandboxing, binary analysis
Development methodology	How is uncertain work divided and validated?	compiler stages, tests, reproducible investigations



Learning rule: an analogy starts a model; an invariant finishes it. Ask what the analogy hides—costs, edge cases, ownership, failure, or adversarial behavior.

2. Complexity Describes Growth

Big-O notation describes an **asymptotic upper bound**. It is often used for worst-case time, but Big-O itself does not mean “worst case”; best, average, and amortized behavior must be named separately.

Growth	Example	What happens as input doubles
$O(1)$	indexed array access	roughly unchanged
$O(\log n)$	binary search	one extra step
$O(n)$	scan every token	roughly doubles
$O(n \log n)$	comparison sort	a little more than doubles
$O(n^2)$	compare every pair	roughly quadruples
$O(2^n)$	enumerate all subsets	roughly squares the work

For a compiler, complexity is only part of the cost:

```
total cost ≈ algorithmic growth
            × constant work per item
            × allocation/cache behavior
            × number of repeated passes
```

3. Data Structures Encode Operations

Structure	Strength	Language-tool example
contiguous array/ <code>Vec</code>	iteration and indexed access	tokens, bytecode, constant pools
linked structure	stable links, local insertion	intrusive runtime lists in rare cases
stack	last-in-first-out	parser states, calls, expression eval
queue	first-in-first-out	BFS, event processing
hash table	expected fast key lookup	symbols, interners, caches
tree	hierarchy or ordered partition	syntax, scopes, search indexes
graph	arbitrary relationships	control flow, call graph, dependencies
heap/priority queue	repeatedly select best candidate	schedulers, Dijkstra, best-first search

Choose a structure from the operations and invariants, not from its familiarity.

4. Search Strategies Trade Certainty for Speed

Strategy	Decision rule	Typical weakness
greedy	take the best immediate choice	local choice may block global optimum
hill climbing	move to a better neighboring state	local maxima and plateaus
random restart	run local search from several initial states	more work; still no general guarantee
simulated annealing	sometimes accept worse moves, less often later	schedule-sensitive and approximate
dynamic programming	reuse overlapping subproblem results	state space may still be too large
exhaustive search	inspect every candidate	combinatorial explosion

Compiler register allocation, instruction selection, inlining, and layout can all use heuristics because globally optimal solutions may be too expensive. Record when a pass is exact, approximate, or nondeterministic.

5. Computability Is Not Complexity

Question	Meaning
Computable?	Can any algorithm solve the problem for every valid input?

Decidable?	Can an algorithm always halt with yes/no?
Tractable?	Can it be solved with acceptable resources?
Verifiable?	Can a proposed answer be checked efficiently?

The **halting problem** shows that no general algorithm can decide whether every arbitrary program will halt. A practical analyzer can still prove termination for a restricted language or return **unknown** when its proof is insufficient.

```
enum Analysis<T> {
    Proven(T),
    Disproven { reason: String },
    Unknown { limit_reached: bool },
}
```

This three-way result is more honest than treating "not proven" as "false."

6. Concurrency Is Not Parallelism

Concept	Precise idea
concurrency	tasks have overlapping lifetimes/progress
parallelism	work executes at the same physical instant
race condition	behavior depends on an uncontrolled ordering
mutex	one holder at a time enters a critical section
semaphore	a bounded number of permits guard a resource
deadlock	tasks wait in a cycle that cannot make progress

A compiler can be concurrent without being parallel—for example, an event loop can interleave file reads and diagnostics on one thread. Parallel compilation needs actual simultaneous workers and data dependencies that permit them to run safely.

7. Security Is Boundary Engineering

Principle	Concrete question
least privilege	What is the smallest capability this component needs?
defense in depth	Which independent checks contain one failure?
complete mediation	Is every sensitive operation authorized?
secure defaults	Is the safe behavior selected without extra setup?
fail closed	Does an error deny access rather than silently allow it?
auditability	Can important decisions be reconstructed later?

Cryptography provides tools for confidentiality, integrity, authentication, and signatures; it does not automatically make the surrounding parser, protocol, key storage, randomness, or authorization correct. Use established cryptographic libraries and protocols rather than inventing production cryptography.

8. Methodology Is a Feedback Loop

Style	Useful when	Main risk
sequential plan	requirements and interfaces are stable	late discovery invalidates earlier work
iterative delivery	uncertainty is high and feedback is available	unmanaged scope and architectural drift
prototype	testing a risky idea or interface	prototype accidentally becomes production
experiment	comparing measurable hypotheses	weak controls produce misleading results

The compiler pipeline benefits from small, testable increments:

```
define one contract
→ implement one stage
→ inspect its artifact
→ test boundaries and failures
→ connect it to the next stage
```

 **Next:** mathematical foundations make the map precise by defining sets, relations, logic, proof, probability, graphs, linear transformations, and finite numerical representations.

 Mathematical Foundations for Computer Science

Computer science uses mathematics to describe **state, relationships, change, and limits** precisely. The purpose is not to memorize symbols. It is to gain a small set of models that let you predict what an algorithm, machine, protocol, or program can do before you run it.

 **Core mental model:** a mathematical model deliberately ignores some physical detail so one important relationship becomes easier to reason about. An implementation then chooses finite representations—integers, floats, arrays, graphs, bit fields—and must account for the details the model ignored.

Use the same six questions throughout this chapter:

Question	What it reveals
What are the objects?	numbers, bits, sets, nodes, vectors, events
What operations are legal?	addition, comparison, traversal, composition
Which relationships matter?	order, reachability, dependence, distance

What remains invariant?	the fact that proves each transition is safe
What is finite on a computer?	width, precision, memory, time, samples
How can the model fail?	overflow, approximation, ambiguity, state explosion

1. Mathematical Objects Answer Different Questions

Tool	Question it answers	Where it appears in a computer
sets	Which distinct values belong to a collection?	type domains, permissions, query results
relations	Which objects are connected or comparable?	database rows, graphs, dependency edges
functions	How does each valid input determine an output?	instructions, compiler passes, APIs
logic	Which conclusions follow from known facts?	branches, circuits, type checking
counting	How many states or arrangements are possible?	complexity, testing, key spaces
modular arithmetic	How do values behave when they wrap?	clocks, hashes, ring buffers, cryptography
probability	How should uncertainty be quantified?	retries, randomized algorithms, inference
graph theory	How does information move through relationships?	ASTs, calls, networks, build graphs
linear algebra	How do linear quantities and transforms compose?	graphics, audio, simulation, ML
calculus	How does a quantity change or accumulate?	motion, optimization, signal processing

These are not rival descriptions. A single system uses several at once. A network router has a graph of routes, finite packet buffers, Boolean policy predicates, probabilistic loss, and counters that wrap.

2. Sets, Relations, and Functions Describe Structure

A **set** is a collection whose members are considered without duplicates. A **relation** is a set of ordered pairs. A **function** is a relation in which each input in the function's domain maps to exactly one output.

Idea	Example	Computer-science meaning
membership, $x \in S$	a token kind belongs to the expression-start set	parser decision
subset, $A \subseteq B$	admin permissions contain reader	authorization

	permissions	
union, $A \cup B$	values accepted by either type	sum/union type
intersection, $A \cap B$	capabilities supported by both peers	protocol negotiation
Cartesian product, $A \times B$	every state paired with every input	state-machine table
partial function	lookup may have no result	<code>Option<T></code>
fallible function	decoding may reject bytes	<code>Result<T, E></code>

Rust makes the difference between total, partial, and fallible computation visible:

```
use std::collections::HashMap;

fn total_square(value: i64) -> i64 {
    // Every i64 input has a mathematical square, although machine overflow still
    // requires a policy if this runs outside checked arithmetic.
    value * value
}

fn partial_lookup<'a>(
    table: &'a HashMap<String, u32>,
    name: &str,
) -> Option<&'a u32> {
    // Absence is a normal part of this relation, so the type exposes it.
    table.get(name)
}

fn fallible_decode(bytes: &[u8]) -> Result<u16, &'static str> {
    // The function's accepted domain is all byte slices, but only two-byte
    // slices satisfy this particular representation contract.
    let pair: [u8; 2] = bytes.try_into().map_err(|_| "expected two bytes")?;
    Ok(u16::from_be_bytes(pair))
}
```

 **Library lens:** a parser combinator represents languages as sets of accepted strings and combines them with operations resembling union, sequence, and repetition. A database query represents relations and transforms them with selection, projection, joins, grouping, and ordering.

3. Logic Turns Facts into Decisions

Propositional logic treats complete statements as true or false. **Predicate logic** adds variables and quantifiers such as “for every” and “there exists.” A program’s conditionals, type rules, protocol guards, and hardware gates all implement logical decisions.

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Two useful equivalences are **De Morgan's laws**:

```
NOT (A AND B) = (NOT A) OR (NOT B)
NOT (A OR B) = (NOT A) AND (NOT B)
```

They explain why guard clauses can replace deeply nested conditions and why bit masks can implement sets efficiently.

```
const READ: u8 = 0b001;
const WRITE: u8 = 0b010;
const EXECUTE: u8 = 0b100;

fn has_all(actual: u8, required: u8) -> bool {
    // AND keeps only bits present in both sets. Equality proves every required
    // permission survived the intersection.
    actual & required == required
}

fn remove(actual: u8, denied: u8) -> u8 {
    // NOT flips the denied mask; AND then keeps every non-denied permission.
    actual & !denied
}
```

Logic is only as sound as its premises. If an authorization check trusts an unverified claim, flawless Boolean algebra still produces the wrong security decision.

4. Proofs and Invariants Explain Why a Program Works

A test checks selected examples. A proof explains why a claim follows for every input covered by its assumptions.

Reasoning method	Shape	Programming use
direct proof	assumptions → valid steps → conclusion	derive a bound or type rule
contradiction	assume the opposite and derive impossibility	uniqueness and impossibility results
induction	base case plus size-preserving step	recursion and tree/list algorithms
case analysis	cover every member of a finite set	exhaustive <code>match</code>
loop invariant	fact true before and after every iteration	searching, sorting, parsing

A binary search is correct because of an invariant, not because it “cuts the array in half”:

```

fn binary_search(sorted: &[i32], target: i32) -> Option<usize> {
    let mut low = 0;
    let mut high = sorted.len();

    // Invariant: if target exists, it is somewhere in sorted[low..high].
    while low < high {
        let middle = low + (high - low) / 2;

        match sorted[middle].cmp(&target) {
            std::cmp::Ordering::Less => {
                // Everything through `middle` is too small.
                low = middle + 1;
            }
            std::cmp::Ordering::Greater => {
                // `middle` and everything after it are too large.
                high = middle;
            }
            std::cmp::Ordering::Equal => return Some(middle),
        }
    }

    // The candidate interval is empty, so the invariant proves absence.
    None
}

```

When reading code, look for the invariant each mutation is meant to preserve. Ownership types, database constraints, protocol sequence numbers, and GPU resource states are all ways of making invariants executable.

5. Counting Explains State Explosion

The **sum rule** counts mutually exclusive choices. The **product rule** counts independent sequences of choices.

Situation	Count
choose one of n alternatives	n
choose an A and then a B	$ A \times B $
subsets of n independent flags	2^n
arrangements of n distinct items	$n!$
choose k items without order	$n! / (k!(n-k)!)$

This is why exhaustive testing becomes impossible quickly. Twenty independent Boolean options already describe $2^{20} = 1,048,576$ configurations. Protocols and APIs therefore try to reduce independent flags, eliminate illegal combinations with types, and test equivalence classes rather than every raw input.

The **pigeonhole principle** says that placing more than n objects into n boxes forces at least one collision. A finite hash has more possible inputs than outputs, so collisions are mathematically unavoidable; a hash table manages them, while a cryptographic design makes chosen collisions computationally difficult.

6. Number Theory and Modular Arithmetic Model Finite Cycles

Two integers are congruent modulo m when they leave the same remainder:

$$a \equiv b \pmod{m} \quad \text{when } m \text{ divides } (a - b)$$

Modular arithmetic models clocks, sequence numbers, ring buffers, integer wraparound, checksums, and much of public-key cryptography. But machine overflow and mathematical modulo are not automatically the same contract—signed remainder rules and overflow behavior vary by language and operation.

```
fn ring_index(sequence: usize, capacity: usize) -> Option<usize> {
    if capacity == 0 {
        return None;
    }
    // Congruence classes map an unbounded logical sequence into finite slots.
    Some(sequence % capacity)
}

fn gcd(mut left: u64, mut right: u64) -> u64 {
    // Euclid's invariant: gcd(left, right) is unchanged when
    // (left, right) becomes (right, left mod right).
    while right != 0 {
        let remainder = left % right;
        left = right;
        right = remainder;
    }
    left
}
```

Wrapping sequence numbers need a bounded-distance comparison. Ordinary $<$ cannot tell which of two wrapped counters is logically newer without assuming they are less than half the number space apart.

7. Probability Quantifies Uncertainty

A **sample space** lists possible outcomes. An **event** is a subset of outcomes. A **random variable** maps outcomes to values. The **expected value** is a probability-weighted average, not a promise about any one run.

Concept	Meaning	Systems example
independence	learning one event does not change the other's probability	idealized separate coin flips
conditional probability	probability of A given evidence B	failure given a warning signal
Bayes' rule	update a belief using evidence and base rates	alert interpretation
expectation	long-run weighted average	retries, latency, memory cost
variance	spread around the mean	jitter and tail instability
distribution	probability assigned across outcomes	request sizes, packet delay

For independent attempts with failure probability p , the probability that all n attempts fail is p^n . This explains why a small number of retries can improve success, but it does **not** justify unbounded retry storms. Retries also add load and can make failures correlated.

Bayes' rule highlights the base-rate problem:

$$P(\text{cause} \mid \text{signal}) = \frac{P(\text{signal} \mid \text{cause}) \times P(\text{cause})}{P(\text{signal} \mid \text{cause}) \times P(\text{cause}) + P(\text{signal} \mid \text{no cause}) \times P(\text{no cause})}$$

A scanner with a low false-positive rate can still produce mostly false alerts when the condition it detects is extremely rare.

8. Information Theory Connects Uncertainty to Encoding

Information content increases as an event becomes less likely. Shannon entropy measures the average uncertainty of a source:

$$H(X) = -\sum p(x) \log_2 p(x)$$

Idea	Mechanism
compression	assign shorter representations to more likely patterns
checksum	detect accidental changes; not designed against an attacker
cryptographic hash	compress arbitrary input into a fixed fingerprint with security goals
encoding	choose a representation; does not imply secrecy
encryption	transform data under a key to provide confidentiality

Lossless compression cannot make every possible input shorter: if all n -bit strings compressed to fewer than n bits, there would not be enough shorter outputs to represent them uniquely. Compressors win by exploiting nonuniform structure in actual data.

9. Graph Theory Models Relationships and Reachability

A graph contains **vertices** and **edges**. Edges may be directed, undirected, weighted, labeled, cyclic, or acyclic.

Graph	Vertices	Edges	Useful operation
AST	syntax nodes	parent/child relation	traversal and rewriting
call graph	functions	possible calls	reachability and inlining
control-flow graph	basic blocks	possible jumps	data-flow analysis
dependency graph	packages/tasks	"requires"	topological ordering
object graph	heap objects	references	garbage-collection tracing

network	machines/routers	links	routing and fault analysis
---------	------------------	-------	----------------------------

Breadth-first search explores by distance in unweighted edges. Depth-first search explores one path deeply and is natural for dependency analysis, cycle detection, and recursive structures. A **topological order** exists only for a directed acyclic graph.



Deep connection: tracing garbage collection, module dependency resolution, reachability-based linker elimination, and “find every node reachable from here” are the same graph problem with different vertex and edge meanings.

10. Linear Algebra Describes Composable Transformations

A vector can represent a position, direction, velocity, color, audio frame, or feature row. A matrix represents a linear transformation. Matrix multiplication composes transformations, and its order matters.

Operation	Meaning
vector addition	combine displacements or signals
scalar multiplication	change magnitude
dot product	measure alignment or projection
matrix-vector product	transform one value
matrix-matrix product	compose transforms
norm	measure magnitude
basis change	express one vector in another coordinate system

```
#[derive(Clone, Copy)]
struct Vec3 {
    x: f32,
    y: f32,
    z: f32,
}

impl Vec3 {
    fn dot(self, other: Self) -> f32 {
        // The dot product accumulates component-wise agreement.
        self.x * other.x + self.y * other.y + self.z * other.z
    }

    fn try_normalize(self) -> Option<Self> {
        let squared_length = self.dot(self);
        if !squared_length.is_finite() || squared_length <= f32::EPSILON {
            return None;
        }

        // Dividing by length changes representation while preserving direction.
        let scale = squared_length.sqrt().recip();
        Some(Self {
            x: self.x * scale,
            y: self.y * scale,
            z: self.z * scale,
        })
    }
}
```

Types such as `Point3`, `Direction3`, `Normal3`, `Radians`, and `Degrees` distinguish objects that may share a numeric representation but obey different legal operations.

11. Calculus and Difference Equations Model Change

A **derivative** measures local rate of change. An **integral** accumulates a quantity over an interval. A **gradient** collects partial derivatives and points in the direction of steepest increase.

Computers usually approximate continuous models with discrete samples:

```
velocity ≈ (new_position - old_position) / time_step
new_position = old_position + velocity × time_step
```

```
#[derive(Clone, Copy)]
struct Motion {
    position: f64,
    velocity: f64,
}

fn integrate(mut state: Motion, acceleration: f64, dt: f64) -> Motion {
    // Semi-implicit Euler: update velocity, then use the new velocity.
    // `dt` is part of the model; a huge or variable step can make it unstable.
    state.velocity += acceleration * dt;
    state.position += state.velocity * dt;
    state
}
```


The approximation has an error model. Smaller time steps usually reduce numerical error but require more work; floating-point rounding and unstable equations can still accumulate error.

Continuous idea	Discrete implementation
derivative	finite difference
integral	weighted sum of samples
differential equation	recurrence / update rule
continuous signal	sampled sequence
gradient descent	repeated parameter update

12. Floating Point Is an Approximation Contract

An IEEE-style floating-point value stores a **sign**, a scaled **significand**, and an **exponent**. This provides large dynamic range, but most real values—including `0.1` — cannot be represented exactly in binary.

Property	Consequence
finite precision	nearby real values round to the same bit pattern
nonuniform spacing	large-magnitude values have coarser gaps
special values	<code>NaN</code> , positive/negative infinity, and signed zero exist
non-associativity	<code>(a + b) + c</code> can differ from <code>a + (b + c)</code>
gradual underflow	subnormal values trade precision for range
<code>NaN != NaN</code>	ordinary equality is not a validity test

```
fn approximately_equal(left: f32, right: f32, absolute: f32, relative: f32) -> bool {
    if !left.is_finite() || !right.is_finite() {
        return false;
    }

    let error = (left - right).abs();
    // Absolute tolerance protects comparisons near zero. Relative tolerance
    // scales with magnitude. Neither is a universal epsilon.
    error <= absolute.max(relative * left.abs().max(right.abs()))
}
```

Use integers or fixed-point values when exact cents, ticks, sample positions, or wire fields matter. Use floating point for continuous quantities, but define units, non-finite handling, tolerances, and determinism requirements.

13. Complexity Is a Model of Growth, Not a Stopwatch

Asymptotic notation describes how resource use grows as input size grows:

Class	Typical shape	Example

$O(1)$	fixed work	array indexing
$O(\log n)$	repeatedly shrink the remaining space	binary search
$O(n)$	visit each item	scan
$O(n \log n)$	divide plus linear merge/work	comparison sorting
$O(n^2)$	compare many pairs	simple nested-loop algorithms
$O(2^n)$	explore subsets	unrestricted backtracking

Big-O hides constants, hardware, allocation, cache locality, vectorization, and input distribution. A complete engineering cost model asks:

- How many bytes move through each memory level?
- Is access contiguous or pointer-chasing?
- How many allocations and system calls occur?
- Is work parallel, serial, blocking, or asynchronous?
- What are the worst-case and tail-latency bounds?
- Which input controls the cost?



Rust/library connection: iterators expose algebraic pipeline composition, slices expose contiguous representation, `Result` exposes partial functions, graph crates expose traversal, Diesel exposes relational operations, and wgpu exposes linear transformations plus asynchronous resource constraints. The library is useful here because its API turns the mathematical model into enforceable program structure.



Next: hardware turns these abstract objects into finite bit patterns, instructions, registers, memory transactions, and timed state changes.



Hardware Fundamentals: How a CPU Changes State

The CPU

If you want to create a “computer” from scratch, you need to start by defining an **abstract model** for your computer. This *abstract model* is also referred to as **Instruction Set Architecture (ISA)**

A **CPU** is an implementation of an **Instruction Set Architecture (ISA)** — an abstract model defining data types, registers, hardware support, and I/O. Together these make up **Machine Language**, the lowest-level language of computing.

The CPU continuously runs a **fetch-decode-execute** loop:



Core loop: **fetch** an instruction, **decode** what it means, then **execute** its effect.

1. **Fetch** — Retrieve the instruction addressed by the **Instruction Pointer (IP)** / Program Counter (PC).
2. **Decode** — Interpret the **opcode** (a unique encoding for an operation — the “atoms of computing”), plus any operands (arguments) and optional prefix (behavioral modifier).
3. **Execute** — The **Control Unit (CU)** dispatches signals to functional units like the **ALU** (Arithmetic Logic Unit), which performs the operation on register values. Results are written back to memory if applicable.

Modern CPUs accelerate this loop with **instruction pipelining** and **speculative execution**.

Number Systems: Binary, Decimal, Hexadecimal

CPUs are built from circuits that are either **on** or **off** — so at the hardware level, everything is represented in **binary (base-2)**: only the digits **0** and **1**.

System	Base	Digits	Example
Binary	2	0, 1	1101
Decimal	10	0 – 9	126
Hexadecimal	16	0 – 9, A – F	0xA1D

Each system is just a different way of writing the same value — e.g. **1101** in binary is **13** in decimal. Binary gets unwieldy fast (the decimal number **250** is **11111010** in binary), so **hexadecimal** (prefixed **0x**) is the standard shorthand in computing: each hex digit maps cleanly to exactly 4 binary bits, so **250** is simply **0xFA**. This is why opcodes, memory addresses, and register dumps are almost always shown in hex.

Opcodes in Practice

Instructions are comprised of instruction code (aka operation code, in short **opcode** or *p-code*), directly executed by the *CPU*. An opcode can either **have operand(s)** or **no operand**.

In an 8-bit machine where instructions are 8 bits:

- LOAD 0101** → **00110101** — the first 4 bits (**0011**) are the opcode for *load*, the second 4 bits (**0101**) are the operand.
- INCREMENT** → **1000** — the opcode for *increment by 1*, no operand needed.

Since opcodes are the atoms of computing, they are presented in an **opcode table** (e.g., the [x86 opcode reference](#)).

A few opcodes worth knowing by name:

- nop** (**0x90** on x86, “no operation”) — the CPU does nothing and advances to the next instruction. It’s a useful placeholder, and understanding it makes reading disassembly less intimidating: not every instruction needs to matter.
- cmp** — compares two values.
- jmp / je / jne** (“jump”, “jump if equal”, “jump if not equal”) — conditionally redirect execution to a different address instead of continuing sequentially. This mechanism — compare, then conditionally jump — is called **branching**, and it’s the opcode-level foundation of every **if / else** you write in a high-level language.

Assembly Language

Because bit-patterns are hard to remember, **Assembly Language** assigns abstract, human-readable symbols to opcodes matching their operations by name:

```
00110101 → LOAD 0101
```

Here, `0011` is abstracted to the symbol `LOAD`, so `00110101` can be written as `LOAD 0101` — a higher-level, human-readable form of the same instruction.

A utility program called an **Assembler** translates Assembly back to Machine Language.

Assembly is *high-level* relative to machine code, but *low-level* relative to C, Rust, or Python. High-level and low-level are **relative terms** that convey the amount of abstraction involved.

Memory Layout

When a program is loaded into memory, it creates a **process** with three key regions:

Region	Purpose
Static	Global variables and constants
Stack	Function frames and local variables — auto-managed, LIFO
Heap	Dynamically allocated data shared across functions and threads

- **Stack analogy:** A notepad — write, tear off when done.
- **Heap analogy:** A whiteboard — write, stays until you explicitly erase it.

Objects live on the heap because they need to *outlive* the function that created them.

How an OS knows to run a program at all: each OS defines its own **executable format** — a file layout the OS knows how to parse and load. On Windows this is the **PE (Portable Executable)** format; on Linux it's **ELF**. Both split the file into named sections, most notably a `.text` section (the compiled opcodes — the actual program code) and a `.data` section (initial values for global variables). This is why an executable built for one OS won't run on another: the OS's loader is looking for a specific format it doesn't recognize.

Representation and Translation: From Source to Machine State

Informally, a **language** is structured text with syntax and semantics. Source code written in a programming language needs:

1. A **translator** of some sort — converts it to another language/format.
2. An **executor** of some sort — runs the translated commands to produce output.

Levels of Abstraction

```
Machine Language → Assembly → IR → Bytecode → Source Language
(lowest)                                     (highest)
```

Level	Description
Machine Language	Raw bit-pattern opcodes directly executed by the CPU

Assembly Language	Human-readable symbols mapped to opcodes; translated by an Assembler
IR (Intermediate Representation)	Any format between source and assembly; converting between IRs is called <i>lowering</i>
Bytecode	An IR emulating a simplified instruction set; executed by a Virtual Machine (VM)

Classifying Languages

Languages can be described along two independent axes:

Axis	Options	Meaning
What it executes on	Compiled vs. Interpreted	Compiled code runs directly on the CPU; interpreted code is translated on the fly by another program
How it executes	Imperative vs. Functional	Imperative runs instructions top-to-bottom; functional resolves through function declaration and evaluation

These combine independently — e.g. C is compiled + imperative, Java is interpreted (its bytecode runs on the JVM) + imperative, Haskell is compiled + functional. An interpreted language can still be *distributed* as something that looks compiled, by bundling the interpreter together with the source/bytecode it runs.

There's no "best" combination — different points on these axes trade off differently for different problems, and (per the Crash Course discussion) every language ultimately answers to the same question: how does this text become something the CPU executes?

Compilers

A **compiler** is any program that translates (maps, encodes) Language A → Language B. Its two core components:

- **Frontend** — Maps source code strings to a structured format called an **Abstract Syntax Tree (AST)**.
- **Backend (Code Generator)** — Translates the AST to Bytecode, IR, or Assembly.

Most often, when we talk about compiler, we mean Ahead-Of-Time (AOT) compiler where the translation happens before execution. Another form of translation is Just-In-Time (JIT) compilation where translation happens right at the time of the execution.

Translation Type	When It Happens	Examples
AOT (Ahead-of-Time)	Before execution	<code>rustc</code> , <code>gcc</code>
JIT (Just-in-Time)	During execution	V8 (JS), PyPy (Python)
Transpiler	Source-to-source	Python → Java

Virtual Machines (VMs)

Hardware instructions are vendor-specific — Intel and AMD instructions differ. A **Virtual Machine (VM)** abstracts away hardware details so that code compiled to the VM’s language becomes **platform-agnostic**.

The most famous example: the **Java Virtual Machine (JVM)**. Any valid Java Bytecode runs on any platform with a **Java Runtime Environment (JRE)**, regardless of where it was compiled.

Memory Systems: Representation, Allocation, and Runtime State

Understanding memory is useful in three directions at once:

1. **Building a language** — you must decide how values, objects, strings, stack frames, and garbage or ownership metadata are represented.
2. **Debugging low-level software** — you must connect a source-level value to bytes, addresses, registers, and instructions.
3. **Reversing a language or binary format** — you infer those same representation decisions from observable behavior.

Use these techniques only on software, memory dumps, packet captures, and systems you own or are explicitly authorized to study. Prefer an isolated VM, disposable test program, local server, or purpose-built target.

 **Safety and authorization boundary:** low-level visibility is powerful. Practice on software and systems you own or have explicit permission to inspect.

1. Bytes, Addresses, Pointers, and Views

A **byte** is data. An **address** is a number naming a location. A **pointer** is a typed or untyped value containing an address. The same bytes may be interpreted through different views:

```
bytes:      48 65 6c 6c 6f 00
ASCII:      H e l l o \0
u16 (LE):  0x6548 0x6c6c 0x006f
```

The bytes do not change; the interpretation does:

Context	Interpretation applied to the same bytes
Disassembler	Opcodes, operands, and instruction boundaries
Executable/packet parser	Header fields, lengths, flags, and sections
Virtual machine	Tagged values, handles, or bytecode instructions
Debugger	Runtime objects, fields, pointers, and strings
File-format reversing	Hypothesized records and relationships

Endianness

For the integer `0x12345678` :

```
big-endian:    12 34 56 78
little-endian: 78 56 34 12
```

Network protocols traditionally call big-endian **network byte order**. x86 and x86-64 normally use little-endian values in memory. Never decode a multi-byte integer without deciding its width and byte order.

```
fn read_u32_be(input: &[u8]) -> Option<u32> {
    let bytes: [u8; 4] = input.get(..4)?.try_into().ok()?;
    Some(u32::from_be_bytes(bytes))
}

fn read_u32_le(input: &[u8]) -> Option<u32> {
    let bytes: [u8; 4] = input.get(..4)?.try_into().ok()?;
    Some(u32::from_le_bytes(bytes))
}
```

Pointer arithmetic is really layout arithmetic

If an array element has size `S`, element `i` begins at:

```
element_address = base_address + i * S
```

For a record or object:

```
field_address = object_address + field_offset
```

A multi-level pointer path repeatedly reads an address and then applies the next offset:

```
root -> object -> component -> field
```

Do not confuse a stable *relationship* with a stable absolute address. ASLR, allocator behavior, and different runs can move stack, heap, libraries, and sometimes the main executable.

2. Memory Regions and Object Lifetimes

Region	Typical contents	Lifetime
Code / <code>.text</code>	Machine instructions	Loaded image
Read-only data	Constants, string literals	Loaded image
Static data	Globals	Process
Stack	Call frames, local values, saved state	Function or scope
Heap	Dynamic objects, collections, closures	Explicit or managed
Memory-mapped region	Files, shared libraries, device memory	Mapping

This table is a model, not a promise that every compiler and OS lays out every value exactly this way. Optimizers may keep a local variable only in a register or remove it entirely.

Stack frame mental model

```
higher addresses
+-----+
| caller's frame |
+-----+
| return address |
| saved registers |
| local variables |
| temporary/spill data |
+-----+ <- stack pointer
lower addresses
```

Calling conventions define where arguments and return values go, which registers a callee must preserve, and how the stack is aligned. A compiler backend must obey the target ABI when calling external functions.

Heap lifetime hazards

Hazard	What went wrong
Leak	Unreachable memory is never released
Double free	One allocation is released twice
Use after free	A stale pointer outlives its allocation
Out-of-bounds access	An index or pointer escapes the allocation
Uninitialized read	Bytes are interpreted before receiving a valid value
Data race	Shared state is accessed without required synchronization

These hazards explain the trade-offs among manual allocation, garbage collection, reference counting, region/arena allocation, and Rust-style ownership.

Arena allocation for compilers

AST nodes often share one phase lifetime: they are all created during parsing and discarded together. An arena makes that lifetime explicit:

```
source text
↓
[AST arena: many small nodes]
↓ type checking / lowering
[IR arena: many small nodes]
↓
discard whole phase at once
```

This can simplify ownership and improve locality, but references into an arena must never outlive it.

3. Layout, Alignment, Padding, and repr

CPUs often prefer or require values at aligned addresses. A `u32` commonly has alignment 4, so a compiler may insert padding:

```
#[repr(C)]
struct Header {
    tag: u8,          // offset 0
                      // offsets 1..3 may be padding
    length: u32,      // offset 4
}

fn main() {
    assert_eq!(std::mem::align_of::<Header>(), 4);
    assert_eq!(std::mem::size_of::<Header>(), 8);
}
```

Key Rust rules:

- Default `repr(Rust)` does not promise a stable C-compatible field layout.
- `#[repr(C)]` requests C ABI layout and is useful at an FFI boundary.
- `#[repr(u8)]` or another integer representation can make an enum discriminant representation explicit.
- `#[repr(packed)]` removes some padding but can create unaligned fields. Creating a normal reference to an unaligned field is invalid; specialized unaligned reads are required.
- A Rust struct is not automatically a serialized file or wire format. Serialize each field explicitly.

For your own language, decide whether object layout is:

- **specified** as part of the language/ABI;
- **implementation-defined** but documented by one compiler;
- **opaque**, accessible only through generated accessors.

Opaque layouts give the runtime freedom to reorder fields or change garbage collectors. Specified layouts make FFI and reversing easier but reduce implementation freedom.

4. Safe Binary Parsing in Rust

A binary parser should treat all offsets, sizes, tags, and counts as untrusted. Prefer slices and checked arithmetic over raw pointers.

```

#[derive(Debug, PartialEq)]
struct Packet<'a> {
    kind: u8,
    payload: &'a [u8],
}

#[derive(Debug, PartialEq)]
enum ParseError {
    Truncated,
    TooLarge,
    TrailingBytes,
}

fn take<'a>(input: &mut &'a [u8], n: usize) -> Result<&'a [u8], ParseError> {
    let (head, tail) = input.split_at_checked(n).ok_or(ParseError::Truncated)?;
    *input = tail;
    Ok(head)
}

fn parse_packet(mut input: &[u8]) -> Result<Packet<'_>, ParseError> {
    let kind = take(&mut input, 1)?[0];
    let length = u16::from_be_bytes(
        take(&mut input, 2)?
        .try_into()
        .map_err(|_| ParseError::Truncated)?,
    ) as usize;

    const MAX_PAYLOAD: usize = 4096;
    if length > MAX_PAYLOAD {
        return Err(ParseError::TooLarge);
    }

    let payload = take(&mut input, length)?;
    if !input.is_empty() {
        return Err(ParseError::TrailingBytes);
    }

    Ok(Packet { kind, payload })
}

#[test]
fn parses_a_small_packet() {
    let bytes = [0x02, 0x00, 0x03, b'c', b'a', b't'];
    assert_eq!(
        parse_packet(&bytes),
        Ok(Packet { kind: 2, payload: b"cat" })
    );
}

#[test]
fn rejects_truncation() {
    assert_eq!(parse_packet(&[2, 0, 5, b'x']), Err(ParseError::Truncated));
}

```

Parser concern	Defensive requirement
Lengths/counts	Bound them before allocating or looping
Arithmetic	Use checked addition, multiplication, and slicing

Tags/enums	Reject unknown mandatory tags and invalid values
Trailing bytes	Define whether they are permitted
Structural limits	Cap recursion depth and decompressed output
Input delivery	Never assume one read contains a complete file or message
Diagnostics	Preserve byte offsets in errors
Robustness testing	Fuzz with arbitrary byte strings

The same parser architecture works for bytecode, object files, save files, image formats, and network frames.

5. Memory Inspection as a Scientific Method

The most general-purpose lesson is not a particular tool; it is the repeated workflow:

1. **Identify** a value or behavior you can control.
2. **Change one variable** while keeping other conditions stable.
3. **Observe** memory, registers, instructions, or packets.
4. **Form a hypothesis** about representation or control flow.
5. **Repeat** with a different input or a fresh run.
6. **Validate** by predicting an observation you have not yet seen.

For a safe toy example, pretend a process snapshot is just a byte vector:

```
fn find_u32_le(haystack: &[u8], wanted: u32) -> Vec<usize> {
    // Make the assumed byte order part of the search contract.
    let needle = wanted.to_le_bytes();

    haystack
        // Windows yields only complete four-byte candidates.
        .windows(needle.len())
        .enumerate()
        // Offsets are hypotheses to refine, not proof of a field's meaning.
        .filter_map(|(offset, window)| (window == needle).then_some(offset))
        .collect()
}

fn retain_changed(old: &[u8], new: &[u8], candidates: &mut Vec<usize>) {
    candidates.retain(|&offset| {
        // Checked addition keeps a malformed candidate from wrapping the end index.
        let Some(end) = offset.checked_add(4) else {
            return false;
        };

        // `get` rejects ranges outside either snapshot without panicking.
        old.get(offset..end)
            .zip(new.get(offset..end))
            .is_some_and(|(a, b)| a != b)
    });
}
```

This demonstrates differential scanning without touching another process. The same reasoning is useful for debugging your VM’s heap, comparing serialized ASTs, and locating a field in a captured memory image.

Static, dynamic, and forensic views

View	Evidence	Best for
Static	File bytes, symbols, strings, disassembly	Possible behavior and structure
Dynamic	Registers, memory, breakpoints, system calls	Actual behavior for one run
Forensic	A saved memory image or capture	Reconstructing prior runtime state

Memory-forensics workflow can be generalized as:

- 1. preserve the original image;
- 2. identify the OS, architecture, and capture context;
- 3. inventory processes, mappings, handles, and connections;
- 4. narrow to relevant artifacts;
- 5. corroborate findings across multiple sources;
- 6. record offsets, hashes, and tool versions so results are reproducible.

Strings are clues, not proof. A string may be unused, encoded, copied, stale, or unrelated.

6. Defensive Memory Safety and Hardening

Defensive memory-safety material is most useful when it helps you recognize what your compiler or runtime must prevent.

Failure	Root cause	Language/runtime defense
Buffer overflow	Write exceeds allocation	Slice bounds, checked APIs
Use after free	Lifetime not enforced	Ownership, GC, handles/generations
Double free	Ambiguous ownership	Move semantics, unique owner
Null dereference	Missing value represented as pointer	<code>Option<T></code> / tagged union
Type confusion	Bytes interpreted as wrong type	Tags, validation, safe casts
Integer overflow	Size calculation wraps	Checked arithmetic and limits
Data race	Unsynchronized shared mutation	Send/sync rules, actors, locks

Platform mitigation	What it hardens
Stack canaries	Detect some stack overwrites before return
NX/DEP	Mark data pages non-executable
ASLR	Randomize important memory regions between runs
PIE	Allow the main executable to be relocated

RELRO	Make selected relocation data read-only after linking
Control-flow integrity	Restrict indirect branches to approved targets

Mitigations are layers, not substitutes for memory-safe design.

Safe C example: bound the read to the actual destination capacity and ensure string termination.

```
#include <stdio.h>

int main(void) {
    char name[64];
    if (fgets(name, sizeof name, stdin) == NULL) {
        return 1;
    }
    printf("Hello, %s", name);
    return 0;
}
```

Testing layer	Recommended practice
Compiler warnings	Enable them and treat new warnings seriously
Native runtime	Use sanitizers on C/C++ components
Input boundaries	Fuzz lexers, parsers, verifiers, and decoders
Numeric limits	Test zero, maximum, and one-past-maximum values
Bytecode	Verify malformed input before execution
Unsafe Rust	Keep it small and document the relied-upon invariants

7. Reversing a Binary, Bytecode VM, or Language

Source compilation and reverse engineering travel in opposite directions:

```
source → tokens → AST → typed IR → machine IR → assembly → bytes
bytes → instructions → control-flow graph → data-flow facts → hypotheses
```

Lost information	Reverse-engineering consequence
Variable/function names	Recovered names are hypotheses or tool-generated labels
Comments/formatting	Intent and original organization cannot be reconstructed
Source distinctions	Several source forms may produce identical instructions
Call boundaries	Inlining can merge a callee into its caller
Expressions	Constant folding can erase the original computation
Source types	Only layout and operation clues may remain

Therefore, decompilation is a **model**, not restoration of the original source.

A disciplined reversing notebook

For each discovery, record:

Field	Example
Location	file offset, virtual address, function symbol
Observation	"four-byte big-endian value precedes payload"
Hypothesis	"value is compressed payload length"
Evidence	controlled inputs of lengths 1, 5, and 20
Confidence	low / medium / high
Falsifier	a capture whose value does not match payload length

Recovering a bytecode format

Start with the smallest programs:

```
0
1
1 + 2
print(1)
if true { print(1) }
```

Then compare outputs:

1. locate a header or magic number;
2. identify version, flags, and section boundaries;
3. find constant pools and encoded strings;
4. change one literal and locate the changed bytes;
5. infer opcode width and operand encoding;
6. trace stack-height changes or virtual registers;
7. build a disassembler before building a decompiler;
8. validate by round-tripping or predicting unseen encodings.

Example of a tiny, safe disassembler:

```

#[derive(Debug, PartialEq)]
enum Instr {
    Push(u8),
    Add,
    Print,
    Halt,
}

fn disassemble(mut bytes: &[u8]) -> Result<Vec<Instr>, &'static str> {
    let mut out = Vec::new();

    while let Some((&opcode, rest)) = bytes.split_first() {
        // Consume the opcode before decoding any instruction-specific operands.
        bytes = rest;

        match opcode {
            0x01 => {
                // `split_first` turns truncation into an ordinary parse error.
                let (&value, rest) = bytes.split_first().ok_or("missing operand")?;
                bytes = rest;
                out.push(Instr::Push(value));
            }
            0x02 => out.push(Instr::Add),
            0x03 => out.push(Instr::Print),
            0xff => {
                out.push(Instr::Halt);

                // This format defines HALT as the final byte, so trailing data is
                // rejected instead of being silently interpreted as another section.
                if !bytes.is_empty() {
                    return Err("bytes after halt");
                }
                return Ok(out);
            }
            _ => return Err("unknown opcode"),
        }
    }

    // Falling off the byte slice is invalid because every program must terminate.
    Err("missing halt")
}

```

Designing for reversibility

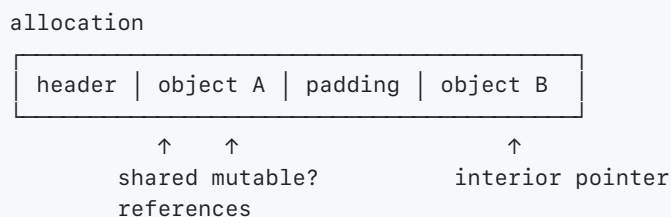
Reversibility feature	Benefit
Magic number + bytecode version	Reliable format identification
Documented section table	Predictable navigation and extension
Source maps/symbol names	Better diagnostics and debugging
Official dumper/disassembler	One canonical inspection path
Separate verifier	Safe validation without execution
Fail-closed versioning	Unknown formats cannot be misinterpreted

8. Separate Values, Objects, Storage, and Allocations

Memory discussions become confusing when several different concepts are all called “the value.” Keep the layers separate:

Term	Precise mental model
value	an abstract member of a type, such as integer <code>42</code>
representation	the bits/bytes used to encode a value
place	a location expression that can hold a value
object	a typed region of storage with a lifetime
allocation	a reserved storage region managed as one unit
address	a numeric location in an address space
pointer	a value used to locate or refer into storage
reference	a pointer-like value with stronger validity/aliasing promises
handle	an indirect stable identifier resolved through a table
lifetime	the interval during which an object may be used
reachability	whether a root can reach an object through runtime references

One allocation may contain several objects, and one object may be viewed through several references:



Conversely, a logical value can span several allocations:

```
Vec<String>
├─ vector header: pointer + length + capacity
├─ allocation containing String headers
└─ one or more allocations containing string bytes
```

Lifetime, Scope, and Storage Duration Are Different

Concept	Question
lexical scope	Where can a source-level name be written?
borrow lifetime	For how long may this reference be used?

object lifetime	When is this typed object initialized and later destroyed?
allocation lifetime	When is the backing storage reserved and released?
storage duration	Is storage automatic, static, dynamic, mapped, or managed?

A borrow may end before its variable leaves lexical scope because the compiler sees its last use. A heap allocation can outlive the function that created it. An arena allocation can stay reserved after individual objects are logically dead.



Debugging rule: before saying “this pointer is valid,” name the allocation, object, offset, type, initialization state, ownership path, and time interval that make the claim true.

9. Choose Ownership by the Shape of the Relationship

Rust ownership containers communicate who keeps an allocation alive:

Type	Ownership model	Thread use	Common fit
<code>T</code>	inline, uniquely owned value	depends on <code>T</code>	ordinary local/field
<code>&T</code>	temporary shared borrow	depends on <code>T: Sync</code>	read-only view
<code>&mut T</code>	temporary exclusive borrow	depends on <code>T: Send</code>	in-place mutation
<code>Box<T></code>	unique ownership of an allocation	depends on <code>T</code>	recursive/large/trait object
<code>Rc<T></code>	non-atomic shared ownership	single-threaded	AST/UI/runtime graph
<code>Weak<T></code>	non-owning observer of <code>Rc / Arc</code>	matches strong pointer	parent/cache/back-reference
<code>Arc<T></code>	atomic shared ownership	cross-thread capable	immutable/shared service state
<code>Cow<'a, T></code>	borrow until mutation requires a copy	depends on owned type	normalization and parsing
<code>Pin<P></code>	restricts moves through pointer <code>P</code>	depends on <code>P / target</code>	self-referential async/FFI states

`Rc::clone` and `Arc::clone` clone a pointer and increment a strong count; they do not deep-copy the inner value. `Arc<T>` makes ownership counting atomic, but it does not make arbitrary mutation of `T` safe.

```
Arc<Config>           shared immutable data
Arc<Mutex<State>>     shared mutation protected by a lock
Arc<AtomicU64>        one atomic integer, no lock
```

Break Ownership Cycles with Weak Edges

```

use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Default)]
struct Node {
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}

fn attach(parent: &Rc<Node>, child: &Rc<Node>) {
    *child.parent.borrow_mut() = Rc::downgrade(parent);
    parent.children.borrow_mut().push(Rc::clone(child));
}

```

```

parent —strong→ child
parent ←weak— child

```

Edge meaning	Usually strong or weak?
child cannot exist without parent	parent owns child
child merely knows its parent	weak back-reference
cache should not keep item alive	weak cache entry
task must finish using shared data	strong task reference

`Weak::upgrade` returns `Option` because the strong owners may already be gone. Reference counting reclaims objects deterministically when the strong count becomes zero, but strong cycles never reach zero.

Pinning Is Not “Permanent Memory”

Pinning restricts moving a value through a particular pointer. It does not:

- prevent the value from eventually being dropped;
- make dangling raw pointers valid;
- synchronize concurrent access;
- promise a stable address for every inner field after arbitrary projection; or
- make ordinary self-references safe without a sound construction API.

Most types implement `Unpin` and do not care about address stability. Reach for `Pin` only when a documented invariant genuinely connects object identity to its address.

10. Pointers Carry More Than a Numeric Address

A useful modern model treats pointer validity as including **provenance**: which allocation and access path authorized the pointer, in addition to its numeric address.

```
pointer use is valid only if:
  allocation is live
  + address is in-bounds (or permitted one-past)
  + alignment fits the access
  + bytes contain a valid initialized value
  + aliasing permissions allow this access
  + access kind matches page/object permissions
```

Two pointers with the same printed integer are not automatically interchangeable. An address recovered from stale storage or guessed from an integer does not by itself prove that dereferencing it is valid.

References Promise More Than Raw Pointers

For the period it is live, a normal Rust reference promises properties such as:

Reference	Key promise
<code>&T</code>	aligned, non-null, initialized, valid shared access to <code>T</code>
<code>&mut T</code>	aligned, non-null, initialized, exclusive access to <code>T</code>
<code>*const T</code> / <code>*mut T</code>	raw address-like value; dereference still needs proof

Creating an invalid reference can already violate Rust’s rules even if code never reads through it. Keep questionable states as raw pointers or `MaybeUninit<T>` until every reference invariant is established.

Interior Mutability Has an Explicit Boundary

`UnsafeCell<T>` is the primitive that permits mutation through a shared reference. Safe types build different policies on top:

Wrapper	Enforcement mechanism	Typical scope
<code>Cell<T></code>	copy/replace values; no references to interior	single thread
<code>RefCell<T></code>	runtime shared/exclusive borrow counter	single thread
<code>Mutex<T></code>	blocking mutual exclusion	multiple threads
<code>RwLock<T></code>	multiple readers or one writer	multiple threads
atomic type	hardware/compiler atomic operations	simple shared values

```
use std::cell::Cell;

struct VisitCounter {
    count: Cell<u64>,
}

impl VisitCounter {
    fn record(&self) {
        self.count.set(self.count.get().saturating_add(1));
    }
}
```

`UnsafeCell` relaxes the immutability assumption of `&T`; it does not permit data races or multiple live `&mut T` references. The wrapper must enforce the remaining rules.

Raw-Pointer Review Card

Before every dereference in `unsafe` code, answer:

1. Which live allocation contains the full access range?
2. How was this pointer derived?
3. Is the address correctly aligned for `T`?
4. Has a valid `T` been fully initialized there?
5. Which references or raw accesses may alias it now?
6. Can a callback, signal, interrupt, GC, or another thread invalidate the proof?
7. Who eventually drops the value and deallocates the storage?

11. Initialization Is a Type Invariant

"The bytes exist" does not mean "a value of type `T` exists." Types restrict valid bit patterns:

Type/category	Example invalid state
reference	null, misaligned, dangling, or invalid provenance
<code>bool</code>	byte pattern other than a valid boolean value
enum	invalid discriminant
<code>NonZeroUsize</code>	zero
<code>String</code> / <code>Vec<T></code>	nonsensical pointer/length/capacity relationship
custom type	broken documented semantic invariant

`MaybeUninit<T>` represents storage that may not yet contain a valid `T`:

```
use std::mem::MaybeUninit;

let mut slot = MaybeUninit::<String>::uninit();
slot.write(String::from("initialized"));

// SAFETY: the preceding `write` stored one valid String in `slot`.
let value = unsafe { slot.assume_init() };
assert_eq!(value, "initialized");
```

Operation	Responsibility
<code>MaybeUninit::uninit</code>	creates storage without claiming a valid <code>T</code>
<code>write(value)</code>	initializes without trying to drop old garbage
<code>assume_init</code>	caller proves a valid <code>T</code> is fully present
dropping <code>MaybeUninit</code>	does not drop a possibly initialized inner <code>T</code>

Zero bytes are valid for some types and invalid for others. Never use “zeroed” as a generic constructor.

Partial Initialization Needs Cleanup State

When initializing an array or object field-by-field, a failure can leave a prefix initialized:

```
[ initialized ][ initialized ][ initialized ][ uninit ][ uninit ]
      0           1           2           3           4
initialized_len = 3
```

A sound implementation tracks how many elements are live and drops exactly that prefix on failure. This is the low-level version of RAI: cleanup state must evolve together with initialization state.

⚠ **Padding rule:** padding may contain unspecified or uninitialized bytes. Do not compare structs byte-for-byte, hash raw struct storage, or write the storage directly to disk/network as a serialization format.

12. Allocator Mechanics

An allocator receives a request described by a **size and alignment**. Rust expresses this with `Layout`.

```
use std::alloc::Layout;

fn value_array_layout(count: usize) -> Result<Layout, &'static str> {
    Layout::array::<u64>(count).map_err(|_| "allocation layout overflow")
}
```

The allocator must return a suitably aligned block or report failure. If raw allocation APIs are used, deallocation must use the corresponding allocator and compatible layout.

What a General-Purpose Allocator Tracks

Mechanism	Purpose
size classes	group similarly sized requests
free lists/bins	find reusable blocks
per-thread cache	reduce global synchronization
allocation metadata	record size/state needed for reuse
page/span management	request larger regions from the OS
coalescing	combine adjacent free blocks
quarantine/debug mode	delay reuse to expose stale-pointer bugs

```
program asks for 24 bytes
  ↓ round to allocator size class, perhaps 32 bytes
reuse cached block or obtain a span/page
  ↓
return aligned user region
```

The requested size, usable size, reserved size, and resident physical memory may all differ.

Fragmentation Has Two Forms

Kind	Meaning
internal fragmentation	unused bytes inside an allocated size class/block
external fragmentation	free space exists but not in a suitable contiguous run

A process can free every logical object and still retain pages in allocator caches or fragmented spans. That is not necessarily a leak, although it can still be a memory budget problem.

Allocation Strategies Encode Lifetime Assumptions

Strategy	Fast path	Best fit	Main limitation
bump arena	advance pointer	phase-scoped AST/IR nodes	individual free usually impossible
pool/slab	pop fixed-size slot	many same-shaped objects	poor fit for varied sizes
free-list heap	find/split reusable block	general dynamic allocation	fragmentation/metadata
reference count	increment/decrement counters	shared acyclic ownership	cycles and counter traffic
tracing GC heap	cheap allocation, periodic tracing	managed object graphs	pauses/barriers/runtime complexity
stack/region	adjust stack/region pointer	nested lifetimes	must obey region lifetime

For compiler phases, arenas often reduce per-node overhead and improve locality. Use typed IDs or arena-borrowed references so nodes cannot silently outlive the arena.

Collections Reserve More Than Their Length

```
let mut bytes = Vec::with_capacity(1024);
bytes.extend_from_slice(b"hello");

assert_eq!(bytes.len(), 5);
assert!(bytes.capacity() >= 1024);
```

Quantity	Meaning
length	initialized elements logically in collection
capacity	elements that fit before reallocation
allocation size	backing storage reserved from allocator

`clear()` drops/removes elements but normally preserves capacity. `shrink_to_fit()` is a request and may reallocate; it is not a real-time guarantee. Reuse capacity when it improves performance, and bound it when retained memory matters more.

13. Thin Pointers, Fat Pointers, and Dynamically Sized Types

Some references need metadata in addition to a data address:

`&T`

data pointer

`&[T] / &str`

data pointer length

`&dyn Trait`

data pointer vtable pointer

The pointee may be dynamically sized even though the pointer itself has a known size. Common examples are slices, `str`, and trait objects.

Type	Metadata	Boundary question
<code>&[T]</code>	element count	Is the whole range live and initialized?
<code>&str</code>	byte length	Are bytes valid UTF-8?
<code>&dyn Trait</code>	vtable for concrete type	Does data match that vtable/type?
<code>Box<[T]></code>	length stored with pointer	Who owns and drops every element?

Never assume a trait object is “just a function pointer.” Its exact ABI/layout is not a stable cross-language interface unless a specific ABI defines it.

Niche Optimization

Some types have invalid bit patterns the compiler can reuse to encode an enum variant. A common example is an optional non-null reference:

```
use std::mem::size_of;

assert_eq!(size_of::<Option<&u8>>(), size_of::<&u8>());
```

The null representation can encode `None` because a Rust reference cannot be null. Niche optimizations are useful, but only rely on guarantees documented for the exact types involved. `repr(Rust)` remains free to change many layout details.

Unaligned Data Must Stay Raw Until Read Correctly

Packed or serialized data may place a multi-byte field at an unaligned address. Creating `&field` would first create an invalid reference. Use raw-address formation and an unaligned read when the format requires it:

```
#[repr(C, packed)]
struct Packed {
    tag: u8,
    value: u32,
}

fn value_of(packed: &Packed) -> u32 {
    let pointer = &raw const packed.value;
    // SAFETY: `pointer` addresses four initialized bytes inside `packed`;
    // unaligned access is intentional.
    unsafe { pointer.read_unaligned() }
}
```

For file and network formats, explicit byte decoding is usually clearer and more portable than mapping bytes onto packed structs.

14. Memory Performance Is Mostly About Movement and Locality

CPU execution is much faster than fetching arbitrary data from main memory. Hardware uses a hierarchy:

```
registers → L1 cache → L2 cache → shared/larger cache → RAM → storage
fastest/smallest                                slowest/largest
```

Locality type	Meaning
temporal locality	recently used data is likely to be used again
spatial locality	nearby data is likely to be used soon
instruction locality	nearby/repeated code paths stay hot

Contiguous iteration usually makes better use of cache lines and prefetching than pointer chasing through scattered objects.

Array of Structures Versus Structure of Arrays

```
AoS:
[x y color][x y color][x y color]

SoA:
[x x x ...][y y y ...][color color color ...]
```

Workload	Often favorable shape
process every field of one object	array of structures
update only <code>x</code> for many objects	structure of arrays
polymorphic sparse graph	handles/indirection may be necessary

Do not choose from slogans. Measure the actual traversal, object count, cache behavior, mutation pattern, and code complexity.

Working Set and Page Locality

The **working set** is the memory actively needed during an interval. A program can have a large virtual address space but a small working set, or a moderate heap with a working set too large for caches/RAM.

```
algorithm touches 1 field in 10 million scattered objects
↓
many cache/TLB misses despite little useful data per object
```

Arena placement, compacting collectors, object splitting, and dense IDs can improve locality by placing related hot data together.

False Sharing

Two threads can modify different variables that happen to occupy the same cache line:

```
one cache line
┌──────── thread A counter ───────── thread B counter ─────────┐
```

The cache-coherence protocol then moves ownership of the line between cores even though the source variables do not logically overlap. Symptoms include poor scaling and high coherence traffic. Padding or per-thread aggregation can help, but increases memory use; measure before changing layout.

15. Garbage Collection Is a Graph Algorithm

A tracing collector treats managed memory as a directed graph:

```
roots: registers, VM stack, globals, native handles
↓
trace outgoing object references
↓
reachable objects survive
unreachable objects may be reclaimed
```

The core invariant is:

Every live managed reference must be discoverable or otherwise protected whenever the collector may run.

Collector Families

Strategy	Strength	Cost/constraint
reference counting	prompt reclamation, simple local reasoning	cycles, count traffic
mark-sweep	handles cycles; objects need not move	tracing pause and fragmentation
mark-compact	improves locality and removes fragmentation	updates references; movement cost
copying/semispace	fast allocation and compaction	extra space; moves live objects

generational	focuses work on usually short-lived objects	remembered sets and write barriers
incremental/concurrent	spreads/reduces pause work	complex barriers and synchronization

"Garbage collected" does not imply objects never move, destructors run immediately, or memory usage stays below a fixed bound.

Open, Closed, and Rooted References

For a VM with closures:

```
active local:
closure → open upvalue → VM register slot

after frame exits:
closure → closed upvalue → heap-managed cell
```

The collector must trace the intermediary in both states. Native code that temporarily holds a managed value also needs a rooting/handle discipline if allocation can trigger collection.

Generational Collection Needs a Write Barrier

If an old object starts pointing to a young object, scanning only young roots would miss the edge:

```
old object —new reference→ young object
```

A write barrier records the old object/card/slot in a remembered set. The next minor collection scans those recorded old-to-young edges.

Barrier failure	Result
missed edge	live young object reclaimed
record everything globally	correct but minor collections lose benefit
barrier runs after unsafe publish	collector may observe inconsistent state

Safepoints and Stack Maps

A precise moving collector must know where references exist in registers and stack slots. Compilers can emit **stack maps** for selected **safepoints**:

Safepoint candidate	Why it is useful
allocation slow path	collection is already possible
function call	register/stack state follows a known convention
loop back edge	long-running computation eventually cooperates
explicit poll	runtime can request suspension

At a safepoint, each live reference location must match the compiler's map. Optimizer and GC metadata must be generated from the same final liveness facts.

Finalizers Are Not Ordinary RAI

Finalization timing may be delayed, reordered, skipped at process exit, or affected by cycles and resurrection rules. Use explicit resource management for files, locks, sockets, transactions, and other scarce external resources. Let GC manage memory reachability, not critical protocol completion.

16. Memory Accounting Uses Several Different Numbers

"The process uses 2 GB" is incomplete without naming the measurement:

Metric	Rough meaning
virtual size	total mapped address ranges
resident set (RSS)	pages currently resident for the process
proportional set (PSS)	shared pages divided among sharers
heap live bytes	allocations still logically live
allocator active bytes	blocks/spans currently serving allocations
allocator reserved bytes	memory retained for future reuse
stack reservation	address space reserved for thread stacks
mapped-file pages	file/library mappings, possibly shared/page-cached
peak usage	high-water mark, not current usage

The exact availability and definition of these metrics is OS/tool-specific.

Leak, Retention, Fragmentation, and Cache Are Different

Pattern	What keeps memory unavailable
true leak	ownership/reference is lost without reclamation
logical retention	reachable collection/cache keeps unneeded objects
reference cycle	ownership counts never reach zero
fragmentation	free pieces cannot satisfy/release useful spans
allocator retention	freed blocks remain cached for later allocation
page cache/mapping	OS keeps file-backed pages or address ranges mapped

A heap profile dominated by reachable cache entries is a policy problem, not a missing `free`. A stable live heap with rising RSS may indicate allocator retention or fragmentation rather than continued object growth.

Ask Four Questions

1. Which object/allocation categories are growing?
2. Are they live, unreachable, freed-but-retained, or file-backed?
3. Which ownership/root path keeps live objects reachable?
4. Does growth stabilize under a fixed workload?

Take comparable measurements after warm-up and after completing the same unit of work. One snapshot shows composition; two or more snapshots show direction.

17. Memory Debugging Needs the Right Observer

Symptom	Useful observer/tool class
invalid native read/write	AddressSanitizer or memory checker
use of uninitialized data	MemorySanitizer/Valgrind-style checking
leak at process exit	LeakSanitizer or allocation tracing
Rust unsafe-code UB	Miri for supported tests
unexpected heap growth	allocation/heap profiler plus retained paths
stack overflow	stack trace, recursion/depth accounting
page-permission fault	debugger plus memory-map/page information
race on shared memory	thread/race sanitizer and synchronization review
cache/TLB bottleneck	hardware performance counters/profiler
wrong VM/GC root	runtime heap verifier and root/handle dump

No one tool proves memory correctness. Dynamic tools observe executed paths; type systems and reviews cover classes of paths; fuzzing/coverage help explore more cases.

Reproducible Memory Investigation

1. record build, allocator, target, optimization, and workload
2. minimize to one repeatable input
3. classify: safety violation, leak, retention, fragmentation, or performance
4. capture a baseline and comparable later snapshot
5. locate the first invalid transition or retaining ownership path
6. fix the source invariant
7. add a regression test and rerun under the relevant observer

Evidence to preserve	Why
exact binary and symbols	optimized layouts differ
sanitizer/tool configuration	detection depends on instrumentation/options
allocator and environment	retention/size classes vary
input and operation count	growth must be normalized by work
stack and allocation trace	connects bytes back to ownership decisions

Runtime Heap Verification

A custom language can check its own heap in debug builds:

```
for every object:
  header tag is valid
  size is aligned and within heap bounds
  every pointer field targets a valid object/handle
  forwarding/mark state is legal for current GC phase

for every free block:
  range is ordered, non-overlapping, and inside its span

for every root:
  type/location agrees with compiler or VM metadata
```

Run verification after collection, after heap growth, and around complicated FFI transitions. A verifier turns silent corruption into an earlier, localized invariant failure.

18. Memory Design Checklist for Your Own Language

Area	Decision to make explicitly
value representation	boxed, tagged union, NaN-boxed, handles, or mixed
object header	type/shape, size, mark state, generation, flags
alignment	minimum object/stack/ABI alignment
ownership	unique, reference-counted, tracing, arena, or hybrid
roots	VM stack, registers, globals, native handles, JIT frames
movement	whether objects can move and how references are updated
allocation failure	exception, result, abort, reserve, or GC retry
stack policy	maximum depth, growth, guard pages, overflow behavior
finalization	explicit close versus nondeterministic cleanup
FFI	pin/copy/handle rules and who owns foreign buffers
concurrency	stop-the-world, per-thread heaps, locks, atomics, barriers
observability	heap dump, object IDs, allocation sites, verifier
limits	heap, object, string, collection, recursion, mapped bytes

```
source-level guarantee
  ↓ compiler representation
runtime allocation/rooting rule
  ↓ OS mapping/page permission
hardware load/store and cache behavior
```

Every layer participates in the language’s memory model. A safe source language can still have an unsafe runtime if bytecode verification, root tracking, layout, FFI, or concurrent mutation breaks the contract.

Operating Systems: Processes, Virtual Memory, Files, and Scheduling

This section connects the compiler’s output to the operating system that actually runs it. It is based on [Computer Science from the Bottom Up](#), especially its chapters on the operating system, processes, virtual memory, ELF, and dynamic linking.

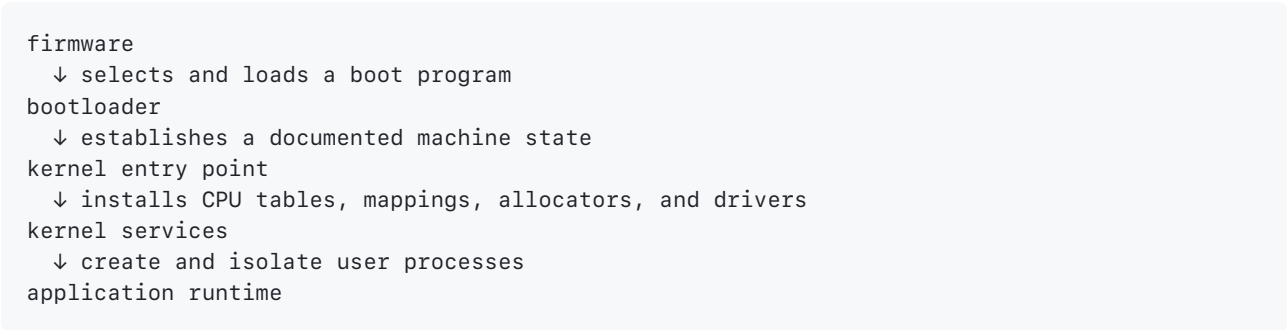
 **OS mental model:** the kernel owns privileged resources and exposes controlled operations to processes through system calls.

Kernel Case Study: Booting Into Protected Execution

[Writing an OS in Rust](#) is useful here because a tiny kernel removes the services that a normal Rust program silently inherits. Its code is not just an operating-system recipe: each step reveals a computer-science contract.

 **Library lens — Phil Opp’s kernel:** read each module as an executable model. The bootloader models **representation transfer**, an interrupt descriptor table models **event dispatch**, paging models **address translation**, an allocator models **resource management**, and a tiny async executor models **cooperative scheduling**.

The Boot Path Is a Chain of Representations



Each arrow is a **contract boundary**. The next stage cannot assume that an ordinary language runtime already exists.

Boundary	State that must be agreed on
Firmware → bootloader	Boot medium, executable format, entry location
Bootloader → kernel	CPU mode, stack, memory map, framebuffer, page tables
Kernel → driver	Register layout, interrupt route, DMA ownership
Kernel → process	Virtual address space, privilege level, initial registers
Runtime → <code>main</code>	ABI, arguments, environment, initialized language services

A freestanding Rust binary therefore needs deliberate answers to questions a hosted program delegates to its operating system:

Hosted assumption	Kernel replacement
<code>std</code> can call the operating system	Use <code>core</code> , then add only the services that exist
A stack already exists	Boot code or bootloader supplies one
Panics can print and unwind	Install a panic policy and an output path
Heap allocation exists	Map heap pages and install an allocator
Threads or an executor exist	Implement scheduling and wake-up policy
Exiting returns to a parent process	Halt, reset, or report through a machine interface

This is the same reasoning needed for a language runtime. A VM cannot use garbage collection, files, exceptions, or native calls until its host has established those services.

CPU Tables Turn Events Into Controlled Transfers

An exception or interrupt is not an ordinary call chosen by the current function. The processor observes an event, saves enough execution state, changes privilege when required, and transfers control through an architecture-defined table.

```

current instruction
  ↓ fault, trap, timer, or device signal
CPU looks up vector
  ↓
descriptor selects handler and privilege rules
  ↓
handler saves additional state and acknowledges source
  ↓
return-from-interrupt restores interrupted execution

```

Event kind	Cause	Typical use
Fault	Current instruction cannot complete normally	Page mapping, access violation
Trap	Deliberate synchronous event	Breakpoint, system-call entry
Interrupt	External or timer event	Keyboard, network device, preemption
Double fault	Failure while delivering another exception	Emergency failure path

The critical invariant is:

A handler may only return if it has repaired or intentionally accounted for the state that caused the transfer.

For example, a demand-page handler may install a valid mapping and retry the instruction. An invalid user pointer should instead terminate or notify the process. Blindly returning repeats the same fault.

Paging Separates Names From Storage

Phil Opp's paging implementation makes a general abstraction visible:

```
virtual address = stable name used by code
page-table entry = current mapping and permissions
physical frame = storage resource
```

The three are not interchangeable. A virtual page can be unmapped, remapped, shared, read-only, executable, or backed by a file. This indirection enables isolation because two processes can use the same numeric virtual address without naming the same frame.

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct VirtualPage(u64);

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
struct PhysicalFrame(u64);

#[derive(Clone, Copy, Debug)]
struct PagePermissions {
    writable: bool,
    executable: bool,
    user_accessible: bool,
}

#[derive(Clone, Copy, Debug)]
struct Mapping {
    // The type distinguishes a storage frame from the virtual name used by code.
    frame: PhysicalFrame,
    permissions: PagePermissions,
}

fn may_write(mapping: Mapping, from_user_mode: bool) -> bool {
    // User code needs both write permission and permission to cross the privilege
    // boundary. Kernel code still needs the page to be writable.
    mapping.permissions.writable
        && (!from_user_mode || mapping.permissions.user_accessible)
}
```

The newtypes do not merely improve readability. They prevent accidental arithmetic between different address spaces. A production kernel also requires alignment checks, reserved-bit validation, architecture-specific flags, TLB invalidation, and synchronization with other cores.



Compiler connection: a compiler also separates names from locations. A source variable, an SSA value, a stack slot, and a machine register can refer to one logical value at different stages, but they are not the same representation.

An Allocator Is a Policy Over a Finite Resource

Mapping a heap region answers **where allocation may occur**. The allocator answers **which free range to choose, how to record it, and how to reuse it**.

Allocator	State	Strength	Limitation
Bump	One next-free offset	Tiny and fast	Individual deallocation is not reclaimed

Free list	Linked free ranges	Reuses variable-sized blocks	Search cost and fragmentation
Fixed-size classes	Free list per size class	Predictable fast paths	Internal waste for size mismatch
Buddy	Power-of-two split/merge tree	Efficient coalescing	Rounding wastes some space

Every allocator preserves at least these invariants:

Invariant	Failure if violated
Live allocations do not overlap	Two owners mutate the same bytes
Returned address satisfies alignment	Invalid or slow machine access
Free space and allocated space cover only the managed region	Memory corruption
A block is freed at most once	Free-list corruption and later aliasing
Metadata remains reachable and consistent	Leaks or arbitrary allocation results

Rust can make the safe API pleasant, but the allocator implementation is an unsafe proof boundary because raw addresses and uninitialized storage precede ordinary references.

A Future Is a Suspended State Machine

The kernel's small async executor is a particularly clear model of cooperative scheduling. An `async` function is transformed into a value that stores:

Stored state	Why it exists
Current suspension point	Determines which part resumes
Locals live across <code>await</code>	They must survive after the call stack unwinds
Child future state	The parent cannot complete before dependencies do
Completion value	Returned once, after the final state

At the conceptual boundary, an executor repeatedly polls tasks:

```

enum Poll<T> {
    Ready(T),
    Pending,
}

trait TinyFuture {
    type Output;

    // `poll` performs bounded work. Pending means "not ready now", not failure.
    fn poll(&mut self) -> Poll<Self::Output>;
}

fn run_until_stalled(tasks: &mut [Box<dyn TinyFuture<Output = ()>>]) {
    for task in tasks {
        // A real executor polls only tasks whose wakers marked them ready.
        // Polling every task is shown solely to expose the scheduling model.
        let _state = task.poll();
    }
}

```

Real Rust uses `Future::poll`, `Pin`, `Context`, and `Waker`. A pending future registers how it should be awakened; otherwise the executor would have to waste CPU repeatedly polling everything. `Pin` matters because the compiler-generated future may contain references into its own stored state, so moving it after polling could invalidate those relationships.

This reveals a broad principle:

Async does not make work happen by itself. It turns progress into an explicit state machine and relies on an executor plus an event source to schedule transitions.

The same model appears in GUI event loops, network reactors, actors, browser tasks, and language coroutines.

What to Observe While Reading a Tiny Kernel

Question	Concept recovered
Which machine state exists at the entry point?	Boot ABI
Which module contains raw register access?	Unsafe boundary
Which type distinguishes virtual from physical addresses?	Representation safety
What event causes the handler to run?	Control transfer
What state is saved across suspension?	Continuation/state machine
Which allocation strategy chooses a free block?	Resource policy
What happens when a finite table or heap is full?	Failure semantics
Which output path works before drivers exist?	Minimal observability

This is also a reversing checklist. When looking at an unfamiliar kernel, firmware image, hypervisor, or runtime, recover its entry state, tables, ownership boundaries, event sources, and finite-resource policies before naming individual functions.

1. The Kernel as a Privileged Resource Manager

An application cannot safely control the processor, physical memory, disks, and network devices directly. The **kernel** owns those shared resources and exposes controlled operations to user programs.

```
user program
  ↓ library/API call
system-call boundary
  ↓ privilege transition
kernel validates request
  ↓
device, filesystem, scheduler, or memory manager
```

The boundary changes both **privilege** and **trust**:

Phase	Responsibility
Request	User code supplies an operation and arguments
Transition	The processor enters privileged mode
Validation	Kernel checks pointers, lengths, permissions, handles
Execution	Kernel performs or rejects the operation
Return	Result/error is copied back to user mode

The exact instruction used to enter the kernel depends on the architecture. Older systems often used a general software trap; modern processors usually provide a faster system-call instruction. User code normally reaches it through a standard library wrapper rather than hand-written assembly.

System Calls Versus Library Calls

Kind	Where it runs	What it does
Library call	Current process	Ordinary computation, formatting, buffering
System call	Kernel entry	Requests a privileged operation
Library wrapper	User + kernel paths	May make zero, one, or several system calls

For example, buffered output may append bytes to a user-space buffer many times and perform a `write` system call only when the buffer is full or flushed.

```
printf / println / buffered writer
  ↓ formatting and buffering in user space
write-like system call
  ↓ kernel copies or maps data toward a file/device
```

This distinction explains why source-level operations do not map one-to-one to kernel operations.

A System Call Is Also an ABI Contract

At the lowest boundary, the operation number, argument registers, pointer widths, return convention, and error encoding are architecture/OS-specific.

```
language call
  ↓ runtime/libc wrapper
marshal syscall number + scalar arguments + user pointers
  ↓ privileged entry
kernel copies/validates referenced user memory
  ↓ operation
return value or encoded error
  ↓ wrapper converts to language-level Result/exception/status
```

Argument kind	Kernel-side question
integer/flags	Are unused bits clear and combinations valid?
file descriptor	Does it name a live object with required permissions?
user pointer	Is the full range mapped with the required access?
pointer + length	Does arithmetic overflow or cross an invalid mapping?
structure	Which ABI version and exact layout does the caller use?
output buffer	How many bytes may be written back safely?

The kernel cannot trust a pointer merely because it is numerically inside user space. Another thread may race with mutable memory, mappings can change, and a range may cross page boundaries. Kernel interfaces therefore copy, pin, validate, or otherwise control how user memory is accessed.

 **Reversing connection:** a syscall instruction is only the bottom edge. Recover the wrapper's ABI, argument construction, error translation, and higher-level resource lifetime before assigning source-level meaning.

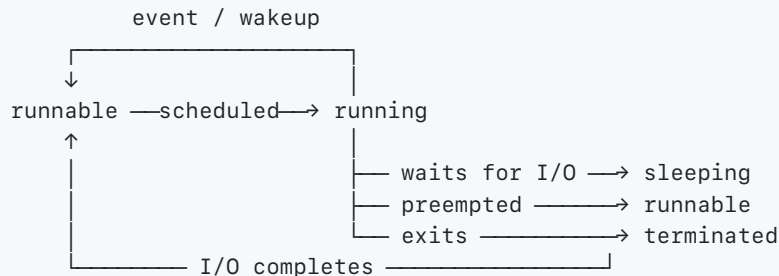
2. What a Process Contains

A **program** is an executable representation stored somewhere. A **process** is a running instance plus all state required to stop and resume it.

Process component	What it represents
Process ID	Kernel-visible identity
Virtual address space	Code, data, heap, stack, mappings
Register state	Instruction pointer, stack pointer, general registers
File-descriptor table	Open files, pipes, sockets, devices
Credentials	User/group identity and permissions
Signal state	Masks, pending signals, handlers
Scheduling state	Running, runnable, sleeping, stopped
Accounting data	CPU time, memory use, I/O statistics

The kernel does not need to leave every process's register values in the CPU. During a context switch it saves the outgoing process's state to memory and restores the incoming process's saved state.

Common Process States



Names vary by operating system, but the distinction is important:

Process state	Meaning
Running	Currently executing on a CPU
Runnable	Able to execute but waiting for CPU time
Sleeping/blocked	Waiting for an event, timer, lock, or I/O
Stopped	Deliberately suspended
Terminated/zombie	Finished; exit information may await parent collection

3. File Descriptors: Small Handles to Kernel Objects

On Unix-like systems, a file descriptor is a small process-local integer that indexes a kernel-maintained table.

Conventional descriptor	Meaning
0	Standard input
1	Standard output
2	Standard error

The descriptor is not the file itself. It is a handle to an open kernel object with associated access mode and current state.

```

process-local fd 3
  ↓
kernel open-file description
  ↓
filesystem object, pipe, socket, terminal, or device
  
```

This uniform interface is why many different resources support operations resembling `read`, `write`, and `close`.

Descriptor consequence	Practical meaning
------------------------	-------------------

Process-local scope	Inheritance/transfer requires an explicit OS mechanism
Reference semantics	<code>close</code> releases one reference, not always the object
Aliasing	Multiple descriptors can share one open-file state
Number reuse	A closed descriptor value may later name something else
No global identity	The integer is meaningful only with its process context

4. `fork`, `exec`, and Process Creation

Unix process creation is often explained as two separate operations:

1. `fork` creates a child based on the calling process.
2. `exec` replaces the current process image with a new program.

Conceptually:

```

shell process
  ↓ fork
shell parent + child copy
                    ↓ redirect descriptors / set environment
                    ↓ exec
                requested program

```

The child does not normally require an eager physical copy of every memory page. Operating systems use **copy-on-write (COW)**:

Event	COW behavior
Immediately after <code>fork</code>	Parent and child map the same read-only frames
Read	Both processes continue sharing the frame
First write	A protection fault transfers control to the kernel
Fault handling	Kernel copies the page and grants private writable access

This makes `fork` much cheaper when the child quickly calls `exec`.

`exec` does not create a second process ID. It replaces the current address space, register setup, and executable image while preserving selected process state such as the process identity and descriptors that are not marked close-on-exec.

5. Context Switching and Scheduling

A **context switch** changes which thread or process owns a CPU:

1. enter the kernel because of a timer, interrupt, system call, or blocking event;
2. save the current execution context;
3. update the current process's state;
4. choose a runnable process;
5. restore its saved context;

6. return to user mode at its saved instruction pointer.

Context switches are necessary but not free:

Cost source	Why it costs time
Register save/restore	Execution context moves between CPU and memory
Scheduler work	Kernel chooses and accounts for the next task
Cache disruption	New code/data displaces useful cache lines
TLB effects	Address translations may need refilling
Branch prediction	Predictor history may no longer match
Core migration	Warm state may not exist on the destination CPU

Preemptive scheduling lets the kernel interrupt a running task. **Cooperative scheduling** relies on tasks yielding voluntarily. General-purpose operating systems are normally preemptive; async runtimes often use cooperative tasks inside a process.

6. Signals and Asynchronous Events

A signal is a small asynchronous notification delivered to a process or thread:

Signal category	Example meaning
Control request	Terminate, stop, or continue
Memory fault	Invalid address or protection violation
Arithmetic fault	Invalid numeric operation
Child notification	A child stopped, continued, or exited
Timer	Alarm or scheduled notification
Application event	User-defined signal

Signal handlers run in an unusual context. Ordinary application invariants may be temporarily broken, so only operations documented as safe in a signal handler should be performed there.

Runtime interaction	Signal-related use
Stack overflow	Detect guard-page faults
Memory management	Handle deliberately fault-driven techniques
Profiling	Trigger periodic sampling
Subprocess supervision	Observe child state changes
Graceful shutdown	Request orderly termination
Debugging	Implement traps and breakpoints

Do not use a signal handler as if it were a normal callback.

7. Virtual Memory Is Address Translation

Virtual memory is often described as disk pretending to be RAM. That describes **swap**, which is only one possible consequence. The central idea is that programs use **virtual addresses**, and the operating system plus hardware translate them to physical memory.

```
virtual address used by instruction
    ↓ split into page number + page offset
page table translates virtual page
    ↓
physical frame + unchanged offset
    ↓
physical memory location
```

Virtual-memory term	Meaning
Address space	Addresses a process can potentially name
Virtual page	Fixed-size block in a virtual address space
Physical frame	Fixed-size block in physical memory
Page table	Mapping and permissions from pages to frames
Mapping	Association with memory, a file, or another kernel object

Two processes can use the same virtual address while mapping it to different physical frames. Conversely, two virtual pages can intentionally map the same physical frame for shared memory.

8. Page Tables and Page Permissions

If the page size is (2^n) , the low (n) address bits are the offset inside a page and the remaining bits identify the virtual page.

For a 4 KiB page:

```
4 KiB = 4096 bytes = 2^12
```

```
virtual address:
```

```
+-----+-----+
| virtual page number | 12-bit |
|                     | page offset|
+-----+-----+
```

Translation changes the page number but preserves the offset:

```
virtual page 0x1234, offset 0x056
    ↓ page-table lookup
physical frame 0x09ab, offset 0x056
```

A page-table entry commonly includes:

PTE field category	What it controls
Presence	Whether a valid mapping currently exists
Permissions	Read, write, and execute access
Privilege	User-accessible or kernel-only
Accessed bit	Whether the page has been referenced
Dirty bit	Whether the page has been modified
Cache policy	How accesses interact with CPU caches
Architecture bits	Target-specific translation controls

These permissions provide isolation and enable mitigations such as non-executable data pages.

9. Multi-Level Page Tables

A flat page table for every possible virtual page would waste large amounts of memory, especially for sparse address spaces. Multi-level page tables divide the virtual page number into indexes:

```

virtual address
  ↓
level-1 index → level-2 index → ... → page-table entry
                                   ↓
                               physical frame

```

Intermediate tables only need to exist for populated parts of the address space.

This resembles a radix tree: each group of address bits selects the next table. Exact levels and bit divisions are architecture-specific.

10. The TLB

Walking page tables for every memory access would be slow. The processor therefore uses a **Translation Lookaside Buffer (TLB)**, a small cache of recent virtual-to-physical translations.

```

virtual address
  ↓
TLB hit? — yes → physical address
  |
  no
  ↓
page-table walk → cache translation → retry access

```

A context switch may require translations to be invalidated, tagged by address-space identity, or otherwise managed so one process cannot use another process's mappings.

Performance implications:

- poor locality can increase both data-cache and TLB misses;
- large pages cover more memory per TLB entry but reduce allocation flexibility;

- changing mappings requires synchronization with CPUs that may cache the old translation;
- a compiler's object layout and traversal order can affect page and cache locality.

11. Page Faults

A **page fault** means an address translation or permission check could not complete normally. It is not automatically a bug.

Fault cause	Possible kernel response
Valid file-backed page not loaded	Read or map the page, then retry
Copy-on-write write	Allocate/copy a private frame, then retry
Stack growth within allowed range	Map another stack page
Swapped-out page	Restore it from backing storage
Write to read-only mapping	Deliver an access violation
Unmapped address	Deliver an access violation

This distinction matters when debugging: the hardware mechanism is a page fault in all cases, but the operating system may resolve some transparently and report others to the process.

12. `mmap`, Shared Memory, and the Page Cache

A memory mapping connects virtual addresses to a backing object.

- **Anonymous mapping:** zero-initialized memory not backed by a normal file.
- **Private file mapping:** file content is visible, but writes become private through copy-on-write.
- **Shared file mapping:** writes may become visible through the shared mapping and eventually reach the file.
- **Shared anonymous mapping:** processes intentionally share frames.

The **page cache** lets the kernel cache file contents in memory. Ordinary file I/O and memory-mapped I/O may ultimately interact with the same cached pages.



Page-cache mental model: file I/O and mapped memory can meet in the kernel's cached view of file contents.

```

file on storage
  ↓
kernel page cache
  ↙   ↘
read/write mmap views

```

This helps explain why:

- a recently read file may be fast to read again;
- writing does not always mean bytes reached durable storage immediately;
- mapped file accesses can fault pages in lazily;
- executable code and shared libraries can be backed by cached file pages.

📁 Filesystems: Turning Blocks Into Durable Names

[Writing a File System from Scratch in Rust](#) is a compact demonstration of how layers of metadata turn an unstructured block device into files and directories.

🧠 **Filesystem mental model:** storage supplies numbered blocks. A filesystem adds allocation, object identity, names, byte ranges, permissions, and recovery rules.

A Block Device Does Not Know About Files

At the bottom, a storage interface resembles:

```
read block N → fixed-size bytes
write block N ← fixed-size bytes
flush        → request durability through lower layers
```

It does not inherently understand `/home/notes.md`, ownership, directories, or the end of a file. Those are interpretations imposed by filesystem metadata.

Layer	Question it answers
Block device	Which fixed-size block should be transferred?
Allocation map	Which blocks and object records are free?
Inode/object record	Which metadata and data blocks belong to this object?
Directory	Which name maps to which object identifier?
Path resolver	How is a sequence such as <code>a/b/c</code> traversed?
File API	Which byte range should a process read or write?
Cache and journal	When is the change visible and when is it durable?

This decomposition is important when reversing an unknown disk image: a recognizable filename is high-level evidence, while the underlying allocation and block pointers tell you how the bytes are actually reached.

One Possible On-Disk Layout

The teaching filesystem in the article uses the classic idea of a superblock, bitmaps, an inode table, and data blocks:

```
+-----+-----+-----+-----+
| superblock | inode bitmap | block bitmap | inode table | data blocks |
+-----+-----+-----+-----+
```

Region	Persistent meaning
Superblock	Version, block size, counts, and locations of other regions
Inode bitmap	One bit per object record: free or allocated
Block bitmap	One bit per data block: free or allocated

Inode table	Stable object metadata and block references
Data blocks	File bytes or serialized directory entries

The superblock is a **decoder key**. If its version, block size, or region offsets are misread, every later byte can look plausible while meaning the wrong thing.

```
#[derive(Debug)]
struct Superblock {
    magic: u32,
    version: u16,
    block_size: u32,
    inode_count: u32,
    data_block_count: u32,
    inode_table_start: u32,
    data_region_start: u32,
}

impl Superblock {
    fn validate(&self, device_blocks: u64) -> Result<(), &'static str> {
        // A magic value rejects unrelated or byte-swapped data early.
        if self.magic != 0x4653_5253 {
            return Err("not this filesystem format");
        }

        // Restrict block sizes before using them in allocation or offset arithmetic.
        if !self.block_size.is_power_of_two() || self.block_size < 512 {
            return Err("unsupported block size");
        }

        // Validate region ordering and bounds before trusting any later pointer.
        if self.inode_table_start >= self.data_region_start {
            return Err("overlapping metadata regions");
        }
        if u64::from(self.data_region_start) >= device_blocks {
            return Err("data region starts beyond the device");
        }

        Ok(())
    }
}
```

The structure used in memory need not be identical to the structure on disk. Production parsers normally decode fields explicitly so endianness, versioning, padding, and validation are controlled rather than inherited from a compiler's struct layout.

Bitmaps Are Compact Sets

A bitmap represents a finite set using one bit per possible member:

```
bit i = 0 → resource i is free
bit i = 1 → resource i is allocated
```

For block `i`:

```

byte index = i / 8
bit index  = i mod 8
mask       = 1 << bit index

```

This is a direct use of quotient and modular arithmetic. Allocation searches for a zero bit, changes it to one, and persists the result. Freeing does the reverse.

```

fn mark_allocated(bitmap: &mut [u8], resource: usize) -> Result<(), &'static str> {
    let byte_index = resource / 8;
    let bit_index = resource % 8;
    let byte = bitmap
        .get_mut(byte_index)
        .ok_or("resource is outside the bitmap")?;
    let mask = 1_u8 << bit_index;

    // Reject double allocation rather than silently hiding metadata corruption.
    if *byte & mask != 0 {
        return Err("resource was already allocated");
    }

    *byte |= mask;
    Ok(())
}

```

The operation is simple in memory but becomes a transaction on disk. A crash between marking a block allocated and connecting it to an inode can leak the block. Connecting it first can make two objects believe they own it after a crash.

Inodes Separate Object Identity From Names

An inode-like record stores an object's metadata and data-block references. A directory stores **name** → **inode number** mappings.

```

directory entry "notes.md" → inode 42
inode 42 → length, type, permissions, timestamps, data-block references
data blocks → actual file bytes

```

Consequences:

Consequence	Explanation
Multiple names can reach one object	Hard links store the same inode identifier
Renaming need not move file bytes	A directory mapping changes
Deleting a name need not delete the object immediately	Other links or open handles may remain
An open file can outlive its path	The kernel holds a reference to the object
A directory is structured data	Its contents are mappings, not ordinary prose

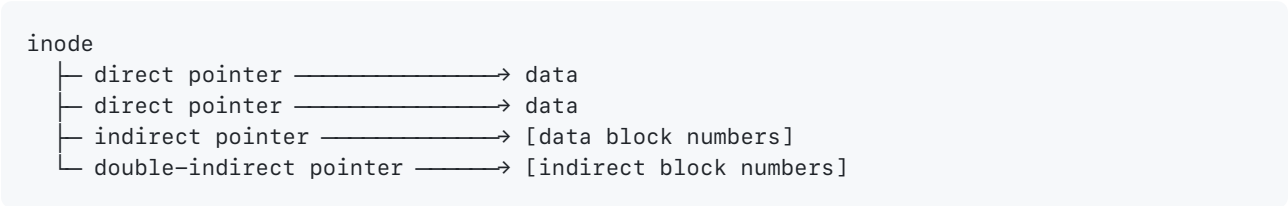
This is another identity-versus-location lesson. A path is a lookup expression, not the file's permanent identity.

File Offsets Map Through Several Levels

For a fixed block size:

```
logical block = file_offset / block_size
offset inside block = file_offset mod block_size
```

Small files can keep direct data-block numbers in the inode. Larger files need indirect blocks that store more block numbers:



Scheme	Lookup depth	Capacity effect
Direct	1	A small fixed number of blocks
Single indirect	2	Pointer count per block
Double indirect	3	Pointer count squared
Extents	Usually shallow	One record describes a contiguous block range

```

const DIRECT_SLOTS: usize = 8;

#[derive(Debug)]
struct Inode {
    length: u64,
    direct_blocks: [u32; DIRECT_SLOTS],
    indirect_block: Option<u32>,
}

fn direct_block_for_offset(
    inode: &Inode,
    file_offset: u64,
    block_size: u64,
) -> Result<(u32, usize), &'static str> {
    // First reject reads beyond the logical end of the file.
    if file_offset >= inode.length {
        return Err("offset is outside the file");
    }

    let logical_block = file_offset / block_size;
    let within_block = file_offset % block_size;
    let slot = usize::try_from(logical_block).map_err(|_| "block index overflow")?;

    // This small example deliberately handles only direct blocks.
    // A complete implementation would parse and validate indirect blocks here.
    let device_block = *inode
        .direct_blocks
        .get(slot)
        .ok_or("offset requires indirect lookup")?;

    let byte_index = usize::try_from(within_block).map_err(|_| "offset overflow")?;
    Ok((device_block, byte_index))
}

```

The comments identify the invariant at each step: logical bounds, checked conversion, lookup depth, and physical block selection. Those are exactly the places where a disk parser should distrust corrupted input.

Path Resolution Is Repeated Graph Traversal

Resolving `/usr/bin/tool` means:

1. start at the root directory object;
2. look up `usr` and verify that the result is a directory;
3. look up `bin` in that directory;
4. look up `tool`;
5. apply permissions, link-following policy, and mount transitions along the way.

```

root inode
├─ "usr" → inode 10
│   └─ "bin" → inode 25
│       └─ "tool" → inode 91

```

Symbolic links introduce another path expression. Mount points can replace the filesystem used for a subtree. `.` and `..` affect traversal. Therefore a safe resolver needs limits on link depth, path component count, and total expansion.



Security connection: checking a path and opening it later can race with renames or link changes. Handle-relative APIs and kernel-enforced traversal policies reduce this time-of-check/time-of-use gap.

Visibility Is Not Durability

A successful write can mean several different things:

```
application buffer
  ↓ flush language buffer
kernel page cache
  ↓ filesystem writeback
device cache
  ↓ device ordering and persistence
durable medium
```

Event	What it normally proves
<code>write</code> accepted bytes	Kernel owns a copy or mapping of the change
Buffered writer flushed	Bytes reached the operating-system interface
Filesystem synchronization completed	Filesystem requested durable ordering
Device reports completion	Meaning depends on device/cache guarantees

Atomicity, consistency, isolation, and durability are separate properties. Renaming a prepared temporary file is a common publication pattern, but crash safety still depends on syncing the file data and the containing directory in the required order.

Crash Consistency Is an Ordering Problem

Suppose growing a file requires:

1. reserve a free data block;
2. write new data;
3. attach the block to the inode;
4. update the inode length.

A crash can occur between any two steps. Filesystem designs constrain the possible post-crash states:

Technique	Core model
Write ordering	Persist prerequisites before pointers that expose them
Journal / write-ahead log	Record intended metadata transaction, then apply it
Copy-on-write tree	Write new nodes, then atomically publish a new root
Checksums	Detect some torn or corrupted records
Recovery scan	Reconstruct or repair invariants after an unclean shutdown

The governing invariant is that every reachable pointer should name initialized, compatible storage, and each allocated resource should have an intended owner.

This resembles a garbage-collected runtime:

Filesystem	Managed-language runtime
Inode or tree root	GC root
Block pointer	Object reference
Reachable block	Live object
Unreferenced allocated block	Leak
Two mutable owners of one block	Aliasing/corruption
Recovery checker	Heap verifier

Filesystems in Language Design and Reversing

Task	Filesystem concept that matters
Module loader	Paths are fallible lookups with encoding and permission rules
Compiler cache	Content identity, invalidation, atomic publication, durability
Package manager	Archive traversal, signatures, checksums, rollback
VM snapshot	Stable serialization, versioning, partial-write recovery
Binary reversing	Executables may be sparse, mapped, compressed, or deleted-but-open
Disk-image reversing	Recover superblock, allocation maps, records, then names

When designing your own language, avoid treating `read_file(path)` as an indivisible primitive. Decide whether the operation follows links, which encoding paths use, how large a file may be, whether reads are streaming, how errors are represented, and whether the runtime exposes synchronous or asynchronous I/O.

When reversing a filesystem, work bottom-up:

```
device geometry
  → format signature and version
  → region boundaries
  → allocation structures
  → object records
  → block mapping
  → directories and paths
  → higher-level file formats
```

Each recovered layer gives constraints that validate the next one. A parser that jumps straight to filenames has skipped the evidence needed to distinguish a real structure from coincidental bytes.

13. From `exec` to `main`

`main` is usually not the first instruction executed.

```

exec request
  ↓
kernel reads executable metadata
  ↓
map loadable segments and create initial stack
  ↓
load requested program interpreter / dynamic linker
  ↓
resolve required libraries and relocations
  ↓
transfer control to executable entry point (`_start`)
  ↓
language/runtime startup
  ↓
call user `main`
  ↓
run termination and destructor logic

```

The initial stack can contain arguments, environment entries, and an **auxiliary vector** with information supplied by the kernel. The runtime startup code uses this state to initialize the process before calling `main`.

Runtime startup responsibility	Purpose
Heap/GC initialization	Prepare managed allocation
Built-in registration	Install core types and native functions
Thread-local state	Establish per-thread runtime context
Arguments/environment	Convert OS input into language values
Standard streams	Connect language I/O to OS handles
Panic/exception handling	Define top-level failure behavior
Module constructors	Initialize global/module state
Language entry point	Call user <code>main</code> or equivalent
Exit conversion	Map the language result to an OS status

14. Bottom-Up Debugging Questions

When a program behaves unexpectedly, walk downward through the abstractions:

1. What source-level operation failed?
2. Which library or runtime function implements it?
3. Did it make a system call?
4. What arguments crossed the kernel boundary?
5. Which descriptor, mapping, or process state did the kernel use?
6. Was the process running, runnable, or blocked?
7. Did an address translation, permission check, or page fault occur?
8. Which executable segment, symbol, or relocation produced the address?

Then walk back upward and explain how the low-level event created the source-level symptom.

Executables, Linkers, ABIs, and FFI

Compiling instructions is only part of producing a usable program. The result must also describe how code and data are arranged, which external names it needs, where it may be loaded, and how functions exchange values.

 **Toolchain handoff:** the compiler emits code and metadata, the linker resolves relationships, and the loader creates the running image.

1. Source Files, Object Files, and Executables

```
source files
  ↓ compiler
object files
  ↓ linker + libraries
executable or shared library
  ↓ operating-system loader
running process
```

An **object file** usually contains:

Object-file component	Purpose
Code/static data	Instructions and compile-time data
Symbol table	Defined and unresolved names
Relocations	Places whose addresses need later repair
Debug information	Maps instructions and values back to source
Target metadata	Identifies architecture, ABI, and file format

The linker combines object files, resolves symbols, lays out sections, applies relocations, and emits the final executable or library.

2. Common Sections

Exact names differ among ELF, PE/COFF, Mach-O, and individual toolchains, but these categories are common:

Section	Typical contents	Usual permissions
<code>.text</code>	Machine instructions	Read + execute
<code>.rodata</code>	String literals and immutable constants	Read
<code>.data</code>	Initialized writable globals	Read + write
<code>.bss</code>	Zero-initialized globals	Read + write
Symbol/string tables	Names used by linking or debugging	Read
Relocations	Places whose addresses need adjustment	Loader/linker data

Debug sections	Source lines, types, local names	Not needed to execute
----------------	----------------------------------	-----------------------

`.bss` does not need to store a file full of zero bytes. The executable records the required size, and the loader supplies zero-initialized memory.

3. Symbols and Name Resolution

A **symbol** associates a name with a function, object, section, or address.

Symbol kind	Linker meaning
Defined	Provided by the current object/library
Undefined	Required from another object/library
Local	Private to one object or translation unit
Global	Participates in cross-object resolution
Weak	May be replaced by a stronger definition

Languages that support overloading or namespaces often use **name mangling** to encode more information into a linker-visible name:

```
source name: math::max(i32, i32)
mangled name: implementation-specific encoded symbol
```

An ABI should not rely on an unstable language-specific mangling scheme unless both sides use compatible toolchains.

4. Relocations and Position Independence

An object file often cannot know final addresses. Instead it contains a placeholder plus a relocation such as:

```
"At offset 0x24, insert the address of symbol print_value."
```

The linker or loader computes the final value after section placement.

Position-independent code (PIC) avoids assuming one fixed load address. Shared libraries normally need PIC, and position-independent executables allow the operating system to randomize the program's base address.

Coordinate	Meaning
File offset	Position inside the executable file
Virtual address (VA)	Address where bytes appear in a process
Relative virtual address	Offset from an image or module base

Never mix these three without an explicit conversion.

5. Static and Dynamic Linking

Model	What is included	Main trade-off
Static linking	Required library code is copied into the executable	Larger files, fewer runtime dependencies
Dynamic linking	Executable refers to libraries loaded at runtime	Smaller files, version and deployment concerns

Dynamic linking involves at least three parties:

1. the compiler emits calls and symbol references;
2. the linker records dependencies and creates the required tables;
3. the loader maps libraries and resolves runtime addresses.

This is why “it compiled” does not guarantee “it launches.” A binary may still have a missing library, incompatible ABI, unsupported architecture, or loader configuration problem.

6. Calling Conventions

A calling convention answers:

ABI question	What must be specified
Arguments	Registers/stack locations by type and position
Return value	Register, memory, or hidden return pointer
Caller-preserved	Registers the caller must save if needed
Callee-preserved	Registers the callee must restore if used
Stack cleanup	Which side removes argument storage
Stack alignment	Required alignment at call boundaries
Variadic arguments	Encoding and retrieval rules

Conceptual call sequence:

```
caller:
  place arguments
  save caller-preserved state if needed
  call function

callee:
  establish frame
  save callee-preserved state
  execute body
  place return value
  restore state
  return
```

Compiler backends must implement the target convention exactly. A mismatch can look like random register corruption even when both functions are individually correct.

7. ABI Versus API

- An **API** describes source-level usage: functions, types, and expected behavior.
- An **ABI** describes binary-level compatibility: data layout, symbol naming, calling convention, alignment, and register rules.

Two libraries may have the same conceptual API but incompatible ABIs. ABI stability matters for shared libraries, plugins, operating-system interfaces, and foreign-function interfaces.

8. Rust FFI Boundary

Use an explicit C ABI and C-compatible types when exposing a simple function:

```
#[unsafe(no_mangle)]
pub extern "C" fn checked_add(left: i32, right: i32, out: *mut i32) -> bool {
    let Some(sum) = left.checked_add(right) else {
        return false;
    };

    let Some(out) = (unsafe { out.as_mut() }) else {
        return false;
    };

    *out = sum;
    true
}
```

The `unsafe` operation is small, and its required condition is obvious: the incoming pointer must either be null or point to a valid writable `i32`.

FFI concern	Required practice
Struct layout	Use <code>#[repr(C)]</code> for C-shared records
Integer widths	Use fixed-width types where layout matters
Allocation	Define who allocates and who frees
Unwinding	Do not let Rust panics cross a foreign ABI
Rust-native types	Do not expose references, <code>String</code> , <code>Vec</code> , or trait objects
Pointer contract	Document nullability, alignment, lifetime, and thread safety
Versioning	Evolve exported interfaces deliberately

9. ELF Has a Linking View and an Execution View

ELF deliberately describes the same file in two ways:

View	Main structure	Consumer	Purpose

Linking view	Sections and section headers	Compiler, linker, debugger	Organize code, data, symbols, relocations
Execution view	Segments and program headers	Kernel and dynamic loader	Map byte ranges with runtime permissions

One or more sections can contribute to one loadable segment.

```
sections: .text + .rodata → read/execute load segment
sections: .data + .bss → read/write load segment
```

This is why “section” and “segment” are related but not interchangeable.

Program-header field	Meaning
p_type	Loadable segment, interpreter, dynamic data, etc.
p_offset	Byte offset in the file
p_vaddr	Virtual address after loading
p_filesz	Bytes present in the file
p_memsz	Bytes required in memory
p_flags	Read/write/execute permissions
p_align	Required file/memory alignment relationship

When `p_memsz` is larger than `p_filesz`, the loader supplies zero-filled memory for the difference. This is how zero-initialized data can occupy memory without occupying the same number of bytes in the file.

10. ELF Entry Point and Program Interpreter

An ELF header includes an entry-point virtual address. A program header of type `PT_INTERP` can name the program interpreter, normally the dynamic linker for a dynamically linked executable.

Conceptually:

```
kernel maps executable segments
  ↓ sees PT_INTERP
kernel maps dynamic linker
  ↓ starts dynamic linker
dynamic linker loads dependencies and resolves relocations
  ↓ transfers control to executable entry point
startup code eventually calls main
```

The executable is still native machine code. Here, “interpreter” means a loader that finishes preparing the native image, not a source-language interpreter.

11. Dynamic Symbols, Relocations, GOT, and PLT

A shared library can be loaded at different addresses in different processes, so some addresses cannot be finalized until runtime.

- `.dynsym` contains symbols needed for dynamic linking.
- `.dynstr` contains names referenced by dynamic symbol entries.
- dynamic relocation sections describe runtime fixups.
- the **Global Offset Table (GOT)** holds runtime-resolved addresses used by position-independent code.
- on architectures/toolchains that use it, the **Procedure Linkage Table (PLT)** provides call stubs that lead through resolver-managed slots.

Simplified lazy-resolution path:

```
first call through PLT stub
  ↓ unresolved GOT slot
dynamic resolver finds symbol in loaded libraries
  ↓ writes resolved address into slot
later calls jump through the resolved slot
```

Systems may instead resolve symbols eagerly at startup. Details vary by architecture, ABI, linker, and hardening settings.

Safe Inspection Commands

For a trusted local ELF file:

```
file ./program
readelf -h ./program      # ELF header
readelf -l ./program      # program headers / segments
readelf -S ./program      # section headers
readelf -s ./program      # symbols
readelf -r ./program      # relocations
readelf -d ./program      # dynamic entries and dependencies
objdump -d ./program      # disassembly
```

Prefer parsers such as `readelf` and `objdump` for initial inspection of untrusted files. Do not execute an unknown binary merely to discover its dependencies.

Embedded Systems: From Reset Vector to Peripheral I/O

[Comprehensive Rust](#) includes specialized material on bare-metal development, unsafe Rust, and using Rust inside Chromium. Together they show three versions of the same lesson: make environmental and safety assumptions explicit at boundaries.

 **Low-level rule:** when the operating system or type system cannot enforce an assumption, **write the assumption down as a contract**.

1. Hosted Versus Bare-Metal Programs

A hosted program starts inside an operating-system process. The OS and standard library provide files, sockets, threads, clocks, allocation, arguments, and process exit.

A bare-metal program may start directly at a reset vector with no OS, loader, filesystem, allocator, or console.


```

reset
↓
startup code establishes stack and memory
↓
initialize hardware and interrupt table
↓
enter firmware main loop or scheduler

```

Rust library layers:

Layer	Provides
<code>core</code>	Language fundamentals, slices, iterators, <code>Option</code> , <code>Result</code> , atomics
<code>alloc</code>	<code>Box</code> , <code>Vec</code> , <code>String</code> , <code>Arc</code> , and other allocation-backed collections
<code>std</code>	OS-backed files, networking, threads, clocks, process APIs, and more

`#![no_std]` removes the dependency on `std`; it does not remove `core`. The `alloc` crate is optional and requires a global allocator supplied by the platform.

2. Minimal `no_std` Shape

The exact entry point and linker setup are target-specific, but the source commonly has this shape:

```

#![no_std]
#![no_main]

use core::hint::spin_loop;
use core::panic::PanicInfo;

#[panic_handler]
fn panic(_info: &PanicInfo<'_>) -> ! {
    loop {
        spin_loop();
    }
}

#[unsafe(no_mangle)]
pub extern "C" fn firmware_entry() -> ! {
    loop {
        spin_loop();
    }
}

```

This is not a complete bootable image:

Target requirement	Responsibility
Linker layout	Place code, data, stack, and device regions
Entry contract	Use the correct reset symbol and calling convention
Stack	Establish a valid initial stack
Static memory	Copy initialized data and zero <code>.bss</code>

Panic strategy	Define terminal failure behavior
Build/runner config	Select target, image format, and launch mechanism
Hardware initialization	Configure clocks, memory, pins, and peripherals

Those responsibilities resemble a language runtime's startup sequence. Building firmware makes the usually hidden path from executable entry point to language-level `main` visible.

3. Hardware-Abstraction Layers

A useful embedded stack:

```

application
  ↓
board support package (specific board and pins)
  ↓
hardware abstraction layer (typed peripheral operations)
  ↓
peripheral access crate (register-level device description)
  ↓
memory-mapped registers

```

Keep application logic testable on the host by depending on small traits:

```

trait OutputPin {
    type Error;

    fn set_high(&mut self) -> Result<(), Self::Error>;
    fn set_low(&mut self) -> Result<(), Self::Error>;
}

fn blink_once<P: OutputPin>(pin: &mut P) -> Result<(), P::Error> {
    pin.set_high()?;
    pin.set_low()
}

```

For real code, return a named error type or the trait's associated error. The important design idea is that the application owns behavior while the board layer owns volatile register details.

4. Memory-Mapped I/O

Peripherals often expose control and status registers at fixed addresses. Ordinary loads and stores are insufficient because the compiler may remove or combine them. Volatile access says the access itself is observable.

```

use core::ptr::NonNull;

#[derive(Clone, Copy)]
struct Register32 {
    address: NonNull<u32>,
}

impl Register32 {
    /// # Safety
    ///
    /// `address` must be aligned, valid for 32-bit volatile accesses for the
    /// lifetime of this value, and refer to a register whose access rules permit
    /// reads and writes performed through this wrapper.
    unsafe fn new(address: usize) -> Option<Self> {
        if address % core::mem::align_of::<u32>() != 0 {
            return None;
        }

        NonNull::new(address as *mut u32).map(|address| Self { address })
    }

    fn read(&self) -> u32 {
        // SAFETY: Construction requires a valid readable MMIO register.
        unsafe { self.address.as_ptr().read_volatile() }
    }

    fn write(&mut self, value: u32) {
        // SAFETY: Construction requires a valid writable MMIO register.
        unsafe { self.address.as_ptr().write_volatile(value) }
    }
}

```

The wrapper is only honest if its constructor’s contract matches the hardware manual.

Volatile does not guarantee	Separate requirement
Data-race safety	Atomics, critical sections, or exclusive access
Multi-register atomicity	Device-specific transaction protocol
CPU/device ordering	Correct memory barriers/fences
Side-effect-free reads	Register semantics from the hardware manual
Peripheral ownership	A higher-level ownership policy

Avoid implementing `Send` or `Sync` automatically for MMIO wrappers. Decide those properties from the hardware sharing rules.

5. Interrupts and Shared State

An interrupt can occur between ordinary instructions, so code shared with an interrupt handler is concurrent code.

Interrupt concern	Preferred approach
Small shared values	Atomics with a justified ordering

Compound state	Target-provided critical section
Data handoff	Interrupt-safe SPSC queue
Handler duration	Record minimal work and return quickly
Forbidden work	Avoid blocking, allocation, and unbounded loops

```
use core::sync::atomic::{AtomicBool, Ordering};

static SAMPLE_READY: AtomicBool = AtomicBool::new(false);

fn interrupt_handler() {
    SAMPLE_READY.store(true, Ordering::Release);
}

fn main_loop_step() {
    if SAMPLE_READY.swap(false, Ordering::Acquire) {
        process_sample();
    }
}

fn process_sample() {
    // Ordinary application work.
}
```

Acquire/release here publishes work associated with the flag, but the exact design must match how the sample data itself is stored. `volatile` is not a substitute for atomics.

6. What `unsafe` Actually Permits

Unsafe Rust enables a small set of operations that the compiler cannot prove safe:

Unsafe capability	Proof obligation moved to the programmer
Dereference raw pointer	Validity, alignment, initialization, and aliasing
Mutable static state	Synchronization and exclusive access
Access union field	Active representation is valid for the field
Call unsafe function	Every documented precondition is satisfied
Implement unsafe trait	The implementation upholds the trait-wide contract

`unsafe` does not disable the borrow checker or turn Rust into C. It transfers proof obligations from the compiler to the programmer.

A good unsafe abstraction has:

1. a documented safety contract;
2. validation performed in safe code where possible;
3. the smallest practical unsafe block;
4. a safe interface that cannot violate the invariant;
5. tests, fuzzing, and dynamic checking appropriate to its risk.

```

/// Views `len` bytes beginning at `pointer`.
///
/// # Safety
///
/// `pointer` must be non-null and aligned for `u8`, reference `len`
/// initialized bytes in one allocation, and remain valid for `a`.
unsafe fn bytes_from_raw<'a>(pointer: *const u8, len: usize) -> &'a [u8] {
    // SAFETY: The caller promises every requirement of `from_raw_parts`.
    unsafe { core::slice::from_raw_parts(pointer, len) }
}

```

Use an explicit unsafe block even inside an `unsafe fn`; the function declares what the caller must prove, while the block marks where the implementation relies on that proof. The `unsafe_op_in_unsafe_fn` lint helps enforce this distinction.

7. Unsafe Review Questions

Review area	Question
Invariant	What exact fact makes the operation memory safe?
Ownership	Who owns the allocation or device?
Lifetime	How long does it remain valid?
Pointer validity	Is it aligned and within one allocation?
Initialization	Are bytes valid before the typed read?
Aliasing	Can mutable access overlap?
Concurrency	Can a thread, signal, interrupt, or FFI call mutate it?
Layout	Is representation guaranteed by <code>repr</code> or a wire format?
Arithmetic	Can an offset/length wrap before validation?
Unwinding	Can panic leave foreign code or hardware invalid?
Safe API	Can any safe caller reach undefined behavior?

Comments should explain the proof, not restate the operation:

```

// Weak:
// SAFETY: dereference pointer.

// Useful:
// SAFETY: `cursor < end` was checked above; both pointers are derived
// from the same allocation, and the parser holds the only mutable borrow.

```

8. Power, Clocks, Reset, and Watchdogs Create the First State

An embedded processor does not begin at `main`. Power and clock circuitry first bring the chip into a state in which instructions can run.

```

power rises
  → power-on-reset holds logic in a known state
  → oscillator/clock source stabilizes
  → reset logic releases the CPU
  → CPU loads initial stack pointer and reset vector
  → startup code initializes RAM and clocks
  → application enters its long-running control loop

```

Mechanism	Why it exists	Failure if misunderstood
power-on reset	establish known register and logic state	execution starts from corrupted state
brownout detector	reset when voltage is too low for reliable logic	silent data or instruction corruption
oscillator	generate a timing reference	CPU and peripherals cannot advance predictably
phase-locked loop	multiply/shape clock frequency	wrong bus and peripheral timing
reset controller	reset selected chip domains	stale peripheral state survives unexpectedly
watchdog	reset if software stops making progress	permanent hang without recovery

Clock configuration is a dependency graph. A peripheral clock is often derived from a system clock through dividers and gates. Changing one frequency affects timers, serial baud rates, flash wait states, power use, and real-time deadlines.



Invariant: code may use a peripheral only after its power domain, clock, reset state, pin routing, and configuration satisfy that peripheral's hardware contract.

9. Embedded Scheduling Is About Deadlines and Bounded Work

Without a desktop operating system, the program still needs a scheduling policy. Common models are:

Model	How work runs	Trade-off
superloop	poll each subsystem repeatedly	simple, but one slow step delays everything
interrupt-driven	hardware preempts ordinary code	responsive, but shared-state reasoning is harder
cooperative executor	tasks yield at explicit wait points	compact and predictable if no task blocks
preemptive RTOS	scheduler switches ready tasks by policy	isolation and priorities with more overhead

Real-time does not mean “fast.” It means the response is useful only if it occurs before a deadline. Important quantities include:

- **latency** — delay before work begins;
- **execution time** — CPU time the work consumes;
- **period** — how often periodic work is released;
- **deadline** — latest acceptable completion time;
- **jitter** — variation in timing;
- **worst-case execution time** — a defensible upper bound.

```
struct PeriodicTask {
    next_release_us: u64,
    period_us: u64,
}

impl PeriodicTask {
    fn take_if_due(&mut self, now_us: u64) -> bool {
        if now_us < self.next_release_us {
            return false;
        }

        // Advance by the period instead of `now + period`. That preserves the
        // intended phase and makes missed deadlines observable as accumulated lag.
        self.next_release_us = self.next_release_us.saturating_add(self.period_us);
        true
    }
}
```

Interrupt handlers should acknowledge the device, capture minimal state, and schedule bounded follow-up work. Long parsing, allocation, logging, and blocking I/O inside an interrupt increase latency for every lower-priority event.

10. GPIO, UART, SPI, I²C, and CAN Are Different Communication Models

An electrical connection, a signaling convention, and a software protocol are separate layers. A driver must satisfy all three.

Interface	Clocking and topology	Data model	Common use
GPIO	individual digital pins	sampled or driven bits	buttons, LEDs, chip selects
UART	asynchronous point-to-point	framed byte stream	console, modules, debugging
SPI	controller-supplied clock, separate select lines	full-duplex shifted words	displays, flash, sensors
I ² C	shared clock/data with addressed devices	half-duplex transactions	configuration and sensors
CAN	shared differential bus with arbitration	prioritized frames	vehicles and industrial control

UART agrees on baud rate and frame shape but carries no address or packet length by itself. **SPI** shifts data in both directions on each clock and usually needs a chip-specific command protocol. **I²C** includes addresses and acknowledgements but requires pull-up resistors and bus arbitration. **CAN** arbitrates by message identifier so a numerically dominant priority can continue without corrupting the bus.

```
trait RegisterBus {
    type Error;

    fn write_register(&mut self, address: u8, value: u8)
        -> Result<(), Self::Error>;

    fn read_register(&mut self, address: u8)
        -> Result<u8, Self::Error>;
}

fn enable_measurement(
    bus: &mut impl RegisterBus<Error = BusError>,
) -> Result<(), BusError> {
    // The sensor-specific state transition is expressed independently of
    // whether the physical transport is SPI or I2C.
    bus.write_register(0x10, 0b0000_0001)?;
    Ok(())
}

#[derive(Debug)]
struct BusError;
```

The trait is not the hardware. It describes the smallest protocol capability the sensor driver needs. A HAL implementation translates that capability into register accesses, DMA requests, interrupts, and pin transitions for one target.

Primary concept references: [Embedded Rust memory-mapped registers](#), [Embedonomicon memory layout](#), and [Arm CMSIS-Core processor and peripheral model](#).

11. Linker Scripts Are Part of the Program

On a microcontroller, the executable must place bytes where the hardware expects them. A linker script normally defines memory regions and output sections:


```

MEMORY
{
    FLASH : ORIGIN = 0x00000000, LENGTH = 256K
    RAM   : ORIGIN = 0x20000000, LENGTH = 64K
}

ENTRY(Reset);

SECTIONS
{
    .vector_table ORIGIN(FLASH) :
    {
        LONG(ORIGIN(RAM) + LENGTH(RAM));
        KEEP(*(.vector_table.reset_vector));
    } > FLASH

    .text :
    {
        *(.text .text.*);
    } > FLASH
}

```

Linker-script element	Purpose
MEMORY	Names address ranges and their sizes
ENTRY(symbol)	Selects the executable entry point
SECTIONS	Maps input sections into output sections
> FLASH / > RAM	Selects the destination memory region
KEEP(...)	Prevents garbage collection of hardware-referenced data
EXTERN(symbol)	Forces the linker to search for a required symbol
/DISCARD/	Removes sections the runtime deliberately does not use

Hardware can reference a vector-table entry without a normal machine-code call. That means the linker may see no ordinary reference; `KEEP` or an equivalent mechanism prevents the required entry from being discarded.

12. Reset and Vector-Table Contracts

For Cortex-M, the first vector-table entries include the initial stack pointer and reset handler. The reset handler has no caller to return to, so its return type is `!`.

```

#![no_std]

#[unsafe(no_mangle)]
pub unsafe extern "C" fn Reset() -> ! {
    // Copy initialized data, zero .bss, initialize runtime state,
    // then call the application entry point.
    loop {
        core::hint::spin_loop();
    }
}

#[unsafe(link_section = ".vector_table.reset_vector")]
#[unsafe(no_mangle)]
pub static RESET_VECTOR: unsafe extern "C" fn() -> ! = Reset;

```

Attribute	Required proof
<code>#[unsafe(no_mangle)]</code>	The global symbol name cannot collide
<code>#[unsafe(export_name = "...")]</code>	The chosen exported name is globally unique
<code>#[unsafe(link_section = "...")]</code>	The item is valid in the destination memory region
<code>extern "C"</code>	The hardware/foreign caller follows that ABI
<code>-> !</code>	Control never returns to a nonexistent caller

The Rust ABI is not a stable hardware boundary. Use the architecture/runtime's required ABI and inspect the final ELF, not merely the source.

Reset Must Create the Language's Memory Invariants

Before ordinary Rust code can assume initialized statics, startup code commonly has to transform the linked image into the runtime image:

```

hardware loads initial SP and Reset address from vector table
↓
copy initialized `.data` bytes: load address in FLASH → run address in RAM
↓
zero `.bss`
↓
initialize clocks/memory/runtime facilities required by the platform
↓
call the non-returning application entry point

```

Linker symbol concept	Meaning during reset
data load start	initialized bytes stored in nonvolatile memory
data run start/end	RAM range that must receive those bytes
BSS start/end	RAM range that must become all zero
stack boundary	initial stack location and permitted extent
heap boundary	optional allocator range that must not overlap data

The linker defines these addresses; reset code consumes them as raw pointers. The `unsafe` proof must cover:

1. source and destination ranges are valid for the required access;
2. byte counts come from ordered, non-overflowing linker symbols;
3. copy regions obey the required overlap rule;
4. startup performs initialization exactly once before safe code reads the statics; and
5. no interrupt or second execution context observes partially initialized state.

Linker assertions can turn layout assumptions into build failures:

```
ASSERT(__data_end <= ORIGIN(RAM) + LENGTH(RAM), "initialized data exceeds RAM");
ASSERT(__heap_start <= __stack_limit, "heap and stack regions overlap");
```



Compiler connection: a source-level `static X: u32 = 7` becomes bytes, section placement, load/run addresses, and startup work. The language guarantee exists only if every handoff preserves it.

13. Exception and Interrupt Vector Tables

When an exception occurs, the processor suspends the current flow, saves architectural state according to its rules, and selects a handler through a vector table.

```
external/internal event
  ↓ hardware exception entry
save required state
  ↓ vector-table lookup
run handler
  ↓ exception return
restore interrupted state
```

Vector-table invariant	Requirement
Placement	Table begins at the architecture's expected address
Entry order	Each slot corresponds to the documented exception
Validity	Every usable slot holds a correctly typed handler address
Defaults	Unimplemented handlers enter a safe diagnostic halt
Override	Application handlers replace defaults at link time
Retention	Link-time garbage collection cannot remove the table

Compile-time defaults can be supplied in the linker script and overridden by an application-defined symbol. This gives static customization without a runtime lookup.

14. Typestate for Peripheral State Machines

The Embedded Rust Book models peripherals as state machines. Consuming a value during a transition ensures the old state cannot still be used.

```

use core::marker::PhantomData;

struct Disabled;
struct Input;
struct Output;

struct Pin<State> {
    number: u8,
    // The zero-sized marker makes the compile-time state part of the type.
    _state: PhantomData<State>,
}

impl Pin<Disabled> {
    fn into_input(self) -> Pin<Input> {
        // Configure hardware before publishing the new typestate.
        configure_as_input(self.number);
        Pin {
            number: self.number,
            _state: PhantomData,
        }
    }

    fn into_output(self) -> Pin<Output> {
        // Consuming `self` prevents use of the old Disabled handle.
        configure_as_output(self.number);
        Pin {
            number: self.number,
            _state: PhantomData,
        }
    }
}

impl Pin<Output> {
    fn set_high(&mut self) {
        // This method does not exist for Disabled or Input pins.
        write_output_high(self.number);
    }
}

fn configure_as_input(_pin: u8) {}
fn configure_as_output(_pin: u8) {}
fn write_output_high(_pin: u8) {}

```

`Pin<Input>` has no `set_high` method, so that invalid operation fails at compile time. The marker types are zero-sized and normally disappear after optimization.

Typestate benefit	Cost/trade-off
Illegal calls disappear	More generic types and <code>impl</code> blocks
Transitions are explicit	State changes may consume and return values
No runtime tag needed	Dynamic state may require an enum instead
Ownership is visible	Shared peripherals need an intentional sharing model

Use typestate when the state graph is small and correctness-critical. Use a runtime enum when states genuinely arrive dynamically or must live in one heterogeneous collection.

15. HAL Portability

`embedded-hal` -style traits separate applications and device drivers from a specific microcontroller implementation:

```
application
  ↓ generic driver
embedded HAL traits
  ↓ chip-family HAL
peripheral access crate
  ↓ volatile registers
hardware
```

Layer	Owns
Application	Product behavior
Driver	Device protocol, such as a sensor over SPI
HAL trait	Portable capability contract
Chip HAL	Pins, clocks, buses, DMA, and peripheral setup
Peripheral access crate	Register-level representation
Board support package	Board-specific pin/component wiring

Generic drivers should depend on the smallest capability trait they need. Avoid leaking one board's concrete pin or peripheral types into reusable application logic.

16. Embedded Concurrency Choices

Interrupt handlers are concurrent with the main loop. A `static mut` increment is not automatically atomic: it may compile into `load → modify → store` and lose an interrupt's update.

Environment	Appropriate synchronization
No interrupts/one loop	Ordinary exclusive ownership
Main + interrupts	Atomics, interrupt-safe queue, or critical section
DMA + CPU	Ownership transfer plus required memory barriers
Multiple cores	SMP-safe atomics/locks; interrupt masking is not enough
Real-time tasks	Priority-aware framework and bounded operations

`Send` means a value can safely move between execution contexts; `Sync` means a shared reference can be used safely from multiple contexts. Treat an interrupt handler as a separate execution context when reasoning about shared state.

Critical sections that merely disable interrupts on one core do not protect against another core. The proof must match the hardware topology.

17. Inspect the Artifact, Not Only the Source

Question	Useful inspection
Is the entry point correct?	ELF header and symbol table
Is the vector table retained?	Section dump and raw bytes
Are code/data in valid regions?	Program/section headers plus linker map
How large is each section?	size / cargo size -style report
What does reset execute?	Disassembly from the reset symbol
Did typestate disappear?	Compare optimized assembly/code size
Does it boot without hardware?	QEMU or another architecture-appropriate emulator

Keep the linker map, target specification, disassembly, and exact firmware hash with a release. They are part of the evidence needed to reproduce a low-level bug.

18. Firmware, Bootloaders, Kernels, and Drivers Form a Chain

```
reset vector
  → immutable/early firmware
  → platform firmware and hardware initialization
  → bootloader chooses and verifies an image
  → kernel initializes memory, interrupts, scheduler, and drivers
  → init/service manager starts user space
  → applications call libraries and system calls
```

Layer	Primary responsibility	Typical artifact
firmware	configure a device below the ordinary OS boundary	flash image, UEFI module
bootloader	initialize enough hardware to locate, verify, and start a kernel	U-Boot, UEFI loader
kernel	privileged resource management and isolation	kernel image plus modules
driver	translate a device model into an OS interface	kernel module or user-mode service
middleware	reusable behavior between application and platform	media, RPC, database layer
backend	server-side policy, data, and integrations	service process
frontend	user-facing presentation and interaction	native/web UI

These are **roles**, not guaranteed processes. Firmware may include an RTOS; a microkernel may run drivers in user space; middleware may exist on the client, server, or both.

A secure-boot chain verifies the next stage before transferring control. A signature answers “was this approved key used over these exact bytes?” It does not prove that the signed code is bug-free.

19. A Driver Is a Concurrent Translator

A driver connects two contracts:

- the OS contract: handles, files, sockets, queues, power states, and errors;
- the hardware contract: registers, interrupts, descriptors, timing, and DMA.

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum DeviceState {
    Reset,
    Ready,
    Busy,
    Fault,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum DriverError {
    WrongState,
    BufferTooLarge,
}

fn begin_transfer(state: &mut DeviceState, bytes: usize) -> Result<(), DriverError> {
    // Model the hardware state transition before touching real registers.
    if *state != DeviceState::Ready {
        return Err(DriverError::WrongState);
    }
    if bytes == 0 || bytes > 4096 {
        return Err(DriverError::BufferTooLarge);
    }
    *state = DeviceState::Busy;
    Ok(())
}
```

The toy state machine is useful even when the final implementation is `unsafe`: it separates protocol validity from the tiny adapter that performs volatile reads/writes.

20. DMA Lets a Device Access Memory Without CPU Copies

Direct memory access changes the data path:

```
CPU virtual address —page tables—> physical memory
device DMA address — IOMMU —> permitted physical pages
```

Term	Meaning
descriptor	record telling a device where and how much to transfer
coherent buffer	CPU/device views obey a platform-defined coherence contract
streaming mapping	temporary mapping for a directional transfer
IOMMU	translates and restricts device-visible addresses
bounce buffer	intermediary used when addressing or isolation requires it
completion interrupt	device reports finished work

An external DMA-capable device can be powerful because it may operate outside the ordinary process/page-table path. Defensive design uses an IOMMU, minimal mapped ranges, short mapping lifetimes, device authorization, signed firmware, and physical security. These notes do not provide bypass or unauthorized acquisition procedures.

21. JTAG and SWD Expose Hardware Debug State

JTAG began as a boundary-scan interface and is also commonly used to reach processor debug infrastructure. ARM's SWD provides a lower-pin-count debug transport.

Authorized debugging may expose:

- halt, resume, and single-step;
- registers and memory;
- hardware breakpoints/watchpoints;
- flash programming;
- trace and peripheral access.

That power makes lifecycle policy essential: production devices may authenticate, limit, or permanently disable debug access. Practice with a development board you own, document voltage and pinout first, and never assume a connector is electrically safe.

22. Real Hardware, Simulator, Emulator, VM, and Container

Environm ent	What is reproduced	Kernel relation	Best use
simulator	selected behavior at an abstract level	implementation-specific	circuits, devices, deterministic teaching
emulator	another machine/ISA or device behavior	may run a guest kernel	compatibility and firmware labs
virtual machine	virtualized hardware for a guest OS	guest has its own kernel	OS isolation and testing
container	isolated host processes	shares host kernel	packaging and service isolation
sandbox	restricted authority, regardless of mechanism	varies	limiting untrusted components
real hardware	actual timing, buses, analog effects	actual target	final validation

[Wokwi](#) simulates boards such as Arduino, ESP32, and STM32 and offers virtual logic analysis, GDB debugging, networking, SD cards, and custom chips. It is excellent for repeatable firmware experiments, but a simulator cannot fully guarantee real electrical behavior, performance, radio conditions, or silicon errata.

23. Hypervisors and Hyper-V

A hypervisor controls privileged execution, second-level address translation, virtual interrupts, and device access so several guest systems can share hardware.

Concept	Meaning
VM exit	guest operation transfers control to the hypervisor
guest virtual address	address used by code inside the guest
guest physical address	"physical" address visible to the guest
host physical address	actual machine memory
SLAT / EPT / NPT	hardware-assisted guest-physical translation
virtual device	emulated or paravirtualized hardware interface
root/parent partition	privileged management environment in Hyper-V
child partition	guest environment managed through the hypervisor

Memory introspection from a hypervisor can observe a guest outside its ordinary OS process model, which helps debuggers, incident response, and malware analysis. It also creates an exceptionally privileged trust boundary. [secret.club's hypervisor article](#) is useful for understanding VM exits and second-level translation; apply those ideas only to owned guests and defensive laboratories.

[Hyper-V's architecture](#) uses a hypervisor plus root and child partitions, with VMBus for synthetic device communication. Nested virtualization adds another translation and scheduling layer, so performance measurements need to state the full nesting topology.

24. Containers Are Process Isolation, Not Tiny VMs

A Docker container packages a process and its filesystem view while normally sharing the host kernel. Namespaces isolate selected resource views; cgroups account for and limit resources; capabilities, seccomp, labels, and read-only mounts reduce authority.

Boundary	Question
image	What exact files and configuration are supplied?
namespace	Which processes, mounts, network interfaces, and IDs are visible?
capability	Which privileged kernel operations remain allowed?
cgroup	What CPU, memory, and I/O limits apply?
secret	How is sensitive configuration supplied and rotated?
supply chain	Are base images pinned, scanned, and reproducible?

Containers improve packaging and isolation but do not turn unsafe host-kernel exposure into a security boundary automatically. Compare this with the [Docker container model](#).

25. Platform Storage and Machine Identity

Mechanism	Scope	Low-level lesson
filesystem	hierarchical byte streams plus	names are not identities; links and races exist

	metadata	
Windows Registry	typed hierarchical configuration database	views, hives, ACLs, and transaction behavior matter
property list (plist)	Apple structured configuration/serialization	XML and binary encodings can represent the same values
environment variable	inherited process string map	observable, size-limited, not a secret vault
secure credential store	OS-mediated secret storage	access policy and user context matter
hardware identifier	property derived from one or more components	can change, collide, or expose privacy

An HWID is not a cryptographic identity. Hardware changes, virtualization, driver normalization, privacy controls, and collisions make it unsuitable as the sole authentication factor. If device enrollment is required, generate a key pair in an OS/hardware-backed keystore, register the public key, rotate it, and provide recovery.

For cross-platform tools, keep a typed configuration model in the core and place Registry/plist/filesystem adapters at the edge. Do not collect broad fingerprints when a random installation ID or authenticated device key meets the requirement.

Sources: [Linux DMA mapping guide](#), [Android bootloader overview](#), [ARM debug access overview](#), [Wokwi documentation](#), and [Docker overview](#).

Concurrency, Atomics, Actors, and Memory Models

 **Library lens:** Actix is used as one concrete actor model: owned state, typed messages, bounded mailboxes, serialized transitions, lifecycle, backpressure, and supervision.

Concurrency adds a second ordering problem. A compiler already reasons about the order of expressions; concurrent code must also reason about what other threads are allowed to observe.

 **Concurrency rule:** shared state is only understandable when **ownership**, **synchronization**, and **ordering** are all explicit.

1. Processes, Threads, Tasks, and Events

Unit	Memory relationship	Scheduled by
Process	Usually isolated address space	Operating system
Thread	Shares process memory	Operating system
Async task	Shares process memory, cooperatively yields	Async runtime
Actor	Communicates through messages	Runtime or application

Concurrency means multiple activities make progress over overlapping time. **Parallelism** means work literally executes at the same instant on different cores or processors.

2. The Core Shared-State Problem

A source expression that looks like one update:

```
counter = counter + 1
```

normally contains several lower-level actions:

```
load counter
add 1
store counter
```

Two threads can interleave those actions and lose an update. This is a **data race** when unsynchronized conflicting accesses violate the language's memory model.

3. Synchronization Tools

Tool	Best mental model
Mutex	One owner of protected state at a time
Read/write lock	Many readers or one writer
Condition variable	Sleep until protected state may have changed
Semaphore	Finite pool of permits
Channel	Transfer values/events between participants
Atomic	Indivisible primitive operation with chosen ordering
Barrier	Wait for every participant to reach one phase

Choose the tool based on the invariant, not on familiarity.

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (send, receive) = mpsc::channel();

    let worker = thread::spawn(move || {
        let result = (1..=100).sum::<u64>();
        send.send(result).expect("receiver should still exist");
    });

    let result = receive.recv().expect("worker should send a result");
    worker.join().expect("worker should not panic");
    assert_eq!(result, 5050);
}
```

Message passing can simplify ownership: the sender transfers a value instead of giving multiple threads mutable access to it.

4. Atomics and Ordering

An atomic operation prevents tearing, but memory ordering controls how operations around it may be observed.

Ordering	Mental model
Relaxed	Atomicity only; no cross-thread ordering guarantee
Acquire	Later operations cannot move before the acquire
Release	Earlier operations cannot move after the release
AcqRel	Acquire + release for a read-modify-write operation
SeqCst	Strongest common ordering; one global order for these operations

Use the weakest ordering only when you can explain the proof. A mutex or channel is often clearer and fast enough.

```
use std::sync::atomic::{AtomicBool, Ordering};

static READY: AtomicBool = AtomicBool::new(false);

// Producer publishes ordinary data first, then:
READY.store(true, Ordering::Release);

// Consumer waits, then may observe data published before the release:
while !READY.load(Ordering::Acquire) {
    std::hint::spin_loop();
}
```

This fragment illustrates ordering but is not a complete queue. Correct lock-free structures also need careful lifetime, progress, and reclamation design.

5. Compiler Implications

A language implementation must define:

Memory-model topic	Required language rule
Atomic operations	Which reads, writes, and updates are indivisible
Data race	Which conflicting accesses constitute one
Race semantics	Error, undefined behavior, or defined outcome
Happens-before	Which synchronization creates ordering
Transfer	Which values may move between threads
Shared mutation	Whether shared references permit mutation

Optimizers rely on these rules. If the language declares data races invalid, the optimizer may assume they never occur and transform code accordingly.

6. Concurrency Failure Modes

Failure mode	Symptom
Deadlock	Participants wait forever in a dependency cycle
Livelock	Work continues but no useful progress occurs
Starvation	One participant repeatedly loses resource access
Priority inversion	High-priority work waits on lower-priority work
Lost wakeup	Notification occurs before the waiter observes it
ABA problem	$A \rightarrow B \rightarrow A$ fools a compare-and-swap equality check
False sharing	Independent data causes needless cache-line coherence

Debug concurrent systems with event traces, stable IDs, timeouts, and explicit state transitions. Adding print statements can change timing and hide the bug.

7. Actors: Owned State Behind a Typed Mailbox

The [Actix actor documentation](#) presents a concurrent application as independently executing actors that cooperate through typed messages. An actor combines four ideas:

```
private state + mailbox + message handlers + lifecycle
```

Other components hold an **address**, not a direct reference to the actor. Sending a message transfers data and asks the actor's context to run the matching handler.

Actix concept	Mental model
<code>Actor</code>	state and behavior owned by one logical participant
<code>Message</code>	typed protocol request with a declared result type
<code>Handler<M></code>	transition function for one message type
<code>Addr<A></code>	capability to send supported messages to actor type <code>A</code>
<code>Recipient<M></code>	narrower capability that exposes only message type <code>M</code>
<code>Context<A></code>	mailbox, lifecycle, timers, streams, and spawned work
<code>Arbiter</code>	single-threaded event-loop environment hosting async work
<code>SyncArbiter</code>	fixed OS-thread pool containing instances of one actor type
<code>Supervisor</code>	restarts a supervised actor context after failure



Actor rule: actors remove direct shared mutation; they do not remove concurrency. Ordering, overload, cancellation, duplicate work, and failure are now protocol questions.

A Typed Symbol-Table Actor

This small actor serializes symbol interning. Each handler receives `&mut self`, so the state transition does not need a mutex inside the actor:

```
use actix::prelude::*;
use std::collections::HashMap;

#[derive(Default)]
struct SymbolTable {
    slots: HashMap<String, u32>,
    next_slot: u32,
}

impl Actor for SymbolTable {
    type Context = Context<Self>;

    fn started(&mut self, context: &mut Self::Context) {
        // Choose capacity from an explicit traffic and memory budget.
        context.set_mailbox_capacity(64);
    }
}

#[derive(Message)]
#[rtype(result = "Result<u32, String>")]
struct Intern(pub String);

impl Handler<Intern> for SymbolTable {
    type Result = Result<u32, String>;

    fn handle(&mut self, message: Intern, _context: &mut Self::Context) -> Self::Result {
        // Interning is idempotent: repeated names receive the original slot.
        if let Some(&slot) = self.slots.get(&message.0) {
            return Ok(slot);
        }

        // Reserve the next value before mutation so overflow leaves state unchanged.
        let following = self
            .next_slot
            .checked_add(1)
            .ok_or_else(|| "symbol table exhausted u32 slots".to_owned())?;
        let slot = self.next_slot;

        // The actor owns both the string and the map; no caller can mutate them.
        self.slots.insert(message.0, slot);
        self.next_slot = following;
        Ok(slot)
    }
}

#[derive(Message)]
#[rtype(result = "Option<u32>")]
struct Lookup(pub String);

impl Handler<Lookup> for SymbolTable {
    type Result = Option<u32>;
```

```

    fn handle(&mut self, message: Lookup, _context: &mut Self::Context) -> Self::Result {
        self.slots.get(&message.0).copied()
    }
}

#[actix::main]
async fn main() {
    // `start` returns an address; callers never receive `&mut SymbolTable`.
    let symbols = SymbolTable::default().start();

    // `send` waits for the typed handler result and can also report mailbox failure.
    match symbols.send(Intern("main".to_owned())).await {
        Ok(Ok(slot)) => println!("main uses slot {slot}"),
        Ok(Err(domain_error)) => eprintln!("interning failed: {domain_error}"),
        Err(mailbox_error) => eprintln!("actor unavailable: {mailbox_error}"),
    }
}

```

Keep handlers short. A real parse, optimize, or code-generation pass may be CPU-bound; running it directly on an async actor's arbiter blocks other work on that event loop. Send the job to a bounded worker pool or a deliberately sized `SyncArbiter`, then send the result back.

Addresses Are Capabilities

An address answers two questions at once:

1. Where may this message be sent?
2. Which protocol is the receiver known to implement?

Send API	Completion/backpressure behavior
<code>send(message)</code>	returns a future for the handler result or mailbox failure
<code>try_send(...)</code>	attempts immediately; reports a full or closed mailbox
<code>do_send(...)</code>	fire-and-forget; bypasses the mailbox limit and hides send errors

`Recipient<M>` erases the concrete actor type while retaining permission to send only `M`. It is useful for subscribers, event sinks, and interfaces that should not gain the actor's full command surface.

⚠ Backpressure warning: a configured mailbox capacity is not a universal memory limit. `do_send`, context notifications, and work spawned outside the mailbox can bypass or outlive that queue. Bound message size, producer rate, in-flight requests, retries, and downstream work as separate resources.

Lifecycle Is a State Machine

Started → Running → Stopping → Stopped

↑ actor may continue under specific context conditions

State	Design question
Started	Which timers, streams, child actors, or resources are attached?
Running	Which messages are legal, and how is overload handled?

Stopping	Are queued messages dropped, drained, rejected, or persisted?
Stopped	Who observes termination and cleans external resources?

An actor can begin stopping when it calls `Context::stop`, when all addresses are dropped, or when no context event sources remain. If stopping completes, the actor is dropped. Shutdown therefore needs a protocol: stop accepting new work, finish or cancel in-flight work, persist required state, then acknowledge termination.

Arbiter Is Placement; Context Is Actor State

Do not merge these concepts:

Layer	Responsibility
System	top-level Actix runtime lifecycle
Arbiter	one event loop on one thread
<code>Context<A></code>	one actor's mailbox, lifecycle, futures, streams, and timers
actor handler	one typed state transition

Actors started on one arbiter are tied to it, but their addresses can communicate across arbiters. Ordinary async arbiters suit short, non-blocking handlers. `SyncArbiter` creates multiple instances of one actor type across a fixed thread pool for synchronous work; its shared address routes messages to available instances, so instance-local state is not automatically global state.

Supervision Restarts Execution, Not History

A supervisor can create a new actor context and call the actor's restart hook, but it cannot guarantee that the message being processed at failure was completed or replayed.

Restart concern	Required application decision
actor state	reconstruct, reload, reset, or reject restart
failed message	retry only if duplicate execution is safe
side effects	use idempotency keys or transactional boundaries
restart loop	apply limits, backoff, and escalation
observability	retain cause, actor identity, message kind, and restart count

Supervision is not a substitute for durable queues or transactions. If work must survive process loss, persist it outside the in-memory actor system before acknowledging acceptance.

When Actors Fit—and When They Do Not

Good actor boundary	Why it fits
one compiler/REPL session	stateful commands need per-session ordering
module registry or symbol service	one owner coordinates shared logical state

protocol connection	messages map to an explicit connection state
build/job coordinator	schedules bounded work and records outcomes
debugger session	commands mutate one observed-process model
cache writer	one serialization point protects invariants

Poor actor boundary	Better starting point
pure stateless calculation	ordinary function or parallel iterator
every tiny AST node	tree plus traversal; mailbox overhead dominates
hot GC pointer update	runtime memory structure and write barrier
bulk numeric loop	contiguous data and data-parallel worker pool
state requiring multi-actor atomic mutation	database transaction or redesigned ownership

For your own language, define actor semantics independently of Actix:

```
spawn → address/capability → enqueue → handle one transition
      → reply/fail → supervise/stop
```

Specify mailbox ordering, capacity, cancellation, failure propagation, restart, location transparency, serialization, and whether messages are processed at-most-once or may be retried. Those choices become part of the language and runtime contract.

Rust as a Model of Ownership, Types, and Machine Boundaries

The notes in this section draw on [Idiomatic Rust Snippets](#), the [Rust Reference](#), and the Rust patterns catalogue. The goal is not merely “code that compiles,” but APIs whose types explain ownership, state, and failure.

The code demonstrations also distill the models illustrated by [Ferrous Systems’ Rust material](#), [100 Exercises to Learn Rust](#), [Rustfinity](#), [Kobzol’s university Rust course](#), [Comprehensive Rust](#), [Rust for C Programmers](#), [Effective Rust](#), and [High Assurance Rust](#). These sources are used for their **code-level explanations of ownership, state machines, I/O, concurrency, unsafe boundaries, and assurance**—not as an exercise checklist.

1. Ownership as a Runtime Design Lesson

Rust’s three basic ownership rules are a useful model for any language:

1. each value has an owner;
2. moving a non-`Copy` value transfers ownership;
3. the value is dropped when its owner leaves scope.

```
fn token_count(source: &str) -> usize {
    source.split_whitespace().count()
}

fn keep_source(source: String) -> Module {
    Module { source }
}

struct Module {
    source: String,
}
```

Borrow when a function only needs temporary access. Take ownership when it must store, consume, transfer, or transform the value.

For parser APIs:

- use `&str` instead of `&String`;
- use `&[Token]` instead of `&Vec<Token>`;
- use `String` or `Vec<T>` when the result owns its data;
- use `Cow<'a, str>` when a transformation may return either borrowed or owned text.

```
use std::borrow::Cow;

fn unescape_identifier(input: &str) -> Cow<'_, str> {
    if input.contains("\\\\-") {
        Cow::Owned(input.replace("\\\\-", "-"))
    } else {
        Cow::Borrowed(input)
    }
}
```

2. Make Illegal States Unrepresentable

Do not represent every compiler stage with one giant struct full of optional fields.

```

struct ParsedModule {
    ast: Ast,
}

struct TypedModule {
    ast: TypedAst,
    symbols: SymbolTable,
}

struct LoweredModule {
    ir: Ir,
}

fn typecheck(parsed: ParsedModule) -> Result<TypedModule, TypeErrors> {
    // Every successful TypedModule is guaranteed to have types and symbols.
    todo!()
}

fn lower(typed: TypedModule) -> LoweredModule {
    todo!()
}

```

The type-state version makes it impossible to call `lower` on an untyped AST without first going through `typecheck`.

Use enums for closed sets of states or instructions:

```

enum Value {
    Int(i64),
    Bool(bool),
    String(String),
    Function(FunctionId),
    Nil,
}

fn truthy(value: &Value) -> bool {
    match value {
        Value::Bool(v) => *v,
        Value::Nil => false,
        Value::Int(_) | Value::String(_) | Value::Function(_) => true,
    }
}

```

Avoid a wildcard arm when adding a new variant should force every semantic decision to be revisited.

3. Newtypes for Offsets, IDs, and Units

Raw `usize` values are easy to mix up:

```
#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct ByteOffset(usize);

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct Register(u16);

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct FunctionId(u32);

fn report_error(at: ByteOffset, message: &str) {
    eprintln!("byte {}: {message}", at.0);
}
```

This prevents accidentally passing a register number where a file offset is required. The wrapper normally has no runtime cost.

Values that look alike	Why they need separate types
Byte offset / virtual address	Different coordinate systems
Source line / source byte	Different location units
Token ID / AST node ID	Different entity domains
VM register / local slot	Different storage namespaces
Host-order / network-order value	Different byte encodings
Compressed / uncompressed size	Different allocation and validation limits

4. Errors Are Part of the Language Tool's Interface

Use `Option<T>` for expected absence and `Result<T, E>` for failure with a reason. Reserve panics for broken internal invariants.

```
#[derive(Debug)]
enum DecodeError {
    UnexpectedEof { at: ByteOffset },
    UnknownOpcode { at: ByteOffset, opcode: u8 },
    InvalidJump { from: ByteOffset, to: ByteOffset },
}

fn opcode_name(byte: u8) -> Option<&'static str> {
    match byte {
        0x01 => Some("push"),
        0x02 => Some("add"),
        0xff => Some("halt"),
        _ => None,
    }
}

fn require_opcode(byte: u8, at: ByteOffset) -> Result<&'static str, DecodeError> {
    opcode_name(byte).ok_or(DecodeError::UnknownOpcode { at, opcode: byte })
}
```

Diagnostic field

Question it answers

Problem	What went wrong?
Location	Where did it happen?
Expectation	What was valid or expected here?
Observation	What did the compiler actually find?
Recovery	What can the user do next?

5. Iterators Express Token and Byte Pipelines

Iterators are lazy and make transformations explicit:

```
fn decimal_literals(tokens: &[Token]) -> impl Iterator<Item = i64> + ' _ {
    tokens.iter().filter_map(|token| match token {
        Token::Integer(text) => text.parse().ok(),
        _ => None,
    })
}

enum Token {
    Integer(String),
    Identifier(String),
    Plus,
}
```

Prefer iterator adapters when they clarify the pipeline. Use a normal loop when control flow, error recovery, or mutable parser state is easier to read that way. “More functional” is not automatically “more idiomatic.”

6. RAI for Files, Sockets, Locks, and Temporary State

RAII means a resource is released when its guard or owner is dropped. Rust’s `File`, `TcpStream`, mutex guards, and collections already follow it.

A compiler can use a guard to restore temporary state even when a function returns early:

```
use std::cell::Cell;

struct DepthGuard<'a>(&'a Cell<usize>);

impl Drop for DepthGuard<'_> {
    fn drop(&mut self) {
        self.0.set(self.0.get() - 1);
    }
}

fn enter_depth(depth: &Cell<usize>, limit: usize) -> Result<DepthGuard<'_>, &'static str> {
    if depth.get() >= limit {
        return Err("nesting limit exceeded");
    }
    depth.set(depth.get() + 1);
    Ok(DepthGuard(depth))
}
```

RAII is also a language-design option: deterministic destructors are convenient, but their exact timing affects semantics, optimization, and cycles.

7. Keep `unsafe` Small and Give It a Contract

`unsafe` does not disable the borrow checker; it permits a small set of operations whose safety the compiler cannot prove. Every `unsafe` block should have a written invariant.

```
fn read_unaligned_u32_le(bytes: &[u8]) -> Option<u32> {  
    // A safe implementation is preferable here.  
    let array: [u8; 4] = bytes.get(..4)?.try_into().ok()?;  
    Some(u32::from_le_bytes(array))  
}
```

Only reach for raw pointers or `read_unaligned` when a safe byte-slice solution cannot meet the requirement. `#[repr(packed)]` is not a shortcut for parsing arbitrary bytes into a struct.

Unsafe concern	Review question
Ownership	Who allocated this memory and who releases it?
Lifetime	How long is the allocation valid?
Initialization	Is every byte valid for the interpreted type?
Alignment	Does the address meet the type's requirement?
Aliasing	Are simultaneous references permitted?
Concurrency	Can another thread or interrupt mutate it?
Bounds	How are the pointer and length proven to be in range?

Rust as a Language for Stating and Checking Invariants

[Rust for C Programmers](#), [Comprehensive Rust](#), [Effective Rust](#), and [High Assurance Rust](#) are most useful when read as four views of the same model:

View	Question
C comparison	Which manual machine-level obligation moved into the type system?
Comprehensive Rust	How do ownership, traits, concurrency, bare metal, and FFI compose?
Effective Rust	How should an API expose those guarantees idiomatically?
High Assurance Rust	What evidence is still needed after the compiler accepts the code?

Ownership Is a Static Resource Protocol

In C, a pointer says where a value might be, but usually not who must free it, whether another pointer aliases it, or how long it remains valid. Rust separates those claims:

Rust form	Permission and obligation

<code>T</code>	Own the value and eventually run its destructor
<code>&T</code>	Temporarily read through a shared borrow
<code>&mut T</code>	Temporarily read/write through an exclusive borrow
<code>Box<T></code>	Own a heap allocation with one responsible owner
<code>Rc<T></code> / <code>Arc<T></code>	Share ownership through reference counts
<code>Cell<T></code> / <code>RefCell<T></code>	Move selected borrow checks to controlled runtime operations
<code>*const T</code> / <code>*mut T</code>	Raw address; validity must be proved outside safe Rust

The important shift is:

C-style question: "Is this pointer probably still valid here?"
Rust-style question: "Which capability does this type grant, and for what lifetime?"

This same model applies beyond memory:

Resource	Owner	Borrow or capability	Release
File	<code>File</code> value	<code>&File</code> Or <code>&mut File</code>	<code>Drop</code> closes handle
Lock	Mutex	Guard value	Guard drop unlocks
Transaction	Transaction object	Scoped operations	Commit or rollback
VM root	Root handle	Borrow into managed heap	Guard drop unregisters
GPU buffer	Device-associated handle	Encoded command use	API/device lifetime policy

RAII means cleanup follows lexical ownership. It does **not** prove that an externally visible operation succeeded: flushing a file, committing a database transaction, or sending a final network frame often needs an explicit fallible method.

Borrowing Is an Aliasing Rule, Not Merely a Syntax Rule

The simplified rule is:

```
many readers
OR
one writer
```

This prevents two classes of problems at once:

1. a reference cannot outlive the value it reaches;
2. mutation cannot race with other active access through safe references.

```
fn split_header_payload(packet: &mut [u8], header_len: usize)
    -> Result<&mut [u8], &mut [u8]>, &'static str>
{
    if header_len > packet.len() {
        return Err("header exceeds packet");
    }

    // `split_at_mut` proves the returned slices do not overlap.
    // Safe callers may mutate both because each owns an exclusive region.
    Ok(packet.split_at_mut(header_len))
}
```

The compiler is not reasoning about the word “header.” It reasons about non-overlapping regions and lifetimes. The domain name is our job; the aliasing proof is encoded by the API.



Low-level connection: optimizer transformations assume that certain references do not alias. Rust’s borrow rules are therefore simultaneously a safety discipline and a source of optimization facts.

Lifetimes Describe Relationships, Not Storage Duration Commands

A lifetime annotation does not keep data alive. It states a relationship that callers must satisfy:

```
fn payload_after<'packet>(
    packet: &'packet [u8],
    header_len: usize,
) -> Result<&'packet [u8], &'static str> {
    // The returned slice is valid for no longer than the borrowed packet.
    packet
        .get(header_len..)
        .ok_or("header exceeds packet")
}
```

Read the signature as:

“If the caller lends a packet for lifetime `'packet`, a successful result is a view whose validity is bounded by that same loan.”

This is relational logic embedded in a type signature. It matters when building parsers, AST views, packet decoders, database rows, or zero-copy file readers.

Algebraic Data Types Define the Valid State Space

Effective Rust emphasizes using types to express data structures. An enum is a tagged sum: the tag selects one variant, and only that variant’s fields exist.


```
#[derive(Debug)]
enum Connection {
    Disconnected,
    Resolving { host: String },
    Connected {
        peer: std::net::SocketAddr,
        // Sequence numbers are meaningful only after a connection exists.
        next_sequence: u64,
    },
    Closing { reason: String },
}
```

A design with unrelated booleans permits nonsense:

```
connected = false
closing = true
peer = present
next_sequence = 900
```

The enum's variants instead define a finite state space, and `match` forces each transition to account for every currently valid state.

Type-system tool	State-space effect
<code>enum</code>	Expresses alternatives
<code>struct</code>	Expresses fields that exist together
<code>Option<T></code>	Makes absence explicit
<code>Result<T, E></code>	Separates success from expected failure
Newtype	Prevents mixing equal representations with different meanings
Private fields	Restricts which code can construct or mutate state
Typestate	Makes protocol phase part of the static type

This is compiler design in miniature. Tokens, AST nodes, typed expressions, bytecode instructions, and VM states all benefit from a representation that contains only valid combinations.

Safe and Unsafe Form a Proof Boundary

Comprehensive Rust describes unsafe Rust as a small set of extra capabilities, including raw-pointer dereference, unsafe calls/traits, mutable statics, and union-field access. `unsafe` does not mean “ignore correctness.” It means the compiler cannot verify all preconditions.

Every unsafe abstraction needs two contracts:

Contract	Audience	Content
Caller contract	Users of an <code>unsafe fn</code>	Preconditions they must establish
Implementation invariant	Maintainers of a safe wrapper	Facts upheld for every safe call

```

/// Reads a little-endian `u32` after a checked bounds test.
fn read_u32_le(bytes: &[u8], offset: usize) -> Result<u32, &'static str> {
    let end = offset.checked_add(4).ok_or("offset overflow"?);
    let field = bytes.get(offset..end).ok_or("truncated field"?);

    // No raw pointer is needed: the safe conversion states the size requirement.
    let array: [u8; 4] = field.try_into().map_err(|_| "wrong field size"?);
    Ok(u32::from_le_bytes(array))
}

```

The safest unsafe block is the one removed by a better safe primitive. If raw access is actually necessary, document:

```

SAFETY:
- why the address is non-null and correctly aligned;
- why the full byte range is initialized and in bounds;
- why the pointee has a valid representation;
- why aliasing rules are satisfied;
- why the value remains alive for the reference's lifetime;
- which thread or interrupt context may access it.

```

FFI Joins Two Different Proof Systems

An `extern "C"` signature agrees on a calling convention. It does not automatically agree on ownership, unwinding, text encoding, struct layout, or thread safety.

```

#[repr(C)]
pub struct ByteView {
    data: *const u8,
    len: usize,
}

/// # Safety
///
/// If `len` is nonzero, `data` must point to `len` readable bytes for the duration of
/// this call. The range must not be concurrently mutated.
pub unsafe extern "C" fn checksum(view: ByteView) -> u32 {
    let bytes = if view.len == 0 {
        &[]
    } else {
        // SAFETY: the foreign caller is required to uphold the documented contract.
        unsafe { std::slice::from_raw_parts(view.data, view.len) }
    };

    bytes
        .iter()
        .fold(0_u32, |sum, byte| sum.wrapping_add(u32::from(*byte)))
}

```

In a production boundary, also decide how errors cross the ABI, prevent unwinding into foreign code, define allocation/free pairs, version structures, and fuzz all decoding paths. Chromium's Rust integration is a useful large-system example: Rust code still enters through build, dependency-review, generated-binding, and C++ interoperability policies rather than existing as an isolated safe island.

High Assurance Means Accumulating Independent Evidence

High Assurance Rust makes an essential correction: memory safety eliminates important bug classes, but it does not prove authentication policy, protocol correctness, availability, confidentiality, or absence of malicious logic.

Evidence source	Finds or prevents	Cannot establish alone
Rust type checker	Many lifetime, aliasing, and data-race violations	Business or security policy
Unit tests	Known examples and local edge cases	All possible values and schedules
Property tests	Broad input classes and algebraic properties	Complete state-space coverage
Fuzzing	Crashes and bad states reachable by generated inputs	Absence of bugs
Differential tests	Disagreement with a reference implementation	Correctness if both share a flaw
Static analysis / lints	Suspicious patterns	Runtime environment behavior
Deductive verification	A stated property under a formal model	That the model and property are complete
Code review	Design errors, intent, maintainability, threat reasoning	Exhaustive execution
Reproducible builds and dependency review	Supply-chain confidence	Runtime correctness

Assurance is an argument:

```
claim
+ explicit assumptions
+ invariants
+ diverse evidence
+ known limitations
= justified confidence, not absolute certainty
```

Threat Modeling Starts at Trust Boundaries

For each boundary, write:

Question	Example
What enters?	Packet, file, AST plugin, database row, FFI pointer
Who controls it?	Local user, remote peer, dependency, device
What must remain true?	Memory bounds, authorization, ordering, availability
Which resources can it consume?	CPU, memory, recursion, file handles, logs

Which failure is acceptable?	Structured error, dropped request, process restart
Which evidence checks the claim?	Parser tests, fuzzing, audit, metrics

This is directly applicable to a language runtime:

Runtime boundary	Assurance focus
Source parser	Bounded input, no panics, useful locations
Bytecode verifier	Valid opcodes, jumps, stack effects, types
Native FFI	Layout, lifetime, ownership, unwinding
Garbage collector	Roots, reachability, barriers, no dangling references
Module loader	Path policy, signatures, dependency identity
JIT	Writable-versus-executable memory, code validation
Debugger/profiler	Permission and privacy boundaries

A Reusable Assurance Workflow

1. **State the claim.** Example: "Every accepted bytecode function has balanced stack effects on all control-flow paths."
2. **Define the representation.** Specify instructions, operands, control-flow graph, and stack-height domain.
3. **Write the invariant.** Each block has one incoming height; every instruction's required operands exist.
4. **Enforce cheap facts statically.** Use enums, newtypes, private fields, and checked constructors.
5. **Validate external data dynamically.** Never deserialize bytes directly into a trusted internal state.
6. **Isolate unsafe or foreign operations.** Give the boundary a small API and explicit safety comment.
7. **Test from several directions.** Examples, properties, fuzzing, malformed corpora, differential execution.
8. **Measure failures in deployment.** Logs and metrics should reveal violated assumptions without exposing secrets.
9. **Record limitations.** The next reviewer must know what remains unproved.

✅ **Internalization prompt:** for every Rust library you read, identify its representation, valid states, ownership graph, transitions, failure values, unsafe boundary, complexity costs, and testing evidence. That turns library documentation into a computer-science lesson.

8. Rust Anti-Patterns to Avoid

Anti-pattern	Better direction
Clone to silence a borrow error	Model ownership and borrowing deliberately
Accept <code>&String</code> or <code>&Vec<T></code>	Accept <code>&str</code> or <code>&[T]</code> when ownership is unnecessary
Use <code>usize</code> for every numeric domain	Introduce newtypes for IDs, offsets, sizes, and slots
<code>unwrap()</code> input-dependent results	Propagate a contextual <code>Result</code>
Index before checking length	Use checked slicing or <code>get</code>

Return only <code>"failed"</code>	Include error kind and byte/source offset
Hold a lock across blocking/ <code>.await</code>	Shorten the guard's scope
Make every AST field <code>pub</code>	Expose a small, stable API
Cast bytes directly to a struct	Parse layout, alignment, and endianness explicitly
Recurse over attacker input	Enforce a depth limit or use an explicit stack

9. Rust and Python: Similar Task, Different Contract

The [Programming Idioms Rust/Python comparison](#) is useful because it holds the **task** constant while changing the language. Do not translate syntax alone: ownership, errors, integers, strings, iteration, and concurrency have different contracts.

Concern	Python tendency	Rust tendency
typing	values checked dynamically	types checked before execution
ownership	garbage-collected object references	ownership, borrowing, and deterministic drop
errors	exceptions	<code>Result<T, E></code> and <code>?</code>
absence	<code>None</code> , often checked dynamically	<code>Option<T></code> forces explicit handling
integers	arbitrary precision by default	fixed-width types; explicit big integers
strings	Unicode string abstraction	UTF-8 <code>String</code> / <code>str</code> ; indexing by byte forbidden
iteration	iterable/generator protocol	zero-cost <code>Iterator</code> adaptors
maps	insertion-ordered <code>dict</code>	choose <code>HashMap</code> or ordered <code>BTreeMap</code>
mutation	object mutability is runtime behavior	binding/reference mutability is explicit

Iteration with Index and Value

```
for (index, value) in items.iter().enumerate() {
    // `value` is borrowed; the collection remains usable after this loop.
    println!("{index}: {value}");
}
```

`iter()` borrows values, `iter_mut()` allows mutation, and `into_iter()` consumes the collection. That choice is part of the API, not incidental syntax.

Parse, Filter, and Collect Errors

Rust makes the "stop at the first invalid conversion" contract explicit:

```
fn parse_nonempty(fields: &[String]) -> Result<Vec<i64>, std::num::ParseIntError> {
    fields
        .iter()
        // Empty fields are intentionally ignored by this particular contract.
        .filter(|text| !text.trim().is_empty())
        // Each parse produces `Result<i64, ParseIntError>`.
        .map(|text| text.parse::<i64>())
        // Collecting results stops and returns the first error.
        .collect()
}
```

Because `Result` implements `FromIterator`, collecting stops on the first error. If all errors matter—such as compiler diagnostics—collect successes and diagnostics into separate typed outputs instead.

Unicode Requires a Unit

```
fn first_five_scalars(text: &str) -> String {
    // This counts Unicode scalar values, not bytes or user-perceived graphemes.
    text.chars().take(5).collect()
}
```

Unit	Rust view/API	Suitable for
byte	<code>as_bytes()</code> , byte ranges	protocols, files, UTF-8 validation
Unicode scalar	<code>chars()</code>	code points and many language rules
grapheme cluster	Unicode-segmentation library	user-perceived characters

Five bytes, five Unicode scalar values, and five visible characters are not equivalent.

Maps Encode Ordering Expectations

```
use std::collections::{BTreeMap, HashMap};

let fast_lookup: HashMap<&str, i32> = [("parse", 1), ("typecheck", 2)]
    .into_iter()
    .collect();

let deterministic_output: BTreeMap<&str, i32> =
    fast_lookup.into_iter().collect();
```

Use a deterministic map or sort keys before emitting compiler artifacts, diagnostics, snapshots, or protocol output that must be reproducible.



Translation habit: ask “what behavior does this idiom promise?” before asking “what Rust syntax looks like the Python syntax?”

Program Design in Rust: State, Errors, and Invariants



Library lens: Rust patterns are treated as executable invariants: builders represent staged construction, enums represent closed state spaces, RAII represents lifetime-coupled cleanup, and traits represent substitutable capabilities.

[Rust Design Patterns](#) separates reusable Rust knowledge into **idioms**, **design patterns**, and **anti-patterns**. The [Rust Cookbook](#) complements that vocabulary with small recipes for common tasks.



Selection rule: use a pattern because its trade-off matches the problem, not because the pattern has a familiar name.

1. Idiom, Pattern, or Recipe?

Kind	Purpose	Example
Idiom	Conventional Rust expression of a common idea	<code>?</code> , iterators, <code>Default</code>
Design pattern	Reusable structure for a recurring design problem	Builder, Command, RAII guard
Anti-pattern	Tempting approach whose costs usually exceed benefits	Clone-to-fix-borrowing
Recipe	Focused example that accomplishes one practical task	Stream a file line by line

Patterns are not automatically abstractions worth keeping. A direct struct constructor is clearer than a builder with no optional choices; an enum can be clearer than a trait-object hierarchy with only two closed variants.

2. Pattern Selection Table

Problem shape	Rust-oriented pattern
Several values share one primitive representation	Newtype
Construction has many optional settings	Builder
Cleanup must happen on every exit path	RAII guard
Actions must be queued, logged, or undone	Command
Behavior varies behind one stable interface	Trait/generic Strategy
Legal operations depend on current state	Typestate or an exhaustive enum
A closed set of variants has different data	Enum + <code>match</code>
Heterogeneous extensions are loaded dynamically	Trait objects

3. Newtypes Encode Meaning

Newtypes prevent values with the same representation from being mixed:

```
#[derive(Clone, Copy, Debug, Eq, PartialEq)]
struct ByteOffset(usize);

#[derive(Clone, Copy, Debug, Eq, PartialEq)]
struct VirtualAddress(u64);

#[derive(Clone, Copy, Debug, Eq, PartialEq)]
struct RequestId(u64);

fn seek_to(offset: ByteOffset) {
    println!("seek to byte {}", offset.0);
}
```

Newtype benefit	Result
Domain separation	Offsets, addresses, counts, and IDs cannot be swapped
Central validation	Constructors can enforce range or formatting rules
API clarity	Signatures explain what a number means
Future evolution	Representation can change behind the public type

Implement only operations that make semantic sense. Adding two byte lengths may be valid; adding two virtual addresses usually is not.

4. Builder for Complicated Construction

Rust has no overloaded functions or default arguments. A builder is useful when a configuration has required fields plus several optional policies:


```

#[derive(Debug)]
struct CompilerOptions {
    target: String,
    optimize: bool,
    warnings_as_errors: bool,
    max_errors: usize,
}

#[derive(Debug)]
struct CompilerOptionsBuilder {
    target: String,
    optimize: bool,
    warnings_as_errors: bool,
    max_errors: usize,
}

impl CompilerOptions {
    fn builder(target: impl Into<String>) -> CompilerOptionsBuilder {
        CompilerOptionsBuilder {
            target: target.into(),
            optimize: false,
            warnings_as_errors: false,
            max_errors: 20,
        }
    }
}

impl CompilerOptionsBuilder {
    fn optimize(mut self, enabled: bool) -> Self {
        self.optimize = enabled;
        self
    }

    fn warnings_as_errors(mut self, enabled: bool) -> Self {
        self.warnings_as_errors = enabled;
        self
    }

    fn max_errors(mut self, limit: usize) -> Self {
        self.max_errors = limit.max(1);
        self
    }

    fn build(self) -> CompilerOptions {
        CompilerOptions {
            target: self.target,
            optimize: self.optimize,
            warnings_as_errors: self.warnings_as_errors,
            max_errors: self.max_errors,
        }
    }
}

```

```

let options = CompilerOptions::builder("wasm32-unknown-unknown")
    .optimize(true)
    .warnings_as_errors(true)
    .max_errors(50)
    .build();

```

Use a builder when...	Prefer a constructor when...
Many settings are optional	Every argument is required
Defaults should remain centralized	There are only a few obvious fields
Construction needs validation or staged preparation	A struct literal is already clear
Adding an option should not break callers	The type is private and construction local

5. RAII Guards Make Cleanup Structural

An RAII guard acquires or changes state during construction and restores/finalizes it in `Drop`. Cleanup then occurs on normal return, early `?`, and panic unwinding.

```
use std::cell::Cell;

struct RecursionGuard<'a> {
    depth: &'a Cell<usize>,
}

impl Drop for RecursionGuard<'_> {
    fn drop(&mut self) {
        self.depth.set(self.depth.get() - 1);
    }
}

fn enter_recursion(
    depth: &Cell<usize>,
    limit: usize,
) -> Result<RecursionGuard<'_>, &'static str> {
    if depth.get() >= limit {
        return Err("recursion limit exceeded");
    }

    depth.set(depth.get() + 1);
    Ok(RecursionGuard { depth })
}
```

The guard should mediate access to the resource when using the resource after finalization would be invalid. Common examples are mutex guards, temporary directory owners, transaction guards, scoped tracing spans, and VM root guards.

⚠ Drop is not a general error channel: destructors cannot conveniently return cleanup errors. Provide an explicit `finish`, `commit`, or `flush` operation when the caller must observe failure.

6. Command Pattern for Compiler Passes

The Command pattern turns an action into a value that can be stored and invoked later. Trait objects suit an open, heterogeneous pass pipeline:

```

#[derive(Debug)]
struct Module {
    instructions: Vec<String>,
}

trait Pass {
    fn name(&self) -> &'static str;
    fn run(&self, module: &mut Module) -> Result<(), String>;
}

struct PassPipeline {
    passes: Vec<Box<dyn Pass>>,
}

impl PassPipeline {
    fn run(&self, module: &mut Module) -> Result<(), String> {
        for pass in &self.passes {
            pass.run(module)
                .map_err(|error| format!("{}", pass.name(), error));
        }
        Ok(())
    }
}

```

Representation	Best fit
Enum of commands	Closed command set; exhaustive dispatch
<code>fn</code> pointers/closures	Small stateless actions
<code>Box<dyn Command></code>	Open plugin-like set with per-command state
Generic command type	One command type on a performance-critical path

Undo requires more than an `undo()` method name. The command must retain enough prior state, and failure halfway through a multi-command transaction needs an explicit rollback policy.

7. Strategy: Static or Dynamic Dispatch

```

trait DiagnosticSink {
    fn emit(&mut self, message: &str);
}

fn type_check<S: DiagnosticSink>(source: &str, sink: &mut S) {
    if source.is_empty() {
        sink.emit("source is empty");
    }
}

```

Dispatch form	Benefit	Cost/constraint
Generic <code>S: Trait</code>	Inlining and static type knowledge	One instantiation per concrete type
<code>&mut dyn Trait</code>	Runtime-pluggable heterogeneous values	Indirect call and object-safety limits

Enum + <code>match</code>	Exhaustive, compact closed set	Adding a variant changes central code
---------------------------	--------------------------------	---------------------------------------

Prefer generics at reusable performance-sensitive boundaries and trait objects where runtime extensibility is the actual requirement.

8. Cookbook Mindset

The Rust Cookbook covers practical categories such as algorithms, command-line tools, encoding, filesystems, concurrency, networking, and asynchronous work.

Recipe category	Transferable lesson
Command line	Preserve non-UTF-8 paths with <code>args_os</code>
File I/O	Stream large input and buffer many small writes
Encoding	Make byte order and text format explicit
Concurrency	Select ownership transfer before shared mutation
Networking	Bound reads, writes, timeouts, and decoded sizes
Compression	Treat expanded size and archive paths as untrusted
Async	Bound task counts, channels, and wait time

Cookbook snippets are starting points. Before adopting a recipe, check current crate documentation, pin an intentional dependency range, test errors, and add limits suitable for the input's trust level.



Rust Demonstration: A Streaming Hash Is a Finite-State Fold

The Rust Cookbook's file and cryptography examples become a computer-science model when the library call is unpacked. A streaming digest does not keep the whole file. It keeps a fixed-size algorithm state and folds each input block into it:

```
initial state
+ chunk 1 → state 1
+ chunk 2 → state 2
+ ...
+ final length/padding → fixed-size digest
```

```

use sha2::{Digest, Sha256};
use std::fs::File;
use std::io::{self, Read};
use std::path::Path;

fn sha256_file(path: &Path) -> io::Result<[u8; 32]> {
    let mut file = File::open(path)?;
    let mut hasher = Sha256::new();
    let mut buffer = [0_u8; 16 * 1024];

    loop {
        // `read` may return any positive prefix length, not necessarily the full
        // buffer. The count is therefore part of the state transition.
        let count = file.read(&mut buffer)?;
        if count == 0 {
            // For files, zero means end-of-file: no more transitions remain.
            break;
        }

        // Hash only initialized bytes actually returned by this read.
        // The unused suffix still contains old or zero data and is not file content.
        hasher.update(&buffer[..count]);
    }

    let digest = hasher.finalize();
    Ok(digest.into())
}

```

Concept	What the example demonstrates
Incremental algorithm	Each chunk advances a compact internal state
Bounded memory	Space is $O(1)$ with respect to file size
Partial I/O	The returned byte count controls which bytes are valid input
Finalization	Padding and total length complete the digest definition
Determinism	Equal byte sequences under the same algorithm produce equal digests
Domain boundary	A file digest detects change; it is not password storage or a signature

SHA-256 is fast by design, which is useful for content integrity and addressing but bad for password verification: an offline attacker could test guesses quickly. Argon2 adds salt and deliberately expensive memory-hard work for that different problem.

The same fold structure appears in:

System	State folded over input
Parser	automaton/stack over tokens
Compression	dictionary/window over bytes
Network checksum	checksum state over packet chunks
Database aggregation	accumulator over rows

Compiler dataflow	facts repeatedly merged across graph edges
Audio filter	delay/filter state over sample blocks

The library therefore illustrates a general model—**bounded state plus a transition function over a stream**—rather than merely a recipe for hashing one file.

9. Stream Large Text Instead of Loading It Whole

```
use std::fs::File;
use std::io::{self, BufRead, BufReader};
use std::path::Path;

fn count_nonempty_lines(path: &Path) -> io::Result<usize> {
    let file = File::open(path)?;
    let reader = BufReader::new(file);
    let mut count = 0;

    for line in reader.lines() {
        if !line?.trim().is_empty() {
            count += 1;
        }
    }

    Ok(count)
}
```

Input shape	Preferred approach
Small trusted UTF-8 configuration	<code>fs::read_to_string</code>
Large line-oriented text	<code>BufReader::lines</code>
Arbitrary binary input	<code>Read</code> into a bounded byte buffer
Many small output writes	<code>BufWriter</code> + explicit <code>flush</code>
In-memory test fixture	<code>Cursor<Vec<u8>></code>

10. Spawn Programs Without Re-parsing a Shell String

```

use std::io;
use std::process::{Command, Output};

fn rustc_version() -> io::Result<Output> {
    Command::new("rustc")
        .arg("--version")
        .output()
}

fn checked_output() -> Result<String, String> {
    let output = rustc_version().map_err(|error| error.to_string())?;
    if !output.status.success() {
        return Err(format!("rustc exited with {}", output.status));
    }

    String::from_utf8(output.stdout).map_err(|error| error.to_string())
}

```

Passing arguments separately avoids shell quoting and injection problems. Use a shell only when shell semantics—pipes, expansion, or redirection—are deliberately required, and never concatenate untrusted text into a shell program.

11. Recipe Adoption Checklist

Review area	Question
Ownership	Can inputs be borrowed rather than cloned?
Errors	Does every failure preserve useful context?
Limits	Are memory, recursion, file, task, and output sizes bounded?
Portability	Do path, process, newline, and filesystem assumptions hold?
Dependencies	Is the crate current, maintained, and necessary?
Security	Which bytes, paths, URLs, and arguments are untrusted?
Testing	Are success, boundary, partial-failure, and cleanup tested?

λ Functional Models: Composition, Effects, and Rust

[fp-core.rs](#) presents functional-programming vocabulary and purely functional data structures for Rust. Most of the underlying ideas also appear directly in standard Rust through closures, iterators, algebraic data types, `Option`, `Result`, and immutable-by-default bindings.

λ **Mental model:** functional programming makes transformation and composition primary. Rust adds explicit ownership, lifetimes, and controlled mutation to that model.

1. Pure Core, Imperative Shell

A **pure** function depends only on its arguments and does not mutate external state. The same input therefore produces the same output.

```
#[derive(Clone, Copy)]
struct Token {
    kind: TokenKind,
    width: usize,
}

fn total_width(tokens: &[Token]) -> usize {
    tokens.iter().map(|token| token.width).sum()
}
```

Pure core	Imperative shell
AST transformations	reading files
type rules	printing diagnostics
byte decoding from a supplied slice	receiving network packets
optimization over an explicit IR	timers, randomness, database operations

Push I/O and nondeterminism to narrow adapters. A pure compiler core is easier to test, cache, parallelize, and compare with another implementation.

2. Higher-Order Functions and Closures

A **higher-order function** accepts or returns another function. A closure can capture state from its environment.

```
fn retain_matching<T>(
    values: Vec<T>,
    mut predicate: impl FnMut(&T) -> bool,
) -> Vec<T> {
    values
        .into_iter()
        .filter(|value| predicate(value))
        .collect()
}

fn at_least(minimum: i64) -> impl Fn(&i64) -> bool {
    move |value| *value >= minimum
}
```

The returned closure **partially applies** `minimum`: it turns a two-input idea into a reusable one-input predicate.

3. Composition Is Typed Pipelining


```
fn compose<A, B, C>(
    first: impl Fn(A) -> B,
    second: impl Fn(B) -> C,
) -> impl Fn(A) -> C {
    move |input| second(first(input))
}

let trim = |text: String| text.trim().to_owned();
let length = |text: String| text.len();
let trimmed_length = compose(trim, length);

assert_eq!(trimmed_length("  AST  ".to_owned()), 3);
```

Pipeline stage	Type
trim	String -> String
length	String -> usize
composition	String -> usize

Compiler passes compose similarly, but fallibility and diagnostics often make the real type `Input -> Result<Output, Error>`.

4. Algebraic Data Types Carry Meaning

Rust enums are **sum types**: a value is one variant. Structs and tuples are **product types**: a value contains all fields.

```
enum Expr {
    Integer(i64),
    Name(SymbolId),
    Call {
        callee: Box<Expr>,
        arguments: Vec<Expr>,
    },
}
```

```
Expr = Integer(i64)
      + Name(SymbolId)
      + Call(Expr × List<Expr>)
```

This algebra explains exhaustiveness, representation choices, serialization tags, and why an enum often models runtime or protocol state better than several booleans.

5. map, and_then, and Context

Operation	Shape	Meaning
map	$F<A> + (A \rightarrow B) \rightarrow F$	transform a value in context
and_then	$F<A> + (A \rightarrow F) \rightarrow F$	chain a context-producing step
unwrap_or	$F<A> + \text{fallback} \rightarrow A$	leave the context deliberately

```
fn identifier_length(source: &str) -> Result<usize, FrontendError> {
    lex_one_identifier(source)
        .map(|token| token.text.len())
}

fn resolve_identifier(
    source: &str,
    symbols: &SymbolTable,
) -> Result<SymbolId, FrontendError> {
    lex_one_identifier(source)
        .and_then(|token| symbols.resolve(token.text).map_err(FrontendError::from))
}
```

`Option` carries possible absence; `Result` carries possible failure. Their combinators preserve that context without nested control flow.

6. Monoids Describe Safe Reduction

A monoid has an associative combine operation and an identity element.

Type/operation	Identity	Compiler use
integers under <code>+</code>	<code>0</code>	instruction counts
strings under append	empty	generated text
sets under union	empty set	data-flow facts
lists under concat	empty list	collected diagnostics

Associativity permits chunked or parallel reduction, but floating-point addition is not mathematically associative at machine precision. Deterministic builds must define reduction order where rounding matters.

7. Referential Transparency Enables Reasoning

If an expression can be replaced by its value without changing behavior, it is referentially transparent. That supports:

Technique	Required caution
memoization	inputs must capture every dependency
common subexpression elimination	operation must be safe to reuse
lazy evaluation	effects and resource lifetime change timing
parallel evaluation	operations must not depend on hidden ordering
property-based testing	generators must respect input invariants

Functional terminology is useful, but do not force abstractions where ownership becomes opaque. Prefer ordinary iterators and explicit enums until a more general abstraction demonstrably improves the code.

➡ **Next:** algorithms apply these transformation, reduction, and invariant ideas to concrete problems with measurable resource costs.

🧩 Algorithms and Data Structures Through Rust

📖 **Library lens:** Rust collections, slices, iterators, ownership, and explicit errors make algorithm state and cost visible. The code illustrates invariants and transitions rather than syntax tricks.

This section expands the algorithm categories from [Idiomatic Rust Snippets](#): sorting, searching, graph traversal, and dynamic programming. The emphasis here is on interfaces and invariants useful in compilers, interpreters, and low-level tools.

🧠 **Algorithm-selection rule:** choose from the **contract and constraints**, not from whichever algorithm name is easiest to remember.

1. Start with the Contract

Before choosing an algorithm, define its contract:

Contract dimension	Decision to make
Input	Shape, ownership, and validity
Output	Required result and failure representation
Comparison	Ordering and equality rules
Mutation	Whether the input may be changed
Scale	Expected data size and worst-case limit
Stability	Whether equal items must preserve relative order
Invalid input	Reject, ignore, repair, or return a partial result

Prefer slices for algorithms that do not need to own or resize data:

```
fn is_sorted<T: Ord>(items: &[T]) -> bool {
    items.windows(2).all(|pair| pair[0] <= pair[1])
}

fn reverse_in_place<T>(items: &mut [T]) {
    items.reverse();
}
```

`&[T]` accepts arrays, vectors, boxed slices, and subslices without allocation.

2. Complexity Is a Growth Model

Complexity	Typical example	Interpretation
$O(1)$	Index an array	Work does not grow with input length

$O(\log n)$	Binary search	Repeatedly halves the search space
$O(n)$	Scan for a token	Visits input once
$O(n \log n)$	Comparison sort	Common efficient general sorting bound
$O(n^2)$	Compare every pair	Often acceptable only for small inputs
$O(2^n)$	Enumerate all subsets	Becomes infeasible quickly

Performance dimension	Why it matters
Space complexity	Bounds extra memory use
Amortized cost	Spreads occasional expensive work over many operations
Worst-case latency	Matters for services, tools, and real-time paths
Constant factors	Separate practical implementations with similar big-O
Locality	Contiguous access can beat pointer-heavy structures

3. Sorting Choices

Algorithm	Average time	Worst time	Extra space	Stable?
Bubble sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Quicksort	$O(n \log n)$	$O(n^2)$	Recursion stack	Usually no
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

In application code, use the standard library unless implementing an algorithm for learning or a specialized constraint:

```
let mut diagnostics = vec![
    ("warning", 20),
    ("error", 5),
    ("note", 12),
];

diagnostics.sort_by_key(|(_, byte_offset)| *byte_offset);
```

Rust slice sorting:

- `sort` and `sort_by_key` preserve the relative order of equal elements;
- `sort_unstable` and `sort_unstable_by_key` may be faster or use less temporary memory when stability is unnecessary.

In-Place Quicksort for Study

```

fn quick_sort<T: Ord>(items: &mut [T]) {
    // Empty and one-element slices are already sorted.
    if items.len() < 2 {
        return;
    }

    // Partition places the pivot in its final sorted position.
    let pivot = partition(items);
    let (left, right_with_pivot) = items.split_at_mut(pivot);

    // Exclude the pivot so neither recursive call sees it again.
    let (_, right) = right_with_pivot.split_first_mut().expect("pivot exists");

    quick_sort(left);
    quick_sort(right);
}

fn partition<T: Ord>(items: &mut [T]) -> usize {
    let last = items.len() - 1;

    // Move the chosen pivot out of the scan range.
    items.swap(items.len() / 2, last);

    // `0..store` contains values known to be smaller than the pivot.
    let mut store = 0;
    for current in 0..last {
        if items[current] < items[last] {
            items.swap(current, store);
            store += 1;
        }
    }

    // Place the pivot between the smaller and greater-or-equal partitions.
    items.swap(store, last);
    store
}

```



Partition invariant: values left of the boundary are smaller than the pivot; unchecked values remain in the middle.

```

[ values < pivot | unchecked values | pivot ]
                ^
                current

```

After partitioning, the pivot is in its final position. The two remaining slices can be sorted independently.

4. Linear and Binary Search

Linear search works on any sequence:

```
fn find_token(tokens: &[Token], wanted: &Token) -> Option<usize> {
    tokens.iter().position(|token| token == wanted)
}

#[derive(PartialEq)]
enum Token {
    Identifier(String),
    Number(i64),
    Plus,
}
```

Binary search requires sorted input:

```
let symbols = ["add", "main", "print", "read"];
assert_eq!(symbols.binary_search(&"print"), Ok(2));
assert_eq!(symbols.binary_search(&"parse"), Err(2));
```

For duplicate values, `binary_search` may return any matching position. Use a boundary search when the first valid insertion point matters:

```
fn lower_bound<T: Ord>(items: &[T], target: &T) -> usize {
    // Search the half-open interval [low, high).
    let mut low = 0;
    let mut high = items.len();

    while low < high {
        // This form avoids overflowing `low + high`.
        let mid = low + (high - low) / 2;
        if &items[mid] < target {
            // `mid` cannot be the first valid insertion position.
            low = mid + 1;
        } else {
            // Keep `mid`: it may be the first value not less than target.
            high = mid;
        }
    }

    low
}

assert_eq!(lower_bound(&[1, 2, 2, 2, 5], &2), 1);
assert_eq!(lower_bound(&[1, 2, 2, 2, 5], &4), 4);
```



Loop invariant: the answer always remains inside the current search interval.

- indexes below `low` contain values less than `target` ;
- indexes at or above `high` are not part of the remaining uncertainty;
- the answer remains in `low..=high` .

5. Two Pointers: Move Boundaries by a Proven Rule

The two-pointer pattern maintains two positions whose movement discards part of the search space. It is useful when the input order makes that movement **monotonic**.

--	--

Shape	Pointer movement
sorted pair search	move left for a small sum, right for a large sum
in-place partition	scan with one pointer, store boundary with the other
palindrome check	compare ends, then move inward
merge two sorted sequences	advance the side containing the next output
remove duplicates in place	read pointer scans; write pointer marks compact output
linked-list fast/slow scan	advance by one and two nodes

```
fn two_sum_sorted(values: &[i64], target: i64) -> Option<(usize, usize)> {
    // The contract requires sorted input. With fewer than two values there is no pair.
    if values.len() < 2 {
        return None;
    }

    let mut left = 0;
    let mut right = values.len() - 1;

    while left < right {
        // Widen before addition so two i64 inputs cannot overflow the comparison.
        let sum = i128::from(values[left]) + i128::from(values[right]);
        let target = i128::from(target);

        match sum.cmp(&target) {
            std::cmp::Ordering::Equal => return Some((left, right)),
            // The slice is sorted, so every pair using this left value is too small.
            std::cmp::Ordering::Less => left += 1,
            // Likewise, every pair using this right value is too large.
            std::cmp::Ordering::Greater => right -= 1,
        }
    }

    None
}
```



Invariant: no discarded index can participate in a valid answer. If you cannot justify that sentence for each pointer movement, the pattern is not correct.

Two pointers do not automatically improve a nested loop. The improvement comes from an ordering property—sorted values, a partition boundary, or a linked-list speed relationship—that prevents revisiting discarded candidates.

6. Sliding Windows: Maintain an Aggregate Over a Contiguous Range

A sliding window is a pair of boundaries plus state describing the half-open range `left..right`. Instead of recomputing every range, update the state when one item enters or leaves.

Window kind	Boundary rule	Common state
fixed-size	add right item; remove item now outside size <code>k</code>	sum, count, rolling hash
variable	expand until invalid; shrink until valid/minimal	sum, frequencies, set

monotonic	remove dominated candidates from a deque	min/max candidates
-----------	--	--------------------

This example finds the shortest contiguous range whose sum reaches a target:

```
fn shortest_sum_window(values: &[u64], target: u64) -> Option<usize> {
    // The empty window already reaches a zero target.
    if target == 0 {
        return Some(0);
    }

    let mut left = 0;
    let mut sum = 0_u64;
    let mut best: Option<usize> = None;

    for right in 0..values.len() {
        // Reject arithmetic overflow rather than silently corrupting the invariant.
        sum = sum.checked_add(values[right])?;

        // Non-negative values make shrinking monotonic: removing an item cannot
        // increase the sum.
        while sum >= target {
            let width = right - left + 1;
            best = Some(best.map_or(width, |old| old.min(width)));

            // Try to make this valid window smaller before advancing `right`.
            sum -= values[left];
            left += 1;
        }
    }

    best
}
```

 **Contract matters:** this variable-window rule depends on non-negative values. Negative inputs destroy the monotonic relationship; use prefix sums with a different data structure or reformulate the problem.

A window is normally contiguous. If the chosen items may be scattered, the problem is more likely a subsequence, subset, graph, or dynamic-programming problem.

7. Linked Lists as an Ownership Laboratory

[Learning Rust With Entirely Too Many Linked Lists](#) uses several list designs to expose how pointer choice determines ownership, mutation, iteration, destruction, aliasing, and unsafe obligations.

Representation	Ownership shape	Main lesson
<code>Option<Box<Node<T>>></code>	one owner per node	moves make safe mutation and reversal natural
<code>Rc<Node<T>></code>	shared, single-threaded	persistent tails can share structure
<code>Arc<Node<T>></code>	shared, thread-safe reference counts	atomic ownership is not automatic data mutation
	shared with runtime-checked	cycles and borrow panics require

<code>Rc<RefCell<Node<T>>></code>	mutation	deliberate care
raw pointers/ <code>NonNull<Node<T>></code>	invariants enforced by the container	unsafe code must prove aliasing and lifetime laws
<code>Vec<T></code> / <code>VecDeque<T></code>	contiguous or ring-buffer allocation	usually faster and simpler for ordinary workloads

The simplest owning link uses `Option` as the empty state instead of a sentinel node:

```
type Link<T> = Option<Box<Node<T>>>;

#[derive(Debug)]
struct Node<T> {
    element: T,
    next: Link<T>,
}

#[derive(Debug, Default)]
struct List<T> {
    head: Link<T>,
}
```

Reverse a Singly Linked List

Reversal is a repeated ownership transfer. At every iteration, three regions are well-defined: the reversed prefix, the current node, and the untouched suffix.

```
fn reverse<T>(mut head: Link<T>) -> Link<T> {
    // `previous` owns the already-reversed prefix.
    let mut previous: Link<T> = None;

    while let Some(mut current) = head {
        // Detach the untouched suffix before redirecting `current.next`.
        head = current.next.take();

        // Move ownership of the reversed prefix behind the current node.
        current.next = previous;

        // The current node becomes the new head of the reversed prefix.
        previous = Some(current);
    }

    previous
}
```

Before one iteration	After one iteration
<code>previous ← current → rest</code>	<code>previous ← current</code> and <code>head → rest</code>

The algorithm runs in $O(n)$ time and uses $O(1)$ auxiliary space. `take()` replaces an `Option` with `None` while returning its old value, which makes the ownership change explicit without cloning or unsafe pointers.

Find the Middle Node with Slow and Fast References

Advance one reference by one edge and the other by two. When the fast reference reaches the end, the slow reference is at the middle:

```
fn middle<T>(head: &Link<T>) -> Option<&T> {
    // `as_deref` turns `Option<Box<Node<T>>>` into `Option<&Node<T>>`.
    let mut slow = head.as_deref();
    let mut fast = head.as_deref();

    loop {
        let Some(first_step) = fast else {
            break;
        };
        let Some(second_step) = first_step.next.as_deref() else {
            break;
        };

        // Fast completed two steps, so slow may safely complete one.
        fast = second_step.next.as_deref();
        slow = slow.and_then(|node| node.next.as_deref());
    }

    // For an even-length list, this convention returns the second middle node.
    slow.map(|node| &node.element)
}
```

This is $O(n)$ time and $O(1)$ space. State the even-length convention because “middle” is ambiguous for lists of length two, four, and so on.

Detect a Cycle with Floyd’s Tortoise and Hare

An exclusively owned `Box` list cannot safely point back to an ancestor, so use an index-based arena to demonstrate cyclic topology without inventing unsafe aliases. Each table entry stores the index of its next node:

```

fn step(next: &[Option<usize>], node: Option<usize>) -> Option<Option<usize>> {
    match node {
        // Reaching the end is a valid list state.
        None => Some(None),
        // An out-of-range edge makes the representation malformed.
        Some(index) => next.get(index).copied(),
    }
}

fn has_cycle(next: &[Option<usize>], head: Option<usize>) -> Option<bool> {
    let mut slow = head;
    let mut fast = head;

    loop {
        // The tortoise advances once; the hare advances twice.
        slow = step(next, slow)?;
        fast = step(next, step(next, fast))?;

        match (slow, fast) {
            // Equal live positions prove that both pointers are inside a cycle.
            (Some(left), Some(right)) if left == right => return Some(true),
            // Either pointer reaching the end proves the chain is acyclic.
            (None, _) | (_, None) => return Some(false),
            _ => {}
        }
    }
}

assert_eq!(has_cycle(&[Some(1), Some(2), Some(1)], Some(0)), Some(true));
assert_eq!(has_cycle(&[Some(1), None], Some(0)), Some(false));

```

The return type separates **malformed edge** (`None`) from **valid acyclic list** (`Some(false)`). After a fast/slow collision, finding the cycle entry takes a second phase: reset one pointer to the head and advance both one step until they meet.

What the Linked-List Designs Teach

Lesson from the list designs	General low-level consequence
recursive structure	recursive destruction may overflow on extreme depth
shared <code>Rc</code> tails	cheap persistence trades mutation for shared identity
<code>Rc</code> cycles	reference counts need <code>Weak</code> non-owning edges
doubly linked mutation	two-way aliases make safe ownership much harder
iterator families	consuming, shared, and mutable iteration differ
raw-pointer queue/deque	safety belongs to a small abstraction boundary
Miri/alias models	passing tests is not proof that unsafe code is valid
variance and <code>Send / Sync</code>	pointer representation affects type-system promises
panic safety	every intermediate mutation must remain droppable
cursor APIs	mutation location and iterator validity are contracts

An iterative destructor avoids recursively dropping one node per stack frame:

```
impl<T> Drop for List<T> {
    fn drop(&mut self) {
        // Detach the head so `self` no longer owns the chain.
        let mut current = self.head.take();

        while let Some(mut boxed_node) = current {
            // Detach `next` before `boxed_node` is dropped. This makes destruction
            // a loop instead of a recursive cascade through `Node::next`.
            current = boxed_node.next.take();
        }
    }
}
```

 **Practical default:** linked lists are excellent for studying ownership and specialized intrusive/persistent structures. For everyday stacks, queues, and indexed data, start with `Vec` or `VecDeque` and change only after measurement or a structural requirement justifies it.

8. Breadth-First Search

BFS explores by increasing edge distance. Use `VecDeque`, not `Vec::remove(0)`.

```
use std::collections::VecDeque;

fn bfs_distances(graph: &[Vec<usize>], start: usize) -> Vec<Option<usize>> {
    // `None` means the node has not been discovered.
    let mut distance = vec![None; graph.len()];
    let mut queue = VecDeque::new();

    // Treat an invalid root as an empty traversal instead of indexing.
    if start >= graph.len() {
        return distance;
    }

    // Mark when enqueueing so each node enters the queue at most once.
    distance[start] = Some(0);
    queue.push_back(start);

    while let Some(node) = queue.pop_front() {
        // FIFO order guarantees this is the shortest unweighted distance.
        let next_distance = distance[node].expect("queued nodes have a distance") + 1;

        for &neighbor in &graph[node] {
            // Ignore malformed edges and skip already discovered nodes.
            if neighbor < graph.len() && distance[neighbor].is_none() {
                distance[neighbor] = Some(next_distance);
                queue.push_back(neighbor);
            }
        }
    }

    distance
}
```

BFS application	Why breadth-first order helps

Unweighted shortest path	First arrival uses the fewest edges
Nearest state	Explores states by increasing transition count
Dependency distance	Assigns layers from a starting node
Level-order traversal	Visits a tree one depth at a time
Minimum transformations	Finds the smallest number of uniform-cost steps

Mark a node visited when enqueueing it, not when dequeuing, to avoid duplicate queue entries. To reconstruct a shortest path, store `parent[neighbor] = Some(node)` at the same moment that the neighbor is first discovered, then walk parents backward from the goal.

9. Depth-First Search

DFS explores one path before backtracking. An explicit stack avoids overflowing the Rust call stack on a deep graph:

```
fn dfs_order(graph: &[Vec<usize>], start: usize) -> Vec<usize> {
    let mut order = Vec::new();
    let mut visited = vec![false; graph.len()];
    let mut stack = vec![start];

    while let Some(node) = stack.pop() {
        if node >= graph.len() || visited[node] {
            continue;
        }

        visited[node] = true;
        order.push(node);

        // Reverse so the first listed neighbor is visited first.
        stack.extend(graph[node].iter().rev().copied());
    }

    order
}
```

DFS application	Useful traversal property
AST/syntax-tree walk	Follow a subtree before its siblings
Reachability	Explore an entire connected region
Cycle detection	Track active recursion/visit state
Strongly connected components	Use DFS ordering in classic SCC algorithms
Postorder code generation	Process children before parents
GC marking	Trace object references to completion

Choose recursive DFS when the input depth is strictly bounded and recursion clarifies the algorithm. Otherwise prefer an explicit stack.

10. Backtracking and DFS with Memoization

DFS describes a traversal order. Two important refinements change what happens when a state is revisited:

Technique	Revisited state	Best fit
plain DFS	skip if globally visited	reachability and component traversal
backtracking	undo the current choice and try another	enumerate constrained candidates
memoized DFS	return the cached answer for the same subproblem	optimization/counting with overlap
BFS	keep the first discovery at minimum unweighted depth	shortest number of uniform-cost steps

Backtracking Is Choose → Explore → Unchoose

Backtracking searches a decision tree and abandons branches that cannot produce a valid answer. This example generates size- k combinations from $0..n$:

```

fn combinations(n: usize, k: usize) -> Vec<Vec<usize>> {
    fn search(
        next: usize,
        n: usize,
        k: usize,
        chosen: &mut Vec<usize>,
        output: &mut Vec<Vec<usize>>,
    ) {
        if chosen.len() == k {
            // Clone only at a complete answer; partial states reuse one buffer.
            output.push(chosen.clone());
            return;
        }

        let needed = k - chosen.len();
        let available = n.saturating_sub(next);
        if available < needed {
            // Even choosing every remaining value cannot fill the combination.
            return;
        }

        for candidate in next..n {
            // Choose.
            chosen.push(candidate);

            // Explore. `candidate + 1` prevents reuse and duplicate orderings.
            search(candidate + 1, n, k, chosen, output);

            // Unchoose, restoring exactly the state this loop iteration received.
            chosen.pop();
        }
    }

    let mut output = Vec::new();
    let mut chosen = Vec::with_capacity(k);
    search(0, n, k, &mut chosen, &mut output);
    output
}

```



Backtracking invariant: after the recursive call returns, all mutable search state must be exactly as it was before the choice.

Prune from constraints before branching. Useful compiler/reversing examples include grammar alternatives, instruction-pattern selection, register assignments, symbolic input constraints, and competing type or calling-convention hypotheses.

Memoized DFS Turns a Repeated Search Tree into Dynamic Programming

Memoization caches a result by the complete state that determines future work. This top-down coin-change example uses an outer `Option` for **not computed yet** and an inner `Option` for **computed but impossible**:

```

fn min_coins(coins: &[usize], amount: usize) -> Option<usize> {
    fn solve(
        coins: &[usize],
        amount: usize,
        memo: &mut [Option<Option<usize>>],
    ) -> Option<usize> {
        if amount == 0 {
            return Some(0);
        }

        // Reuse both successful and impossible results.
        if let Some(cached) = memo[amount] {
            return cached;
        }

        let best = coins
            .iter()
            .copied()
            // Positive coins make every recursive edge decrease the state.
            .filter(|&coin| coin > 0 && coin <= amount)
            .filter_map(|coin| {
                solve(coins, amount - coin, memo)
                    .and_then(|count| count.checked_add(1))
            })
            .min();

        // Cache before returning so every amount is solved at most once.
        memo[amount] = Some(best);
        best
    }

    let mut memo = vec![None; amount.checked_add(1)?];
    solve(coins, amount, &mut memo)
}

assert_eq!(min_coins(&[1, 3, 4], 6), Some(2));
assert_eq!(min_coins(&[2, 4], 7), None);

```

Without the memo table, many amounts are recomputed along different DFS paths. With memoization, there are at most `amount + 1` states and each tries every coin, giving ($O(\text{amount} \times \text{coins})$) time and ($O(\text{amount})$) table space.

Memoization design question	Requirement
state key	include everything that changes future answers
base case	terminate without consulting an invalid index
progress	recursive edges must approach a base case
impossible result	cache it too, or failed states will be recomputed
cycles	use an “in progress” state if the subproblem graph cycles
depth	prefer tabulation for adversarial or very deep state DAGs

This is dynamic programming because the recursive call graph is a DAG of overlapping subproblems. **Top-down memoization** evaluates only reachable states; **bottom-up tabulation** avoids call-stack depth and makes dependency order explicit.

11. Topological Sorting

A directed acyclic graph can be ordered so every dependency appears before its users. Kahn's algorithm repeatedly removes nodes with zero incoming edges:

```
use std::collections::VecDeque;

fn topological_sort(graph: &[Vec<usize>]) -> Option<Vec<usize>> {
    // Count prerequisites for every node.
    let mut incoming = vec![0; graph.len()];

    for neighbors in graph {
        for &neighbor in neighbors {
            // Reject edges outside the declared node domain.
            if neighbor >= graph.len() {
                return None;
            }
            incoming[neighbor] += 1;
        }
    }

    // Nodes with no remaining prerequisites are ready to emit.
    let mut ready: VecDeque<_> = incoming
        .iter()
        .enumerate()
        .filter_map(|(node, &count)| (count == 0).then_some(node))
        .collect();

    let mut order = Vec::with_capacity(graph.len());

    while let Some(node) = ready.pop_front() {
        order.push(node);
        for &neighbor in &graph[node] {
            // Removing `node` satisfies one prerequisite of each neighbor.
            incoming[neighbor] -= 1;
            if incoming[neighbor] == 0 {
                ready.push_back(neighbor);
            }
        }
    }

    // A cycle leaves at least one node with a nonzero incoming count.
    (order.len() == graph.len()).then_some(order)
}
```

If not every node is emitted, the graph contains a cycle.

Topological-sort use	Dependency represented by each edge
Module build order	Imported module must be available first
Constant initialization	Dependency value must be initialized first
IR scheduling	Producer operation must precede its consumer

Pass/migration ordering	Prerequisite step must run first
Cycle diagnosis	Unemitted nodes reveal a dependency cycle

12. Dynamic Programming

Dynamic programming applies when:

- the problem decomposes into subproblems;
- subproblems overlap;
- the best answer can be built from smaller answers.

Two styles:

- **memoization**: recursive, cache results on demand;
- **tabulation**: iterative, fill a table in dependency order.

Example: longest common subsequence length with rolling rows:

```
fn lcs_length<T: Eq>(left: &[T], right: &[T]) -> usize {
    // Only the previous DP row is needed to construct the current row.
    let mut previous = vec![0usize; right.len() + 1];
    let mut current = vec![0usize; right.len() + 1];

    for left_item in left {
        for (column, right_item) in right.iter().enumerate() {
            current[column + 1] = if left_item == right_item {
                // Matching items extend the best smaller-prefix solution.
                previous[column] + 1
            } else {
                // Otherwise discard one side and keep the better prefix.
                current[column].max(previous[column + 1])
            };
        }

        // Reuse allocations instead of constructing a fresh row each iteration.
        std::mem::swap(&mut previous, &mut current);
        current.fill(0);
    }

    previous[right.len()]
}

assert_eq!(lcs_length(b"compiler", b"compare"), 5);
```

The full ($O(mn)$) table is unnecessary when each row depends only on the previous row. This reduces extra space to ($O(n)$).

DP use in language tools	Optimized subproblem
Sequence/tree diff	Best alignment of smaller prefixes/subtrees
Parser recovery	Lowest-cost repair path
Instruction selection	Cheapest covering of IR patterns

Parenthesization	Lowest-cost evaluation order
Diagnostic edit distance	Minimum edits between names
Layout/line breaking	Best formatting up to each position

13. Weighted Graphs and Dijkstra's Algorithm

Use Dijkstra's algorithm when edge weights are non-negative and the goal is the shortest distance from one source. The key operation is **relaxation**:

```
candidate = distance[current] + edge_weight
if candidate < distance[neighbor]:
    distance[neighbor] = candidate
```

Rust's `BinaryHeap` is a max-heap, so reverse the comparison to make the cheapest pending state come out first:

```
use std::cmp::Ordering;
use std::collections::BinaryHeap;

#[derive(Clone, Copy, Debug, Eq, PartialEq)]
struct Edge {
    to: usize,
    cost: u64,
}

#[derive(Clone, Copy, Debug, Eq, PartialEq)]
struct State {
    node: usize,
    cost: u64,
}

impl Ord for State {
    fn cmp(&self, other: &Self) -> Ordering {
        other
            .cost
            .cmp(&self.cost)
            .then_with(|| self.node.cmp(&other.node))
    }
}

impl PartialOrd for State {
    fn partial_cmp(&self, other: &Self) -> Option<Ordering> {
        Some(self.cmp(other))
    }
}

fn dijkstra(graph: &[Vec<Edge>], start: usize) -> Option<Vec<Option<u64>>> {
    if start >= graph.len() {
        return None;
    }

    let mut distance = vec![None; graph.len()];
    let mut frontier = BinaryHeap::new();
    distance[start] = Some(0);
    frontier.push(State {
        node: start,
```

```

        cost: 0,
    });

    while let Some(State { node, cost }) = frontier.pop() {
        if distance[node].is_some_and(|best| cost > best) {
            continue; // Stale queue entry.
        }

        for edge in &graph[node] {
            if edge.to >= graph.len() {
                return None;
            }

            let candidate = cost.checked_add(edge.cost)?;
            let improves = distance[edge.to].is_none_or(|best| candidate < best);

            if improves {
                distance[edge.to] = Some(candidate);
                frontier.push(State {
                    node: edge.to,
                    cost: candidate,
                });
            }
        }
    }

    Some(distance)
}

```

Important details:

- Skip stale heap entries instead of trying to decrease a key in place.
- Use checked addition when costs can come from untrusted input.
- `None` represents infinity without reserving a magic integer value.
- Dijkstra is incorrect for negative edges; use Bellman–Ford or reformulate the problem.
- With an adjacency list and binary heap, the usual bound is $O((V + E)\log V)$.

Compiler and reversing uses include weighted instruction selection, cheapest conversion paths, control-flow navigation, and finding a minimum-cost sequence of state transitions.

14. Disjoint Sets (Union-Find)

A disjoint-set structure maintains equivalence classes. It supports:

- `find(x)` — return the representative of `x`;
- `union(a, b)` — merge the two sets.

Path compression and union by rank make a long sequence of operations nearly linear:

```

#[derive(Debug)]
struct DisjointSet {
    parent: Vec<usize>,
    rank: Vec<u8>,
}

impl DisjointSet {
    fn new(len: usize) -> Self {
        Self {
            parent: (0..len).collect(),
            rank: vec![0; len],
        }
    }

    fn find(&mut self, item: usize) -> Option<usize> {
        let parent = *self.parent.get(item)?;
        if parent != item {
            let root = self.find(parent)?;
            self.parent[item] = root;
        }
        Some(self.parent[item])
    }

    fn union(&mut self, left: usize, right: usize) -> Option<bool> {
        let mut left_root = self.find(left)?;
        let mut right_root = self.find(right)?;

        if left_root == right_root {
            return Some(false);
        }

        if self.rank[left_root] < self.rank[right_root] {
            std::mem::swap(&mut left_root, &mut right_root);
        }

        self.parent[right_root] = left_root;
        if self.rank[left_root] == self.rank[right_root] {
            self.rank[left_root] = self.rank[left_root].saturating_add(1);
        }

        Some(true)
    }
}

```

Union-find application	Equivalence being maintained
Kruskal's algorithm	Vertices already connected by selected edges
Type unification	Type variables known to be equal
Alias analysis	Values that may refer to the same region
Connected components	Nodes belonging to one component
Symbols/relocations	Records known to represent one entity

The invariant is that every parent chain ends at a root whose parent is itself.

15. More Dynamic Programming: 0/1 Knapsack

The 0/1 knapsack problem chooses each item at most once while maximizing value under a capacity limit. A one-dimensional table is enough if capacities are visited in reverse:

```
#[derive(Clone, Copy, Debug)]
struct Item {
    weight: usize,
    value: u64,
}

fn knapsack(items: &[Item], capacity: usize) -> Option<u64> {
    let mut best = vec![0u64; capacity.checked_add(1)?];

    for item in items {
        if item.weight == 0 {
            return None; // Define zero-weight semantics explicitly.
        }

        for remaining in (item.weight..=capacity).rev() {
            let with_item = best[remaining - item.weight].checked_add(item.value)?;
            best[remaining] = best[remaining].max(with_item);
        }
    }

    best.last().copied()
}
```

Reverse iteration is the essential invariant. Forward iteration would allow the newly updated entry to be reused, silently changing the problem into **unbounded** knapsack.

Language-tool analogies:

- select optimizations under a compile-time budget;
- choose instructions under a byte-size limit;
- pack constants or sections under an alignment/size constraint;
- choose diagnostic repairs with bounded cost.

The table uses ($O(\text{capacity})$) memory and the algorithm uses ($O(\text{items} \times \text{capacity})$) time. This is pseudo-polynomial: a numerically large capacity can still make it impractical.

16. Cryptographic Algorithms: Learn the Shape, Use a Library

Cryptography combines algorithms with strict protocol rules. A mathematically correct primitive can still be insecure because of nonce reuse, weak randomness, timing leaks, bad serialization, missing authentication, or poor key handling.

Goal	Appropriate primitive
Detect accidental corruption	Non-cryptographic checksum
Integrity without a key	Cryptographic hash
Integrity with a shared key	Message authentication code (MAC)
Confidentiality + integrity	Authenticated encryption with associated data

Store passwords	Password KDF with salt and work factor
Establish a shared secret	Authenticated key-exchange protocol
Prove who signed data	Digital signature

Cryptographic rule	Practical meaning
Authenticate encryption	Confidentiality alone does not prevent tampering
Respect nonce rules	Public does not mean reusable
Distinguish salts/keys	A salt is neither a password nor an encryption key
Hashes are one-way	Hashing is not encryption
Use a password KDF	Fast general hashes are unsuitable for password storage
Compare in constant time	Use the library's authenticator comparison
Minimize key exposure	Keep keys scoped and never log them
Version the format	Record algorithms and parameters alongside ciphertext
Use audited constructions	Do not invent production ciphers or protocols

Educational Building Block: Modular Exponentiation

RSA and Diffie–Hellman use modular arithmetic, but real implementations also require large integers, carefully generated parameters, padding/encoding rules, side-channel resistance, and secure randomness. The following is only the square-and-multiply arithmetic pattern:

```

fn mod_pow(mut base: u128, mut exponent: u128, modulus: u128) -> Option<u128> {
    // Modulo zero is undefined, so reject it instead of panicking.
    if modulus == 0 {
        return None;
    }

    // Keep intermediate values in the smallest equivalent residue class.
    base %= modulus;
    // One is the multiplicative identity; `% modulus` also handles modulus == 1.
    let mut result = 1 % modulus;

    while exponent != 0 {
        // A set low bit means the current power contributes to the answer.
        if exponent & 1 == 1 {
            // This detects u128 overflow; production big-integer code needs a
            // multiplication method that can reduce without overflowing first.
            result = result.checked_mul(base)? % modulus;
        }

        // Consume the bit just processed.
        exponent >>= 1;
        if exponent != 0 {
            // Move from base^(2^k) to base^(2^(k+1)).
            base = base.checked_mul(base)? % modulus;
        }
    }

    Some(result)
}

```

This function is useful for learning exponentiation by squaring, but it is **not** constant time and ordinary `u128` multiplication can overflow before reduction. It is not a production cryptographic implementation.

Cryptography as a Parser and Protocol Problem

Treat authenticated data as a precisely encoded structure:

```

version || algorithm_id || nonce || associated_data || ciphertext || tag

```

The decoder should:

1. bound all lengths before allocating;
2. reject unknown or forbidden algorithm identifiers;
3. authenticate before releasing plaintext;
4. reject duplicate/replayed nonces when the protocol requires it;
5. keep parsing errors from becoming a useful oracle;
6. preserve exact bytes used as associated data;
7. fail closed.

This is directly relevant to designing a language runtime: signed packages, bytecode modules, plugin manifests, update files, and network messages are all binary formats with trust boundaries.

17. Where Algorithms Appear in Language Tools

Language-tool problem	Useful algorithm/data structure
Token lookup	Hash table or trie
Source-offset lookup	Binary search
AST walk	DFS
Shortest unweighted path in CFG	BFS
Module dependency order	Topological sort
Recursive call groups	Strongly connected components
Type equivalence classes	Union-find/disjoint set
Data-flow analysis	Worklist fixed-point iteration
Register allocation	Graph coloring or linear scan
Constant propagation	Lattice + worklist
Garbage-collector marking	DFS/BFS over object graph
Diagnostic suggestions	Edit distance
Instruction scheduling	Dependency DAG

18. Worklist Fixed-Point Pattern

Many compiler analyses repeatedly propagate facts until nothing changes:

```

initialize facts
enqueue affected nodes

while worklist is not empty:
    node = pop
    new_fact = transfer(predecessor facts)
    if new_fact changed:
        store it
        enqueue successors

```

Fixed-point component	Requirement
Fact domain	Finite height or another convergence argument
Transfer function	Monotone with respect to the fact ordering
Change detection	Reliable equality or improvement test
Dependencies	Precise successors to re-enqueue
Tests	Deterministic expected final facts

19. Algorithm Engineering Checklist

Area	Preferred practice
------	--------------------

Standard tools	Use library sorting/searching in production
Invariants	State them beside non-obvious loops
Arithmetic	Check indexes, capacities, and input-derived costs
Depth	Avoid recursion over unbounded adversarial input
Allocation	Preallocate only from trustworthy estimates
Queues/heaps	Use <code>VecDeque</code> for FIFO and <code>BinaryHeap</code> for priority
Membership	Use a set or boolean vector instead of repeated scans
Reproducibility	Preserve stable ordering when output must be deterministic
Edge cases	Test empty, duplicate, cyclic, disconnected, and maximum inputs
Performance	Benchmark realistic workloads; big-O is not the whole result
Generics	Generalize only when reuse improves without hiding the algorithm
Failure	Return <code>Option / Result</code> when absence or failure is part of the contract



Machine Learning: Optimization, Probability, and Generalization

The [Machine Learning Glossary](#) organizes the field around linear/logistic regression, calculus, linear algebra, forward propagation, backpropagation, activations, losses, optimizers, regularization, and model architectures. The goal here is a systems-level mental model, not a substitute for a statistics course or a production ML framework.

1. Learning Means Fitting a Parameterized Function

```

input features x
  ↓
model f(x; parameters θ)
  ↓
prediction ŷ
  ↓ compare with target y
loss L(ŷ, y)
  ↓ optimizer updates θ

```

Term	Meaning
feature	measured or derived model input
label/target	desired output used for supervised learning
parameter	value learned during training
hyperparameter	configuration chosen outside training
inference	compute a prediction with fixed parameters

training	adjust parameters to reduce an objective
epoch	one pass over the training data
batch	subset used for one update

2. Split Data by Purpose

Split	Used for	Must not be used for
training	fitting parameters	final performance claim
validation	selecting models and hyperparameters	repeated unbiased final reporting
test	final estimate after choices are frozen	tuning

Data leakage occurs when training receives information that would not exist at real prediction time or indirectly sees validation/test targets. Split related samples by user, machine, project, time, or other dependency when random rows would leak context.

3. Linear Regression in Small Rust

For one feature:

```
prediction = weight × x + bias
error      = prediction - target
MSE        = mean(error2)
```

```
#[derive(Debug, Clone, Copy)]
struct LinearModel {
    weight: f64,
    bias: f64,
}

impl LinearModel {
    fn predict(self, x: f64) -> f64 {
        self.weight * x + self.bias
    }

    fn train_step(&mut self, samples: &[(f64, f64)], rate: f64) {
        if samples.is_empty() {
            return;
        }

        let count = samples.len() as f64;
        let (weight_gradient, bias_gradient) =
            samples.iter().fold((0.0, 0.0), |(dw, db), &(x, target)| {
                let error = self.predict(x) - target;
                (dw + (2.0 / count) * error * x, db + (2.0 / count) * error)
            });

        self.weight -= rate * weight_gradient;
        self.bias -= rate * bias_gradient;
    }
}
```

This example is intentionally small. Production training needs numerical checks, vectorized/tensor operations, reproducibility controls, efficient batching, and mature libraries.

4. Gradient Descent Is Local Iterative Optimization

$$\theta(\text{next}) = \theta(\text{current}) - \text{learning_rate} \times \nabla L(\theta)$$

Symptom	Possible cause
loss explodes	rate too high, poor scaling, numerical issue
loss barely moves	rate too low, saturation, weak features
training good/test poor	overfitting or leakage
training and test both poor	underfitting, bad representation, bad labels
results vary wildly	seed, ordering, nondeterminism, small dataset

Feature normalization changes optimization geometry. It does not create information that the features do not contain.

5. Classification and Regression Need Different Outputs

Task	Output	Common loss/metric examples
regression	continuous value	MSE, MAE, RMSE
binary classification	probability/logit for two classes	binary cross-entropy, precision/recall
multiclass classification	class distribution/logits	cross-entropy, top-k accuracy
ranking	relative ordering	pairwise/listwise objectives
clustering	unlabeled grouping	distance/internal validity measures

Accuracy can be misleading on imbalanced data. A detector that predicts “safe” for 99.9% of inputs can score highly while missing every rare failure. Choose metrics from the cost of false positives and false negatives.

6. Neural Networks Compose Differentiable Layers

input
→ affine transform (weights × input + bias)
→ activation
→ more layers
→ output
→ loss

Concept	Role
weight	learned connection strength

bias	learned offset
activation	nonlinear transformation
forward pass	compute predictions and intermediates
backpropagation	apply chain rule from loss toward earlier layers
optimizer	turn gradients into parameter updates
regularization	discourage brittle overfit solutions

ReLU is simple and common; sigmoid maps to $(0, 1)$ but can saturate; softmax converts logits into a normalized multiclass distribution. Match the final activation, loss, and target representation.

7. Regularization Controls Generalization

Technique	Intuition
L1 penalty	encourage sparse parameters
L2 penalty/weight decay	discourage large parameters
dropout	randomly omit activations during training
early stopping	stop when validation performance stops improving
data augmentation	encode valid input-preserving transformations
ensembling	combine models to reduce some variance

Regularization cannot repair mislabeled data, a broken split, or a metric unrelated to the real objective.

8. Numerical and Systems Boundaries

Boundary	Engineering question
data loader	Are order, decoding, and augmentation reproducible?
tensor shape	Are dimensions and broadcasting intentional?
numeric precision	Can overflow, underflow, or cancellation occur?
accelerator transfer	Is copy/synchronization cost measured?
checkpoint	Are model, optimizer, schema, and version recorded?
inference service	Are latency, batching, memory, and fallback bounded?
monitoring	Can drift and quality degradation be detected?

9. ML and Language Tools

Possible use	Deterministic guardrail
code completion/ranking	parser/type checker rejects invalid output

optimization heuristic	semantic equivalence tests
decompiler naming suggestion	evidence and analyst confirmation
fuzz-input prioritization	coverage and reproducible seeds
anomaly detection in traces	retain raw evidence and explain uncertainty
performance autotuning	benchmark budget and safe fallback

Do not replace a cheap exact rule with a probabilistic model merely because ML is available. Use ML when approximation creates value and its uncertainty can be contained.

➡ **Next:** formal-language and compiler chapters show how grammars, types, IR, and runtimes turn human-readable rules into executable state transitions. The same optimization and approximation ideas reappear there without statistical uncertainty.

Formal Languages and Grammar: Defining Valid Structure

A **formal grammar** is a set of rules that define what makes valid code in a programming language. Think of it like the grammar of English: "The cat sat" is valid, but "Cat the sat" is not.

🧠 **Mental model:** a grammar is the language's structural contract; the parser is the program that checks that contract and turns matching text into structure.

For programming languages, a grammar specifies:

- **What tokens are valid** - keywords like `def`, `if`, `return`; operators like `+`, `-`, `*`; literals like `42`, `true`
- **How tokens can be combined** - `1 + 2` is valid, `++ 1` is not
- **The structure of programs** - functions contain statements, statements contain expressions

Parsing: From Tokens to Syntax Trees

A parser takes raw source code (a string) and converts it into a structured **Abstract Syntax Tree (AST)** that a computer can evaluate or compile. Think of source code as a sentence and the AST as its grammatical diagram. It captures *structure*, not just text.

🕒 **Parser invariant:** every accepted input produces a well-formed structure, and every rejected input produces a bounded, useful diagnostic rather than a crash.

Here is the step-by-step process of building one from scratch using Rust and the `pest` crate:

1. Setting Up the Lexer/Grammar

Grammar rules are built from expressions (hence "parsing expression grammar"). These expressions are a terse, formal description of how to parse an input string.

Expressions are composable: they can be built out of other expressions and nested inside of each other to produce arbitrarily complex rules (although you should break very complicated expressions into multiple rules to make them easier to manage).

PEG expressions are suitable for both *high-level meaning*, like “a function signature, followed by a function body”, and *low-level meaning*, like “a semicolon, followed by a line feed”. The combining form “followed by”, the sequence operator, is the same in either case.

Macro Captures Used in Grammar Code

Within Macros	Explanation
<code>\$x:ty</code>	Macro capture (here a <code>\$x</code> is the capture and <code>ty</code> means <code>x</code> must be type).
<code>\$x:block</code>	A block <code>{ }</code> of statements or expressions, e.g., <code>{ let x = 5; }</code>
<code>\$x:expr</code>	An expression, e.g., <code>x</code> , <code>1 + 1</code> , <code>String::new()</code> or <code>vec![]</code>
<code>\$x:ident</code>	An identifier, for example in <code>let x = 0;</code> the identifier is <code>x</code> .
<code>\$x:item</code>	An item, like a function, struct, module, etc.
<code>\$x:lifetime</code>	A lifetime (e.g., <code>'a</code> , <code>'static</code> , etc.).
<code>\$x:literal</code>	A literal (e.g., <code>3</code> , <code>"foo"</code> , <code>b"bar"</code> , etc.).
<code>\$x:meta</code>	A meta item; the things that go inside <code>#[...]</code> and <code>#![...]</code> attributes.
<code>\$x:pat</code>	A pattern, such as <code>Some(x)</code> , <code>(17, 'a')</code> , or an OR-pattern.
<code>\$x:pat_param</code>	A pattern accepted as a parameter without a top-level OR-pattern.
<code>\$x:path</code>	A path (e.g., <code>foo</code> , <code>::std::mem::replace</code> , <code>transmute::<_, int></code>).
<code>\$x:stmt</code>	A statement, e.g., <code>let x = 1 + 1;</code> , <code>String::new();</code> or <code>vec![];</code>
<code>\$x:tt</code>	A single token tree.
<code>\$x:ty</code>	A type, e.g., <code>String</code> , <code>usize</code> or <code>Vec<u8></code> .
<code>\$x:vis</code>	A visibility modifier; <code>pub</code> , <code>pub(crate)</code> , etc.

Pest

First, you need rules that define what makes valid code. Using a parser generator (like the `pest` crate in Rust), you define a grammar file (e.g., `grammar.pest`). The tool uses this grammar to validate the text and tokenize the raw string into an iterator of parsed “pairs” (tokens) — e.g., pulling out operators, numbers, and nested expressions.

```
#[derive(pest_derive::Parser)]
#[grammar = "grammar.pest"]
struct CalcParser;
```

Syntax	Meaning	Example
<code>"text"</code>	Match exact text	<code>"def"</code> matches the keyword <code>def</code>
<code>~</code>	Sequence (then)	<code>"if" ~ "(" ~ Expr ~ ")"</code> matches <code>if</code> followed by <code>(</code>

	Choice (or)	"true" "false" matches either
*	Zero or more (Kleene star)	Stmt* matches any number of statements
+	One or more	ASCII_DIGIT+ matches one or more digits
?	Optional	ReturnType? matches zero or one return type
{ }	Rule definition	Add = { "+" } defines a rule
_ { }	Silent rule	_ { Expr } matches but does not appear in AST
@ { }	Atomic rule	@ { ASCII_DIGIT+ } matches as a single token
SOI	Start of input	Beginning of the source code
EOI	End of input	End of the source code

Example: Calculator Grammar

```

num = @ { int ~ ( "." ~ ASCII_DIGIT* )? ~ ( ^ "e" ~ int )? }
int = { ( "+" | "-" )? ~ ASCII_DIGIT+ }

operation = _ { add | subtract | multiply | divide | power }
  add      = { "+" }
  subtract = { "-" }
  multiply = { "*" }
  divide   = { "/" }
  power    = { "^" }

expr = { term ~ ( operation ~ term )* }
term = _ { num | "(" ~ expr ~ ")" }

calculation = _ { SOI ~ expr ~ EOI }

WHITESPACE = _ { " " | "\t" }

```

Cheat sheet

Syntax	Meaning	Syntax	Meaning
foo = { ... }	regular rule	baz = @ { ... }	atomic
bar = _ { ... }	silent	qux = \$ { ... }	compound-atomic
#tag = ...	tags	plugh = ! { ... }	non-atomic
"abc"	exact string	^ "abc"	case insensitive
'a' .. 'z'	character range	ANY	any character
foo ~ bar	sequence	baz qux	ordered choice
foo*	zero or more	bar+	one or more
baz?	optional	qux { n }	exactly <i>n</i>
qux { m, n }	between <i>m</i> and <i>n</i> (inclusive)		

<code>&foo</code>	positive predicate	<code>!bar</code>	negative predicate
<code>PUSH(baz)</code>	match and push	<code>PUSH_LITERAL("a")</code>	push without match
<code>POP</code>	match and pop	<code>PEEK</code>	match without pop
<code>DROP</code>	pop without matching	<code>PEEK_ALL</code>	match entire stack

2. Defining the Abstract Syntax Tree (AST)

The AST captures the *meaning* and *structure* of the code, not just the raw text. In Rust, this is typically done using `enum` to represent different node types:

- **Terminal Nodes (Leaves):** e.g., `Node::Int(i32)` for raw numbers.
- **Unary Expressions:** e.g., `Node::UnaryExpr { op, child }` for operations like `-5`.
- **Binary Expressions:** e.g., `Node::BinaryExpr { op, lhs, rhs }` for operations like `1 + 2`. (Note: `Box<Node>` is used for the children to allow recursive nesting).

The tree naturally encodes the order of operations through its nesting.

```

Program = _{ SOI ~ Expr ~ EOF }

Expr = { BinaryExpr | UnaryExpr | Term }

Term = { Int | "(" ~ Expr ~ ")" }

UnaryExpr = { Operator ~ Term }

// Allow expressions to start with a unary expression (e.g., -1 + 2)
BinaryExpr = { (UnaryExpr | Term) ~ (Operator ~ Term)+ }

Operator = { "+" | "-" }

Int = @{ ASCII_DIGIT+ }

WHITESPACE = _{ " " | "\t" | "\r" | "\n" }

EOF = _{ EOI | ";" }

```

```
pub enum Operator {
    Plus,
    Minus,
}

pub enum Node {
    // Terminal Nodes (Leaves)
    Int(i32),
    // Unary Expressions (e.g., -5)
    UnaryExpr {
        op: Operator,
        child: Box<Node>,
    },
    // Binary Expressions (e.g., 1 + 2)
    BinaryExpr {
        op: Operator,
        lhs: Box<Node>,
        rhs: Box<Node>,
    },
}
```

Note: `Box<Node>` is required for the children to allow recursive nesting. Without `Box`, Rust wouldn't be able to calculate the size of the `Node` enum at compile time.

3. The Main Parsing Loop

The main entry point (often just `parse(source: &str)`) takes the raw string, feeds it to the parser generator, and loops through the resulting top-level tokens. When it finds a top-level expression, it passes that token to a recursive builder function to construct the AST.

```
pub fn parse(source: &str) -> std::result::Result<Vec<Node>, pest::error::Error<Rule>> {
    let mut ast = vec![];
    let pairs = CalcParser::parse(Rule::Program, source)?;

    for pair in pairs {
        if let Rule::Expr = pair.as_rule() {
            ast.push(build_ast_from_expr(pair));
        }
    }
    Ok(ast)
}
```

4. Parsing Expressions & Associativity (Composite Nodes)

This is the core logic that converts generic `pest` tokens into your custom AST nodes by looking at the rule:

- **Unary Expressions:** Grab the operator and the single child term, then construct the Unary node.

- **Binary Expressions:** Handling chained operations (like `1 + 2 + 3`) requires **left-associativity** so it evaluates as `(1 + 2) + 3`. Instead of using pure recursion (which might result in right-associativity), this is often handled using a `loop` :
 - i. Parse the initial Left-Hand Side (LHS), operator, and Right-Hand Side (RHS).
 - ii. Build the first `BinaryExpr` node.
 - iii. **The Loop:** If there is another operator immediately following in the token stream, take the expression you *just built* and make it the **new LHS**, grab the next number as the RHS, and build a new, nested `BinaryExpr`.

```
fn parse_unary_expr(pair: pest::iterators::Pair<Rule>, child: Node) -> Node {
    Node::UnaryExpr {
        // Grammar validation guarantees that only supported unary operators arrive.
        op: match pair.as_str() {
            "+" => Operator::Plus,
            "-" => Operator::Minus,
            _ => unreachable!(),
        },
        // Box the recursive child so `Node` has a finite compile-time size.
        child: Box::new(child),
    }
}

fn parse_binary_expr(pair: pest::iterators::Pair<Rule>, lhs: Node, rhs: Node) -> Node {
    Node::BinaryExpr {
        // Keep this mapping exhaustive as the grammar gains new operators.
        op: match pair.as_str() {
            "+" => Operator::Plus,
            "-" => Operator::Minus,
            _ => unreachable!(),
        },
        // The AST owns both operand subtrees.
        lhs: Box::new(lhs),
        rhs: Box::new(rhs),
    }
}
```

```

fn build_ast_from_expr(pair: pest::iterators::Pair<Rule>) -> Node {
    match pair.as_rule() {
        // `Expr` is a grouping rule; its single child carries the useful shape.
        Rule::Expr => {
            let child = pair.into_inner().next().expect("Expr has one child");
            build_ast_from_expr(child)
        }
        Rule::UnaryExpr => {
            let mut parts = pair.into_inner();
            let op = parts.next().expect("unary expression has an operator");
            let child = parts.next().expect("unary expression has an operand");
            parse_unary_expr(op, build_ast_from_term(child))
        }
        Rule::BinaryExpr => {
            let mut parts = pair.into_inner();
            let first = parts.next().expect("binary expression has a left operand");

            // The first operand may itself be unary.
            let mut lhs = if first.as_rule() == Rule::UnaryExpr {
                build_ast_from_expr(first)
            } else {
                build_ast_from_term(first)
            };

            // Fold from the left: `a - b - c` becomes `(a - b) - c`.
            while let Some(op) = parts.next() {
                let rhs_pair = parts.next().expect("operator has a right operand");
                let rhs = build_ast_from_term(rhs_pair);
                lhs = parse_binary_expr(op, lhs, rhs);
            }

            lhs
        }
        Rule::Term => build_ast_from_term(pair),
        unknown => panic!("unexpected expression rule: {unknown:?}"),
    }
}

```

5. Parsing Terms (Leaf Nodes)

When the parser drills down to a basic term (`Rule::Term`), it hits the bottom of the tree. It expects one of two things:

1. **A raw number:** It parses the string into an `i32` and returns a `Node::Int`.
2. **A nested expression:** If it hits parentheses, it recurses back up to the expression parsing logic to evaluate the inside of the parentheses first.

```
fn build_ast_from_term(pair: pest::iterators::Pair<Rule>) -> Node {
    // `Term` is a wrapper rule; unwrap its one meaningful child.
    let pair = pair.into_inner().next().expect("Term has one child");
    match pair.as_rule() {
        Rule::Int => {
            // The grammar already restricted this text to an integer token.
            let int: i32 = pair.as_str().parse().expect("validated integer");
            Node::Int(int)
        }
        // Parentheses return us to expression precedence parsing.
        Rule::Expr => build_ast_from_expr(pair),
        unknown => panic!("unexpected term rule: {unknown:?}"),
    }
}
```

Parsing Theory Through PEGs and Pest

 **Library lens:** Pest is used here to expose PEG recognition, ordered choice, lookahead, concrete parse trees, precedence, and the difference between accepted syntax and semantic meaning.

Pest turns a **parsing expression grammar** (PEG) into a Rust parser. It is a good fit for a small language because the grammar remains readable while Rust code handles AST construction, validation, diagnostics, and later compiler phases.

 **Mental model:** a Pest grammar recognizes concrete syntax. Your Rust conversion code decides which recognized details become meaningful AST nodes.

1. PEG Choice Is Ordered

In a PEG, `a | b` means **try `a` first; try `b` only if `a` fails**. This differs from thinking of alternatives as an unordered mathematical set.

```
keyword_if = { "if" }
identifier = @{ ASCII_ALPHA ~ (ASCII_ALPHANUMERIC | "_")* }

// Put the more specific alternative first.
atom = { keyword_if | identifier }
```

Operator	Meaning	Compiler use
<code>~</code>	Sequence	<code>let</code> then name then <code>=</code> then value
<code> </code>	Ordered choice	statement form or expression form
<code>*</code>	Zero or more	arguments, statements, suffixes
<code>+</code>	One or more	digits, identifier characters
<code>?</code>	Optional	type annotation or trailing comma
<code>&rule</code>	Positive lookahead	Require a prefix without consuming
<code>!rule</code>	Negative lookahead	Reject reserved words or terminators

Repetition binds more tightly than predicates, then sequence, then ordered choice. Add parentheses when a reader might otherwise have to remember that precedence.

2. Whitespace Is a Grammar Policy

When rules named `WHITESPACE` or `COMMENT` exist, Pest can insert them implicitly between sequence and repetition expressions. That is convenient for language-level tokens, but dangerous inside indivisible tokens such as identifiers and numbers.

```
WHITESPACE = _{ " " | "\t" | NEWLINE }
COMMENT = _{ "//" ~ (!NEWLINE ~ ANY)* }

identifier = @{ (ASCII_ALPHA | "_") ~ (ASCII_ALPHANUMERIC | "_")* }
integer = @{ "-"? ~ ASCII_DIGIT+ }
```

Modifier	Parse-tree effect	Whitespace effect	Typical use
<code>_</code>	Silent: omit this rule	Normal	punctuation, whitespace
<code>@</code>	Atomic: inner rules become silent	Disable implicit whitespace	identifiers, numbers
<code>\$</code>	Compound atomic: keep inner rule pairs	Disable implicit whitespace	structured string token
<code>!</code>	Non-atomic inside an atomic context	Re-enable implicit whitespace	interpolation

Atomic does not mean thread-safe. Here it means “match this grammar region as one whitespace-sensitive unit.”

3. Anchor Complete Inputs

Without `SOI` and `EOI`, a parser may recognize only a valid prefix and leave unexpected text behind.

```
program = { SOI ~ statement* ~ EOI }

statement = _{ let_stmt | print_stmt }
let_stmt = { "let" ~ identifier ~ "=" ~ expression ~ ";" }
print_stmt = { "print" ~ expression ~ ";" }

expression = { term ~ (add_op ~ term)* }
add_op = { "+" | "-" }
term = _{ integer | identifier | "(" ~ expression ~ ")" }
```

Entry rule	Appropriate contract
<code>program</code>	Must consume the whole source file
<code>expression</code>	May parse a nested region selected by another rule
editor fragment	May intentionally accept incomplete, recoverable text

4. `Pair` Is a Concrete-Syntax View, Not the AST

Each `Pair<Rule>` identifies the matched rule, matched text, byte span, and nested pairs. Convert it promptly into your own AST so the rest of the compiler is independent of Pest's tree shape.

```
use pest::iterators::Pair;
use std::ops::Range;

#[derive(Debug)]
struct Spanned<T> {
    node: T,
    bytes: Range<usize>,
}

#[derive(Debug)]
enum Expr {
    Integer(i64),
    Name(String),
    Add(Box<Spanned<Expr>>, Box<Spanned<Expr>>),
}

fn byte_range(pair: &Pair<'_, Rule>) -> Range<usize> {
    let span = pair.as_span();
    span.start()..span.end()
}
```

Pest spans are **byte offsets** into the original UTF-8 input. If diagnostics also need line and column, compute or cache that view deliberately; never treat a byte offset as a character index.

5. Keep Syntax and Semantic Errors Separate

Failure layer	Example	Best owner
Grammar	Missing <code>)</code>	Pest parse error
AST lowering	Integer does not fit in <code>i64</code>	AST conversion error
Name binding	Unknown variable	Resolver
Type checking	Add integer to function	Type checker
Codegen	Unsupported target representation	Backend

```
#[derive(Debug)]
enum FrontendError {
    Syntax(Box<pest::error::Error<Rule>>),
    IntegerOutOfRange { text: String, bytes: Range<usize> },
}

fn parse_integer(pair: Pair<'_, Rule>) -> Result<Spanned<Expr>, FrontendError> {
    let bytes = byte_range(&pair);
    let text = pair.as_str();
    let value = text.parse::<i64>().map_err(|_| {
        FrontendError::IntegerOutOfRange {
            text: text.to_owned(),
            bytes: bytes.clone(),
        }
    })?;

    Ok(Spanned {
        node: Expr::Integer(value),
        bytes,
    })
}
```

6. Use Pratt Parsing for Expression Precedence

Pest's `PrattParser` maps a flat operator/operand stream into the intended precedence and associativity. Define operators from **lowest to highest precedence**.

```
use pest::pratt_parser::{Assoc, Op, PrattParser};
use std::sync::LazyLock;

static PRATT: LazyLock<PrattParser<Rule>> = LazyLock::new(|| {
    PrattParser::new()
        .op(Op::infix(Rule::add, Assoc::Left)
            | Op::infix(Rule::subtract, Assoc::Left))
        .op(Op::infix(Rule::multiply, Assoc::Left)
            | Op::infix(Rule::divide, Assoc::Left))
        .op(Op::infix(Rule::power, Assoc::Right))
        .op(Op::prefix(Rule::negate))
});
```

Expression	Required shape	Reason
2 + 3 * 4	2 + (3 * 4)	Multiplication is higher
10 - 3 - 2	(10 - 3) - 2	Subtraction is left-associative
2 ^ 3 ^ 2	2 ^ (3 ^ 2)	Power is commonly right-associative
-x ^ 2	Language-defined	Prefix precedence must be explicit

 **Invariant:** after AST construction, precedence is encoded by tree shape. Later compiler phases should not need to remember the grammar’s precedence table.

7. Test the Grammar at Three Scales

--	--	--

Scale	Test input	Assertion
Rule	one identifier, literal, or operator	exact match and rejection cases
Construct	one expression or statement	AST shape, span, associativity
Program	a small file	complete parse and diagnostics
Adversarial	truncation, deep nesting, huge literals	bounded behavior and clear rejection

```
#[test]
fn multiplication_binds_more_tightly_than_addition() {
    let ast = parse_expression("2 + 3 * 4").expect("valid test expression");
    assert_eq!(format!("{ast:?}"), "Add(Integer(2), Multiply(Integer(3), Integer(4)))");
}

#[test]
fn program_rejects_trailing_garbage() {
    assert!(LanguageParser::parse(Rule::program, "print 1; ???").is_err());
}
```

Snapshotting a debug string is useful early, but mature compilers should compare typed AST values so formatting changes do not break semantic tests.

8. Rule Modifiers Control Both Whitespace and Tree Shape

Pest rule modifiers are not cosmetic. They change whether implicit whitespace is allowed and which nodes appear in the concrete parse tree.

Rule form	Meaning	Typical use
<code>name = { ... }</code>	normal rule; produces a pair	meaningful syntax construct
<code>name = _{ ... }</code>	silent rule; matches but produces no pair	whitespace or grouping helper
<code>name = @{ ... }</code>	atomic; disables implicit whitespace and silences inner rules	identifiers and simple literals
<code>name = \${ ... }</code>	compound atomic; disables whitespace but keeps inner pairs	structured string/token forms
<code>name = !{ ... }</code>	non-atomic even inside an atomic caller	interpolation with normal syntax

```

WHITESPACE = _{ " " | "\t" | NEWLINE }
COMMENT    = _{ "//" ~ (!NEWLINE ~ ANY)* }

identifier = @{ ASCII_ALPHA ~ (ASCII_ALPHANUMERIC | "_")* }

string = ${ "\"" ~ string_part* ~ "\"" }
string_part = {
    escape
    | interpolation
    | (!("\"" | "\\") ~ ANY)
}
interpolation = !{ "${" ~ expression ~ "}" }

```

The identifier is atomic because `hello world` must not become one identifier merely because the grammar defines implicit whitespace. The interpolation rule becomes non-atomic so an expression such as `${left + right}` can use ordinary spacing.

⚠ **Tree-shape trap:** changing `{ ... }` to `_ { ... }` can make correct matching code fail later because the corresponding `Pair` disappears. Treat the concrete parse tree as an interface and cover it with tests.

9. Predicates Inspect Without Consuming

Positive (`&`) and negative (`!`) predicates are PEG lookahead operations. They test the next input but leave the parser at the same byte position.

```

keyword_if = { "if" ~ !ASCII_ALPHANUMERIC }

triple_string = {
    "'''"
    ~ (!"'" ~ ANY)*
    ~ "'''"
}

```

Predicate	Reads input?	Consumes input?	Use
<code>&rule</code>	yes	no	require a following shape
<code>!rule</code>	yes	no	reject a following shape
<code>!terminator ~ ANY</code>	yes	one character	consume until a delimiter

Every successful repetition must make progress. A repeated expression that can succeed without consuming input is a design smell because the parser cannot advance:

```

// Risky idea: `item?` may match zero bytes.
// list = { (item?)* }

list = { item* }

```

10. The Grammar Stack Handles Matched Delimiters

Pest's grammar stack stores **matched text**, not arbitrary semantic values. `PUSH` records text; `PEEK` checks it without removing it; `POP` checks and removes it; `DROP` removes it without matching.

One useful case is a raw string whose closing delimiter must contain the same number of `#` characters as its opening delimiter:

```
raw_string = {
  "r" ~ PUSH("#"*) ~ "\""
  ~ (!("\"" ~ PEEK) ~ ANY)*
  ~ "\"" ~ POP
}
```

```
r###"the " character is allowed here"###
^^^                                ^^^
pushed delimiter                  must match exactly
```

Good grammar-stack use	Better handled elsewhere
matching a repeated delimiter	symbol tables and name resolution
indentation prefix tracking	type checking
opener-dependent closing token	arbitrary runtime state
exact textual equality later	semantic equality of decoded values

Keep the stack localized to one grammar feature. If many distant rules depend on it, the grammar becomes difficult to reason about and error recovery becomes fragile.

 **Next:** the execution-pipeline section treats the AST as the trusted input to an interpreter or backend. Pest-specific details stop here; the language model continues.

Abstract Syntax Trees and the Execution Pipeline

The Pipeline

```
Source → Tokens → AST → Output
```

Walking through `1 + 2` :

- 1. **Grammar** — Rules defining valid syntax. `1 + 2` matches; `++1` doesn't.
- 2. **Lexer (Tokenizer)** — Breaks source into meaningful chunks: `"1 + 2"` → `[1, +, 2]`. Tracks source location for error messages.
- 3. **Parser** — Builds a tree structure from tokens: `+` at the root, `1` and `2` as children — this is the **AST**.
- 4. **Interpreter/Evaluator** — Walks the AST recursively and computes the result.

AST analogy: Think of source code as a sentence and the AST as its grammatical diagram. Just as "The cat sat" is diagrammed into subject/verb, `1 + 2` is diagrammed into left/operator/right. The AST captures **structure**, not just text.

We need a systematic way to turn any AST into LLVM IR.

The answer is recursive tree traversal. We already do this in the interpreter - walk the tree, evaluate each node. For code generation, we walk the tree and emit instructions for each node instead of computing values.

Two common patterns help structure this:

- Builder pattern - Used for LLVM IR generation
- Visitor pattern - for AST transformations

Builder Pattern

Think of the LLVM builder like a cursor in a text editor. You position it somewhere in your code, then "type" instructions at that position. The builder keeps track of where you are and ensures instructions are added in the right place.

Let's compare our interpreter's recursive evaluation to the new JIT approach: the interpreter walks the tree and computes values directly, while the builder walks the tree and emits instructions instead.

The Interpreter & Recursion

The CPU is the *ultimate* interpreter — it executes opcodes one at a time. Our software interpreter does the same at a higher level by **walking the AST recursively**:

To evaluate `1 + 2` :

1. Evaluate left (`1`) → `1`
2. Evaluate right (`2`) → `2`
3. Apply operator (`+`) → `3`

If the left side were `(3 + 4)`, we'd recursively evaluate it first. This is why trees are powerful: **structure determines evaluation order**. Parse → AST → recursive eval is the foundation of every interpreter — Python, Ruby, JavaScript all do this with more node types.

State & Functions

A calculator is **stateless** — input goes in, output comes out, nothing persists. A real language is a **state machine**:

Language feature	Example	Role in execution
Variables	<code>n</code>	Named storage
Functions	<code>fib</code>	Abstraction and reuse
Parameters	passing <code>n</code>	Data flow into a call
Conditionals	<code>if / else</code>	Control-flow branching
Operators	<code><, -</code>	Computation

Recursion	<code>fib</code> calling <code>fib</code>	Self-reference
Call stack	one frame per call	Tracks local execution and returns

The AST becomes a *program to execute*, not just an expression to evaluate.

Variables

Variables let us store values and refer to them by name. Without variables, every computation would need to repeat its literal inputs — variables give us **memory**.

Reassigning a variable means any code referencing it automatically uses the new value, which makes code reusable and readable.

Where variables live: the *environment* (storage) — a `HashMap` inside a *frame*.

- **Local** variables exist inside a function.
- **Global** variables exist outside all functions.
- Variables can be **reassigned**.

This simple mechanism — storing and looking up names — underlies all programming:

Concept	Role
Parameters	Variables bound from function call arguments
Loop counters	Variables that change each iteration
Object fields	Variables attached to an object

The environment is one of the most important data structures in any interpreter.

Functions

Functions are the heart of any programming language. Without them, code must be copy-pasted every time it's reused. Functions let us:

1. **Name** a computation, to refer to it later
2. **Parameterize** it with inputs, so it works with different values
3. **Reuse** it — write once, call anywhere

Parameters: names for the inputs a function expects. When we bind *parameters*, we are associating names with argument values.

Functions enable **abstraction**: calling `fibonacci(10)` hides *how* Fibonacci is computed behind a simple name.

How Function Calls Work

Evaluating `add(3, 4)` triggers a fixed sequence:

1. **Look up** the function by name

2. **Evaluate** the arguments (`3` , `4`)
3. **Create** a new frame for local variables
4. **Bind** parameters to arguments (`a = 3` , `b = 4`)
5. **Execute** the function body
6. **Return** the result and pop the frame

The Call Stack

Call Stack: Runtime data structure tracking function calls. Each call pushes a frame; return pops it.

Each **frame** on the stack represents one in-progress function call. The stack **grows** on call and **shrinks** on return — last in, first out (LIFO).

One frame per call: two calls to `foo` each get their own `x`. Call 1's `x = 5` never interferes with call 2's `x = 10`, because they live in separate frames. This isolation is essential for recursion, where the same function appears on the stack multiple times, each with its own variables.

Control Flow

Programs need to **decide** ("if logged in, show dashboard") and **repeat** ("while items remain, sum their prices"). Without control flow, code only runs straight-line, top to bottom.

Conditionals: `if / else`

An `if` expression evaluates a condition and picks a branch:

1. **Evaluate** the condition (e.g. `x > 0`)
2. **Choose** a branch — `then` if true, `else` if false
3. **Return** whatever that branch produces

Loops: `while`

A `while` loop repeats its body while a condition holds true, using Rust's `loop` construct with a condition check each iteration:

After the body runs, execution returns to the top and re-checks the condition — this is what creates repetition.

Control Flow in Functions

- **Nested conditionals** — for when one condition isn't enough
- **`return` in loops** — exits the function immediately, even from deep inside a loop; useful for "search" patterns

Control Flow as Branching

Both `if` and `while` change the flow of execution, letting code:

- **Skip** code (untaken `if` branch)
- **Repeat** code (`while` body)
- **Exit early** (`return` inside a loop)

In compiled languages, these become branch instructions — the CPU jumps to different memory locations. (In Secondlang, `if` compiles to LLVM IR's `br` instruction.)

Recursion, Stacks, and Recursive Structure

Recursion is when a function calls itself. It seems paradoxical — how can a function call itself mid-execution? — but the call stack makes it work cleanly.

Analogy: Russian nesting dolls (matryoshka). Each doll contains a smaller version of itself, down to the smallest doll, which contains nothing.

The Key Insight

Every recursive function needs two parts:

Part	Purpose
Base case	Returns directly, no further recursion — the “smallest doll”
Recursive case	Calls itself on a smaller problem, then combines the result

- No base case → recursion never stops → **stack overflow**
- No recursive case → not actually recursion, just a regular function

Why Recursion Works

Three design decisions make it work:

1. **Functions are stored globally** — `factorial` can look itself up by name mid-call.
2. **Each call gets its own frame** — `factorial(3)` called from within `factorial(4)` has an `n` fully independent of the outer `n = 4`.
3. **Return values propagate correctly** — `factorial(1)` returns `1` → used by `factorial(2)` as `2 * 1 = 2` → and so on up the stack.

Using a single global `n` instead of per-call frames would break recursion: each call would overwrite the shared `n`.

Mutual recursion: functions can call each other recursively.

 **Stack overflow risk:** every recursive call consumes stack memory; very deep recursion exhausts it.

Some languages implement **tail call optimization** to run certain recursive functions in constant stack space — not implemented here, but a notable optimization.

When to Use Recursion

Recursion fits problems with recursive structure:

- **Trees** (each subtree is a smaller tree)
- **Mathematical sequences** (Fibonacci, factorial)
- **Divide-and-conquer algorithms** (merge sort, quicksort)

- **Parsing nested structures** (JSON, HTML, an AST)

For simple loops, iteration is usually clearer — but for inherently recursive problems, recursion is more natural.

Operators

An **operator** tells us what to do. Values and operands are the things we operate on.

Operator	Example	Explanation	Overloadable?
!	<code>ident!(...), ident!{...}, ident![...]</code>	Macro expansion	
!	<code>!expr</code>	Bitwise or logical complement	Not
!=	<code>expr != expr</code>	Nonequality comparison	PartialEq
%	<code>expr % expr</code>	Arithmetic remainder	Rem
%=	<code>var %= expr</code>	Arithmetic remainder and assignment	RemAssignment
&	<code>&expr, &mut expr</code>	Borrow	
&	<code>&type, &mut type, &'a type, &'a mut type</code>	Borrowed pointer type	
&	<code>expr & expr</code>	Bitwise AND	BitAnd
&=	<code>var &= expr</code>	Bitwise AND and assignment	BitAndAssignment
&&	<code>expr && expr</code>	Short-circuiting logical AND	
*	<code>expr * expr</code>	Arithmetic multiplication	Mul
*=	<code>var *= expr</code>	Arithmetic multiplication and assignment	MulAssignment
*	<code>*expr</code>	Dereference	Deref
*	<code>*const type, *mut type</code>	Raw pointer	
+	<code>trait + trait, 'a + trait</code>	Compound type constraint	
+	<code>expr + expr</code>	Arithmetic addition	Add
+=	<code>var += expr</code>	Arithmetic addition and assignment	AddAssignment
,	<code>expr, expr</code>	Argument and element separator	
-	<code>- expr</code>	Arithmetic negation	Neg
-	<code>expr - expr</code>	Arithmetic subtraction	Sub

<code>--</code>	<code>var -= expr</code>	Arithmetic subtraction and assignment	SubAssign
<code>-></code>	<code>fn(...) -> type, ... -> type</code>	Function and closure return type	
<code>.</code>	<code>expr.ident</code>	Field access	
<code>.</code>	<code>expr.ident(expr, ...)</code>	Method call	
<code>.</code>	<code>expr.0, expr.1, and so on</code>	Tuple indexing	
<code>..</code>	<code>.., expr.., ..expr, expr..expr</code>	Right-exclusive range literal	PartialOrd
<code>..=</code>	<code>..=expr, expr..=expr</code>	Right-inclusive range literal	PartialOrd
<code>..</code>	<code>..expr</code>	Struct literal update syntax	
<code>..</code>	<code>variant(x, ..), struct_type { x, .. }</code>	"And the rest" pattern binding	
<code>...</code>	<code>expr...expr</code>	(Deprecated, use <code>..=</code> instead) In a pattern: inclusive range pattern	
<code>/</code>	<code>expr / expr</code>	Arithmetic division	Div
<code>/=</code>	<code>var /= expr</code>	Arithmetic division and assignment	DivAssign
<code>:</code>	<code>pat: type, ident: type</code>	Constraints	
<code>:</code>	<code>ident: expr</code>	Struct field initializer	
<code>:</code>	<code>'a: loop {...}</code>	Loop label	
<code>;</code>	<code>expr;</code>	Statement and item terminator	
<code>;</code>	<code>[...; len]</code>	Part of fixed-size array syntax	
<code><<</code>	<code>expr << expr</code>	Left-shift	Shl
<code><<=</code>	<code>var <<= expr</code>	Left-shift and assignment	ShlAssign
<code><</code>	<code>expr < expr</code>	Less than comparison	PartialOrd
<code><=</code>	<code>expr <= expr</code>	Less than or equal to comparison	PartialOrd
<code>=</code>	<code>var = expr, ident = type</code>	Assignment/equivalence	
<code>==</code>	<code>expr == expr</code>	Equality comparison	PartialEq
<code>=></code>	<code>pat => expr</code>	Part of match arm syntax	

>	expr > expr	Greater than comparison	PartialOrd
>=	expr >= expr	Greater than or equal to comparison	PartialOrd
>>	expr >> expr	Right-shift	Shr
>>=	var >>= expr	Right-shift and assignment	ShrAssignment
@	ident @ pat	Pattern binding	
^	expr ^ expr	Bitwise exclusive OR	BitXor
^=	var ^= expr	Bitwise exclusive OR and assignment	BitXorAssignment
	pat pat	Pattern alternatives	
	expr expr	Bitwise OR	BitOr
=	var = expr	Bitwise OR and assignment	BitOrAssignment
	expr expr	Short-circuiting logical OR	
?	expr?	Error propagation	

State machine

If you pause a program in a debugger, you are looking at its current state. The call stack tells you how the program got to this state and what local data it currently has access to within that specific function frame. In computer science terms, adding a stack to a finite state machine turns it into a **Pushdown Automaton**—allowing it to handle nesting, recursion, and scoped memory.

- **Concise definition:** A mathematical model of computation representing a system that can be in exactly one of a finite number of conditions (states) at any given time. The machine changes from one state to another (a transition) in response to specific inputs or events.
- **Programming definition:** A system that maintains a “memory” of its current condition and changes that condition based on incoming events. Unlike a stateless system (like a basic calculator), a state machine’s output depends on its accumulated history — which in a programming language is tracked via variables, memory heaps, and the call stack.

Core concepts:

- **State** — the current condition or snapshot of the system’s memory (e.g., current variable values, active call stack).
- **Inputs/Events** — the actions or instructions fed into the system (e.g., executing the next line of code).
- **Transitions** — the rules that dictate how the system moves from one state to the next based on the input.

Function Call Sequence

For `add(3, 4)`:

1. **Look up** — Find the function value by name.
2. **Evaluate arguments** — Compute `3` and `4`.
3. **Push frame** — Create a fresh environment (scope) on the call stack.
4. **Bind parameters** — `a = 3`, `b = 4`.
5. **Execute body** — Run function instructions.
6. **Pop frame** — Return result to the caller; the frame below becomes current again.

Call stack analogy: A stack of sticky notes. Each call writes variables on a new note and puts it on top. Returning tears it off. This is why `inner()`'s variables don't overwrite `outer()`'s — they're on different notes.

Each **frame** is just a `HashMap` pushed and popped like any stack. Grammar grows, AST nodes multiply, but execution is still recursive tree traversal — now with scoped state.

The REPL

A **Read-Eval-Print Loop** provides an interactive environment:

1. **Read** — Get a line of input.
2. **Eval** — Parse and execute it.
3. **Print** — Show the result.
4. **Loop** — Return to step 1.

A working REPL with variables, functions, conditionals, operators, recursion, and a call stack means you have a **real programming language**. The culminating proof is recursive Fibonacci:

```
def fib(n) {  
  if (n < 2) { return n } else { return fib(n - 1) + fib(n - 2) }  
}  
fib(10)    # → 55
```

Multi-line input: a REPL needs to know when a statement is *incomplete* (e.g., mid function body) versus ready to evaluate. One approach: track **bracket depth** — count unmatched `{`, `(`, `[` (ignoring characters inside strings). While depth is positive, keep reading lines and show a continuation prompt (`...`); once it returns to zero, evaluate the accumulated input.

Types, Constraints, and Type Inference

Static vs. Dynamic Typing

Types act as **contracts** — when you write `a: int`, you promise `a` will always be an integer. The compiler enforces that promise.

🟡 **Type-system goal:** reject impossible or unsafe programs early while preserving useful expressiveness.

Approach	When Checked	Examples
Static	Compile time (before running)	Rust, C, Java, Haskell
Dynamic	Runtime (while running)	Python, JavaScript, Ruby

Trade-offs:

- **Static** — Catches bugs early, enables better performance, requires upfront annotations.
- **Dynamic** — More flexible, faster to prototype, but bugs can hide until runtime.

Types Enable Fast Code

When the compiler knows `x` and `y` are `i64`, it generates a **single CPU instruction** for `x * y`.

Without types, a dynamic interpreter must at runtime:

1. Check what type `x` is.
2. Look up the multiplication operation for that type.
3. Check operand compatibility.
4. Handle potential type errors.
5. Finally, multiply.

This overhead makes dynamically typed languages **10–100x slower** for numeric work. This is why JIT compilers like **V8** (JavaScript) and **PyPy** (Python) invest heavily in *type speculation* — guessing types to generate fast paths.

Type Inference

Inference means the compiler deduces types automatically — no annotations needed in most cases. Types **flow forward** from known sources (literals, annotated parameters) through operations into variables.

```
let x = 1 + 2    // x inferred as Int – no annotation needed
```

Type inference analogy: Like solving a crossword puzzle. Some squares have letters (explicit annotations); others are blank (`Unknown`). Constraints like “this is added to an int, so it must be int” fill in the blanks.

Key mechanisms:

- **Unification** — Checks if two types are compatible and finds a common type. Resolving `Unknown` with a concrete type is how the compiler *learns* what an unknown type should be.
- **Type Environment** — A `HashMap<String, Type>` mapping names to types. Also called symbol table or context

The environment is:

- **Extended** when we declare a variable or enter a function (adding new bindings)
- **Queried** when we reference a variable (looking up its type)
- **Scoped** — inner scopes can shadow outer bindings

Two-Pass Function Type Checking

Functions are trickier because we need to handle:

- **Parameters** (types come from annotations)
- **Local variables** (types are inferred)
- **Return value** (must match declared return type)

Functions can call each other (mutual recursion), requiring two passes:

1. **Pass 1 — Collect Signatures** — Scan all function definitions and record their type signatures *before* checking any bodies. This ensures `foo` can call `bar` even if `bar` is defined later.
2. **Pass 2 — Check Bodies** — Walk each function body, inferring and unifying types throughout.

Per-expression pattern: recursively type-check sub-expressions → apply typing rule → set type on this node.

Type inference works by:

1. Starting with **known types** — literals (`42` → `Int` , `true` → `Bool`) and annotated parameters.
2. **Flowing types** through expressions — operators, calls, assignments.
3. **Recording** types in the environment so variables can be looked up later.
4. **Unifying** types — checking compatibility and resolving `Unknown` .
5. **Reporting errors** when types conflict.

Type Inference Approaches

Approach	Description
Hindley-Milner	Infers polymorphic types like <code>fn identity<T>(x: T) -> T</code> without any annotations
Local Inference	Requires annotations at function boundaries; infers types <i>within</i> function bodies

The Typed (Decorated) AST

Secondlang's type checker doesn't just validate types — it produces a new tree where **every expression carries its inferred type**. This is called a **decorated** or **annotated** AST:

```
/// A typed expression: expression + its inferred type
pub struct TypedExpr {
  pub expr: Expr,
  pub ty: Type,
}
```

Nodes start as `Type::Unknown` and get filled in during inference. Grammar-wise, Secondlang barely changes from Firstlang — the expression rules (`Expr`, `Comparison`, `Additive`, ...) are identical. Only `Function`, `TypedParam`, `ReturnType`, `Type`, and `Assignment` gain optional type annotations; the real growth happens in the compiler, not the grammar.



Bytecode and Virtual Machines

Bytecode is another technique for translating source code toward machine code: it emulates the instruction set with a new, simplified, human-friendly encoding — an intermediate representation that sits between source and assembly, lower-level than source but higher-level than assembly language. It's executed by a **VM**.

Why Bytecode?

Approach	Trade-off
Direct AST interpretation	Simple to implement, but slow
Bytecode + VM	Moderate complexity, significantly faster, highly portable
JIT to native machine code	Fastest, but complex (LLVM dependency, platform-specific)

- **Simpler than JIT** — No LLVM dependency; works everywhere.
- **Faster than AST** — Bytecode is compact and cache-friendly.
- **Portable** — Same bytecode runs on any machine with your VM.

This is how Python (CPython), Java (JVM), and Ruby (YARV) work: compile source to bytecode once, run the bytecode interpreter wherever you need it.

Is Python (or a language X) Compiled or Interpreted?

Being AOT compiled, JIT compiled or interpreted is implementation-dependent. For example, the standard Python implementation is CPython which compiles a Python source code (in CPython VM) to CPython Bytecode (contents of .pyc) and interprets the Bytecode. However, another implementation of Python is PyPy which (more or less) compiles a Python source code (in PyPy VM) to PyPy Bytecode and JIT compiles the PyPy Bytecode to the Machine Code (and is usually faster than CPython interpreter).

Bytecode Structure

- `instructions` - A flat array of bytes. Opcodes and their arguments, all mixed together.
- `constants` - A table of literal values. Instead of encoding 42 in the instruction stream, we store it in the constants table and reference it by index.

```
pub struct Bytecode {
    pub instructions: Vec<u64>, // 64 bit instructions
    pub constants: Vec<Node>,
}
```

The Stack Machine

We've seen two ways to execute code: interpret the AST directly, or JIT compile to native machine code. There's a third approach that sits in between: **compile to bytecode, then interpret that**. Our VM is a **Stack Machine** — it keeps intermediate values on a stack.

```
const STACK_SIZE: usize = 2048;

pub struct VM {
    bytecode: Bytecode,
    stack: [Node; STACK_SIZE], // nodes are leafs in the abstract syntax tree
    stack_ptr: usize, // points to the next free space
}
```

```
pub enum OpCode {
    OpConstant(u16), // pointer to constant table
    OpPop,           // pop is needed for execution
    OpAdd,
    OpSub,
    OpPlus,
    OpMinus,
}
```

A **stack machine** has three components:

- **Bytecode** — the program to execute.
- A **stack array** supporting push/pop, used to hold intermediate values. A fixed-size array is used for speed.
- An **Instruction Pointer (IP)** and **Stack Pointer (SP)**, tracking which instruction is executing and what's next.

Why stacks? They handle *any* nesting automatically. For $(1 + 2) * (3 + 4)$:

- Push 1, push 2, add → stack: [3]
- Push 3, push 4, add → stack: [3, 7]
- Multiply → stack: [21]

Every operation pops its inputs and pushes its output. The stack naturally tracks what's "in progress."

Think of bytecode as a simplified assembly language designed for our VM.

- OpConstant(index) Push a constant onto the stack
- OpAdd Pop two values, push their sum
- OpSub Pop two values, push their difference
- OpPop Pop and discard the top value

Real assembly has hundreds of instructions.

VM execution of $1 + 2$ (fetch-decode-execute):

These four carry the arithmetic (OpSub not used in this example):

```
OpConstant(1) → push 1           [stack: 1  ]
OpConstant(2) → push 2           [stack: 1, 2]
OpAdd         → pop 2, pop 1, push 3 [stack: 3  ]
OpPop         → return 3
```

The VM reads instructions left-to-right with the **Instruction Pointer** (`ip`):

1. **Fetch** — Read next byte from `bytecode.instructions[ip]`
2. **Decode** — Match on the opcode
3. **Execute** — Manipulate the stack
4. **Repeat** — Increment IP; continue until out of instructions

Runtime Architecture Through Rust-Hosted Languages

 **Library lens:** Rust-hosted language designs expose arenas, handles, call frames, roots, allocation, and host/runtime boundaries. The crate shape matters only because it makes those runtime mechanisms inspectable.

This section draws from [Writing Interpreters in Rust: a Guide](#). Its central problem is deeper than parsing: how can a Rust host safely implement a dynamic guest language whose objects, aliases, and garbage-collection rules are not known to Rust's borrow checker?

 **Host/guest boundary:** Rust owns the runtime implementation; the guest language defines the behavior of values that live inside it.

1. Host Language Versus Guest Language

- The **host language** implements the runtime. Here, the host is Rust.
- The **guest language** is parsed, compiled, and executed by that runtime.
- The **mutator** is ordinary guest-program execution that allocates and changes objects.
- The **collector** traces, moves, marks, or reclaims guest objects.

Rust statically enforces Rust ownership. It cannot automatically prove the guest language's runtime ownership model.

```
Rust compiler proves host-level invariants
↓
runtime abstraction validates guest references
↓
guest program manipulates dynamic objects
```

This means runtime authors must design a safe public abstraction around a small, carefully reviewed unsafe core.

2. Separate Runtime Layers

A useful architecture has explicit boundaries:


```

source text
  ↓ parser
guest syntax / AST
  ↓ compiler
bytecode + literal pools
  ↓ virtual machine
dynamic values + call frames
  ↓ allocation interface
heap objects + object headers
  ↓ collector
mark / trace / reclaim

```

Each layer should depend on the narrowest interface below it:

Layer	Should understand	Should not depend on
Parser	tokens, grammar, AST nodes	heap-block layout
Compiler	AST, constants, bytecode	raw-pointer manipulation
VM	instructions, values, handles	parser internals
Allocator	sizes, alignments, object kind	source-language grammar
Collector	roots and tracing metadata	syntax or source-level declarations

3. Allocation Is More Than Getting Bytes

A runtime allocation request must answer:

Allocation property	Question the runtime must answer
Size	How many bytes are required?
Alignment	At which address boundary may the object begin?
Type	Which runtime object layout is being allocated?
Tracing	Does it contain references to other managed objects?
Initialization	When do all fields become safe for the GC to observe?
Header lookup	How does a payload locate its metadata?
Movement	May collection change the object's address?
Failure	Does failure trigger GC, return an error, or terminate?

Conceptually:

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
enum RuntimeType {
    Text,
    Pair,
    Function,
    Bytecode,
}

#[derive(Clone, Copy, Debug)]
struct ObjectHeader {
    kind: RuntimeType,
    size_bytes: u32,
    marked: bool,
}
```

The real header layout is part of the runtime ABI. Changing it affects allocation, tracing, debugging, and possibly serialized heap images.

4. Bump Allocation Mental Model

A bump allocator keeps a cursor inside a block:

```

block start                                block end
| allocated objects | free space          |
|                   | ^                   |
|                   | cursor              |

```

To allocate:

1. round the cursor up to the required alignment;
2. check whether `aligned_cursor + size` fits;
3. reserve that range;
4. advance the cursor;
5. initialize the object before exposing it.

Bump allocation is fast because the common path is arithmetic plus a bounds check. It does not individually reclaim objects; a collector or arena reset handles reclamation.

Checked alignment helper:

```
fn align_up(offset: usize, alignment: usize) -> Option<usize> {
    if !alignment.is_power_of_two() {
        return None;
    }
    offset.checked_add(alignment - 1).map(|n| n & !(alignment - 1))
}

assert_eq!(align_up(13, 8), Some(16));
```

5. Runtime Values

A dynamically typed language binds type information to values at runtime:

```
#[derive(Clone, Debug)]
enum Value {
    Nil,
    Bool(bool),
    Int(i64),
    Float(f64),
    Text(GcRef),
    Function(FunctionId),
}

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct GcRef(u32);

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
struct FunctionId(u32);
```

This handle-based sketch keeps raw heap addresses out of most code. A real runtime may use direct pointers, indices, generations, tagged words, or a mixture.

Representation decision	Common choices
Small integers	Inline/tagged or boxed
Booleans and <code>nil</code>	Immediate singleton values or heap objects
Strings	Interned, uniquely allocated, or mixed
Object addresses	Stable or movable during collection
Equality	Identity, contents, or type-dependent
Stale handles	Generation counters, tables, or validation

6. Tagged Values and Tagged Pointers

Aligned pointers have low bits that are normally zero. Some runtimes use those bits as a small type tag:

```
aligned pointer: ppppppppppppp000
tagged value:    pppppppppppppttt
```

Possible tags might mean:

```
000 → heap object pointer
001 → small integer
010 → symbol/interned identifier
011 → special value
```

Tagged-value advantage	Tagged-value cost
Identify common types without reading an object header	Masking and shifting require carefully reviewed unsafe code
Keep small integers out of the heap	The inline integer range becomes smaller

Fit a complete dynamic value in one machine word	Available tag bits depend on guaranteed alignment
Reduce indirection for common values	Debuggers and sanitizers see encoded pointers
Improve locality for value stacks and register arrays	Portability and pointer-provenance rules need deliberate design

Keep the encoded representation private. Expose safe constructors and accessors that validate tags before decoding.

7. Object Headers Versus Inline Tags

Inline tags cannot represent unlimited types. A hybrid runtime may use:

```

value word
├── immediate integer/special value
├── directly tagged common object
└── generic object pointer → header contains full RuntimeType

```

The header is also a natural place for:

Header field	Purpose
Mark bits	Record tracing state
Size or size class	Locate boundaries and choose reclamation bin
Generation or age	Support generational collection
Forwarding information	Redirect references after movement
Immutable/frozen state	Enforce guest-language mutation rules
Identity hash	Preserve stable identity-based hashing

Every extra header field costs memory for every object, so header design is a workload trade-off.

8. Mutator Scopes and Rooting

The mutator and collector have conflicting views:

- normal execution wants stable access to individual objects;
- a moving collector may need exclusive access to the whole heap and must update every reference.

A safe design ensures they cannot run simultaneously.

```

enter mutator scope
  ↓ obtain scoped handles
parse / compile / execute / allocate
  ↓ all temporary borrows end
leave mutator scope
  ↓ collector may trace or move objects

```

Long-lived guest references should normally be represented as traceable roots or handles, not as unconstrained Rust references.

Root source	Why it keeps objects alive
VM value stack/register array	Contains currently executing values
Active call frames	Hold locals, closures, and return state
Globals and modules	Remain reachable by name
Interned symbols	Are retained by the runtime’s intern table
Host-visible native handles	Represent guest objects borrowed by native code
Pending exceptions	Carry values until handled
Executable compiler constants	Remain reachable from live functions

If one live reference is absent from the root graph, a collector may reclaim a reachable object.

9. Garbage-Collection Invariants

The exact collector can vary, but these invariants transfer:

GC invariant	Required property
Reachability	Every live object is reachable through traceable edges
Initialization	Traceable fields are initialized before GC can observe them
Layout knowledge	The collector knows each object’s layout or tracing procedure
Write barriers	Mutator writes preserve the collector’s required metadata
Movement	Every reference is updated when an object moves
Reclamation	No valid handle can reach reclaimed memory
Finalization	Objects cannot resurrect through an untracked reference
Collection failure	The heap remains in a defined state

Basic tracing shape:

```
roots
  ↓ mark
reachable object graph
  ↓ sweep unmarked
free space
```

Do not hold a normal Rust reference across an operation that may allocate if allocation can trigger a moving collection.

10. Register-Based Bytecode

The hosted-languages guide uses a register-oriented VM influenced by Lua. Each compiled function owns bytecode and literals, while call frames provide that function's registers.

```
#[derive(Clone, Copy, Debug)]
struct Register(u16);

#[derive(Clone, Copy, Debug)]
struct LiteralId(u16);

#[derive(Clone, Debug)]
enum Opcode {
    LoadLiteral {
        dst: Register,
        literal: LiteralId,
    },
    Move {
        dst: Register,
        src: Register,
    },
    Add {
        dst: Register,
        left: Register,
        right: Register,
    },
    Return {
        src: Register,
    },
}

struct Bytecode {
    code: Vec<Opcode>,
    literals: Vec<Value>,
    register_count: u16,
}
```

The compiler sees a construction interface:

- append an instruction;
- append/deduplicate a literal;
- reserve a register;
- patch a forward jump after its destination is known.

The VM sees an execution interface:

- fetch the instruction at the instruction pointer;
- advance or replace the instruction pointer;
- access frame registers;
- switch bytecode when calling or returning.

These are two views of the same data, much like ELF's linking and execution views.

11. Call Frames

A register VM frame may contain:

```
struct CallFrame {
    function: FunctionId,
    instruction_pointer: usize,
    registers: Vec<Value>,
    return_destination: Option<Register>,
}
```

Real implementations often store all registers in one contiguous VM stack and let each frame refer to a window. That avoids one `Vec` allocation per call.

Frame invariant	Required relationship
Instruction pointer	Refers to the frame’s own bytecode
Register index	Is below the function’s declared register count
Arguments	Occupy the calling convention’s expected registers
Return destination	Belongs to the caller and remains valid
Live values	Are visible to the garbage collector

12. Safe-Unsafe Boundary Checklist

Area	Review requirement
Isolation	Keep raw allocation and pointer arithmetic in one small module
Safety contract	State alignment, initialization, aliasing, and lifetime requirements
Tagged pointers	Never expose an encoded value as directly dereferenceable
Domain types	Use newtypes for IDs, sizes, offsets, handles, and registers
GC coordination	Make collection and mutation mutually exclusive
Edge tests	Cover zero, maximum, overflow, and alignment-edge allocations
Dynamic checks	Run Miri and sanitizers where applicable
Fuzzing	Fuzz bytecode verification and object decoding
Stable identity	Prefer handles when raw addresses may move
Allocation audit	Treat every allocation point as a possible collection point

13. Keep VM State Explicit and Split by Responsibility

The hosted-languages guide models one execution thread with distinct structures for call frames, register values, open upvalues, globals, and the current instruction stream. Keeping these roles separate prevents one overloaded “stack” from hiding different invariants.

```

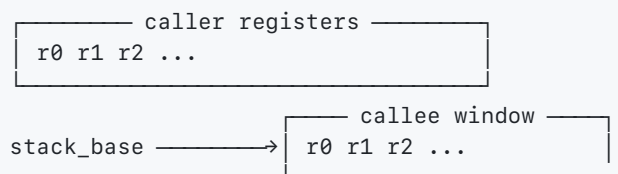
struct VmThread {
    frames: Vec<CallFrame>,
    registers: Vec<Value>,
    stack_base: usize,
    open_upvalues: Vec<UpvalueHandle>,
    globals: GlobalTable,
    current_code: FunctionId,
    instruction_pointer: usize,
}

```

State	Primary invariant
frames	top frame owns the active return context
registers	active window fits within the allocated register stack
stack_base	points to the first register of the active call
open_upvalues	each captured stack slot has shared intermediary state
globals	keys obey the language's name/environment policy
instruction location	belongs to the active function's bytecode

A register VM can reserve a fixed-width window for each call:

one contiguous value stack



This is still a stack of activation storage, even though bytecodes name registers instead of pushing and popping every operand.

Call and Return Are Coordinated State Transitions

call:

- validate callee and arity
- save caller function, return IP, base, and destination
- establish callee register window
- copy/place arguments
- switch current bytecode and IP

return:

- collect declared return values
- close upvalues owned by the departing window
- discard or invalidate callee registers
- restore caller frame, base, bytecode, and IP
- write results into caller destination(s)

Failure mode	Required defense
--------------	------------------

bytecode names register 250 of 8	verify register operands before execution
malformed return underflows frames	verifier plus runtime boundary check
captured local dies with frame	open/closed upvalue intermediary
collection misses a live register	trace active windows and saved frame values
native call re-enters the VM	document re-entrancy and rooting discipline

Bytecode verification can move many checks out of the dispatch loop, but only if unverified bytecode can never reach the fast path.

Language Semantics Through a Minimal Lua Interpreter

 **Library lens:** The small Lua implementation is a semantic microscope: each Rust type corresponds to a value representation, environment rule, call transition, or runtime invariant.

This section follows the incremental idea from [Build a Lua Interpreter in Rust](#): implement one tiny program through the complete pipeline, then grow the language without discarding the architecture.

 **Growth strategy:** make one tiny program work **end to end**, then expand one language feature and one test at a time.

1. A Small Program Contains Many Concepts

```
print "hello, world!"
```

Even this one line crosses the complete language pipeline:

Stage	Work required for <code>print "hello, world!"</code>
Tokenizer	Recognize an identifier and string literal
Parser	Build a function-call expression
Value model	Represent a dynamic string and callable value
Name resolution	Resolve the global name <code>print</code>
Compiler	Store the string constant and emit call instructions
Calling convention	Place the argument where the callee expects it
Native boundary	Invoke the standard-library implementation
VM	Fetch, decode, and execute the generated bytecode

This is a good vertical slice: tiny surface area, complete pipeline.

2. Define Bytecode Before Filling in the Pipeline

Bytecode becomes a contract between compiler and VM:

```
source
  ↓ lexer + parser + compiler
bytecode chunk
  ↓ virtual machine
observable result
```

For the tiny program:

```
#[derive(Clone, Copy, Debug)]
struct Register(u8);

#[derive(Clone, Copy, Debug)]
struct ConstantId(u16);

#[derive(Clone, Debug)]
enum Instruction {
    GetGlobal {
        dst: Register,
        name: ConstantId,
    },
    LoadConstant {
        dst: Register,
        constant: ConstantId,
    },
    Call {
        function: Register,
        argument_count: u8,
        result_count: u8,
    },
    Return,
}
```

Semantic trace:

GetGlobal r0, "print"	r0 = built-in print function
LoadConstant r1, "hello"	r1 = string value
Call r0, 1, 0	call r0 with one argument, no results
Return	finish the chunk

The exact bytecode is an implementation decision, not part of the Lua language specification. Another compatible Lua implementation may use different instructions.

3. Chunk, Constants, and Globals Are Different

```
struct Chunk {
    code: Vec<Instruction>,
    constants: Vec<Value>,
    register_count: u8,
}

type Globals = std::collections::HashMap<String, Value>;
```

- The **constant table** belongs to compiled code and is addressed by compact indexes.
- The **global table** belongs to execution state and is addressed by language-level names.

- Bytecode can first load a name from constants, then use that name to query globals.

Keeping these separate prevents compile-time representation from being confused with runtime state.

4. Dynamic Values as an Enum

```
type Builtin = fn(&[Value]) -> Result<Vec<Value>, RuntimeError>;

#[derive(Clone)]
enum Value {
    Nil,
    Bool(bool),
    Integer(i64),
    Float(f64),
    String(String),
    Builtin(Builtin),
}

#[derive(Debug)]
enum RuntimeError {
    UnknownGlobal(String),
    RegisterOutOfRange,
    ConstantOutOfRange,
    NotCallable,
    WrongArgumentCount,
}
```

The enum makes every operation explicitly handle the dynamic type. Later, heap-backed strings, tables, closures, and userdata may use reference-counted or garbage-collected handles instead of owned `String`.

5. Keep the First VM Loop Obvious

```
fn execute(chunk: &Chunk, globals: &Globals) -> Result<(), RuntimeError> {
    let mut registers = vec![Value::Nil; chunk.register_count as usize];
    let mut ip = 0;

    while let Some(instruction) = chunk.code.get(ip) {
        ip += 1;

        match instruction {
            Instruction::GetGlobal { dst, name } => {
                let name = match chunk.constants.get(name.0 as usize) {
                    Some(Value::String(name)) => name,
                    _ => return Err(RuntimeError::UnknownGlobal("<invalid name>".into())),
                };
                let value = globals
                    .get(name)
                    .cloned()
                    .ok_or_else(|| RuntimeError::UnknownGlobal(name.clone()))?;
                *registers
                    .get_mut(dst.0 as usize)
                    .ok_or(RuntimeError::RegisterOutOfRange)? = value;
            }
            Instruction::LoadConstant { dst, constant } => {
                let value = chunk
                    .constants
                    .get(constant.0 as usize)
                    .cloned()

```

```

        .ok_or(RuntimeError::ConstantOutOfRange)?;
    *registers
    .get_mut(dst.0 as usize)
    .ok_or(RuntimeError::RegisterOutOfRange)? = value;
}
Instruction::Call {
    function,
    argument_count,
    result_count,
} => {
    if *result_count != 0 {
        return Err(RuntimeError::WrongArgumentCount);
    }
    let function_index = function.0 as usize;
    let args_start = function_index + 1;
    let args_end = args_start + *argument_count as usize;
    let args = registers
        .get(args_start..args_end)
        .ok_or(RuntimeError::RegisterOutOfRange)?;

    match registers.get(function_index) {
        Some(Value::Builtin(function)) => {
            let results = function(args)?;
            if !results.is_empty() {
                return Err(RuntimeError::WrongArgumentCount);
            }
        }
        _ => return Err(RuntimeError::NotCallable),
    }
}
Instruction::Return => return Ok(()),
}
Ok(())
}

```

This version favors clarity over optimization. Before adding features, add a bytecode verifier so malformed instructions cannot index arbitrary registers or constants.

6. Module Layout That Can Grow

```

src/
├── main.rs           # command-line entry point
├── lexer.rs          # source → tokens
├── parser.rs         # tokens → syntax / emitted bytecode
├── bytecode.rs       # instructions, chunk, indexes
├── value.rs          # dynamic runtime values
├── vm.rs             # execution state and dispatch loop
├── builtins.rs       # print and later standard-library functions
└── error.rs          # lexical, parse, compile, runtime errors

```

Split by runtime domain and responsibility. Do not make every internal type public.

7. Lexer and Parser Contract

The tiny grammar can be:

```

program := statement* EOF
statement := IDENTIFIER STRING

```

Tokens:

```

#[derive(Debug, PartialEq)]
enum Token<'a> {
    Identifier(&'a str),
    StringLiteral(String),
    End,
}

```

Even at this stage, retain source spans:

```

#[derive(Clone, Copy, Debug)]
struct Span {
    start: usize,
    end: usize,
}

struct Spanned<T> {
    item: T,
    span: Span,
}

```

Spans let later compiler and runtime errors point back to the original source.

8. Grow by Preserving Invariants

Feature	Compiler/runtime addition
Multiple statements	Loop until end-of-file
Local variables	Scope table + register allocation
Assignment	L-value representation + store instruction
Arithmetic	Numeric coercion rules + arithmetic opcodes
Tables	Heap object + indexed/name lookup
Conditionals	Conditional jump + patching
Loops	Backward jumps + break targets
Functions	Prototypes, frames, call/return
Closures	Upvalues and open/closed capture state
Garbage collection	Roots, tracing, reclamation

A jump offset may begin as forward-only and later need to become signed when loops add backward edges. Representation choices should evolve when language semantics demand it.

9. Closures and Escaping Upvalues

An inner function may outlive the outer frame it references:

```
function make_counter()  
  local count = 0  
  return function()  
    count = count + 1  
    return count  
  end  
end
```

While the outer call is active, `count` can be an **open upvalue** referring to a stack slot. When that frame returns, the runtime must **close** the upvalue by moving or copying the captured value into heap-managed storage.

```
active outer frame:  
closure → open upvalue → stack slot  
  
after outer return:  
closure → closed upvalue → heap value
```

All closures sharing the same captured local must share the same upvalue cell, not independent copies.

10. Differential and Layered Testing

Test layer	Assertion
Lexer	Exact token and span sequence
Parser/compiler	Exact instructions and constants
Verifier	Malformed register, constant, and jump indexes are rejected
VM	Hand-built chunks execute independently of parsing
End-to-end	Source text produces expected output
Differential	Visible behavior matches reference Lua where intended
Snapshot	Bytecode listings and diagnostics remain readable/stable

When a test fails, ask which boundary first differs:

```
source → tokens → chunk → VM state → output
```

11. Lua Tables Combine Array and Hash Representations

Lua presents one table abstraction, but an implementation can split storage:

```
use std::collections::HashMap;

struct LuaTable {
    array: Vec<Value>,
    map: HashMap<ValueKey, Value>,
}
```

Key shape	Likely representation
dense positive integer sequence	array part
sparse integer	hash part
string or other permitted key	hash part
<code>nil</code> or invalid numeric key	reject according to semantics

The split is an optimization hidden behind one semantic interface. Reads and writes must behave consistently even when a key migrates between representations.

```
local key = "language"
local t = {
    10, 20, 30,      -- list-style fields
    name = "tiny-lua", -- record-style field
    [key] = "rust",   -- general expression key
}
```

Source form	Semantic operation
<code>t.name</code>	<code>t["name"]</code>
<code>t[i]</code>	evaluate <code>t</code> , evaluate <code>i</code> , then index
<code>t.x.y</code>	chained reads with intermediate values
<code>t.x = v</code>	l-value/indexed store, not a local assignment

Table objects have reference identity. Assigning a table value normally copies a reference, not all entries. That affects equality, hashing, mutation visibility, and garbage-collector edges.



Implementation rule: specify table semantics first; only then optimize dense integer keys, constant fields, inline caches, or hash layouts.

12. Control Flow Turns Labels into Patched Offsets

The compiler often emits a jump before it knows the destination:

```
fn patch_relative_jump(
    code: &mut [Instruction],
    jump_at: usize,
    target: usize,
) -> Result<(), CompileError> {
    // Relative branches are measured from the instruction *after* the jump.
    // Checked addition keeps a corrupt/oversized program from wrapping to zero.
    let next = jump_at
        .checked_add(1)
        .ok_or(CompileError::ProgramTooLarge)?;

    // Convert both indices before subtraction so a backward edge becomes negative.
    // A failed conversion means this program cannot fit the VM's offset model.
    let distance = isize::try_from(target)
        .ok()
        .and_then(|target| isize::try_from(next).ok().map(|next| target - next))
        .ok_or(CompileError::ProgramTooLarge)?;

    // The instruction encoder performs its own final width/range validation.
    code[jump_at].set_relative_offset(distance)?;
    Ok(())
}
```

if condition then body end

evaluate condition

JumpIfFalse ??? 

target: ←

patch ??? = target - next_instruction

Construct	Patch information to retain
if	false branch and optional end branch
while	exit branch plus backward loop edge
break	list of exits for the nearest loop
goto	label, scope depth, and unresolved jump
short-circuit op	true/false target and whether a value is required

Forward-only offsets stop being sufficient once loops add backward edges. Store signed, range-checked displacements and define whether they are relative to the current instruction or the following instruction.

Logical expressions have two compilation modes:

condition mode: branch directly to true/false targets

value mode: produce a boolean or operand value in a destination register

Compiling `a` and `b` directly as branches can avoid materializing a temporary boolean, but the result must still preserve Lua's value semantics when the expression is used as a value.

Object Models, Dispatch, and Classes

Why Classes?

Without classes, related data is **scattered**. This works, but causes problems:

- **No semantic grouping** — nothing says `x1` and `y1` belong together; they're just two independent integers.
- **Easy to mix up** — accidentally using `x1` with `y2` isn't caught by the compiler.
- **Can't pass as a unit** — you can't write `distance(p1, p2)`; you need `distance(x1, y1, x2, y2)`.
- **No encapsulation** — the distance formula is scattered across your code, so changing it means finding every place it was computed.

We need a way to group related data and attach behavior to it — that's what classes give us.

```
# Without classes — error-prone
distance(x1, y1, x2, y2)

# With classes — clear, grouped, safe
p1.distance(p2)
```

Classes analogy: Think of a filing cabinet. Without classes you have loose papers (`x1`, `y1`). Classes are folders that group related papers together — and know what operations to perform on them.

OOP Vocabulary

Concept	Description	Example
Class	Blueprint for creating objects	<code>class Point { ... }</code>
Object	An instance of a class	<code>p = new Point(1, 2)</code>
Field	Data stored in an object	<code>self.x</code> , <code>self.y</code>
Method	Function attached to a class	<code>def distance(self)</code>
Constructor	Initializes a new object	<code>def __init__(self)</code>
Destructor	Cleans up before deletion	<code>def __del__(self)</code>

Classes as Custom Types

Classes define **new types** in a **nominal type system** — types are identified by their names:

```
class Point { x: int y: int ... }
class Counter { count: int ... }

def move(p: Point, dx: int) -> Point { ... }
#      ^^^^^ Point is now a first-class type like int
```

Design Decisions

We implement a **subset** of OOP — deliberately simple:

No Inheritance — Many OOP languages support `class B extends class A`. We skip this because it adds significant complexity (vtables, dynamic dispatch), and composition over inheritance is often preferred anyway. The core concepts are clearer without it.

Explicit Memory Management — Instead of GC, we use explicit `delete` :

```
p = new Point(1, 2)
delete p    # programmer's responsibility
```

This mirrors C++ and teaches how memory actually works. Understanding manual management helps you appreciate what GC, reference counting, and ownership models solve.

What we include vs exclude:

Included	Excluded
Class definition with fields	Inheritance (vtables, dynamic dispatch)
Methods with <code>self</code>	Interfaces / Traits
Constructor <code>__init__</code>	Visibility (<code>public</code> / <code>private</code>)
Destructor <code>__del__</code>	Static methods
<code>new</code> / <code>delete</code>	Operator overloading
Field access <code>p.x</code> / <code>self.x</code>	
Method calls <code>p.method()</code>	
Classes as types <code>other: Point</code>	

Memory Model: Stack vs Heap

In most primitive/variable contexts, values live on the **stack**. Objects live on the **heap** because they must outlive the function that created them and can be shared across references.

	Stack	Heap
Management	Automatic (LIFO with call frames)	Manual (<code>new</code> / <code>delete</code>)
Speed	Fast (just move a pointer)	Slower (system call to OS)
Size	Limited (~few MB)	Large (all available RAM)
Lifetime	Tied to function scope	Until explicitly freed

Analogy: The stack is a stack of cafeteria trays — same size, add/remove from top only. The heap is a parking lot — park anywhere, leave as long as you want, but you must retrieve it or it stays forever (memory leak).

Constructors & new

The **constructor** (`__init__`) initializes a new object. It always takes `self` as the first parameter.

When you write `p = new Point(10, 20)` :

1. **Calculate size** — `Point` has two `i64` fields → 16 bytes.
2. **Call `malloc`** — Ask the OS for 16 bytes; get back a pointer.
3. **Initialize fields** — Zero-initialize all fields as a safety baseline.
4. **Call `__init__`** — Runs with the new pointer as `self` ; sets `self.x = 10` , `self.y = 20` .
5. **Return pointer** — `p` now holds the heap address of the object.

Memory layout:

```
class Point {
    x: int    # offset 0,  8 bytes (i64)
    y: int    # offset 8,  8 bytes (i64)
}            # total: 16 bytes

LLVM: %Point = type { i64, i64 }
```

Field order (tracked via `field_order` in `ClassInfo`) determines the memory layout — order matters!

Constructor Patterns

Default values:

```
class Config {
    value: int
    enabled: bool

    def __init__(self) {
        self.value = 42      # Default
        self.enabled = true  # Default
    }
}
```

Computed initialization:

```
class Square {
    side: int
    area: int

    def __init__(self, side: int) {
        self.side = side
        self.area = side * side    # Computed from input
    }
}
```

Validation (clamping, since we have no exceptions):

```
class PositiveInt {
    value: int

    def __init__(self, v: int) {
        if (v < 0) {
            self.value = 0    # Clamp to valid range
        } else {
            self.value = v
        }
    }
}
```

Failure: Our constructors cannot fail. Real languages handle this via exceptions (Java, Python), `Result / Option` (Rust), or factory methods. We keep it simple — constructors always succeed.

The `self` Parameter

`self` is the **receiver** for the object the method was called on. At the machine-code boundary it becomes an ordinary pointer or reference argument. Rust exposes the receiver explicitly and therefore makes the lowering easy to see:

```
struct Point {
    x: i64,
}

impl Point {
    fn get_x(&self) -> i64 {
        // `self` is a shared reference to the receiving object.
        self.x
    }
}

// A compiler can lower `point.get_x()` to an ordinary function whose first
// argument is the receiver. The exact symbol name is an implementation choice.
fn point_get_x(self_ref: &Point) -> i64 {
    self_ref.x
}
```

Language	Self/This
Python	<code>def method(self):</code> — explicit

Rust	<code>fn method(&self)</code> — explicit
Java / C++	<code>this</code> — implicit
Thirdlang	<code>def method(self)</code> — explicit

Explicit `self` makes it clear: **methods are just functions that receive the object as their first argument.**

In **codegen**, `self` is just stored as a local variable pointer, exactly like any other parameter — there's no special-casing:

```
Expr::SelfRef => {
    let ptr = self.variables.get("self").ok_or("'self' not in scope")?;
    self.builder.build_load(ptr_type, *ptr, "self").unwrap()
}
```

Methods & Field Access

Methods compile to regular functions with the naming convention `ClassName__methodName` — avoiding collisions between classes and making ownership clear.

Field access via `getelementptr` (GEP):

GEP calculates the memory address of a struct field without reading memory — it is pointer arithmetic that knows struct layouts:

```
; return self.x
%x_ptr = getelementptr %Point, ptr %self, i32 0, i32 0 ; pointer to field 0
%x      = load i64, ptr %x_ptr
ret i64 %x

; self.x = 42
%x_ptr = getelementptr %Point, ptr %self, i32 0, i32 0
store i64 42, ptr %x_ptr
```

Method call `p.get_x()` → `call @Point__get_x(ptr %p)` — the object is passed as the first argument.

Important: When you pass an object as a parameter, you pass a **pointer** — not a copy. Modifying `other.x` inside a method modifies the original object. This is **reference semantics**.

Method Patterns

Calling methods on `self` — methods can call other methods on the same object:

```
class Calculator {
    value: int

    def __init__(self) { self.value = 0 }

    def add(self, n: int) {
        self.value = self.value + n
    }

    def double(self) {
        self.add(self.value)    # Call another method on self
    }
}
```

Returning self's type — enables builder pattern:

```
class Builder {
    value: int

    def __init__(self) { self.value = 0 }

    def set_value(self, v: int) -> Builder {
        self.value = v
        return self    # Return the same object
    }
}

b = new Builder()
b.set_value(10)
```

Method naming convention (`ClassName__methodName` in LLVM):

Method	LLVM Function
<code>Point.__init__</code>	<code>@Point____init__</code>
<code>Point.get_x</code>	<code>@Point__get_x</code>
<code>Counter.increment</code>	<code>@Counter__increment</code>

This avoids name collisions between classes and lets the JIT find the right function at call sites.

Destructors & `delete`

The **destructor** (`__del__`) is called automatically when you `delete` an object:

```
delete p
```

Behind the scenes:

1. Call `Point__del(p)` — runs any cleanup code.
2. Call `free(p)` — returns memory to the OS.

```

; delete p
call void @Point__del(ptr %p)    ; destructor (if defined)
call void @free(ptr %p)          ; free heap memory

```

After `delete`, `p` still holds the old address — but accessing it is **undefined behavior**.

LLVM Codegen for Classes — Summary

Thirdlang	LLVM IR
<code>class Point { x: int }</code>	<code>%Point = type { i64 }</code>
<code>new Point(10)</code>	<code>call @malloc + call @Point____init__</code>
<code>p.x</code>	<code>getelementptr + load</code>
<code>p.x = 5</code>	<code>getelementptr + store</code>
<code>p.method()</code>	<code>call @Point__method(ptr %p)</code>
<code>delete p</code>	<code>call @Point____del__ + call @free</code>

Class compilation happens in six phases:

1. **Declare libc functions** — `malloc` and `free`.
2. **Create class struct types** — define an LLVM struct for each class.
3. **Declare methods** — create function signatures (enables cross-method and forward calls).
4. **Compile class bodies** — generate method implementations.
5. **Compile top-level code** — generate the `@__main` wrapper.
6. **Verify module** — check the IR is well-formed.

Complete Codegen Example

Tracing `Counter` from source to LLVM IR:

```

class Counter {
    count: int
    def __init__(self) { self.count = 0 }
    def increment(self) -> int {
        self.count = self.count + 1
        return self.count
    }
}
c = new Counter()
c.increment()

```

Generated IR:

```

%Counter = type { i64 }

define void @Counter____init__(ptr %self) {
entry:
    %count_ptr = getelementptr %Counter, ptr %self, i32 0, i32 0
    store i64 0, ptr %count_ptr
    ret void
}

define i64 @Counter__increment(ptr %self) {
entry:
    %count_ptr = getelementptr %Counter, ptr %self, i32 0, i32 0
    %count = load i64, ptr %count_ptr
    %new_count = add i64 %count, 1
    store i64 %new_count, ptr %count_ptr
    %result = load i64, ptr %count_ptr
    ret i64 %result
}

define i64 @__main() {
entry:
    %raw = call ptr @malloc(i64 8)
    call void @Counter____init__(ptr %raw)
    %result = call i64 @Counter__increment(ptr %raw)
    ret i64 %result
}

```

The code that runs isn't interpreted — it's real compiled machine code, the same as if you'd written it in C or Rust. Objects really live on the heap. Methods really jump to function addresses. When this calls `malloc`, it's calling the actual C `malloc`.

Performance considerations:

What We Do	What Real Compilers Add
Direct field access via GEP (fast)	Inline method calls when possible
Static method calls — no vtable lookup	Escape analysis (stack-allocate short-lived objects)
Objects laid out contiguously in memory	Field alignment optimization, dead field elimination

Memory Layout Trace

Tracing `new Point(10, 20)` followed by `delete p` at the address level:

new Point(10, 20) :

- `malloc(16)` → returns pointer `0x1000`
- `Point____init__(0x1000, 10, 20)` — sets `x=10` at offset 0, `y=20` at offset 8
- `p` holds `0x1000`

delete p :

- `Point____del__(0x1000)` — runs destructor cleanup
- `free(0x1000)` — returns memory to OS

3. `p` still holds `0x1000` — but it is **invalid** (dangling pointer)

Best Practices (Manual Memory)

1. Delete What You Allocate

Every `new` has a matching `delete`.

```
p = new Point(1, 2)
# ... use p ...
delete p
```

2. Keep One Clear Owner

Have one clear owner responsible for deletion:

```
def make_point() -> Point {
    return new Point(1, 2)  # Caller owns this
}
p = make_point()
delete p  # Caller is responsible
```

3. Clear or Invalidate After Release

Set the reference to null after deletion when the language supports it:

```
delete p
p = null  # Mark as invalid — prevents use-after-free
```

Memory Safety Bugs

Languages without GC (C, C++) require explicit `new` / `delete`, introducing high-risk bugs:

Bug	Description	Code
Memory Leak	Forgot <code>delete</code> ; memory consumed until exit	<code>p = new Point(1,2)</code> — never freed
Use After Free	Access after <code>delete</code> — undefined behavior	<code>delete p; p.x</code>
Double Free	<code>delete</code> twice — corrupts allocator	<code>delete p; delete p</code>
Dangling Pointer	Two refs to same object; one deletes it	<code>q = p; delete p; q.x</code>

```
// Memory Leak
def leak() {
    p = new Point(1, 2)
    // forgot delete p - memory lost until program exits
}

// Use After Free - undefined behavior
p = new Point(1, 2)
delete p
p.x          // BUG

// Double Free - undefined behavior
delete p
delete p      // BUG

// Dangling Pointer
q = p        // q and p point to same object
delete p
q.x          // BUG - q points to freed memory
```

Memory Management Approaches

Approach	Pros	Cons
Manual (C, C++)	Fast, predictable, teaches fundamentals	Error-prone
Garbage Collection (Java, Python)	Safe, convenient	Runtime overhead, GC pauses
Reference Counting (Swift, Python)	Predictable cleanup	Cycle leaks, overhead
Ownership (Rust)	Safe with no runtime cost	Complex ownership rules

Compiler Optimization: Preserving Meaning While Changing Form

Optimizations simplify the AST before code generation, producing faster output, cleaner debug trees, and less work for the backend. They are chained into a **pass pipeline** — order matters!

 **Optimization invariant:** the transformed program must preserve every behavior the language specification says is observable.

AST → [Constant Folding] → [Algebraic Simplification] → [Dead Code Elimination] → Optimized AST

Optimization	Description
Constant Folding	<code>1 + 2 → 3</code> at compile time
Algebraic Simplification	<code>x * 0 → 0</code> , strength reduction

Dead Code Elimination	Remove unreachable branches
Common Subexpression Elimination	Compute identical sub-expressions once
Loop Unrolling	Replace loops with repeated sequential code
Inlining	Substitute function body directly at call site
Tail Call Optimization	Convert tail recursion into a flat loop

Even when LLVM will optimize downstream, custom passes improve compile speed, debug output readability, and can exploit language-specific knowledge LLVM can't.

Intermediate Representation and Native Code with LLVM

LLVM is like a universal translator for CPUs. You speak LLVM IR; LLVM translates it to x86, ARM, WebAssembly — whatever you need. Write your compiler frontend once; LLVM gives you every platform for free.

 **Backend boundary:** your frontend must produce **valid, well-typed IR**; LLVM cannot repair incorrect language semantics or undefined assumptions.

LLVM IR is a universal, low-level assembly language not tied to any specific CPU. It is used by Rust, Swift, Julia, Kotlin/Native, and more. By targeting LLVM IR, you get world-class optimizations for free.

Core IR Mechanics

SSA (Static Single Assignment)

In LLVM IR, every variable is assigned **exactly once**. This makes optimizations like dead code elimination and constant propagation trivial — the compiler always knows exactly where each value was defined.

```
%1 = add i64 3, 4      ; %1 is assigned once
%2 = mul i64 %1, 2     ; %2 is assigned once
```

Mutability: Alloca / Load / Store

Because SSA values are immutable, mutable variables use stack slots:

```
%x.addr = alloca i64      ; reserve stack space
store i64 5, i64* %x.addr ; write (mutate)
%x = load i64, i64* %x.addr ; read
```

LLVM's `mem2reg` pass promotes these stack slots to fast registers automatically.

Why store parameters too? For `add(%a, %b)`, the pattern is: (1) `alloca` stack space for `%a.addr / %b.addr`, (2) `store` the incoming parameters into those slots, (3) `load` the values back, (4) `add i64` the loaded values, (5) `ret` the result. This looks wasteful — why not use `%a` and `%b` directly? Because in LLVM IR, SSA values like `%a` are immutable, but in most source languages variables *can* change. Storing every variable in a stack slot up front handles mutability uniformly, and `mem2reg` cleans it up afterward when a variable is never actually reassigned.

Basic Blocks & Branching

Conditionals require separate **basic blocks** (`entry`, `then`, `else`, `merge`). Each block ends with a **terminator** — either `ret` (return) or `br` (branch):

```
entry:
  %cmp = icmp sgt i64 %a, %b          ; signed greater than → i1 boolean
  br i1 %cmp, label %then, label %else

then:
  ...
  br label %merge

else:
  ...
  br label %merge

merge:
  ...
```

Phi Nodes (ϕ)

In SSA form, if a variable's value depends on which branch was taken, a **phi node** selects the correct value based on the incoming basic block:

```
merge:
  %x = phi i64 [ %x.then, %then ], [ %x.else, %else ]
```

"If we came from `%then`, use `%x.then`. If we came from `%else`, use `%x.else`." Phi nodes are the **only** way to merge values from different control flow paths in SSA.

Recursion

Recursive functions use the standard `call` instruction:

```
%result = call i64 @fib(i64 %n_minus_1)
```

Function Calls & Return Values

To compile a call: look up the function, compile each argument, then emit a `call` instruction. LLVM calls return either a **basic value** (an int or pointer you can use) or **nothing** (void). The pattern `try_as_basic_value().unwrap_basic()` extracts the basic-value case — safe here since our functions always return `int` — and `.into_int_value()` converts the result to the specific integer type needed downstream.

LLVM State

Component	Role
Context	Workspace for all LLVM objects
Module	Container for functions (one per compilation unit)
Builder	Inserts IR instructions into a basic block
variables map	Maps variable names → stack pointers (<code>alloca</code> results)
functions map	Maps function names → LLVM function objects
current_fn	The function being compiled (needed to create new basic blocks)

The Three-Pass Compilation Process

The Prestage stage:

- `Context::create()` - The top-level container. All LLVM objects belong to a context. It manages memory and ensures thread safety.
- `context.create_module("addition")` - Creates a module (like a compilation unit). Our `add` function will live here.
- `context.i32_type()` - Gets the 32-bit integer type. LLVM is explicitly typed - we need to declare that our function works with `i32`.

Pass 1 — Declare Functions

Announce all function signatures before compiling any body. This enables mutual recursion through forward references.

- `i32_type.fn_type(&i32_type.into(), i32_type.into(), false)` - Creates a function type: returns `i32`, takes two `i32` parameters. The `false` means it's not variadic (doesn't take variable arguments like `printf`).
- `module.add_function("add", fn_type, None)` - Adds a function called "add" with this signature to our module.
- `context.append_basic_block(add_fn, "entry")` - Creates a basic block named "entry". A basic block is a sequence of instructions with no branches in the middle - execution flows straight through.
- `context.create_builder()` - The builder is our "cursor" for adding instructions. We position it at a basic block, then build instructions there.
- `builder.position_at_end(entry)` - Point the builder at our entry block. New instructions will go here.

Pass 2 — Compile Bodies

Generate IR instructions for each function body using the `alloca/load/store` pattern.

- `add_fn.get_nth_param(0)` - Get the first parameter. LLVM functions have an array of parameters, indexed from 0.
- `.unwrap().into_int_value()` - Parameters come as generic "basic values." We know ours are integers, so we convert them.

- `builder.build_int_add(x, y, "result")` - This emits an `add` instruction. The `"result"` is just a name for the output (helps when reading IR).
- `builder.build_return(Some(&sum))` - Emit a `ret` instruction to return our sum.

Pass 3 — Create the `@__main` Wrapper

Wrap top-level expressions, such as `fib(10)`, in `__main` as the JIT entry point, then verify the module.

- `module.create_jit_execution_engine(OptimizationLevel::None)` - Creates a JIT compiler. LLVM takes our IR and compiles it to native x86/ARM code *right now*, in memory.
- `execution_engine.get_function::<unsafe extern "C" fn(i32, i32) -> i32>("add")` - Look up our compiled function. The type signature tells Rust how to call it.
- `add.call(1, 2)` - Call the native function! This jumps directly to machine code - no interpretation, no overhead.

Running a Program: Source to Result

What happens when you run `cargo run -- examples/fibonacci.sl`:

1. **Parse** the source file → Typed AST (with `Unknown` types).
2. **Type check** → Typed AST (all types resolved).
3. **Optimize** (optional) → Simplified AST.
4. **Compile** → LLVM IR.
5. **JIT** → Native machine code.
6. **Execute** → Result.

All in a fraction of a second. Each step transforms the program into a different representation, getting closer and closer to something the CPU can execute directly.

Calling the JIT-compiled code:

1. Create a code generator — sets up the LLVM context, module, and builder (the workspace).
2. Compile the program to IR.
3. Create a JIT execution engine from the verified module.
4. Get a pointer to `@__main` — our entry point.
5. Call it and return the result.

The JIT engine compiles IR to native machine code on the fly, then executes it — this is much faster than interpretation, because the CPU is running actual machine code, not walking a tree.

The `unsafe` block required when calling JIT output signals that we are invoking raw machine code — we must trust that our code generator produced valid IR.

LLVM Optimization Passes

After code generation, LLVM passes transform naive IR into efficient code. Passes are chained in a **pipeline string** (e.g., `"mem2reg,dce,instcombine,simplifcfg"`).

Why optimize ourselves if LLVM will do it anyway?

Reason	Benefit
Learning	Implementing optimizations teaches how production compilers actually work
Simplicity	A simpler AST means simpler, less error-prone code generation
Debug output	Optimized IR is easier to read when printed for debugging
Specialized optimizations	You may know language-specific facts LLVM cannot infer
Compile time	A simpler AST means less work for LLVM, so compilation is faster

Pass	What It Does
mem2reg	Promotes <code>alloca</code> stack slots to SSA registers — the most impactful pass
dce	Dead Code Elimination — removes instructions whose results are never used
instcombine	Merges redundant instructions; constant folds; strength-reduces (<code>mul x, 2</code> → <code>shl x, 1</code>)
simplifycfg	Removes empty blocks, merges single-predecessor blocks, simplifies trivial branches

Before/after example — `increment` method (14 → 4 instructions):

```
; BEFORE (naive codegen)
define i64 @Counter__increment(ptr %self) {
entry:
    %self1 = alloca ptr           ; alloca for parameter
    store ptr %self, ptr %self1   ; store param to stack
    %self2 = load ptr, ptr %self1  ; load self
    %field_ptr = getelementptr %Counter, ptr %self2, i32 0, i32 0
    %field = load i64, ptr %field_ptr ; load self.value
    %add = add i64 %field, 1
    %self3 = load ptr, ptr %self1   ; load self AGAIN
    %field_ptr4 = getelementptr %Counter, ptr %self3, i32 0, i32 0
    store i64 %add, ptr %field_ptr4
    %self5 = load ptr, ptr %self1   ; load self A THIRD TIME
    %field_ptr6 = getelementptr %Counter, ptr %self5, i32 0, i32 0
    %field7 = load i64, ptr %field_ptr6 ; load for return
    ret i64 %field7
}

; AFTER mem2reg → instcombine → dce
define i64 @Counter__increment(ptr %self) {
entry:
    %field = load i64, ptr %self           ; no alloca, GEP simplified away
    %add = add i64 %field, 1
    store i64 %add, ptr %self
    ret i64 %add                          ; return register directly
}
```

Each pass in isolation:

`mem2reg` — promotes an alloca'd variable straight to a value:

```
; before                                ; after
%x = alloca i64                          %val = 42
store i64 42, ptr %x
%val = load i64, ptr %x
```

`dce` — drops instructions whose results are never read:

```
; before                                ; after
%unused = add i64 %a, %b                  %result = mul i64 %c, %d
%result = mul i64 %c, %d                  ret i64 %result
ret i64 %result
```

`instcombine` — simplifies arithmetic patterns: `sub i64 %x, 1` → `add i64 %x, -1`; `mul i64 %x, 2` → `shl i64 %x, 1` (shift left); constant-folds `add i64 3, 4` → `7`.

`simplifycfg` — removes empty basic blocks, merges blocks with a single predecessor, and simplifies trivially-true/false branches.

LLVM preset pipelines:

Level	Description
<code>default<00></code>	No optimization (verification only)
<code>default<01></code>	Light optimization
<code>default<02></code>	Standard optimization (recommended)
<code>default<03></code>	Aggressive optimization

CLI usage:

```
thirdlang examples/point.tl                # Run without optimization
thirdlang -O examples/point.tl              # Run with optimization
thirdlang --passes "mem2reg,dce" examples/point.tl # Custom pass pipeline
thirdlang --ir examples/point.tl            # Print unoptimized IR
thirdlang --ir -O examples/point.tl          # Print optimized IR
thirdlang --passes "default<02>" examples/point.tl # LLVM O2 preset
```

Setting up the pass manager requires:

1. **Initialize Native Target** — required before creating a `TargetMachine`.
2. **Get Target Triple** — the host machine description (e.g., `x86_64-apple-darwin`).
3. **Create TargetMachine** — needed for target-specific optimizations.
4. `run_passes` — takes the comma-separated pass list (or a preset like `"default<02>"`) and applies it to the module.





Library lens: Inkwell is used as a typed view over LLVM concepts. The lesson is dominance, SSA, terminators, verification, ABI shape, and semantic equivalence—not memorizing builder calls.

The `tests/all` suite in Inkwell is useful as **executable API documentation**. Its files are organized around LLVM concepts—builders, blocks, contexts, modules, targets, types, values, passes, debug information, object files, and the execution engine—rather than around one large demo.

1. Read Upstream Tests as a Capability Map

Upstream test area	Question for your compiler
<code>test_types.rs</code> / <code>test_values.rs</code>	Are guest types represented exactly as intended?
<code>test_builder.rs</code> / <code>test_basic_block.rs</code>	Is every block well formed and terminated?
<code>test_module.rs</code>	Can modules verify, link, clone, and serialize?
<code>test_execution_engine.rs</code>	Does generated IR compute the correct result?
<code>test_targets.rs</code> / <code>test_object_file.rs</code>	Can the backend emit for the selected target?
<code>test_passes*.rs</code>	Does optimization preserve behavior?
<code>test_debug_info.rs</code>	Can source spans survive into debugger metadata?
<code>test_attributes.rs</code>	Are calling and optimization contracts attached?
<code>test_intrinsics.rs</code>	Are target-independent LLVM operations declared?
<code>test_memory_buffer.rs</code>	Can IR/bitcode enter through a byte-oriented API?

Inkwell's `tests/all/main.rs` declares these as modules so Cargo builds them into a single integration-test binary. It also uses `cfg` conditions for LLVM-version-sensitive tests. That is a strong model for a backend supporting several LLVM versions:

```
shared semantic contract
├─ always-on type, value, builder, and module tests
├─ target-dependent emission tests
└─ LLVM-version-gated pass and feature tests
```

2. Verify Every Constructed Module

LLVM IR is typed, but a sequence of individually valid builder calls can still produce an invalid function or module. Run the verifier immediately after the smallest useful construction.

```

use inkwell::context::Context;
use inkwell::module::Module;

fn build_add<'ctx>(context: &'ctx Context) -> Module<'ctx> {
    // Context owns LLVM's uniqued types; the module owns generated IR.
    let module = context.create_module("arithmetic");
    let builder = context.create_builder();
    let i64_type = context.i64_type();

    // Declare: i64 add_i64(i64, i64).
    let function_type = i64_type.fn_type(
        &[i64_type.into(), i64_type.into()],
        false,
    );
    let function = module.add_function("add_i64", function_type, None);

    // Every instruction must be inserted into a basic block.
    let entry = context.append_basic_block(function, "entry");
    builder.position_at_end(entry);

    // Parameters are generic LLVM values until converted to integer values.
    let lhs = function
        .get_nth_param(0)
        .expect("parameter 0 exists")
        .into_int_value();
    let rhs = function
        .get_nth_param(1)
        .expect("parameter 1 exists")
        .into_int_value();
    let sum = builder
        .build_int_add(lhs, rhs, "sum")
        .expect("builder is positioned in a block");

    // `ret` terminates the entry block and must match the declared return type.
    builder
        .build_return(Some(&sum))
        .expect("return type matches function type");

    // Builder calls can succeed while the combined module is still invalid.
    module.verify().expect("generated module must be valid");
    module
}

```



Backend invariant: each reachable basic block ends with exactly one terminator, every SSA use is dominated by its definition, and instruction/result types agree.

3. Test at Increasing Levels of Meaning

Test level	What it catches	Typical assertion
Builder result	Missing insertion point or invalid operation	<code>Result</code> is <code>Ok</code>
Function verify	Local control-flow/type defects	<code>function.verify(true)</code>
Module verify	Cross-function/module defects	<code>module.verify().is_ok()</code>
IR inspection	Names, structure, attributes, calling form	selected normalized IR fragments

JIT execution	Wrong semantics despite valid IR	returned value equals oracle
Object emission	Target/data-layout/relocation mistakes	object parses and symbols exist
Differential run	Optimization or backend disagreement	interpreter result equals JIT/AOT

Do not snapshot an entire LLVM module for every test. Full snapshots are noisy across LLVM versions. Prefer semantic execution tests plus focused IR assertions for features whose representation matters.

4. JIT Lookup Is an Unsafe ABI Boundary

`ExecutionEngine::get_function` is unsafe because Rust must promise that the requested function-pointer type matches the generated function's ABI and signature. The engine must also outlive the pointer.

```
use inkwell::OptimizationLevel;

type AddI64 = unsafe extern "C" fn(i64, i64) -> i64;

#[test]
fn generated_add_has_the_expected_semantics() {
    let context = Context::create();
    let module = build_add(&context);
    let engine = module
        .create_jit_execution_engine(OptimizationLevel::None)
        .expect("native JIT is available");

    // SAFETY:
    // - generated `add_i64` uses the C ABI;
    // - it accepts exactly two i64 values and returns one i64;
    // - `engine` remains alive while the JIT function is called.
    let add = unsafe {
        engine
            .get_function:::<AddI64>("add_i64")
            .expect("generated function exists")
    };

    // SAFETY: the lookup contract above establishes the callable signature.
    assert_eq!(unsafe { add.call(20, 22) }, 42);
}
```

Wrap this unsafe lookup behind a small, typed backend API. Do not let arbitrary strings and arbitrary Rust function-pointer types spread through the compiler.

5. Build a Compiler Compatibility Matrix

Dimension	Minimum matrix
LLVM version	each version/feature combination you claim
Optimization	unoptimized plus the production pass pipeline
Target	host plus every cross-target you distribute
Execution mode	interpreter/reference, JIT, and AOT where applicable
Numeric edges	zero, extrema, overflow policy, NaN where applicable

Control flow	both branch arms, loop zero/one/many iterations
Debug mode	verifier and debug-info checks enabled

For your own language, a high-value oracle is:

```
source
├── tree-walking interpreter → expected value
├── unoptimized LLVM JIT → actual value A
└── optimized LLVM JIT/AOT → actual value B

require expected == A == B
```

When those disagree, preserve the source, seed, IR before/after optimization, LLVM version, target triple, and data layout. That turns a mysterious codegen bug into a reproducible backend test.

 **Next:** debugging treats every compiler stage as a hypothesis with observable inputs, outputs, invariants, and failure evidence.

Debugging as Evidence-Guided Model Correction

Compiler Pipeline Debugging

Your language is a pipeline: `Source → Tokens → AST → Output`. When something breaks, find which stage produced the wrong output.

 Systematic approach:

- 1. **Reproduce** — Find the smallest input that triggers the bug.
- 2. **Isolate** — Which stage is producing wrong output?
- 3. **Inspect** — Print the data at that stage.
- 4. **Fix** — Change the code.
- 5. **Verify** — Re-run the test.

Symptom	Action
Parse error	Simplify input; inspect lexer tokens
Wrong result	Print the AST; check grammar operator precedence
Program crash	Add debug prints; run <code>cargo clippy</code>
Infinite loop	Add print statements inside the <code>eval</code> loop
Precedence wrong	Print AST and verify grammar rule order

Debugging tips:

- Test each feature in isolation — don't write 100 lines then debug.
- Use the REPL for quick experiments.
- Print the AST — structure reveals bugs that output doesn't.

- Check operator precedence by inspecting which node is the AST root.

Testing the Pipeline

Test at multiple levels:

Level	What to Test
Unit	Individual functions — parser output, type checker rules
Integration	Multiple components — parse + typecheck + codegen
End-to-end	Full programs from source to result
Snapshot	IR and error message output — catch regressions

Start with integration tests for full programs, then add unit tests for complex logic. Add a regression test for every bug fixed.

Reverse Engineering & Debugging Compiled Binaries

Everything above is about debugging *your own* compiler pipeline while you have the source. A different (and complementary) skill is understanding **someone else's already-compiled binary**, where you only have opcodes — this is the mirror image of everything in [Hardware Fundamentals](#): instead of going source → assembly → opcodes, you're going opcodes → assembly → (partial) understanding.

Static vs. dynamic analysis:

Approach	What it means	When it's used
Static analysis	Disassemble a binary <i>without running it</i> — read the opcodes as text	Auditing a binary before executing it (e.g. static inspection)
Dynamic analysis	Attach a debugger to a <i>running</i> process and observe it live	Understanding behavior that only appears at runtime
Decompiling	Attempt to reconstruct high-level source from disassembly	Best-effort only — decompilers guess at structure and can mislead

Disassembly always reflects what's actually executing; decompiling does not — it's a reconstruction, not a fact.

Debugger fundamentals (same core loop regardless of tool — gdb, LLDB, x64dbg, WinDbg):

- **Attach** — connect the debugger to a running process (or launch the process under it).
- **Breakpoint** — mark an address so execution pauses when the CPU reaches it. Breakpoints can be **code breakpoints** (pause on an instruction) or **memory/data breakpoints** (pause when a memory address is read or written).

- **Step** — advance execution one instruction (or line) at a time once paused.
 - **Step into** a `call` — follow execution into the called function.
 - **Step over** a `call` — run the whole function without descending into it, landing on the next instruction after it returns.
 - **Execute until return** — run until the current function's `ret`, useful for “bubbling up” from a low-level instruction to the higher-level function that led there.
- **Inspect** — read registers, the call stack, and memory (typically shown as hex + ASCII) while paused.

Reading assembly you didn't write: the general strategy is to establish *context* first — find a landmark (a known string, a known memory value, a function you already understand) — then work outward from it one instruction at a time. `mov`, `add`, `sub`, `cmp`, and `jmp / je / jne` (conditional branches) cover the majority of what you'll need to trace through simple control flow; not every instruction needs to be understood to follow the logic.

Why this connects back to compilers: a debugger is doing, by hand, exactly what your [interpreter's eval loop](#) and the [CPU's fetch-decode-execute cycle](#) do automatically — stepping through instructions one at a time and tracking state. Understanding one deepens the others.

Following Unfamiliar Call Chains

A function rarely acts alone — it calls other functions, which call others still. Visualizing this as a **function chain** makes a deep call stack easier to hold in your head:

```
handle_input() → validate() → normalize_data() → apply_rule() → write_result()
```

When stepping through a debugger, working from a low-level instruction *back up* through a chain like this — using **execute-until-return** repeatedly — is sometimes called **bubbling up**: you keep surfacing to the next caller until you reach the higher-level function that actually matters to you. It's the same recursive-structure idea as [the call stack](#), just observed from the outside instead of written from the inside.

A general four-step approach for making sense of *any* unfamiliar system's behavior — a program, an API, a bug report — mirrors this:

1. **Identify** what you're trying to understand or verify.
2. **Understand** what data or code path is actually involved.
3. **Locate** it — find the concrete function, variable, or address.
4. **Observe/verify** — confirm your understanding matches what actually happens.

Pointers & Shared Libraries

Two more building blocks that show up constantly once you're reading real systems code:

- **Pointers** — a variable that stores *the address of* another value rather than the value itself. `int *y = &x;` makes `y` point at `x`; dereferencing (`*y`) reads or writes through that address. Pointers are how languages like C/C++/Rust let code directly reference and manipulate memory rather than only working with copies.

- **Shared/dynamic libraries** (`.dll` on Windows, `.so` on Linux, `.dylib` on macOS) — compiled code that's loaded into a program *at runtime* rather than baked into the executable at compile time, so multiple programs can share one copy of common functionality (e.g. a UI toolkit) without each program bundling its own copy. This is the runtime cousin of the [Assembler/linker relationship](#) — a linker resolves *static* dependencies at build time, while a loader resolves *dynamic* ones each time the program starts.



System Design: Queues, Feedback, Failure, and Scale

[DesignDeck](#) presents system-design decisions as a connected set of trade-offs: traffic, data, consistency, caching, partitioning, messaging, and failure handling. This section translates several of those decisions into Rust-shaped interfaces.



Design principle: begin with **quantities, guarantees, and failure behavior**; only then choose components.

1. Start with Quantities and Guarantees

Before drawing services and databases, write down:

Design input	Questions to answer
Traffic	Peak reads and writes per second?
Payloads	Typical and maximum request/response sizes?
Storage	Daily growth and retention period?
Latency	Acceptable median and tail latency?
Reliability	Availability, recovery time, and recovery point targets?
Consistency	What does each operation guarantee?
Message behavior	Is loss, duplication, or reordering acceptable?
Security	Where are trust and authorization boundaries?
Dependency failure	What happens when a dependency is slow or unavailable?

Useful rough estimates:

```
bandwidth      ≈ requests/second × bytes/request
daily storage  ≈ writes/second × bytes/write × 86,400
concurrent work ≈ arrival rate × average service time
```

These are estimates, not capacity promises. Validate them with measurement and leave headroom for bursts, retries, replication, metadata, indexes, and uneven partitions.

2. Cache-Aside

In cache-aside, the application checks the cache, loads a miss from the source of truth, and then fills the cache. The cache does not independently query storage.

```
read:
  cache hit  → return
  cache miss → read database → populate cache → return

write:
  update database → invalidate or update cache
```

A small interface keeps policy separate from storage:

```
trait KeyValue<K, V> {
  type Error;

  // `None` is a normal miss; `Err` means the storage operation itself failed.
  fn get(&self, key: &K) -> Result<Option<V>, Self::Error>;
  // Taking owned values lets an implementation retain them after this call.
  fn put(&mut self, key: K, value: V) -> Result<(), Self::Error>;
}

fn cache_aside<K, V, E>(<
  cache: &mut impl KeyValue<K, V, Error = E>,
  source: &impl KeyValue<K, V, Error = E>,
  key: &K,
) -> Result<Option<V>, E>
where
  K: Clone,
  V: Clone,
{
  // A hit never consults the slower source of truth.
  if let Some(value) = cache.get(key)? {
    return Ok(Some(value));
  }

  // Preserve the distinction between "missing" and "storage failed."
  let Some(value) = source.get(key)? else {
    return Ok(None);
  };

  // Clone only at the ownership boundary: the cache retains one copy while
  // the caller receives the value loaded from the source.
  cache.put(key.clone(), value.clone())?;
  Ok(Some(value))
}
```

Cache decision	Question
Staleness	How long may an old value remain visible?
Negative caching	Should missing values be cached?
Stampede control	What protects a hot key during a miss?
Fill failure	Does cache failure fail the request or only reduce speed?
Write policy	Write-through, write-back, or invalidation?

Eviction	LRU, LFU, FIFO, or a domain-specific policy?
----------	--

The database remains the source of truth unless the design explicitly chooses a different consistency model.

3. Token-Bucket Rate Limiting

A token bucket allows controlled bursts while enforcing a long-term rate. Tokens refill over time up to a fixed capacity; each accepted operation spends tokens.

```
use std::time::Instant;

#[derive(Debug)]
struct TokenBucket {
    capacity: f64,
    available: f64,
    refill_per_second: f64,
    last_update: Instant,
}

impl TokenBucket {
    fn new(capacity: u32, refill_per_second: f64, now: Instant) -> Option<Self> {
        if capacity == 0 || !refill_per_second.is_finite() || refill_per_second <= 0.0 {
            return None;
        }

        Some(Self {
            capacity: f64::from(capacity),
            available: f64::from(capacity),
            refill_per_second,
            last_update: now,
        })
    }

    fn try_take(&mut self, amount: u32, now: Instant) -> bool {
        let elapsed = now.saturating_duration_since(self.last_update);
        self.available = (self.available + elapsed.as_secs_f64() * self.refill_per_second)
            .min(self.capacity);
        self.last_update = now;

        let amount = f64::from(amount);
        if amount > self.available {
            return false;
        }

        self.available -= amount;
        true
    }
}
```

Rate-limit decision	Choices to define
Identity	User, IP, token, tenant, endpoint, or resource
Scope	Per process or coordinated across replicas
Store failure	Fail open, fail closed, or use a local fallback
Over-limit action	Reject, queue, or degrade

Client feedback

Retry timing, remaining quota, and reset metadata

A leaky-bucket queue smooths output at a regular rate. It is useful when work can wait; a token bucket is useful when short bursts are legitimate.

4. Circuit Breakers, Timeouts, and Backoff

A circuit breaker prevents repeated calls to a dependency that is already failing:

```
use std::time::Instant;

#[derive(Clone, Copy, Debug)]
enum CircuitState {
    Closed { consecutive_failures: u32 },
    Open { retry_after: Instant },
    HalfOpen,
}
```

State transitions:

```
Closed --failure threshold--> Open
Open  --cooldown elapsed---> HalfOpen
HalfOpen --probe succeeds---> Closed
HalfOpen --probe fails-----> Open
```

Use the patterns together:

Resilience pattern	Responsibility
Timeout	Limit one attempt
Retry budget	Limit total extra work
Exponential backoff	Increase spacing between attempts
Jitter	Prevent synchronized retry storms
Circuit breaker	Stop calls during sustained dependency failure
Load shedding	Reject low-priority work before global collapse

Retry only operations that are safe to repeat, or attach an idempotency key. Never let every layer retry independently without a shared budget; multiplicative retries can turn a small outage into overload.

5. Delivery Is Not Processing

Networks and processes can fail between any two observable steps. Messages may be delayed, duplicated, dropped, or reordered.

Delivery model	What the consumer must expect
At most once	Loss is possible; duplicates are suppressed
At least once	Duplicates are possible; retry covers some losses

Effectively once

Deduplication makes repeated processing harmless

Exactly-once network delivery is not a general guarantee. Systems usually approach exactly-once **effects** through idempotency, durable deduplication, and atomic state changes.

```
use std::collections::HashSet;

#[derive(Clone, Copy, Debug, Eq, Hash, PartialEq)]
struct EventId(u128);

#[derive(Debug)]
struct Event {
    id: EventId,
    delta: i64,
}

fn apply_once(
    event: &Event,
    processed: &mut HashSet<EventId>,
    total: &mut i64,
) -> Result<bool, &'static str> {
    if processed.contains(&event.id) {
        return Ok(false);
    }

    let next = total.checked_add(event.delta).ok_or("total overflow")?;
    *total = next;
    processed.insert(event.id);
    Ok(true)
}
```

This in-memory example explains the invariant but is not crash safe. In a real system, the deduplication record and business update must be committed atomically in durable storage, or a crash can occur between them.

An **event log** retains ordered records for replay and multiple consumers. A **queue** typically distributes work so one consumer handles each message. Choose based on replay, fan-out, ordering, and retention needs.

6. Consistent Hashing and Partition Movement

Ordinary `hash(key) % node_count` remaps most keys when the number of nodes changes. Consistent hashing places keys and nodes in a hash space so adding or removing a node moves a smaller portion of the data.

```
hash ring:
0 — node A — node B — node C — max
   ↑ key x           ↑ key y
```

Virtual nodes give each physical node multiple positions, which can improve balance. Jump consistent hashing is another option when a compact computation is preferable to maintaining a ring.

Partition concern	Required decision
Hash stability	Is the mapping stable across versions and languages?

Hot keys	How are skewed workloads isolated or replicated?
Replication	How many owners does each partition have?
Membership	How are node changes proposed and observed?
Rebalancing	Can reads and writes continue while data moves?
Recovery	How are partially moved partitions detected and repaired?

If the key is a compiler module, bytecode package, or analysis artifact, stable partitioning can also support distributed builds and content-addressed caches.

7. Consistency Is Per Operation

Do not label an entire system merely “strong” or “eventual.” State what each operation guarantees:

Consistency guarantee	Meaning
Read-your-writes	A client observes its own completed writes
Monotonic reads	A client does not move backward to older state
Causal ordering	Causally related events are observed in order
Linearizable compare-and-set	The update behaves as one atomic real-time operation
Bounded staleness	Reads lag by no more than a defined time/version bound
Eventual convergence	Replicas agree once updates and failures stop

The CAP trade-off matters during a network partition: a distributed system cannot simultaneously guarantee both availability for every request and linearizable consistency across the partition. Different operations may make different choices.

8. System-Design Review Checklist

Area	Review criterion
Data ownership	Source of truth and derived copies are explicit
Remote calls	Every call has a timeout
Resource bounds	Queues, retries, bodies, fan-out, and concurrency are capped
Overload	Degraded and rejected behavior is defined
Idempotency	Repeated messages are safe or durably detected
Versioning	Wire formats and stored schemas carry versions
Deployment	Migration, rollback, and mixed-version behavior are defined
Observability	Saturation, errors, tail latency, and dropped work are measured
Privacy	Logs and cache keys exclude secrets and sensitive data
Failure testing	Slowness and partial failure are tested

Recovery	Recovery point and recovery time objectives are stated
Simplicity	Operators can explain the important failure modes

Data Models, APIs, Caches, and Distributed Consistency

A storage engine turns logical operations into bytes on media. A distributed system adds copies, messages, clocks, and partial failure. The useful question is never merely “SQL or NoSQL?” It is **which operations, invariants, access paths, failure behavior, and consistency guarantees are required?**

1. Database Families Optimize Different Access Patterns

Family	Natural model	Strength	Typical cost
relational	tables, rows, keys, constraints	joins, transactions, integrity	schema and query planning discipline
document	nested records such as BSON/JSON	aggregate-shaped reads and flexible fields	duplicated data and cross-document work
key-value	opaque value by key	predictable direct lookup	secondary relationships are application work
wide-column	sparse rows partitioned by key	large distributed write/read workloads	queries must follow partition design
graph	vertices, edges, properties	relationship traversal	partitioning and global traversal cost
time-series	timestamped measurements	range aggregation, retention, compression	specialized query shape
search index	tokens, postings, ranking	text search and relevance	usually not source of truth
object/blob store	named immutable-ish byte objects	durable large-file storage	limited transactional/query semantics
embedded	library and local file, often SQL/KV	simple deployment and local durability	one-machine concurrency boundaries

“NoSQL” is a broad label for several of these models, not a single consistency or schema policy. Document databases still have schemas—the schema may be enforced by the application, validation rules, or migration code rather than a traditional DDL.

2. Relational Design Connects Identity and Constraints

Concept	Meaning
primary key	stable row identity inside a relation
foreign key	declared reference to another relation

unique constraint	database-enforced non-duplication
index	auxiliary structure that accelerates selected access paths
transaction	group of operations with an atomicity boundary
isolation	which concurrent intermediate states can be observed
MVCC	keep versions so readers and writers can overlap under defined rules
migration	versioned transformation of schema and stored data

An index trades write amplification and space for faster lookup. A foreign key is not an index in every database. Query plans are algorithms over cardinality estimates; use the database's plan inspector rather than assuming a query is "obviously fast."

3. NULL, Missing, Empty, and Unknown Are Different

SQL `NULL` represents an absent/unknown marker and uses three-valued logic. `x = NULL` does not test nullness; SQL uses `IS NULL`. At an application boundary, distinguish:

State	Example meaning
missing field	client made no statement
explicit <code>null</code>	client asks to clear or declares unknown
empty string/list	present value with zero content
default	value inferred by policy
tombstone	record intentionally deleted

```
#[derive(Debug, Clone, PartialEq, Eq)]
enum FieldPatch<T> {
    // The request omitted the field, so preserve the stored value.
    Unchanged,
    // The request explicitly supplied null, so clear the stored value.
    Clear,
    // The request supplied a replacement.
    Set(T),
}

fn apply_patch<T>(slot: &mut Option<T>, patch: FieldPatch<T>) {
    match patch {
        FieldPatch::Unchanged => {},
        FieldPatch::Clear => *slot = None,
        FieldPatch::Set(value) => *slot = Some(value),
    }
}
```

This enum is also a language-design lesson: one nullable pointer cannot express every business state without conventions.

4. GraphQL Is an API Language, Not a Database

GraphQL defines a typed schema and lets a client request a response shape. Resolvers may read SQL, documents, caches, services, or generated values.

Benefit	Required discipline
client selects fields	depth/complexity limits
schema enables validation and tooling	authorization at every resolver/data edge
one request can traverse relationships	batching to avoid N+1 lookups
types document possible values	explicit nullability and error semantics
schema evolves additively	deprecation and usage measurement

REST, RPC, and GraphQL are API styles, not mutually exclusive transport protocols. GraphQL commonly travels over HTTP, while the database behind it remains an independent choice. The [GraphQL overview](#) explicitly describes it as a query language and runtime that is not tied to a storage engine.

5. Serialization Is a Versioned Boundary

Serialization maps typed state to bytes; deserialization attempts the inverse. It is also a **parser for untrusted input**.

Decision	Questions
framing	where does one value end and the next begin?
schema	which fields, types, tags, and defaults exist?
encoding	JSON, XML, CBOR, Protobuf, custom binary?
compatibility	can old readers skip new fields?
limits	maximum depth, size, collection count, and allocation?
canonicalization	is there exactly one byte representation for signing/hashing?
validation	which semantic invariants follow syntactic parsing?

```
#[derive(Debug, serde::Serialize, serde::Deserialize)]
struct WireMessage {
    version: u16,
    request_id: String,
    payload: Vec<u8>,
}

fn validate_message(message: WireMessage) -> Result<WireMessage, &'static str> {
    // Deserialization proves only that the syntax matched the Rust shape.
    if message.version != 1 {
        return Err("unsupported wire version");
    }
    if message.request_id.is_empty() || message.request_id.len() > 64 {
        return Err("invalid request ID length");
    }
    if message.payload.len() > 1024 * 1024 {
        return Err("payload exceeds one-megabyte policy");
    }
    Ok(message)
}
```

XML adds namespaces, attributes, mixed text, and entity-processing policy. Disable external entity resolution unless a tightly controlled use case requires it; bind values to a domain model instead of interpreting arbitrary object types.

6. UUIDs Are Identifiers, Not Authentication

A UUID is 128 bits with a version/variant structure. Different versions make different tradeoffs:

Kind	Useful property	Caution
random-style	decentralized generation	collisions are improbable, not impossible
time-ordered	index locality and rough order	may reveal timing; ordering is not trust
name-derived	deterministic for namespace/name	changes when either input changes

Use a current implementation of [RFC 9562](#). A UUID does not prove identity, authorization, secrecy, or unpredictability unless the specific version and generator provide the property—and even then it should not replace a cryptographic session token.

7. Cache Layers Trade Freshness for Time

Layer	Near	Typical contents	Invalidation signal
CPU cache	processor core	memory cache lines	coherence protocol
OS page cache	kernel/filesystem	file-backed pages	writes, reclaim
in-process	one service instance	parsed/computed values	TTL/version/event
Redis/network cache	several service instances	shared hot records	TTL/delete/event
HTTP browser/proxy	user or intermediary	responses	validators/directives

CDN edge	geographically near users	cacheable web/media objects	TTL/purge/versioned URL
----------	---------------------------	-----------------------------	-------------------------

Every cache needs:

- a **key** and namespace;
- a value plus version/expiry;
- a miss/loading policy;
- an invalidation or freshness model;
- stampede protection;
- size/eviction limits;
- observability for hits, misses, stale results, and origin load.

Redis provides typed structures—not only strings—including hashes, lists, sets, sorted sets, streams, and probabilistic structures. It can persist, but treating it as disposable cache or durable primary storage are different architectures. Review its [data types](#) and [persistence choices](#) before relying on recovery behavior.

8. HTTP Caching Is a Protocol Between Participants

Mechanism	Meaning
Cache-Control: max-age=N	response is fresh for a defined time
no-store	do not store this response
no-cache	storage may occur, but revalidate before reuse
private	shared caches must not store it
ETag / If-None-Match	validator for a selected representation
Last-Modified / If-Modified-Since	time-based validator
Vary	request headers that select different representations
304 Not Modified	reuse stored body after successful validation

Cache keys must include every representation selector described by policy. Personalized responses accidentally stored in shared caches are security incidents, not merely staleness bugs. See [RFC 9111](#) and the [HTTP caching guide](#).

9. CDNs Combine Reverse Proxying, Caching, and Routing

A CDN places edge servers closer to clients. On a cache hit, the edge answers without origin work; on a miss, it retrieves or forwards to the origin and may store a copy.

Benefit	New concern
lower latency	cache-key correctness
origin offload	purge/propagation delay
traffic absorption	provider and control-plane dependency

TLS termination near users	certificate/key lifecycle
geographic presence	data residency and logging

Versioned immutable asset names make invalidation simple: deploy a new URL instead of trying to retract an ambiguous old value. Cloudflare's [caching overview](#) provides a useful edge/origin mental model.

10. Consistency Is a Per-Operation Promise

Model	Reader may observe	Suitable when
strong / linearizable	latest completed write in one global order	locks, ownership, critical coordination
sequential	one order consistent with each client's program order	some replicated state machines
causal	causally related operations in order	collaboration and messaging
eventual	replicas converge after writes stop	caches, feeds, derived views
weak	few ordering/freshness promises	telemetry or best-effort data
read-your-writes	a client sees its own accepted updates	user-facing settings

"The database is eventually consistent" is too vague. Name the operation, object, failure case, and session guarantee. The [consistency patterns guide](#) is a useful comparison; verify the actual database configuration and API semantics.

11. Consensus Is Agreement Under Failure

A consensus algorithm lets a group agree on a sequence/value despite delays and some failures. A leader-based replicated log usually separates:

```
leader election → log replication → quorum acknowledgement → commit → state-machine apply
```

Term	Meaning
quorum	overlapping majority or configured voting set
term/epoch	logical leadership generation
commit index	highest entry known to meet the commit rule
state machine	deterministic application of committed commands
split brain	more than one side incorrectly acts authoritative
fencing token	monotonically increasing proof that rejects stale leaders

Consensus does not make a service automatically available, fast, or correct. The state machine can contain bugs; clients need retry/idempotency semantics; quorum loss may choose safety over progress.

12. Fault Tolerance Starts with a Failure Model

Failure	Design response
process crash	supervisor, durable checkpoint, restart budget
request timeout	deadline, cancellation, bounded retry
duplicate delivery	idempotency key and deduplication window
message loss	acknowledgement and redelivery policy
reordering	sequence/term numbers where order matters
network partition	explicit availability/consistency choice
slow dependency	bulkhead, circuit breaker, backpressure
corrupt data	checksum/signature, validation, replica repair
region loss	independent failure domains and tested recovery

Reliability is the probability a system meets its specified behavior over time. Availability is the fraction of time it can serve according to the contract. Durability concerns accepted data surviving. Do not merge all three into “uptime.”

13. Heartbeats Detect Silence, Not Death

A heartbeat periodically reports liveness or lease renewal. Missing heartbeats means “the observer did not receive timely evidence,” which could be a crashed peer, network delay, scheduler pause, overload, or clock issue.

```
use std::time::{Duration, Instant};

#[derive(Debug)]
struct Heartbeat {
    last_seen: Instant,
    generation: u64,
}

impl Heartbeat {
    fn is_suspect(&self, now: Instant, timeout: Duration) -> bool {
        // `saturating_duration_since` remains safe if the supplied clock sample
        // appears earlier; production code should use a monotonic clock source.
        now.saturating_duration_since(self.last_seen) > timeout
    }
}
```

Use several missed intervals, generation/lease numbers, and a **suspect** → **verify** → **failed** state transition. A heartbeat alone must not authorize destructive failover.

14. Event-Driven and Stream Processing

Concept	Meaning
event	immutable statement that something happened

command	request that something should happen
event loop	dispatches readiness/messages to handlers
broker/log	stores and distributes ordered records
stream processor	transforms unbounded event sequences
event time	time occurrence claims to have happened
processing time	time the processor handled it
watermark	estimate that earlier event-time data is mostly complete
window	finite grouping over an unbounded stream

Event-driven development decouples producers and consumers, but it introduces schema evolution, duplicates, ordering domains, backpressure, poison messages, and observability needs. “Exactly once” usually describes a scoped combination of broker, state store, and sink protocol—not a magic end-to-end guarantee.

15. Synchronous, Asynchronous, Futures, and Promises

Term	Meaning
synchronous	caller’s control flow waits for completion
asynchronous	operation can make progress while caller does other work
blocking	thread cannot do other work while waiting
nonblocking	operation returns without waiting for unavailable progress
future	value representing a computation that may complete later
promise	writable/completion side of a future in some ecosystems
task	schedulable async state machine

Rust’s `async fn` compiles into a state machine implementing `Future`; polling must not block. Cancellation often occurs by dropping the future, so partial-effect and cleanup contracts matter. Concurrency allows interleaving; parallelism means simultaneous execution.

16. Load Balancing Selects a Destination

Strategy	Good for	Watch for
round robin	similar healthy instances	unequal work
least connections	long-lived sessions	connections differ in cost
latency-aware	geographically/disparately performing backends	noisy feedback loops
consistent hash	affinity and partitioned caches	skew and rebalancing
weighted	mixed capacity or gradual rollout	stale weights

Health checking, connection draining, overload signals, retry budgets, and session affinity matter as much as the selection algorithm. A retry routed to another backend can duplicate a non-idempotent operation.

17. TDD and Reliability Tests Operate at Different Scales

Test-driven development uses a short feedback loop:

```
write a failing behavioral example → implement the smallest passing change
→ refactor while tests remain green
```

Test	Proves
unit	local transformation/invariant
property	behavior over generated input classes
integration	boundary adapters cooperate
contract	client/provider assumptions match
concurrency/model	allowed interleavings preserve invariants
fault-injection	timeouts, loss, retries, and crashes are handled
load/soak	behavior under volume and time
recovery	backups, failover, and replay actually restore service

Tests are evidence about specified cases, not proof of absence of defects. Keep clocks, randomness, storage, and network adapters injectable so unusual states become repeatable.

Database sources: [PostgreSQL tutorial](#), [PostgreSQL concurrency control](#), [MongoDB data modeling](#), and [MongoDB embedded documents](#).

Networking: Packets, Protocols, Ports, and Wire Formats

Networking is low-level I/O plus a shared language. A protocol has **syntax** (frame layout), **semantics** (what messages mean), and **state** (which messages are valid now). That makes protocol design closely related to compiler design.

 **Mental model:** a network protocol is a language spoken over an unreliable, partial-I/O boundary.

1. Layered Mental Model

Application protocol	your messages, grammar, state machine
Transport	TCP stream or UDP datagrams
Network	IP addressing and routing
Link	local network frames
Physical	electrical/radio/optical signals

An application usually reads and writes through a socket; the OS networking stack handles the lower layers.

--	--	--

Concept	TCP	UDP
Abstraction	Ordered byte stream	Individual datagrams
Delivery	Retransmitted while connection lives	Best effort
Boundaries	Not preserved	Preserved per datagram
Connection state	Yes	No transport connection
Typical use	Web, shells, databases, file transfer	DNS, telemetry, games, real-time media

TCP does **not** deliver “messages.” One `write` may be split across many reads, and several writes may arrive in one read. Your protocol must provide framing.

2. Common Framing Strategies

Strategy	Example	Trade-off
Fixed size	exactly 32 bytes	Simple, wastes space
Delimiter	line ending <code>\n</code>	Human-readable, escaping needed
Length prefix	<code>u32 length</code> + payload	Efficient, must bound length
Type-length-value	tag + length + value	Extensible
Self-describing	JSON/CBOR-like value	Flexible, more overhead

Example binary frame:

```

0               1               2               3
+-----+-----+-----+-----+
| version (u8) | kind (u8)   | payload length (u16, BE) |
+-----+-----+-----+-----+
| payload ...  |
+-----+-----+-----+-----+
```

```

#[derive(Debug)]
struct Frame<'a> {
    version: u8,
    kind: u8,
    payload: &'a [u8],
}

fn decode_frame(bytes: &[u8]) -> Result<Frame<'_, &'static str> {
    // Parse the fixed-size header before trusting any encoded length.
    let header = bytes.get(..4).ok_or("short header")?;
    let length = u16::from_be_bytes([header[2], header[3]]) as usize;

    // Apply the protocol budget before slicing or allocating.
    if length > 4096 {
        return Err("payload too large");
    }

    // Checked slicing rejects truncated input without raw pointer arithmetic.
    let payload = bytes.get(4..4 + length).ok_or("short payload")?;

    // This protocol defines exactly one frame per input buffer.
    if bytes.len() != 4 + length {
        return Err("trailing data");
    }
    Ok(Frame {
        version: header[0],
        kind: header[1],
        payload,
    })
}

```

3. Reading a Length-Prefixed TCP Frame

`read_exact` loops until the requested buffer is filled or an error occurs. It is appropriate for a fixed header and a validated payload length.

```

use std::io::{self, Read};
use std::net::TcpStream;

const MAX_FRAME: usize = 64 * 1024;

fn read_frame(stream: &mut TcpStream) -> io::Result<Vec<u8>> {
    let mut header = [0u8; 4];

    // TCP is a byte stream; `read_exact` may perform several underlying reads.
    stream.read_exact(&mut header)?;

    let length = u32::from_be_bytes(header) as usize;

    // Reject hostile lengths before allocating the payload buffer.
    if length > MAX_FRAME {
        return Err(io::Error::new(
            io::ErrorKind::InvalidData,
            "frame exceeds limit",
        ));
    }

    let mut payload = vec![0u8; length];

    // EOF before `length` bytes is a truncated frame.
    stream.read_exact(&mut payload)?;
    Ok(payload)
}

```

Real services should also use timeouts, connection limits, authentication when needed, and resource budgets. Portable code should not assume every operating system reports socket timeouts with exactly the same error kind.

4. A Safe Loopback Protocol Lab

Bind test services to loopback so they are not exposed to the local network:


```

use std::io::{self, Read, Write};
use std::net::{TcpListener, TcpStream};

fn handle(mut stream: TcpStream) -> io::Result<()> {
    let mut request = [0u8; 4];

    // The toy protocol uses one fixed four-byte request.
    stream.read_exact(&mut request)?;

    // Match bytes, not UTF-8 text, because the wire format is byte-oriented.
    let response = match &request {
        b"PING" => b"PONG",
        _ => b"NOPE",
    };

    // `write_all` handles partial writes for this small response.
    stream.write_all(response)?;
    Ok(())
}

fn main() -> io::Result<()> {
    // Port 0 asks the OS for an unused port; loopback avoids LAN exposure.
    let listener = TcpListener::bind("127.0.0.1:0"?);
    println!("listening on {}", listener.local_addr()?);

    for incoming in listener.incoming() {
        match incoming {
            Ok(stream) => {
                // Isolate per-connection errors from the accept loop.
                if let Err(error) = handle(stream) {
                    eprintln!("connection error: {error}");
                }
            }
            Err(error) => eprintln!("accept error: {error}"),
        }
    }
    Ok(())
}

```

Lab concept	What the example demonstrates
Endpoint	An address combined with a port
Server lifecycle	Listen, accept, read, write, close
Partial I/O	Use helpers that complete or report failure
Protocol semantics	Map a request to a defined response
Error isolation	One connection failure need not stop the listener

5. Socket Lifecycle and Client/Server Roles

[Beej's Guide to Network Programming](#) frames a socket as an operating-system handle used for network I/O. On Unix-like systems it is a file descriptor, so familiar resource and blocking-I/O ideas apply.

Typical TCP server:

```

resolve local address
  ↓
socket → bind → listen → accept
                        ↓
                        read/write
                        ↓
                        close

```

Typical TCP client:

```

resolve remote address
  ↓
socket → connect → read/write → close

```

Rust wraps the resource in `TcpStream`, `TcpListener`, and `UdpSocket`. Their destructors close the underlying socket when ownership ends, but protocol shutdown may still need an explicit `shutdown`.

```

use std::io::{self, Read, Write};
use std::net::{Shutdown, TcpStream};
use std::time::Duration;

fn request(address: &str, bytes: &[u8]) -> io::Result<Vec<u8>> {
    let mut stream = TcpStream::connect(address)?;

    // Bound both directions so a silent peer cannot block forever.
    stream.set_read_timeout(Some(Duration::from_secs(3)))?;
    stream.set_write_timeout(Some(Duration::from_secs(3)))?;

    // Send the complete request, then half-close to signal request EOF.
    stream.write_all(bytes)?;
    stream.shutdown(Shutdown::Write)?;

    // Cap the response even if the peer never sends an application delimiter.
    let mut response = Vec::new();
    stream.take(64 * 1024).read_to_end(&mut response)?;
    Ok(response)
}

```

`shutdown(Write)` sends an end-of-stream signal for the writing direction while leaving the reading direction open. This is useful only when the application protocol defines end-of-stream as the request boundary. Many protocols keep the connection open and use lengths or delimiters instead.

6. Names, Addresses, and Ports

An endpoint combines an IP address with a transport port:

```

IPv4 endpoint: 192.0.2.10:443
IPv6 endpoint: [2001:db8::10]:443

```

Network name/value	Role
IP address	Identifies an interface or routing destination

Port	Identifies a transport endpoint on a host
Host name	Resolves to one or more IPv4/IPv6 addresses
Client local port	Usually assigned ephemeral by the OS
Server port	Bound to a known local address/service port

Do not resolve a name and then use only the first address forever. Try the returned candidates according to policy:

```
use std::io;
use std::net::{TcpStream, ToSocketAddrs};
use std::time::Duration;

fn connect_any(host: &str, port: u16) -> io::Result<TcpStream> {
    // Resolution can produce several IPv4/IPv6 candidates.
    let addresses = (host, port).to_socket_addrs()?;
    let timeout = Duration::from_secs(3);
    let mut last_error = None;

    for address in addresses {
        // Try each candidate until one completes within the per-address timeout.
        match TcpStream::connect_timeout(&address, timeout) {
            Ok(stream) => return Ok(stream),
            Err(error) => last_error = Some(error),
        }
    }

    // Preserve the final connection error, or report an empty resolution result.
    Err(last_error.unwrap_or_else(|| {
        io::Error::new(io::ErrorKind::AddrNotAvailable, "name resolved to no addresses")
    }))
}
```

Resolution is not authentication. A successful DNS lookup says where to connect, not whether the peer is trusted. Secure protocols still authenticate the remote identity.

Rust Demonstration: Observe a Bounded Set of TCP Ports

A TCP port scanner is conceptually a repeated **connection-state experiment**:

```
target IP + candidate port
  ↓ attempt a TCP connection under a deadline
connected      → a listener completed the handshake
refused        → the path answered, but no listener accepted that endpoint
no response    → filtering, packet loss, routing failure, or a slow host
other OS error → preserve the actual observation
```

The last three states must not be collapsed into “closed.” A timeout does not prove why no response arrived.

```

use std::io;
use std::net::{IpAddr, SocketAddr, TcpStream};
use std::time::{Duration, Instant};

#[derive(Debug)]
enum PortState {
    // The operating system completed a TCP connection.
    Open,
    // A TCP reset or equivalent refusal came back.
    Refused,
    // The deadline expired; the cause is deliberately left unknown.
    NoResponse,
    // Preserve unexpected OS classifications instead of inventing a network meaning.
    Error(io::ErrorKind),
}

#[derive(Debug)]
struct PortObservation {
    endpoint: SocketAddr,
    state: PortState,
    elapsed: Duration,
}

fn observe_ports(
    target: IpAddr,
    ports: &[u16],
    timeout: Duration,
) -> Vec<PortObservation> {
    ports
        .iter()
        .copied()
        .map(|port| {
            let endpoint = SocketAddr::new(target, port);
            let started = Instant::now();

            let state = match TcpStream::connect_timeout(&endpoint, timeout) {
                Ok(stream) => {
                    // Dropping the owned stream closes this observation connection.
                    drop(stream);
                    PortState::Open
                }
                Err(error) if error.kind() == io::ErrorKind::ConnectionRefused => {
                    PortState::Refused
                }
                Err(error) if error.kind() == io::ErrorKind::TimedOut => {
                    PortState::NoResponse
                }
                Err(error) => PortState::Error(error.kind()),
            };

            PortObservation {
                endpoint,
                state,
                elapsed: started.elapsed(),
            }
        })
        .collect()
}

```

A harmless local experiment can inspect a short, explicit list:

```

use std::net::{IpAddr, Ipv4Addr};
use std::time::Duration;

let observations = observe_ports(
    IpAddr::V4(Ipv4Addr::LOCALHOST),
    &[22, 80, 443, 8_080],
    Duration::from_millis(250),
);

for observation in observations {
    println!("{observation:?}");
}

```

Use network observation only on systems you own or are authorized to assess. The bounded slice is intentional: it makes work, connection count, and audit scope visible.

Computer-science concept	What the code exposes
Cartesian product	A scan chooses endpoints from hosts × ports
State classification	Results are modeled as an enum rather than a Boolean
Time	A deadline turns unlimited waiting into a bounded observation
Evidence versus inference	<code>NoResponse</code> records evidence without claiming a cause
Ownership	Each successful <code>TcpStream</code> is closed when its value is dropped
Complexity	One host and (p) ports performs (O(p)) connection attempts
Backpressure	A concurrent version must cap in-flight sockets and result buffering

Parallel scanning changes latency, not the amount of work. Spawning one thread or task per possible port can exhaust file descriptors, ephemeral ports, scheduler capacity, or the target. A bounded worker pool or semaphore makes concurrency an explicit resource policy:

```

candidate queue
  ↓ at most N active workers
connect attempts with deadlines
  ↓ bounded result channel
collector

```

This is the same producer–consumer model used by crawlers, compilers with parallel work queues, and servers that limit simultaneous requests.

7. Encapsulation and Byte Order

Each layer wraps the data from the layer above:

```
[link header [IP header [TCP/UDP header [application frame]]]]
```

On receipt, the layers remove their respective headers in reverse order. This separation lets the application use the same socket interface across different link technologies.

Multi-byte integers need a specified wire order. Network protocols commonly use big-endian, traditionally called **network byte order**:

```
let request_id = 0x1234_5678u32;
let encoded = request_id.to_be_bytes();
assert_eq!(encoded, [0x12, 0x34, 0x56, 0x78]);

let decoded = u32::from_be_bytes(encoded);
assert_eq!(decoded, request_id);
```

Never transmit an in-memory Rust struct directly:

In-memory property	Wire-format problem
Padding	Bytes may be unstable or uninitialized
Alignment/layout	Can vary by target and compiler settings
Integer representation	May use the wrong byte order
Pointers/references	Have no meaning in another process
Enum representation	Is unstable without an explicit representation contract

Serialize each field deliberately, or use a documented format and implementation.

8. Partial I/O, EOF, and Blocking

A successful socket read or write may transfer fewer bytes than requested.

Operation/result	Meaning
<code>read</code> → <code>Ok(0)</code>	TCP end-of-stream
<code>read_exact</code>	Fill the buffer or report EOF/error
<code>write_all</code>	Continue until all bytes are accepted or an error occurs
Timeout	Bound waiting; does not create a message boundary

Blocking sockets put the calling thread to sleep until progress or an error is possible. Non-blocking sockets instead report that the operation would block. An event loop uses an OS readiness facility to learn which sockets may make progress.

blocking model:	one waiting thread per operation or connection
readiness model:	poll/select/epoll/kqueue → operate on ready sockets
completion model:	submit I/O → receive completion later
async Rust:	futures + reactor/executor hide much of this bookkeeping

Readiness is a hint, not a promise that an arbitrarily large operation will finish. Drain or fill only until the operation would block again, and keep per-connection buffers bounded.

For TCP framing, test all of these:

```
header split across reads
payload split across reads
header + payload + next frame in one read
peer closes halfway through a frame
declared length exceeds the limit
write accepts only a prefix
```

9. UDP Is Message-Oriented but Not Reliable

UDP preserves datagram boundaries, but an individual datagram may be lost, duplicated, or reordered. If reliability matters, the application protocol must define sequence numbers, acknowledgments, retransmission, deduplication, and congestion behavior—or use an existing transport that already does.

```
use std::io;
use std::net::UdpSocket;
use std::time::Duration;

fn udp_exchange(server: &str, request: &[u8]) -> io::Result<Vec<u8>> {
    let socket = UdpSocket::bind("127.0.0.1:0")?;
    socket.connect(server)?;
    socket.set_read_timeout(Some(Duration::from_secs(1)))?;
    socket.send(request)?;

    let mut buffer = vec![0u8; 1_200];
    let received = socket.recv(&mut buffer)?;
    buffer.truncate(received);
    Ok(buffer)
}
```

This example binds to loopback for a local lab. In a real UDP protocol:

UDP concern	Defensive design
Fragmentation	Keep datagrams within a conservative size
Authenticity	Authenticate before acting
Response matching	Use unpredictable request IDs
Amplification	Bound response size relative to request size
Retransmission	Model timeout/retry as finite protocol state

10. Protocol State Machines

A parser answers “is this frame shaped correctly?” A state machine answers “is this message valid now?”

```

enum Session {
    AwaitHello,
    Ready { user: String },
    Closed,
}

enum Message {
    Hello { user: String },
    Data(Vec<u8>),
    Goodbye,
}

fn transition(state: Session, message: Message) -> Result<Session, &'static str> {
    match (state, message) {
        (Session::AwaitHello, Message::Hello { user }) => {
            Ok(Session::Ready { user })
        }
        (Session::Ready { user }, Message::Data(_)) => {
            // Process data, then remain ready.
            Ok(Session::Ready { user })
        }
        (Session::Ready { .. }, Message::Goodbye) => Ok(Session::Closed),
        _ => Err("message is invalid in the current state"),
    }
}

```

A compiler is also a protocol state machine:

```
source → parsed → typed → lowered → emitted
```

11. Reverse Engineering an Unknown Protocol Safely

packet-analysis workflow generalizes well to your own local server, interoperability research, and authorized captures:

1. identify the transport and endpoints;
2. capture a baseline exchange;
3. change one controllable input;
4. compare packet lengths and changed byte regions;
5. look for magic values, counters, timestamps, strings, and length prefixes;
6. test byte order and compression hypotheses;
7. infer the session state machine;
8. write a decoder that rejects malformed input;
9. validate against fresh captures you did not use to form the hypothesis.

Observed clue	Plausible interpretation
First 2/4 bytes match remaining length	Length-prefixed framing
Many readable bytes	Text or lightly structured data
Tiny change affects most later bytes	Compression, encryption, or integrity data

Repeated fixed prefix	Magic value, version, or header
Monotonically changing field	Sequence number or timestamp
Same message differs each time	Nonce, timestamp, randomization, or encryption

Do not assume traffic is plaintext merely because a few bytes are readable. Do not attempt to defeat encryption or access systems without permission.

12. Designing a Protocol for Your Own Language

Suppose your language has a remote REPL. Separate framing, message syntax, and semantics:

```

Frame:
  version: u8
  message_kind: u8
  payload_length: u32 big-endian
  payload: bytes

Message kinds:
  1 = Evaluate(source_utf8)
  2 = Result(value)
  3 = Diagnostic(code, span, message)
  4 = Cancel(request_id)

```

Protocol area	Design requirement
Versioning	Include a version from the beginning
Integers	Define widths and byte order
Text	Specify UTF-8 validity and normalization
Resource limits	Cap frames, strings, collections, and nesting
Identifier domains	Use newtypes for request IDs and session IDs
Extensibility	Define optional/mandatory unknown-field behavior
Authentication	Authenticate before evaluating code
Sandboxing	Bound CPU, memory, and execution time
Diagnostics	Use structured fields, not scraped prose
Compatibility tests	Publish input bytes with expected decoded values

13. Networking Failure Modes

Failure mode	Safer design
Assume full <code>read / write</code>	Loop with <code>read_exact / write_all</code> or buffered state
Trust a length field	Validate against protocol and resource limits
Omit timeouts	Bound connect, read, write, and idle time

Recurse without a limit	Enforce structural depth
Treat TCP close as a frame	Use explicit framing semantics
Parse before trust checks	Place authentication/authorization deliberately
Trust client type tags	Validate every tag, offset, and enum value
Log entire payloads	Redact and cap diagnostic data
Use host endianness	Specify and encode a wire byte order
Change layouts silently	Version messages and define compatibility

Tests should fragment a frame at every possible byte boundary, combine several frames into one read, truncate every field, and exercise maximum permitted sizes.

14. Address Resolution Produces Candidates, Not One Answer

Beej's socket workflow begins with `getaddrinfo`: a host/service name can resolve to multiple address-family, socket-type, and protocol candidates. Robust clients try compatible candidates until one connects.

```

host + service + hints
  ↓ name/service resolution
candidate 1: IPv6 stream address — connect fails
candidate 2: IPv4 stream address — connect works } — stop
candidate 3: ...

```

Resolution concern	Design requirement
IPv4 and IPv6	avoid hard-coding one address family
multiple records	define candidate ordering and connection timeout
service names	separate human service input from numeric port
canonical/display name	do not treat reverse lookup as authentication
DNS change	cache with an explicit lifetime, not forever
untrusted hostname	cap length and log safely

In Rust, `ToSocketAddrs` is convenient, but the same mental model applies:

```

use std::io;
use std::net::{TcpStream, ToSocketAddrs};
use std::time::Duration;

fn connect_any(host: &str, port: u16) -> io::Result<TcpStream> {
    let timeout = Duration::from_secs(3);
    let addresses = (host, port).to_socket_addrs()?;
    let mut last_error = None;

    for address in addresses {
        match TcpStream::connect_timeout(&address, timeout) {
            Ok(stream) => return Ok(stream),
            Err(error) => last_error = Some(error),
        }
    }

    Err(last_error.unwrap_or_else(|| {
        io::Error::new(io::ErrorKind::AddrNotAvailable, "no addresses resolved")
    }))
}

```

A production client should usually apply one **overall deadline**, not a full timeout for an unbounded number of candidates.

15. Readiness Is Not Completion

`poll` / `select` -style APIs report that an operation can make progress without blocking; they do not promise that a whole logical message can be transferred.

```

socket becomes readable
  ↓ read some bytes
append to connection buffer
  ↓ zero or more complete frames?
parse bounded frames
  ↓ preserve incomplete suffix for the next readiness event

```

Readiness event	Possible interpretation
readable	data, orderly EOF, or an error is available
writable	at least some send-buffer capacity may exist
error/hangup	inspect the socket result; buffered data may remain

For a nonblocking connection, keep explicit state:

```

struct ConnectionState {
    read_buffer: Vec<u8>,
    write_buffer: Vec<u8>,
    written: usize,
    peer_closed_write: bool,
}

```

```
write_buffer[written..]
  ↓ write as much as accepted
advance `written`
  ↓ if incomplete, wait for writable again
compact/clear only after every byte is accepted
```

Bug	Consequence
discard suffix after partial send	truncated protocol message
treat <code>WouldBlock</code> as fatal	disconnect under ordinary load
parse before full frame exists	false malformed-input error
keep growing read buffer	memory exhaustion
ignore EOF with partial frame	ambiguous truncation
assume writable means peer is alive	delayed failure or lost application semantics

`shutdown` can close one direction while leaving the other usable. Protocols that rely on half-close must define whether EOF means “end of request,” “end of response,” or an unexpected truncation; explicit length framing is usually easier to validate.

Cryptography, Identity, Network Services, and Web Security

This chapter connects wire protocols to trust. The recurring rule is:

Parse and bound bytes first, authenticate the peer and message second, authorize the requested action third, and only then perform a side effect.

1. Client, Server, Proxy, and Load Balancer Are Roles

Role	Initiates/accepts	Main responsibility
client	initiates a request/connection	asks for a service
server	accepts and responds	owns a service contract
forward proxy	client connects to it	reaches destinations for clients
reverse proxy	appears to be the server	fronts one or more origins
load balancer	selects a healthy destination	distributes connections/requests
gateway	translates policy/protocol/domain boundary	mediates unlike systems
CDN edge	reverse proxy near users	caches/serves/forwards content

One process can play several roles. A browser is an HTTP client and may also accept local debugging connections; a reverse proxy is a server to the client and a client to the origin. The [proxy-server guide](#) compares forward and reverse placement.

2. Ports Identify Transport Endpoints

An IP address selects an interface/host route; a transport port helps select an endpoint within that network stack. A socket is identified by protocol and addressing state, often summarized for TCP by:

```
(source IP, source port, destination IP, destination port, transport protocol)
```

Ports are 16-bit numbers, not security identities. A familiar port suggests an application protocol but does not prove it; TLS, authentication, and application parsing determine what the peer is actually speaking.

Protocol	Typical role	Important distinction
IP	packet addressing/routing	best effort; no application message boundary
TCP	reliable ordered byte stream	no record boundaries
UDP	message-oriented datagrams	loss, duplication, and reordering possible
QUIC	secure multiplexed transport over UDP	application still defines messages
HTTP	request/response semantics	may run over TCP/TLS or QUIC
DNS	names and typed resource records	caching and authority are core
SSH	secure remote shell/tunneling protocol	not the same protocol as TLS
FTP	legacy file-transfer control/data channels	plaintext unless separately secured

3. HTTP Is a Typed Message Exchange

Simplified HTTP/1.1 shape:

```
GET /notes/42 HTTP/1.1
Host: example.test
Accept: application/json
If-None-Match: "revision-7"
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 31
Cache-Control: private, max-age=60
ETag: "revision-8"

{"id":42,"title":"Low level"}
```

The blank line ends headers. `Content-Length` counts body **octets**, not Unicode characters. HTTP/2 and HTTP/3 encode frames differently but retain HTTP method, status, field, representation, and caching semantics.

Header	Boundary question
<code>Host</code> / <code>:authority</code>	which virtual service is addressed?

Content-Type	how should body bytes be interpreted?
Content-Encoding	which representation coding must be decoded?
Accept	which response media types can the client process?
Authorization	what credential accompanies this request?
Cookie / Set-Cookie	which browser-managed state is sent or stored?
Origin	which web origin initiated the request?
Cache-Control	where/how long may the response be reused?
Forwarded	what proxy-origin information is asserted, and by whom?

Trust forwarding headers only from proxies you control and after stripping untrusted copies at the edge.

4. DNS Resolves Typed Names Through Delegation

```
application → stub resolver → recursive resolver cache
                                miss → root → TLD → authoritative server
```

Record	Purpose
A / AAAA	IPv4 / IPv6 address
CNAME	alias to another canonical name
NS	authoritative server delegation
MX	mail exchanger
TXT	arbitrary text used by several higher-level protocols
SRV	service location with port/priority/weight
CAA	certificate-authority issuance policy

A TTL bounds how long a cached record should be reused; it is not a propagation guarantee or lease on the whole Internet. A recursive resolver answers from cache or pursues referrals; an authoritative server answers from the zones it serves. DNSSEC authenticates signed DNS data and denial proofs, but it does not encrypt ordinary DNS queries. See [RFC 7719 DNS terminology](#).

5. FTP, FTPS, SFTP, and File APIs Are Not Synonyms

Name	Built on	Security note
FTP	separate TCP control and data channels	credentials/data are plaintext by default
FTPS	FTP protected with TLS	still retains FTP's channel/firewall complexity
SFTP	SSH file-transfer subsystem	not FTP over SSH

SCP	SSH-based copying protocol/tooling	different semantics from SFTP
HTTPS object API	HTTP/TLS	common for cloud/object storage

For new systems, prefer a maintained secure protocol with explicit authentication, integrity, limits, resumability, atomic destination behavior, and path containment.

6. VPN, Tunnel, and Shadowsocks Cover Different Traffic

Mechanism	Usually captures	Endpoint sees
application proxy	traffic from configured applications	proxy as network peer
SOCKS proxy	supported TCP/UDP destinations	proxy as relay
Shadowsocks	traffic explicitly sent through local secure split proxy	remote relay
VPN	selected IP routes or an interface	VPN gateway as routed hop
SSH tunnel	explicitly forwarded ports/connections	SSH server as relay

[Shadowsocks](#) describes itself as a secure split proxy loosely based on SOCKS5: local and remote components protect the relay leg, then the remote connects to the target. It is not automatically a device-wide VPN, anonymity system, or substitute for end-to-end TLS.

A tunnel changes routing and observation points. It does not make a malicious endpoint safe, and DNS or other traffic can escape if routing policy is incomplete.

7. Encoding, Compression, Encryption, and Obfuscation

Transformation	Goal	Key required?	Reversible?	Security property
encoding	represent data under a syntax	no	yes	none
decoding	recover represented data	no	yes	none
compression	remove redundancy	no	yes/lossy	none
encryption	confidentiality under a key	yes	yes with key	confidentiality; integrity only with authenticated mode
hashing	fixed-size one-way digest	no	not generally	integrity building block
MAC	authenticate bytes with	yes	verification	integrity/authenticity to key

	shared key			holders
digital signature	authenticate bytes with private/public keys	private to sign	verification	integrity, signer-key authenticity
obfuscation	make understanding harder	usually no	intentionally difficult	delay, not a trust boundary

Base64, hex, URL encoding, and character encodings are not encryption. Packing, minification, symbol stripping, control-flow flattening, and string hiding are forms of obfuscation, not permission checks.

For data that will be both compressed and encrypted, protocols generally **compress before encryption**, because good ciphertext should not retain compressible patterns. Compression of attacker-influenced data beside secrets can leak through output length, so security protocols need a threat-specific policy.

8. Symmetric and Asymmetric Cryptography Work Together

Family	Key relationship	Good at	Main operational risk
symmetric encryption	same secret protects/recovers	fast bulk confidentiality	safely sharing/rotating the secret
MAC	shared secret authenticates	fast message integrity	every verifier can also forge
asymmetric encryption/KEM	public/private pair	establishing a shared secret	key validation and algorithm misuse
digital signature	private signs; public verifies	software/document identity and integrity	private-key compromise

Real protocols are commonly **hybrid**: authenticate peers and establish a fresh shared secret using asymmetric techniques, then protect bulk records with an authenticated symmetric cipher. Use a maintained protocol/library and an AEAD construction; do not invent a cipher mode, nonce scheme, padding scheme, or certificate validator.

9. Signing Protects Exact Bytes and Context

A safe signing design defines:

```
domain label || version || algorithm identifiers || canonical metadata || payload hash
```

The domain label prevents a valid signature for one protocol from being replayed as a different kind of authorization. Canonical encoding prevents multiple byte representations of the “same” logical value. Verification must also check purpose, identity, time policy, revocation/status policy, and key trust—not only the math.

10. Certificates Bind Keys to Claims

A digital certificate is a signed structure binding a public key to names/attributes under an issuer and validity policy.


```
trust anchor
  signs intermediate CA certificate
  signs leaf/service certificate
  contains service public key and permitted names/usages
```

TLS certificate validation commonly checks:

- a chain to a trusted anchor;
- signatures and allowed algorithms;
- current validity interval;
- requested hostname/IP against subject alternative names;
- key usage and extended key usage;
- basic constraints/path limits;
- status/revocation policy when applicable.

Installing a certificate does not itself prove control of the corresponding private key; TLS proves possession during the handshake. A self-signed certificate can encrypt a connection but provides no external identity unless its fingerprint/key is trusted through another channel.

11. TLS and SSH Protect Different Application Boundaries

TLS provides a secure transport for protocols such as HTTP, database connections, and custom services. SSH defines secure remote login, channels, port forwarding, and file transfer.

Question	TLS	SSH
common identity	certificate name/key	host key plus user credential
common trust store	public/private CAs	<code>known_hosts</code> , enterprise CA, explicit key
application	layered beneath an application protocol	includes channel/subsystem model
first-use risk	PKI/domain validation policy	host-key verification/TOFU policy

Both require correct host identity verification. Disabling certificate or host-key checks turns active interception into expected behavior. See [TLS 1.3](#) and the [TLS-versus-SSH guide](#).

12. Authentication, Authorization, and API Keys

Step	Question
identification	which identity is being claimed?
authentication	what evidence supports that claim?
session establishment	how is repeated proof represented safely?
authorization	may this identity perform this action on this object now?
accounting/audit	what security-relevant decision occurred?

An API key is usually a bearer secret identifying an integration/project; it is not automatically an end-user identity. Store only a verifier/hash when possible, show the secret once, scope it, rotate it, rate-limit it, and keep it out of URLs and logs.

Authorization belongs at the object/action boundary. "User is logged in" does not imply "user may read record 42."

13. Password Storage Uses Slow Verification, Salt, and Policy

Passwords should be stored with a maintained **password-hashing function**, not reversible encryption or a fast general-purpose hash.

Component	Purpose
per-password salt	makes equal passwords produce different stored verifiers
memory-hard work factor	makes each offline guess expensive
pepper (optional)	separate server-held secret outside the database
algorithm/version fields	allow verification and future upgrades
rehash-on-login	migrate accepted old parameters to current policy
rate limiting/MFA	reduce online-guessing and credential-reuse damage

A salt is not secret and must be unique enough for each password. A pepper is secret and must be rotatable with an incident plan. Follow current [OWASP password-storage guidance](#) rather than copying fixed parameters forever.

Rust Demonstration: Hash and Verify a Password With Argon2id

 **Terminology correction:** authentication systems normally **hash passwords**; they do not encrypt them. Encryption is reversible with a key. Password verification should compare against a deliberately expensive, one-way verifier.

The current RustCrypto `argon2` password API demonstrates the full data flow:

```
password bytes + fresh random salt + cost parameters
  ↓ Argon2id performs memory-hard repeated mixing
derived verifier
  ↓ encoded with algorithm, version, parameters, and salt
PHC string stored in the database
```

The stored string contains enough public information to repeat verification. It does not contain the original password or a decryption key.

```

use argon2::{
    password_hash::{
        rand_core::OsRng,
        PasswordHash,
        PasswordHasher,
        PasswordVerifier,
        SaltString,
    },
    Argon2,
};

fn hash_password(
    password: &[u8],
) -> Result<String, argon2::password_hash::Error> {
    // The salt is public but must be fresh for each stored password.
    // OsRng obtains cryptographic randomness from the operating system.
    let salt = SaltString::generate(&mut OsRng);

    // The default RustCrypto context uses Argon2id and encodes its selected
    // version and cost parameters into the returned PHC string.
    let encoded = Argon2::default()
        .hash_password(password, &salt)?
        .to_string();

    // Store this encoded verifier, never `password`.
    Ok(encoded)
}

fn password_matches(
    candidate: &[u8],
    stored_phc: &str,
) -> Result<(), argon2::password_hash::Error> {
    // Parsing is separate from verification so malformed database state,
    // unsupported algorithms, and mismatches remain typed library results.
    let parsed = PasswordHash::new(stored_phc)?;

    // Verification reads the algorithm/version/parameters/salt from the PHC value,
    // recomputes the candidate verifier, and compares through the library.
    Argon2::default().verify_password(candidate, &parsed)
}

```

Example lifecycle:

```

let stored = hash_password(b"correct horse battery staple"?);

assert!(password_matches(
    b"correct horse battery staple",
    &stored,
).is_ok());
assert!(password_matches(b"wrong password", &stored).is_err());

```

Do not log either the password or the full authentication request. In a production service, use generic externally visible failure messages and separately rate-limit attempts.

PHC field	Why verification needs it
Algorithm identifier	Selects Argon2id rather than an unrelated function

Version	Reconstructs the algorithm definition used originally
Memory/time/parallelism costs	Repeats the intended amount of work
Salt	Makes equal passwords produce unrelated stored values
Derived verifier	Value against which the candidate is checked

The salt prevents attackers from amortizing one precomputed table across many accounts. The memory-hard cost makes each guess consume meaningful memory and time. Neither salt nor cost makes a weak password strong, so online rate limits, MFA, breach detection, and password screening remain separate controls.

Rehash-on-Login Is a Format Migration

Because the PHC string carries its parameters, old records remain verifiable after the service adopts stronger parameters:

```

parse stored PHC value
  ↓ verify using its recorded parameters
candidate accepted?
  ↓ yes
parameters below current policy?
  ↓ yes
generate a fresh salt and replace the record with a new PHC value

```

This is the same versioned-decoding pattern used for network messages, filesystem records, bytecode, and serialized ASTs: decode the old representation, validate it, then deliberately emit the current representation.

 **Boundary:** fixed cost parameters copied from an old example are not a permanent security policy. Benchmark current guidance on the deployment hardware, cap concurrent hashing work, and plan how parameter upgrades affect service capacity.

14. Sessions, Cookies, Tokens, and JWTs

Item	Meaning
session	server-recognized continuity of authenticated state
cookie	browser storage/attachment mechanism scoped by attributes
bearer token	possession grants the represented authority
refresh token	longer-lived credential used to obtain new access tokens
JWT	signed and/or encrypted claims container with defined registered claims

JWT is a **format**, not an authentication architecture. A signed JWT is normally readable; the signature prevents undetected modification but does not encrypt claims. Validate the allowed algorithm, signature, issuer, audience, time claims, token type, key selection, and application-specific authorization. See [RFC 7519](#).

For browser session cookies:

- use `Secure` so HTTPS is required;

- use `HttpOnly` to reduce script access;
- choose `SameSite` deliberately;
- set narrow `Path / Domain` ;
- rotate the session ID after authentication/privilege changes;
- enforce idle and absolute expiry server-side;
- revoke or version sensitive sessions.

Follow the [OWASP session-management guide](#).

15. CORS, CSRF, and XSS Solve Different Problems

Risk/control	What it concerns	Core defense
same-origin policy	browser script reading another origin	browser-enforced isolation
CORS	server opts selected origins into cross-origin reads	exact origin/method/header policy
CSRF	victim's browser sends authenticated unwanted request	SameSite, CSRF token, origin checks, safe methods
XSS	attacker-controlled content executes as trusted page script	contextual output encoding, safe DOM APIs, CSP

CORS is not authentication, and it does not stop non-browser clients from sending requests. CSRF primarily matters when credentials are attached automatically, such as cookies. XSS can often defeat CSRF tokens by acting inside the trusted origin, so preventing injection remains essential. Use the [OWASP CSRF guide](#), [OWASP XSS guide](#), and [MDN CORS guide](#).

16. Fingerprinting and Canvas Are Privacy Boundaries

Fingerprinting combines observable features—headers, screen properties, fonts, rendering differences, timing, device capabilities, and storage state—to correlate a browser/device. Canvas fingerprinting renders controlled content and hashes or analyzes the resulting pixels, whose small differences may reflect graphics and font stacks.

Defensive/product guidance:

- collect only what a documented purpose requires;
- prefer authenticated account/device keys over probabilistic fingerprints;
- obtain consent where required and define retention;
- treat a fingerprint as a noisy risk signal, not identity proof;
- give users recovery and appeal paths;
- avoid silently expanding the attributes collected.

17. Random Numbers Must Match the Domain

Need	Appropriate source
simulation/test	

reproducibility	explicitly seeded deterministic PRNG
randomized algorithm	fast PRNG with documented seed
session/token/key/nonce	operating-system cryptographic randomness through a maintained library
unique database ID	UUID/sequence according to collision/order policy

Never use timestamps, process IDs, ordinary hashes, or a non-cryptographic PRNG for secrets. Never reuse a nonce where the selected AEAD/signature scheme forbids it. Randomness tests cannot prove that a secret generator is unpredictable; entropy source, state protection, reseeding, and construction matter. NIST maintains current [cryptographic standards and random-bit-generation guidance](#).

Rust Demonstration: Reproducible and Secret Randomness Are Different APIs

Random-number generation is a state machine:

```
seed + current generator state
  ↓ transition/output function
sample + next generator state
```

The same seed reproduces a pseudorandom sequence for the same generator definition. That is useful for tests and simulations, but it is precisely why a public fixed seed cannot protect a token or cryptographic key.

The current `rand` API makes the generator choice visible:

```
use rand::rngs::SmallRng;
use rand::{RngExt, SeedableRng};

fn simulated_die_rolls(seed: u64, count: usize) -> Vec<u8> {
    // A fixed seed makes a failed simulation reproducible.
    // SmallRng is fast, but it is not a secret generator and its exact sequence
    // is not promised to remain portable across crate versions or platforms.
    let mut rng = SmallRng::seed_from_u64(seed);

    (0..count)
        .map(|_| rng.random_range(1..=6))
        .collect()
}

let first_run = simulated_die_rolls(42, 8);
let replay = simulated_die_rolls(42, 8);
assert_eq!(first_run, replay);
```

Modulo bias occurs when the raw generator range is not an exact multiple of the desired range. For a byte mapped with `% 6`, the 256 possible bytes cannot be divided evenly among six outcomes, so some results occur more often. A maintained range sampler uses a correct reduction strategy such as rejection sampling.

For secret material, request bytes from the operating system and handle failure:

```

use rand::rand_core::TryRng;
use rand::rngs::{SysError, SysRng};

fn secret_token_bytes() -> Result<[u8; 32], SysError> {
    let mut token = [0_u8; 32];

    // SysRng is a fallible interface to the operating system's preferred source.
    // The error remains explicit because silently falling back to a weak seed would
    // destroy the security claim.
    SysRng.try_fill_bytes(&mut token)?;

    // Encode these bytes before transporting them; do not print or log real tokens.
    Ok(token)
}

```

`SysRng` is the `rand` 0.10 name for the system source formerly exposed as `OsRng`. Password-hashing crates may re-export a compatible `OsRng` through their own `rand_core` dependency, as the Argon2 example above does. Follow the imports documented for the exact crate version rather than assuming names from a different version compose.

Property	Seeded simulation PRNG	Operating-system secret source
Reproducible	Yes, within the generator's stability contract	No
Failure	Construction can often be infallible	Entropy access can fail
Fast bulk sampling	Yes	Often seed a CSPRNG for bulk work
Appropriate for test replay	Yes	Usually no
Appropriate for keys/tokens	Only when securely seeded CSPRNG policy says so	Yes
Seed may be logged	Only for non-secret simulations	Never

Distribution Is Separate From Entropy

An RNG produces raw unpredictable or pseudorandom state. A **distribution** maps that state into the domain the program needs:

Desired sample	Distribution problem
Fair die	Uniform integers 1 through 6
Random array index	Uniform integers 0 through length minus one
Simulation noise	Often normal, exponential, or domain-specific
Weighted choice	Probability mass proportional to weights
Shuffle	Unbiased permutation, normally Fisher–Yates

This separation mirrors compilers:

```
raw entropy : distribution :: source bytes : parser
```

The left side supplies undifferentiated input; the right side interprets it under a mathematical contract.

A Token Also Needs an Encoding

Random bytes are not necessarily printable or safe in URLs. Encoding changes their representation, not their entropy:

```
32 random bytes
  ↓ base64url without padding
43 printable characters
```

Hex would use 64 characters for the same 256 bits. Do not “improve” randomness by hashing a timestamp or concatenating a process ID. Those values can be predictable even when their encoded output looks irregular.

18. Sandboxing Is Authority Reduction

A sandbox restricts what code can observe or mutate. Mechanisms include:

Mechanism	Restricts
process boundary	address space and crash containment
OS user/ACL	files, objects, services
capability handle	specific resource operations
syscall filter	kernel entry points
namespace	resource view
VM/hypervisor	guest machine boundary
language/Wasm runtime	instructions, memory, imports, fuel
broker process	privileged operations through validated messages

A sandbox is only as strong as its escape surface: syscalls, parsers, shared memory, GPU/driver interfaces, JITs, IPC brokers, and configuration. Start with no authority and grant narrow handles rather than passing ambient global access.



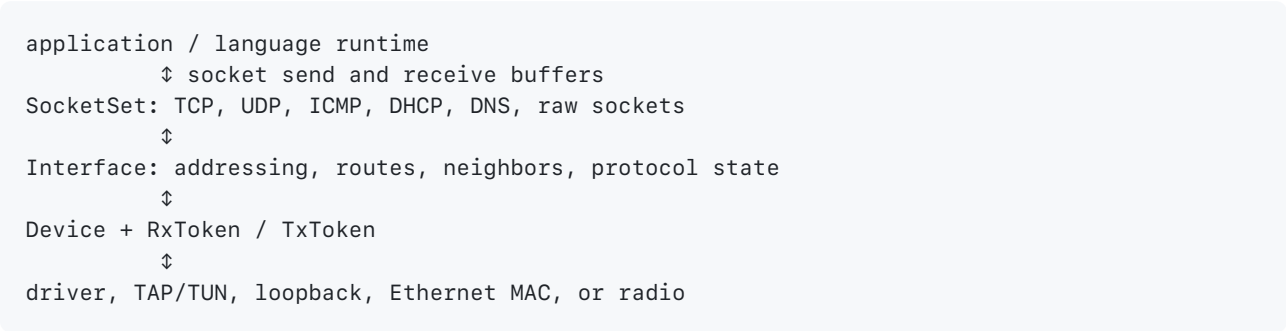
TCP/IP State Machines and Bounded Buffers Through smoltcp



Library lens: smoltcp removes the hosted socket illusion and exposes packet buffers, polling, timers, device tokens, socket sets, and TCP state. That makes protocol mechanics and resource bounds directly observable.

`smoltcp` is a standalone, event-driven TCP/IP stack designed for bare-metal and real-time systems. It can operate without heap allocation, making queues, socket buffers, timing, and packet work visible rather than hiding them behind an operating-system socket API.

1. Stack Architecture



Layer	Owns or describes
Device	frame receive/transmit capability, MTU, medium
Interface	IP addresses, routes, neighbor/protocol state
SocketSet	heterogeneous sockets identified by stable handles
socket	protocol state plus explicitly supplied buffers
application	when to poll, consume, produce, retry, and time out

This resembles a language runtime: the device is the host boundary, the interface is runtime-wide state, sockets are managed objects, and `poll` is the scheduler step.

2. Memory Is Part of the Network Contract

In hosted code, examples may provide `Vec<u8>` storage. In heapless code, the same buffer abstractions can borrow fixed slices.

```
use smoltcp::socket::tcp;

static mut RX_BYTES: [u8; 1024] = [0; 1024];
static mut TX_BYTES: [u8; 1024] = [0; 1024];

// In real embedded code, obtain unique mutable access during initialization.
// The surrounding platform code must ensure these buffers are never aliased.
let socket = unsafe {
    // SAFETY: initialization runs once before interrupts/tasks can access the buffers.
    let rx = tcp::SocketBuffer::new(&mut *core::ptr::addr_of_mut!(RX_BYTES));
    let tx = tcp::SocketBuffer::new(&mut *core::ptr::addr_of_mut!(TX_BYTES));
    tcp::Socket::new(rx, tx)
};
```

Prefer linker- or platform-provided single-init cells where available; the snippet makes the low-level ownership contract visible, not idealizes `static mut` as an everyday API.

Fixed resource	Behavior when exhausted

TCP receive buffer	advertised window shrinks; application must read
TCP transmit buffer	send capacity disappears until packets advance
UDP packet metadata/storage	datagram send/receive can fail
neighbor cache	entries are replaced or resolution is delayed
routes/addresses	insertion may fail at configured capacity
reassembly storage	oversized/excess fragmented packets are dropped



Invariant: buffer size is observable protocol behavior. Treat it as a deliberate capacity and backpressure decision, not an incidental allocation.

3. Polling Makes Scheduling Explicit

`Interface::poll(now, device, sockets)` transmits queued socket data and processes queued device packets. `poll_at` or `poll_delay` tells an event loop when protocol timers want attention.

```
use smoltcp::iface::{Interface, SocketSet};
use smoltcp::phy::Device;
use smoltcp::time::Instant;

fn drive_once(
    iface: &mut Interface,
    device: &mut impl Device,
    sockets: &mut SocketSet<'_,>,
    now: Instant,
) {
    let _changed = iface.poll(now, device, sockets);

    // Inspect socket readiness and do a bounded amount of application work here.
    // Then sleep/yield until the device is ready or iface.poll_at(...) is due.
}
```

Timing mistake	Consequence
Poll too late	retransmissions and quality of service suffer
Busy-poll continuously	wasted CPU and energy
Use a non-monotonic clock	broken timeout/retransmission reasoning
Drain unlimited ingress	other real-time work may starve
Ignore application backpressure	buffers fill and useful work is displaced

The current interface API warns that a full `poll()` may process an unbounded number of queued ingress packets. In a cooperative or real-time scheduler, use `poll_ingress_single`, `poll_egress`, and `poll_maintenance` to create explicit yield points and bounded work.

4. Socket State Is a Protocol State Machine

```

use smoltcp::iface::{SocketHandle, SocketSet};
use smoltcp::socket::tcp;

fn service_echo(
    sockets: &mut SocketSet<'_>,
    handle: SocketHandle,
) -> Result<(), tcp::ListenError> {
    let socket = sockets.get_mut::<tcp::Socket<'_>>(handle);

    if !socket.is_open() {
        socket.listen(7000)?;
    }

    if socket.can_recv() {
        let _ = socket.recv(|bytes| {
            let consumed = bytes.len();
            // Parse or copy only what the bounded application budget permits.
            (consumed, ())
        });
    }

    Ok(())
}

```

Readiness/state query	Meaning for application logic
<code>is_open()</code>	socket is not fully closed
<code>is_active()</code>	connection has active protocol state
<code>can_recv()</code>	receive would produce application data
<code>may_recv()</code>	peer may still send data in the future
<code>can_send()</code>	transmit buffer currently has capacity
<code>may_send()</code>	connection may still permit future sends

Do not equate `can_recv() == false` with EOF. Like a VM or coroutine, a socket can be temporarily unable to progress while remaining live.

5. A Networked Language Runtime

Runtime feature	<code>smoltcp</code> integration decision
remote REPL	frame parser, source limit, authentication, timeout
debugger protocol	request IDs, cancellation, bounded diagnostic output
package transfer	chunking, hash verification, storage budget
actor/message runtime	UDP loss policy or TCP backpressure
embedded web endpoint	HTTP parser limits and one/many-connection policy
telemetry	sampling, queue eviction, reconnect/backoff

Keep protocol parsing independent from the socket. Feed it byte slices and test it with fragmentation, coalescing, truncation, invalid lengths, and capacity exhaustion. Then test the stack in a loopback or TAP-based lab you control.

6. Test Faults, Not Only Happy Packets

The upstream hosted examples support packet dropping, corruption, size limits, rate limits, and PCAP capture. These are valuable language-runtime tests too:

Injected condition	Runtime property to verify
packet loss	retry/timeout does not duplicate semantic work
corruption	checksums and parsers reject damaged input
tiny MTU	fragmentation assumptions are not baked in
tiny socket buffer	backpressure reaches the producer
slow polling	deadlines fail predictably
connection reset	pending requests finish with structured errors

Run raw-interface and fault-injection experiments only on an isolated lab interface or network you are authorized to control.

➡ **Next:** QUIC keeps UDP's user-space flexibility while adding secure connections, reliable streams, loss recovery, and congestion control.

⚡ Reliable Multiplexed Transport Through QUIC and Quinn

🔍 **Library lens:** Quinn exposes QUIC's actual model: one encrypted connection, independent ordered streams, datagrams, congestion/loss feedback, flow control, and cancellation.

The [Quinn networking introduction](#) compares TCP, UDP, and QUIC, then motivates QUIC through latency and head-of-line blocking. Quinn is an async Rust implementation of QUIC.

1. Transport Comparison

Property	TCP	UDP	QUIC
connection	yes	no	yes
reliability/order	one reliable ordered byte stream	neither provided	reliable streams; optional datagrams
implementation	usually kernel	usually kernel primitive	mainly user space over UDP
security	separate TLS	application-defined	TLS integrated into the protocol
multiplexing	application protocol must	application-	multiple independent streams

	add it	defined	
migration	connection tied to address tuple	application-defined	connection IDs can support migration

QUIC is not “reliable UDP.” It is a connection protocol built over UDP with its own handshake, encryption, stream, loss-recovery, and congestion-control behavior.

2. Head-of-Line Blocking Has Layers

TCP exposes one ordered stream. If a packet containing earlier bytes is lost, later bytes cannot be delivered to the application even if they belong to an unrelated logical request.

```

TCP connection:
request A bytes  ┌
request B bytes ┤ — one ordered delivery sequence
request C bytes └

QUIC connection:
stream A — independent ordered sequence
stream B — independent ordered sequence
stream C — independent ordered sequence

```

Loss on one QUIC stream does not prevent complete data on another stream from becoming available to the application. Congestion control still couples traffic at the connection/path level, so “independent” does not mean resource-free.

3. Quinn’s Core Objects

```

Endpoint
├── Connection
│   ├── bidirectional stream: SendStream + RecvStream
│   ├── unidirectional SendStream
│   ├── unidirectional RecvStream
│   └── unreliable datagrams

```

Object/API	Responsibility
Endpoint	bind UDP socket, accept/connect, own QUIC state
Connection	secure peer session and shared transport state
open_bi / accept_bi	create/accept bidirectional streams
open_uni / accept_uni	create/accept one-way streams
datagram API	bounded unreliable messages
connection stats	observe loss, RTT, congestion, and traffic

4. A Bounded Request on an Existing Connection

```

use anyhow::{Context, Result};
use tokio::io::AsyncWriteExt;

const MAX_RESPONSE_BYTES: usize = 256 * 1024;

async fn request(
    connection: &quinn::Connection,
    payload: &[u8],
) -> Result<Vec<u8>> {
    // One bidirectional stream gives this exchange an independent byte sequence.
    let (mut send, mut receive) = connection
        .open_bi()
        .await
        .context("open QUIC stream")?;

    // `write_all` handles partial writes; a single `write` would not.
    send.write_all(payload)
        .await
        .context("write request")?;

    // Finish only the sending half. The peer may still return a response.
    send.finish().context("finish request stream")?;

    // The cap turns a peer-controlled response into a bounded allocation.
    receive
        .read_to_end(MAX_RESPONSE_BYTES)
        .await
        .context("read bounded response")
}

```

The connection setup is deliberately separate because certificate trust, server name, transport configuration, socket address, timeouts, and resumption policy belong to a larger security contract.

5. Streams Need Application Framing

A QUIC stream is still a byte stream. Opening one stream per request can provide a natural boundary, but a long-lived stream still needs explicit framing.

Choice	Appropriate when
one stream per request	independent request/response exchanges
long-lived framed stream	ordered session, REPL, incremental protocol
unidirectional stream	logs, asset push, one-way result
datagram	stale data is worse than missing data

Never allocate directly from an untrusted length field. Cap stream count, total buffered bytes, message length, decompressed size, and work per connection.

6. Security and Replay

Concern	Design requirement
certificate trust	verify the intended identity; do not disable verification

ALPN	negotiate the exact application protocol
0-RTT	early data may be replayed; allow only replay-safe work
authentication	transport identity may not equal application user
migration	authorization must not rely only on source IP
key logging/tracing	keep diagnostic secrets out of production logs

A safe 0-RTT operation is idempotent or otherwise replay-tolerant. “Charge account,” “delete record,” and “run build” are not safe merely because the transport accepts them early.

7. Backpressure and Concurrency

```
use std::sync::Arc;
use tokio::sync::Semaphore;

async fn serve_connection(
    connection: quinn::Connection,
    permits: Arc<Semaphore>,
) -> anyhow::Result<()> {
    loop {
        // Accepting a stream does not yet start unbounded application work.
        let (send, receive) = connection.accept_bi().await?;

        // Acquire before spawning so queued streams cannot create unlimited tasks.
        let permit = Arc::clone(&permits).acquire_owned().await?;

        tokio::spawn(async move {
            // Keep the guard in the task; dropping it releases capacity.
            let _permit = permit;

            // A stream failure is isolated and logged instead of ending the
            // entire connection's accept loop.
            if let Err(error) = handle_stream(send, receive).await {
                tracing::warn!(%error, "stream failed");
            }
        });
    }
}
```

The semaphore bounds active stream handlers. A production server also needs connection limits, idle timeouts, cancellation, task supervision, per-peer policy, and a strategy for expected connection closure.

8. QUIC for a Language Runtime

Runtime feature	Useful QUIC mapping
compile request	one bidirectional stream
diagnostic stream	server-initiated unidirectional stream
debugger session	long-lived framed bidirectional stream
cancellation	reset/stop the request stream plus request ID

live telemetry	datagrams when old samples have no value
package/object transfer	independent reliable streams with hashes

Define semantic cancellation separately from transport reset: stopping byte delivery does not automatically roll back compilation, database work, or external effects.

 **Next:** graphics and media systems explain the vectors, transforms, sampling, buffers, GPU queues, and timing models that browsers later use to produce pixels, audio, and video.

Graphics and Media Systems: 3D, GPUs, Audio, and Video

 **Library lens:** wgpu and SDL are used to make linear algebra, resource layouts, GPU queues, audio timing, and presentation contracts concrete.

This chapter applies the earlier mathematical foundations to systems that must produce perceptual output under strict memory, bandwidth, latency, and device constraints. The equations describe the intended transformation; APIs such as wgpu and SDL expose the resource ownership, queues, formats, and synchronization that make the transformation run on real hardware.

 **Prerequisite:** read [Mathematical Foundations](#) first. Here, vectors, matrices, sampling, approximation, and probability become concrete graphics and media pipelines.

1. The 3D Transform Pipeline

```
model space → world space → view/camera space → clip space
               → perspective divide → normalized device coordinates
               → viewport/screen space → rasterized fragments
```

Stage	Question	Frequent reversing clue
model	Where is a vertex relative to its object?	mesh vertex/index buffers
world	Where is the entity in the scene?	transform/component structures
view	Where is it relative to the camera?	camera basis or view matrix
projection	How does depth map to a viewing volume?	field of view, near/far planes
clipping	Which primitives intersect the visible volume?	clip-space w , plane tests
rasterization	Which pixel samples does a triangle cover?	viewport and depth state
shading	What color/material result is produced?	shader constants, textures
compositing	How are layers combined for display?	render targets, blend state

Matrices compose transforms, and **order matters**. With column-vector notation, `projection * view * model * position` applies model first. An engine using row vectors may write the apparent reverse. Recover convention by testing a known translation, not by guessing from storage order.

Homogeneous coordinates add `w` : points usually use `w = 1` , directions use `w = 0` , and perspective projection makes the later divide possible. A quaternion compactly represents orientation and composes rotations without the same singularity problems as Euler angles, but it still needs normalization and a documented multiplication order.

2. Scene Entities Are Relationships, Not Just Coordinates

An entity/component system often separates **identity** from **storage**:

Part	Purpose	Low-level pattern
entity ID	stable logical identity	index plus generation counter
component pool	dense values of one type	contiguous array / sparse set
transform hierarchy	parent-child spatial relation	graph or indexed parent
system	behavior over a component query	loop or scheduled function
resource	shared world-level state	singleton handle or typed store

When reversing an authorized build, repeated access patterns such as `base + index * stride + field_offset` suggest an array of fixed-layout records. Pointers from one record to several component pools suggest indirection. Do not name a field from one observation: vary position, health, animation, and ownership separately and record which bytes change.

For your own language, an entity declaration can lower to ordinary component schemas and queries. The compiler can validate component names while the runtime chooses dense storage for cache-friendly iteration.

3. A GPU Is an Asynchronous Throughput Machine

The CPU records work; the driver/runtime validates and translates it; the GPU consumes commands later.

```
application → graphics/compute API → user-mode driver → kernel driver
               → command queue → GPU cores/caches/memory → fence/event → CPU
```

GPU concept	Mental model	Failure mode
shader	small program executed across many invocations	divergent control flow
buffer	typed interpretation over bytes	wrong stride/alignment/lifetime
texture	multidimensional sampled storage	wrong format/color space/layout
command buffer	recorded work for later execution	resource freed too early
pipeline state	compiled collection of fixed and programmable stages	incompatible formats
barrier	visibility/order constraint between uses	stale or racing data

fence	completion signal visible to the host	CPU reads before completion
staging memory	host-friendly transfer area	excess copies and bandwidth

Synchronization is part of correctness. “The submit call returned” usually means the queue accepted work, not that rendering finished. The same distinction appears in async I/O, DMA, databases, and distributed acknowledgements.

wgpu as an Executable Model of GPU Work

Current wgpu APIs make the abstract GPU pipeline visible as a set of separate capabilities and timelines:

wgpu object	Computer-science model it exposes
Instance	discovery of available graphics backends
Adapter	one implementation’s capabilities and limits
Device	validated resource and pipeline factory
Queue	ordered submission timeline for asynchronous work
CommandEncoder	a mutable builder for a later immutable command buffer
render/compute pipeline	compiled state machine describing legal execution
bind-group layout	type-like contract for shader-visible resources
bind group	concrete resources satisfying that contract
Surface	finite pool of presentation images owned with the window system

The CPU does not call a shader once per vertex. It records commands and resource relationships, submits them to a queue, and continues while the GPU consumes that work. Validation checks that bindings, formats, usage flags, and pipeline stages agree before the backend reaches a less forgiving driver API.

```

struct RecordedFrame {
    // These are logical handles, not proof that the GPU has finished using memory.
    command_count: usize,
    writes_presentable_image: bool,
}

enum FrameState {
    Acquired,
    Recorded(RecordedFrame),
    Submitted { submission_index: u64 },
    Presented,
}

```

This conceptual enum shows why “the Rust function returned” and “the GPU finished” are different events. Correct resource reuse needs a completion signal or an API guarantee that internally provides the same ordering.

The render pipeline also demonstrates **separation of static and dynamic state**: shader entry points, vertex formats, color formats, depth rules, and primitive topology are compiled into a reusable pipeline; buffers, textures, offsets, and draw counts vary per submission. Moving stable validation out of the hot loop is the same idea used by prepared database statements, compiled regular expressions, and bytecode VMs.

4. Digital Audio Is Timed Sampled Data

Term	Meaning
sample	one amplitude value for one channel at one instant
sample rate	audio frames per second, such as 48,000
channel	one signal lane: left, right, center, and so on
audio frame	one sample for every channel at a time point
bit depth / format	integer or float representation of each sample
buffer	contiguous frames exchanged with a device or codec
latency	time from production/input to audible output/observation
underrun	consumer needs frames that the producer has not supplied
clipping	amplitude exceeds the representable range
resampling	change sample rate by interpolation/filtering

```
fn mix_pcm_f32(left: &[f32], right: &[f32], output: &mut [f32]) -> Result<(), &'static str>
{
    // A real mixer must also agree on sample rate, channel layout, and timestamps.
    if left.len() != right.len() || output.len() != left.len() {
        return Err("buffers describe different frame counts");
    }

    for ((left_sample, right_sample), mixed) in
        left.iter().zip(right).zip(output.iter_mut())
    {
        // Scaling leaves headroom before clamping to the normalized PCM range.
        let value = 0.5 * left_sample + 0.5 * right_sample;
        *mixed = value.clamp(-1.0, 1.0);
    }
    Ok(())
}
```

Audio callbacks may run on a real-time-sensitive thread. Avoid blocking I/O, unbounded allocation, slow locks, and logging in the callback. Feed it from a bounded ring buffer and decide whether overload drops old data, inserts silence, or applies backpressure.

5. Frequency, Windows, and Compression

A time-domain waveform says **amplitude over time**. A Fourier transform expresses the same finite window as weighted frequencies. The sampling theorem links the highest representable frequency to sample rate, while a reconstruction/anti-alias filter makes the physical boundary practical.

Operation	Purpose
windowing	reduce edge discontinuities in a finite analysis block
FFT	compute a discrete Fourier transform efficiently
convolution	apply an impulse response/filter
dynamic range compression	reduce amplitude range; not file compression
lossless codec	reproduce original samples exactly
lossy codec	discard perceptually less important information

"Compression" is overloaded. Always specify **data compression**, **audio dynamics compression**, or **geometric compression**.

6. Video Separates Frames, Codecs, and Containers

Layer	Contains	Example concern
raw frame	pixels plus dimensions, format, stride, timestamp	RGB vs YCbCr
codec	compressed sequence syntax	keyframes and inter-frame references
elementary stream	encoded chunks for one track	decoder configuration
container	tracks, timestamps, metadata, seek indexes	MP4, WebM, Matroska
transport	how chunks move between systems	file, HTTP, RTP, QUIC

A **codec** compresses/decompresses media; a **container** multiplexes tracks and metadata. Demuxing finds encoded chunks; decoding turns chunks into frames. Seeking often starts at a keyframe and decodes forward because predicted frames depend on other frames.

YCbCr separates luma from chroma. Formats such as **4:2:0** store color at lower spatial resolution than brightness, reducing size. A correct frame parser must track:

- coded and display dimensions;
- pixel format and bit depth;
- per-plane stride and offset;
- color primaries, transfer function, and range;
- timestamps and time base;
- ownership while hardware decoding is in flight.

7. Media Reversing Is Layer Identification

For files and owned captures, begin with structure:

1. identify magic bytes and the container;
2. parse bounded lengths and indexes;

3. enumerate tracks, formats, and time bases;
4. hand encoded payloads to a maintained codec library;
5. compare decoded frames or samples, not only file hashes;
6. preserve the original and fuzz only disposable copies.

Do not mistake obfuscated metadata for a new codec. First test ordinary causes: endianness, alignment, compression, encryption, checksums, and versioned headers.

 **Language-design connection:** arrays, SIMD vectors, numeric conversions, foreign-function interfaces, async device queues, and resource lifetimes all become concrete when a language can safely express a small renderer or audio graph.

Sources: [SDL3 audio model](#), [SDL3 pixel formats](#), [digital-audio concepts](#), [digital-video concepts](#), and [frames, codecs, and containers](#).

How Chrome Works: Processes, Parsing, JavaScript, and Pixels

Chrome is not one parser attached to one window. It is an **operating environment for untrusted network programs**. It combines networking, process isolation, compilers, garbage collectors, databases, event loops, graphics, media, accessibility, and device APIs behind the abstraction of a page.

 **Core mental model:** a browser repeatedly transforms untrusted bytes into more structured representations while moving work across security and scheduling boundaries. Each representation exists because the next stage needs different facts.

```
URL
→ DNS / connection / TLS / HTTP
→ response bytes
→ HTML tokens
→ DOM + discovered resources
→ CSS rules and computed styles
→ layout geometry
→ paint display items
→ composited layers / raster tiles
→ GPU commands
→ presented pixels
```

JavaScript can mutate earlier stages, so this is not a one-way batch compiler. It is an incremental, concurrent pipeline with invalidation: a mutation marks affected derived state dirty, and later work reconstructs only what is needed.

1. Navigation Is a Distributed State Machine

Entering a URL begins a navigation whose state is coordinated by the privileged browser process:

1. Parse and normalize the URL.
2. Check policy, existing service workers, caches, and navigation history.
3. Resolve a host name through DNS when a network connection is needed.
4. Establish or reuse a transport connection; HTTPS authenticates the server and negotiates encryption with TLS.

5. Send an HTTP request and validate response headers.
6. Choose or create a renderer process allowed to host the destination's site.
7. Commit the navigation, stream response data, and start parsing.
8. Discover and fetch dependent scripts, style sheets, fonts, images, and media.

```
enum NavigationState {
    Created { url: String },
    Fetching { request_id: u64 },
    ResponseStarted { status: u16, mime: String },
    ReadyToCommit { renderer_id: u32 },
    Committed { document_id: u64 },
    Failed { stage: &'static str, reason: String },
}

fn begin_response(
    state: NavigationState,
    status: u16,
    mime: String,
) -> Result<NavigationState, &'static str> {
    match state {
        NavigationState::Fetching { .. } => {
            // The transition is legal only after a request exists. Encoding the
            // states as variants prevents fields from belonging to the wrong phase.
            Ok(NavigationState::ResponseStarted { status, mime })
        }
        _ => Err("response arrived in an invalid navigation state"),
    }
}
```

The real browser supports redirects, authentication challenges, downloads, back/forward cache, speculative preloading, cancellation, process swaps, error pages, and races between them. The useful lesson is that an async operation is a state machine whose late results must be checked against current identity and generation.

2. Chrome Uses Processes as Protection Domains

Chromium separates major responsibilities into operating-system processes. Exact placement varies by platform and configuration, but the conceptual boundaries remain:

Process/service	Main responsibility	Why isolate it
browser process	windows, tabs, navigation, permissions, process policy	privileged coordinator and authority broker
renderer process	Blink, page DOM/layout, V8 execution for hosted documents	contains untrusted site code
network service	HTTP, proxy, DNS integration, cache, socket/TLS work	central policy and fault isolation
Viz/GPU process	raster, compositing, GPU command submission, presentation	isolates complex drivers and aggregates surfaces
utility		

processes	decoding and narrowly scoped services	least privilege and crash containment
-----------	---------------------------------------	---------------------------------------

A process has a separate virtual address space. A renderer cannot directly dereference a browser-process object. It must request an operation through an IPC interface, which creates a place to validate identity, ownership, size, and permission.

Multi-process design trades memory and IPC cost for:

- **fault isolation** — one renderer crash need not terminate every tab;
- **security isolation** — a compromised renderer receives limited operating-system authority;
- **parallelism** — unrelated sites can run on different CPU cores;
- **resource accounting** — memory and CPU use can be attributed and reclaimed.

Threads solve scheduling within a protection domain; processes also change what memory and operating-system capabilities are reachable.

3. Origin, Site, Frame, and Process Are Different Boundaries

An **origin** is normally the tuple `(scheme, host, port)`. A **site** is a broader grouping used by parts of the process model. A page contains a tree of **frames**, and a cross-site iframe may live in a different renderer process from its parent.

Object	Meaning
document	one parsed HTML document with script and rendering state
frame	a position in the page's nested browsing-context tree
origin	principal used by the same-origin security model
site instance	group of related documents that may need synchronous access
renderer process	OS protection domain hosting one or more permitted site instances

Site Isolation tries to prevent one renderer from receiving sensitive documents belonging to another site. The browser process retains authoritative frame and security state, and uses remote-frame proxies when related frames reside in different renderers.



Invariant: a process locked to one site must not gain document data or privileged capabilities belonging to another principal merely by forging an IPC request.

4. Mojo IPC Makes Cross-Process Authority Explicit

Chromium commonly uses **Mojo** interfaces for typed inter-process communication. An interface defines legal messages; endpoints grant the capability to send those messages. Payloads are serialized because sender and receiver do not share ordinary object pointers.

```

enum BrowserRequest {
    OpenUrl { frame_id: u64, url: String },
    ReadClipboard { frame_id: u64, user_gesture: bool },
    RequestCamera { frame_id: u64, origin: String },
}

fn authorize(request: &BrowserRequest, policy: &Policy) -> Result<(), Denied> {
    match request {
        BrowserRequest::OpenUrl { frame_id, url } => {
            policy.check_navigation(*frame_id, url)
        }
        BrowserRequest::ReadClipboard {
            frame_id,
            user_gesture,
        } => policy.check_clipboard(*frame_id, *user_gesture),
        BrowserRequest::RequestCamera { frame_id, origin } => {
            policy.check_camera(*frame_id, origin)
        }
    }
}

struct Policy;
struct Denied;

impl Policy {
    fn check_navigation(&self, _frame: u64, _url: &str) -> Result<(), Denied> {
        Ok(())
    }
    fn check_clipboard(&self, _frame: u64, _gesture: bool) -> Result<(), Denied> {
        Ok(())
    }
    fn check_camera(&self, _frame: u64, _origin: &str) -> Result<(), Denied> {
        Ok(())
    }
}

```

The enum is only a teaching model. The important ideas are transferable:

- deserialize into bounded, validated data;
- authenticate the sender's process/frame identity out of band;
- do not trust the sender's claim about its own origin or permissions;
- grant narrow endpoints rather than global ambient authority;
- handle endpoint closure, process death, cancellation, and version skew.

5. Blink and V8 Cooperate but Solve Different Problems

The renderer contains two major language systems:

Component	Model
Blink	implements HTML, CSS, DOM, layout, painting, accessibility, and web-platform bindings
V8	parses, interprets, compiles, executes, and garbage-collects JavaScript/WebAssembly

Blink owns host objects such as DOM nodes. V8 owns JavaScript values and execution. Bindings translate calls such as `element.appendChild(child)` across the JavaScript ↔ Blink boundary while preserving identity, lifetime, exceptions, and security policy.

This is an FFI problem inside one process: two runtimes have different object models and garbage collectors but expose one apparent programming environment.

6. HTML Parsing Is Tokenization Plus Error-Recovering Tree Construction

HTML parsing does more than split on angle brackets:

```
bytes → character decoding → tokenizer states → tokens → tree-builder states → DOM
```

The tokenizer has states because the meaning of a character depends on context. `<` in ordinary text, a script, a comment, or an attribute does not always begin the same token. The tree builder uses insertion modes and a stack of open elements to recover a deterministic DOM even from malformed markup.

Mechanism	Why it exists
encoding detection/decoding	bytes must become Unicode input
tokenizer state	characters have context-dependent meaning
insertion mode	tokens mean different tree operations in tables, body, templates, etc.
stack of open elements	track nesting and recovery
speculative preload scanner	discover likely resources before the main parser reaches them

Parser-blocking scripts can pause tree construction because they may inspect or mutate the partially built document. Stylesheets can delay script execution when computed style must be available. Therefore network order, parser order, and script semantics interact.

7. DOM, Style, Layout, Paint, and Layers Are Separate Representations

One tree cannot efficiently answer every rendering question.

Representation	Stores	Omits or differs from
DOM	document nodes and attributes	final visual geometry
style data	winning computed property values	pixel placement
layout objects/fragments	boxes, line fragments, sizes, positions	final painted pixels
paint records/display items	drawing operations in order	physical screen surfaces
property trees/layers	transform, clip, effect, scroll relationships	document semantics
raster tiles	pixel data for regions	high-level DOM meaning

CSS processing includes selector matching, cascade origin, importance, specificity, inheritance, custom properties, and computed-value resolution. Layout then solves box geometry under constraints such as available width, formatting context, intrinsic size, font metrics, and fragmentation.

A DOM node may produce no layout object (`display: none`), one or many fragments, or a relationship that differs from DOM ancestry. This is why “the render tree” is a useful introductory phrase but not one universal data structure.

8. Invalidation Makes Rendering Incremental

When state changes, Chrome records which derived stages are no longer valid:

```
#[derive(Default)]
struct Dirty {
    style: bool,
    layout: bool,
    paint: bool,
    composite: bool,
}

fn class_attribute_changed(dirty: &mut Dirty) {
    // A class can change any matching selector.
    dirty.style = true;
    // Later stages depend on computed style, so they may also need rebuilding.
    dirty.layout = true;
    dirty.paint = true;
    dirty.composite = true;
}

fn opacity_changed(dirty: &mut Dirty) {
    // If the element is already isolated appropriately, opacity can often be
    // updated by compositing without re-running layout or repainting content.
    dirty.composite = true;
}
```

The exact invalidation rules are far more selective. The model explains performance:

- changing geometry can require layout, paint, raster, and composite;
- changing color may require paint without layout;
- changing a compositor-managed transform may avoid main-thread layout and paint;
- reading layout immediately after a write can force pending work synchronously.

9. RenderingNG Splits Main-Thread and Compositor Work

The renderer’s main thread runs scripts, document lifecycle work, style, layout, paint, and event dispatch. A compositor thread can handle eligible scrolling and animations without waiting for page JavaScript. The Viz process aggregates compositor frames, rasters tiles, submits GPU work, and presents the final image.

```
renderer main thread
  DOM/style → layout → paint records
    ↓ commit
renderer compositor thread
  layers/property trees → tiles → compositor frame
    ↓ surface submission
Viz process
  aggregate frames → raster/draw → GPU queue → display
```

Compositing does not mean every element owns a permanent texture. Layerization is a resource decision balancing independent movement against memory, raster work, and composition overhead.

Stage	Typical bottleneck
style	broad selector invalidation and large affected subtrees
layout	geometry dependencies and forced synchronous reads
paint	many or expensive display items
raster	large/complex tiles and image decode
composite	too many surfaces/layers or GPU synchronization
present	missed frame deadline or display backpressure

10. Input and the Event Loop Are Scheduling Systems

Raw mouse, touch, keyboard, and wheel events begin in platform input systems. The browser routes them to the correct frame and hit-tested target. Some scrolling can run on the compositor thread; events requiring JavaScript reach the renderer main thread.

The web event loop coordinates several queues and checkpoints:

1. take one runnable task;
2. execute JavaScript to completion for that task;
3. drain the microtask queue at the specified checkpoint;
4. perform rendering work when the document and scheduler allow;
5. wait for more input, timers, network completion, or other work.

“Run to completion” prevents two JavaScript callbacks on one agent from interleaving at arbitrary instructions, but callbacks can observe mutations made by earlier tasks, and `await` divides a function into multiple scheduled continuations.

11. V8 Uses Tiered Execution to Balance Startup and Peak Speed

Current V8 uses multiple execution tiers because no single compiler optimizes simultaneously for fastest startup, lowest compilation cost, and highest peak speed:

```
JavaScript source
→ parser / AST
→ Ignition bytecode interpreter
→ Sparkplug baseline machine code
→ Maglev mid-tier optimized code
→ top-tier optimized code
↘ deoptimization when assumptions fail
```

Mechanism	Computer-science idea
bytecode	compact instruction set independent of host CPU
interpreter	fetch/decode/execute loop over virtual instructions

inline cache	remember shapes/types observed at an operation
hidden class / map	shared structural description of object properties
speculative optimization	generate fast code under observed assumptions
deoptimization	reconstruct a less-specialized execution state when an assumption fails
tiering	spend optimization cost only on code that becomes hot

For `object.x`, V8 can observe repeated objects with the same shape and cache where `x` is stored. Optimized code checks the shape, loads from the known offset, and takes a slow path or deoptimizes when the shape differs.

 **Compiler connection:** this combines parsing, bytecode, profiling, SSA/CFG optimization, machine-code generation, stack maps, garbage-collector metadata, and reversible mappings needed to reconstruct interpreter-visible state.

12. Garbage Collection Preserves Reachability Across Runtimes

V8 uses a generational heap because most newly allocated objects die young. Young objects can be collected frequently; surviving objects move or promote into older storage. Major collection traces reachable objects, reclaims unreachable storage, and may compact fragmented pages.

GC mechanism	Why it is needed
roots	begin the reachability graph from stacks, globals, and runtime handles
young/old generations	exploit different object lifetime distributions
remembered sets	find old-to-young references without scanning all old objects
write barriers	record graph mutations during generational/incremental/concurrent GC
marking	discover reachable objects
sweeping	return unreachable regions to free storage
compaction	move live objects together and update references

Blink also manages garbage-collected objects. Cross-runtime bindings must ensure that a live JavaScript wrapper keeps the corresponding host object valid when required, and that cycles crossing runtime boundaries do not become invisible to tracing.

“Garbage collected” does not mean “memory is free immediately.” Reachable caches, listener registrations, pending tasks, DOM references, and native resources can retain large graphs indefinitely.

13. Caches and Storage Have Different Consistency and Lifetime Rules

Chrome coordinates multiple forms of state:

Storage	Purpose	Important boundary
memory cache	reuse decoded or fetched resources quickly	process/document lifetime
	reuse responses under protocol freshness	request method, headers,

HTTP cache	rules	validators
cookies	small request-associated state	domain/path/SameSite/Secure policy
Cache Storage	service-worker-controlled response storage	origin and application policy
IndexedDB	transactional structured object storage	origin and schema/version
local/session storage	small synchronous key/value APIs	origin and browsing-session rules
back/forward cache	preserve an entire page for fast history navigation	page eligibility and lifecycle

A service worker is a programmable request intermediary, not a magical offline flag. It has an event-driven lifetime, can intercept in-scope requests, and must coordinate version changes and durable state without assuming it runs forever.

HTTP caching is a protocol between client, intermediary, and origin server. Freshness, validators, request headers, response headers, partitioning, and invalidation determine whether a stored response is reusable.

14. Browser Security Is Layered Boundary Enforcement

Boundary	Mechanism
site ↔ site	same-origin policy, Site Isolation, frame/process policy
renderer ↔ operating system	sandbox and brokered capabilities
document ↔ network response	CORS, CORB/ORB, content-type and fetch rules
text ↔ executable script	CSP, Trusted Types, parsing contexts
user ↔ site	permission prompts, gestures, profile policy
JavaScript ↔ native engine	checked bindings, V8 sandboxing, memory safety defenses

No one layer proves the browser safe. The model assumes a parser, JIT, renderer, or GPU component can contain bugs, then limits what a compromise can reach. This is **defense in depth**: validation, least privilege, process isolation, origin policy, memory protection, and exploit mitigations overlap.

15. Diagnose Browser Performance by Finding the Delayed Stage

Symptom	Likely investigation
slow initial content	DNS, connection, TLS, response timing, parser-blocking resources
long input delay	main-thread task length and event scheduling
animation jank	layout/paint invalidation, raster cost, compositor eligibility
memory growth	retained DOM/listeners, caches, JS heap, images, GPU resources
slow script after warm-up	unstable object shapes, deoptimization, allocation/GC

visual update appears late

frame deadline, compositor commit, raster, GPU/presentation

Measure before naming the cause. A “GPU problem” can originate in main-thread layout; a “JavaScript problem” can be network blocking; a “memory leak” can be intentional cache retention. Performance tools are useful because their tracks correspond to real queues, threads, representations, and transitions in the architecture.

16. Chrome Is a Capstone for Computer Science

Earlier concept	Chrome manifestation
grammars and automata	HTML/CSS/JavaScript parsing
compilers and VMs	V8 bytecode, tiering, speculation, deoptimization
graph algorithms	DOM traversal, dependency graphs, GC reachability
operating systems	processes, virtual memory, scheduling, permissions
networking	DNS, HTTP, TLS, proxying, caching, QUIC
databases	origin-scoped durable storage and transactions
distributed systems	IPC, failure, cancellation, versioning, partial results
graphics and linear algebra	transforms, clipping, raster, compositing
security	sandboxing, principals, capabilities, validation
real-time reasoning	frame deadlines, input latency, media synchronization

For your own language or runtime, do not copy Chrome’s scale. Copy the separation of models:

```
source/bytes → validated representation → explicit state transition
               → bounded queue → isolated capability → observable result
```

Current primary references: [Chromium multi-process architecture](#), [process model and Site Isolation](#), [RenderingNG architecture](#), [Chromium network service](#), [V8 high-level overview](#), and [V8 garbage collection](#).



Next: GUI and TUI architectures generalize the same event, state, invalidation, layout, rendering, and presentation models outside the browser.



User Interfaces: Events, State, Layout, Windows, GUI, and TUI



Library lens: winit, egui, Ratatui, and wgpu are used to compare event sources, retained versus immediate state, layout, diffing, GPU command recording, and OS presentation.

A user interface is a feedback loop over state:

```
OS/terminal input → normalize event → update application state
→ layout visible elements → generate drawing commands → render/present
→ wait for the next input, timer, or completed task
```

A GUI targets windows and pixels. A TUI targets a terminal's character-cell protocol. Their output devices differ, but both need event handling, state, layout, focus, incremental redraw, accessibility, testing, and cleanup.

1. A GUI Sits on Several Layers

Layer	Responsibility	Rust examples
application model	domain state and commands	your structs, enums, reducers
UI toolkit	widgets, layout, focus, accessibility, styling	<code>egui</code> , <code>iced</code> , Slint, GTK4
text/2D scene	shaping, paths, images, glyphs, clipping	Skia, Cairo, <code>tiny-skia</code>
graphics API	buffers, textures, shaders, command queues	<code>wgpu</code> , Vulkan, Metal, D3D12, OpenGL
window/event layer	windows, monitors, input, IME, DPI, lifecycle	<code>winit</code> , SDL3
compositor/window system	combines application surfaces on screen	Wayland/X11, DWM, Quartz
driver/GPU/display	executes commands and scans completed images out	platform hardware stack

A full toolkit may own most layers; a focused library may own one. `winit` creates windows and dispatches events but is not a widget toolkit. `wgpu` submits graphics work but does not provide buttons, layout, or text editing.

2. The Event Loop Owns Platform Lifecycle

Desktop platforms deliver resize, scale-factor, keyboard, pointer, touch, IME, theme, focus, suspend/resume, redraw, and close events. The event loop often has thread and lifecycle restrictions, so create windows only when the platform says it is valid.

```
resumed → create/recreate window and graphics surface
window/input event → update state and mark dirty
redraw requested → acquire surface image → paint → submit → present
suspended → release resources the platform may invalidate
close requested → save/confirm policy → exit
```

Current `winit` uses `ApplicationHandler` with `EventLoop::run_app`; windows are commonly created through `ActiveEventLoop` during `resumed`. A redraw request means “produce a frame when dispatched,” not “draw immediately inside every input callback.”

Separate raw platform events from domain messages:

```

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum Message {
    Increment,
    Decrement,
    Reset,
    WindowCloseRequested,
}

#[derive(Debug, Default, PartialEq, Eq)]
struct CounterState {
    value: i64,
    should_close: bool,
}

fn update(state: &mut CounterState, message: Message) {
    // The reducer is independent of winit, egui, a terminal, and the GPU.
    match message {
        Message::Increment => state.value = state.value.saturating_add(1),
        Message::Decrement => state.value = state.value.saturating_sub(1),
        Message::Reset => state.value = 0,
        Message::WindowCloseRequested => state.should_close = true,
    }
}

```

This pure update core can be driven by a GUI, TUI, tests, network command, or your own language runtime.

3. Retained, Immediate, and Declarative UI

Model	Programmer describes	Toolkit retains	Good fit
retained mode	create/update a persistent widget/object tree	widgets and invalidation state	document/form-style applications
immediate mode	rebuild desired widgets each frame/pass	interaction memory keyed by IDs	tools, editors, debug panels
declarative /reactive	desired UI as a function of state	dependency graph or virtual/retained representation	data-driven apps
Elm/reducer style	messages, pure update, view	runtime effects/subscriptions/tree	explicit state transitions

“Immediate” does not mean the GPU redraws every pixel for every method call. A toolkit can collect a frame’s UI description, tessellate once, and repaint only when requested. “Retained” does not eliminate application state; it means the toolkit retains UI objects and schedules updates.

Stable widget IDs matter in immediate-mode systems: focus, drag state, open/closed state, and text editing must remain attached to the same logical control across frames.

4. Layout Is Constraint Solving Over a Tree

Most layout engines perform some variation of:

```

parent supplies constraints → child measures intrinsic/preferred size
→ parent allocates rectangles → child lays out descendants

```


Input	Examples
constraints	minimum/maximum width and height
intrinsic size	text, image, control content
policy	row/column/grid, flex, alignment, padding
environment	DPI scale, font metrics, locale, theme
output	logical rectangles plus clip and transform

Use **logical units** for layout and convert to physical pixels at the rendering boundary. Fractional scale factors mean rounding each child independently can create gaps or drift; distribute rounding with a documented policy.

Cycles require rules. “Parent sizes to child while child fills parent” cannot be solved by naive recursion. Real toolkits use bounded passes, priority constraints, or well-defined flex/intrinsic algorithms.

5. Painting Produces a Display List

Widgets should usually emit drawing primitives rather than talk directly to the GPU:

```
filled rectangle → clipped rounded path → shaped glyph run → image
```

The renderer then:

1. resolves clips/transforms and z-order;
2. shapes text and locates glyphs;
3. batches compatible primitives;
4. uploads changed buffers/textures;
5. records graphics commands;
6. submits and presents;
7. retains resources until the GPU has finished.

Dirty-region/damage tracking avoids repainting unaffected areas. Immediate-mode toolkits may still cache fonts, meshes, textures, and unchanged windows.

6. Text Is the Hardest “Simple” Widget

Text input/rendering involves:

Concern	Why bytes or scalar values are insufficient
shaping	several characters may become one glyph or contextual forms
grapheme clusters	one perceived character may contain several scalars
bidi	visual order can differ from logical storage order
line breaking	language and punctuation rules affect breaks
font fallback	one run may require several fonts

cursor selection	byte, scalar, grapheme, glyph, and visual positions differ
IME	composition text is provisional until committed
accessibility	semantic label/value/role differs from painted pixels

Keep source text and semantic selection separate from shaped glyph positions. A glyph atlas is a cache of rasterized glyph images, not the authoritative text.

7. Input Becomes Routing, Focus, and Gestures

```
device event → window coordinates → inverse transforms → hit test
→ pointer capture/focus → widget event → domain message
```

Mechanism	Purpose
hit testing	choose visual target under a point
focus	choose keyboard/IME target
pointer capture	keep receiving drag events outside original bounds
event bubbling/capture	route through ancestors under toolkit policy
gesture recognition	convert temporal pointer/touch sequences into intent
command mapping	turn shortcuts/menu actions into domain messages

Do not identify keyboard shortcuts only by produced text; physical key, logical key, modifiers, keyboard layout, and IME composition are different representations.

8. Accessibility Is a Parallel Semantic Tree

A painted rectangle does not tell assistive technology that it is a checked, disabled button named "Mute." Expose a semantic tree containing:

- role, label, value, description, and state;
- parent/child relationships and reading order;
- focus and selection;
- available actions;
- live-region/change notifications;
- bounds when the platform requires them.

Keyboard operation, visible focus, contrast, scalable text, reduced motion, and screen reader semantics are architectural requirements. Test the accessibility tree and real assistive technology; a mouse-only visual test is insufficient.

9. GUI and Graphics Library Selection

Library	Layer/model	Choose it when	It does not replace
	cross-platform	building a custom renderer or	widgets, text,

winit	windows/events	toolkit	renderer
wgpu	safe cross-platform GPU API	custom 2D/3D/compute rendering	window/event loop and UI
tiny-skia	small pure-Rust CPU 2D rasterizer	paths and pixels without a GPU stack	full widgets and windowing
Skia / Cairo bindings	mature 2D graphics/text ecosystem	complex vector/text rendering or integration	application UI architecture
egui / eframe	immediate-mode Rust GUI	tools, inspectors, editors, rapid native/web UI	native platform widgets
iced	typed Elm-inspired GUI	message/update/view architecture and async tasks	OS-native widget toolkit
Slint	declarative UI language plus Rust API	designer-friendly declarative apps, embedded/desktop targets	arbitrary platform-native widgets
gtk4-rs / Relm4	bindings / Elm-like layer over GTK4	established toolkit, accessibility, GNOME integration	small dependency footprint
Tauri	system webview plus Rust backend/capabilities	HTML/CSS UI with a Rust desktop boundary	custom native/GPU widget toolkit
SDL3	window/input/audio/2D/GPU -oriented platform layer	games, media tools, custom loops	complete application widget set
Bevy	ECS game/application engine	data-oriented scenes, games, visual tools	traditional desktop widget semantics

No toolkit is universally “best.” Decide using:

Requirement	Verify before choosing
target platforms	desktop, web, mobile, embedded, remote
look/behavior	native toolkit, custom brand, game/tool style
accessibility/IME	real support on every required platform
renderer	CPU, GPU, webview/DOM, remote
application model	immediate, retained, reducer, reactive
packaging	runtime libraries, binary size, system dependencies
licensing	toolkit and transitive/native dependencies
testability	headless/test backend, event injection, accessibility snapshots
maturity	releases, migration cost, issue responsiveness

10. A Small Immediate-Mode `egui` View

```
fn counter_view(ui: &mut egui::Ui, state: &mut CounterState) {
    ui.heading("Counter");
    ui.label(format!("Current value: {}", state.value));

    ui.horizontal(|ui| {
        // Each button response is converted into the same domain messages a TUI
        // or test could send. The reducer remains the source of behavior.
        if ui.button("-").clicked() {
            update(state, Message::Decrement);
        }
        if ui.button("Reset").clicked() {
            update(state, Message::Reset);
        }
        if ui.button("+").clicked() {
            update(state, Message::Increment);
        }
    });
}
```

The function describes the current frame's controls. `egui` remembers interaction state through stable IDs and returns responses such as `clicked`. Long-running work belongs in a task/thread; send a result back and request repaint rather than blocking the UI thread.

11. How a TUI Reaches the Screen

A terminal user interface usually writes a stream of characters and control sequences to a **terminal emulator**:

```
TUI process ⇔ pseudo-terminal/console ⇔ terminal emulator
                                   → glyph cells → GUI compositor → display
```

Terminal concept	Meaning
cell grid	rows/columns containing glyph, style, and width state
escape/control sequence	protocol command for cursor, color, erase, mode, and more
raw mode	deliver key bytes/events without ordinary line editing/echo
alternate screen	temporary full-screen buffer used by many TUIs
cursor state	position, visibility, and style
terminal size	current rows/columns; can change asynchronously
PTY	bidirectional pseudo-device connecting program and terminal/remote session

The terminal ultimately draws pixels too, but the TUI normally controls **cells**, not the terminal emulator's font rasterizer or compositor.

12. TUI Rendering Uses a Back Buffer and Diff

For each frame:

1. query the current terminal area;
2. lay out widgets in cell coordinates;
3. render all visible widgets into an in-memory cell buffer;
4. compare with the previous buffer;
5. emit control sequences only for changed runs;
6. flush, then wait for events/ticks.

Wide Unicode graphemes may occupy two cells; combining marks and terminal capability differences complicate width. Test with the terminal families and locales you support.

13. TUI Library Selection

Library	Role	Use
Ratatui	immediate-rendered widgets, layout, terminal/frame abstraction	build the TUI
Crossterm	cross-platform terminal modes, cursor, style, input events	common Ratatui backend and lower-level control
Termion	Unix-oriented terminal control/backend	alternative backend in Unix-focused apps
Termwiz	terminal model/input/rendering utilities	advanced terminal behavior or alternate backend
<code>tui-textarea</code> and widget crates	reusable higher-level widgets	avoid rewriting complex editing behavior

Ratatui 0.30's main crate is the recommended application entry point, and its `run` helper initializes and restores the terminal around the application closure.

14. A Small Ratatui Application

```
use crossterm::event::{self, Event, KeyCode, KeyEventKind};
use ratatui::{
    layout::{Constraint, Layout},
    widgets::{Block, Borders, Paragraph},
    DefaultTerminal, Frame,
};

#[derive(Debug, Default)]
struct App {
    counter: CounterState,
    quit: bool,
}

impl App {
    fn run(&mut self, terminal: &mut DefaultTerminal) -> std::io::Result<()> {
        while !self.quit {
            // Ratatui draws the complete desired UI into an intermediate buffer,
            // then the backend emits the terminal-cell differences.
            terminal.draw(|frame| self.render(frame));
        }
    }
}
```

```

        self.handle_event()?;
    }
    Ok(())
}

fn render(&self, frame: &mut Frame) {
    let [header, body, help] = Layout::vertical([
        Constraint::Length(3),
        Constraint::Min(1),
        Constraint::Length(3),
    ])
    .areas(frame.area());

    frame.render_widget(
        Paragraph::new("Low-Level Counter")
            .block(Block::default().borders(Borders::ALL)),
        header,
    );
    frame.render_widget(
        Paragraph::new(format!("Value: {}", self.counter.value)),
        body,
    );
    frame.render_widget(
        Paragraph::new("<=> change · r reset · q quit")
            .block(Block::default().borders(Borders::TOP)),
        help,
    );
}

fn handle_event(&mut self) -> std::io::Result<()> {
    if let Event::Key(key) = event::read()? {
        // Some backends also report release/repeat; act only on key presses.
        if key.kind == KeyEventKind::Press {
            match key.code {
                KeyCode::Left => update(&mut self.counter, Message::Decrement),
                KeyCode::Right => update(&mut self.counter, Message::Increment),
                KeyCode::Char('r') => update(&mut self.counter, Message::Reset),
                KeyCode::Char('q') => self.quit = true,
                _ => {}
            }
        }
    }
    Ok(())
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let mut app = App::default();
    // `run` restores terminal modes even when the closure returns an error.
    ratatui::run(|terminal| app.run(terminal))?;
    Ok(())
}

```

For periodic animation or background work, poll with a deadline or use an event channel instead of blocking forever on `event::read`. Keep the channel bounded and coalesce frequent progress updates.

15. Terminal Cleanup and Escape-Sequence Safety

If a TUI enables raw mode, hides the cursor, or enters the alternate screen, restore the terminal on normal return, errors, and panics where practical. A stuck terminal is usually recoverable, but it is an avoidable failure.

Untrusted text may contain control sequences that change colors, titles, hyperlinks, clipboard-related controls, or apparent log content. A widget string API that escapes or treats content as text is safer than printing raw bytes. Bound input length and filter control characters according to the display contract.

16. UI Performance and Correctness Checklist

Symptom	Inspect
idle CPU/GPU usage	repaint requests, polling control flow, animations
typing lag	synchronous work on UI thread, shaping/layout invalidation
resize flicker	surface reconfiguration and retained frame policy
blurry text	logical-to-physical scale and half-pixel transforms
lost clicks	hit-test transforms, clipping, pointer capture
wrong key input	physical/logical key and IME composition handling
TUI corruption	terminal restore, width calculation, partial writes
screen-reader gaps	missing role/name/state/action semantics
memory growth	texture/glyph/widget caches and stale IDs
GPU validation errors	resource lifetime, format, alignment, barriers

Measure event-to-present latency separately from frames per second. A form that redraws only on input may feel excellent at zero idle FPS; a game may need continuous, well-paced frames.

17. Designing a UI System for Your Own Language

A small UI language can lower through familiar compiler stages:

```
declarative source → parsed UI AST → type/style validation → widget/scene tree
→ constraints and layout → display list or terminal-cell buffer → backend
```

Language feature	Runtime representation
component/function	constructor plus state slot
property	typed field or reactive value
event handler	closure/function reference plus environment
conditional/list	tree diff or immediate iteration with stable key
async command	future/task that sends a typed message
style	resolved rules/tokens converted to paint/layout values

accessibility	semantic nodes emitted beside visuals
backend	DOM, native toolkit, GPU display list, or terminal cells

Define whether updates are synchronous, batched, or transactional; whether event handlers may re-enter layout; how stable identity works; and which thread owns UI objects. Make resource handles and backend capabilities explicit.

When reversing a UI runtime, look for the same stages in reverse: event dispatch tables, widget/component trees, string/font caches, layout rectangles, display lists, GPU buffers, and platform accessibility/IME calls.

18. Native Window API Basics: Win32 as a Concrete Example

“Window API” can mean a portable layer such as `winit`, or the operating system’s native windowing interface. Win32 makes the lower-level pieces unusually explicit:

Win32 concept	Meaning
<code>HINSTANCE</code>	handle identifying the loaded application/module instance
window class	registered metadata shared by windows, including a window procedure
<code>HWND</code>	opaque handle identifying one live window; it is not an owned Rust reference
window procedure	ABI callback that receives messages for a window
message queue	per-thread queued input/lifecycle work retrieved by the pump
sent message	synchronous call into a target procedure; may cause re-entrancy
client area	application-painted content inside the non-client frame
update region	invalid portion that needs repainting
device context	GDI drawing state/target used during painting

The basic lifetime is:

```
register class → create HWND → show/update → pump and dispatch messages
→ receive close/destroy → post quit → leave pump → release application state
```

Window Classes and `HWND`

`RegisterClassW` registers a name plus behavior such as the `WNDPROC`. A window class is not an object-oriented class and is not one visible window. `CreateWindowExW` creates an instance and returns its `HWND`.

Handles are opaque tokens owned under platform rules. After destruction, an `HWND` must not be used; the numeric value may eventually be reused for another object.

The Window Procedure and Message Pump

```
GetMessageW → TranslateMessage → DispatchMessageW → your WNDPROC
```


`GetMessageW` has three outcomes: positive means a message was retrieved, zero means `WM_QUIT`, and negative means an error. Do not write `while GetMessageW(...).as_bool()` without preserving the error case.

The window procedure receives `(HWND, message ID, WPARAM, LPARAM)`. The meaning and ownership of both parameter-sized values depend entirely on the message. Delegate unhandled messages to `DefWindowProcW`; default behavior includes ordinary activation, non-client interaction, and close processing.

Paint, Resize, DPI, and Close

Message	Typical responsibility
<code>WM_NCCREATE</code> / <code>WM_CREATE</code>	attach and initialize instance state
<code>WM_SIZE</code>	update logical size and resize/reconfigure render targets
<code>WM_PAINT</code>	validate the update region and paint the requested client area
<code>WM_DPICHANGED</code>	update scale and usually apply the suggested window rectangle
<code>WM_CLOSE</code>	confirm/save policy or allow default destruction
<code>WM_DESTROY</code>	release window resources and post quit for a single-window app
<code>WM_NCDESTROY</code>	final point to detach per-window state

`BeginPaint` and `EndPaint` define the ordinary `WM_PAINT` validation scope. Continuous renderers may use a swap chain and explicit frame scheduling, but they still need to respect invalidation, occlusion, resize, and device-loss behavior.

Set DPI awareness through the supported manifest/context policy before creating windows. Use logical layout units, query the window's current DPI, and handle `WM_DPICHANGED` when crossing monitors rather than assuming 96 DPI forever.

Minimal windows-rs Skeleton

This example exposes the raw boundary for learning. A production application will usually prefer `winit`, `windows-window`, `GTK`, `egui / eframe`, `iced`, `Slint`, or another wrapper that owns more lifecycle invariants.

```
#[cfg(target_os = "windows")]
mod win32_window {
    use windows::{
        core::{w, Error, Result},
        Win32::{
            Foundation::{HINSTANCE, HWND, LPARAM, LRESULT, WPARAM},
            Graphics::Gdi::{BeginPaint, EndPaint, PAINTSTRUCT},
            System::LibraryLoader::GetModuleHandleW,
            UI::WindowsAndMessaging::{
                CreateWindowExW, DefWindowProcW, DispatchMessageW, GetMessageW,
                PostQuitMessage, RegisterClassW, TranslateMessage, CW_USEDEFAULT, MSG,
                WINDOW_EX_STYLE, WM_DESTROY, WM_PAINT, WNDCLASSW, WS_OVERLAPPEDWINDOW,
                WS_VISIBLE,
            },
        },
    },
};

// Win32 calls this function using the system ABI. Every pointer-sized message
```

```

// parameter must be interpreted only according to that message's documentation.
unsafe extern "system" fn window_proc(
    hwnd: HWND,
    message: u32,
    wparam: WPARAM,
    lparam: LPARAM,
) -> LRESULT {
    match message {
        WM_PAINT => {
            let mut paint = PAINTSTRUCT::default();
            // `BeginPaint` returns a temporary device context and validates the
            // dirty region when paired with `EndPaint`.
            let _device_context = unsafe { BeginPaint(hwnd, &mut paint) };

            // Draw client content here, or hand the HWND/surface to a renderer.

            let _ = unsafe { EndPaint(hwnd, &paint) };
            LRESULT(0)
        }
        WM_DESTROY => {
            // For a one-window application, end the thread's message loop.
            unsafe { PostQuitMessage(0) };
            LRESULT(0)
        }
        _ => {
            // Preserve the operating system's default behavior for messages this
            // small program does not handle.
            unsafe { DefWindowProcW(hwnd, message, wparam, lparam) }
        }
    }
}

pub fn run() -> Result<> {
    // All raw Win32 interaction stays inside this reviewed adapter.
    unsafe {
        let module = GetModuleHandleW(None)?;
        let instance = HINSTANCE(module.0);
        let class_name = w!("LowLevelNotesWindow");

        let window_class = WNDCLASSW {
            hInstance: instance,
            lpzClassName: class_name,
            lpfnWndProc: Some(window_proc),
            ..Default::default()
        };

        // Unlike many windows-rs functions, RegisterClassW returns zero on failure
        // rather than a `Result`, so convert the platform error explicitly.
        if RegisterClassW(&window_class) == 0 {
            return Err(Error::from_win32());
        }

        let _window = CreateWindowExW(
            WINDOW_EX_STYLE::default(),
            class_name,
            w!("Window API Basics"),
            WS_OVERLAPPEDWINDOW | WS_VISIBLE,
            CW_USEDEFAULT,
            CW_USEDEFAULT,
            960,
            640,

```

```

        None,
        None,
        Some(instance),
        None,
    )?;

    let mut message = MSG::default();
    loop {
        let status = GetMessageW(&mut message, None, 0, 0).0;
        if status == -1 {
            return Err(Error::from_win32());
        }
        if status == 0 {
            break; // `WM_QUIT`
        }
        let _ = TranslateMessage(&message);
        DispatchMessageW(&message);
    }
}
Ok(())
}
}

```

The `windows` crate needs only the Win32 feature groups used by the adapter. Prefer the Unicode `W` APIs, keep UTF-16 conversion at the boundary, and use `windows::core::Error::from_win32()` immediately when a sentinel-returning function fails so a later call cannot overwrite the thread's last-error value.

Attaching Rust State Safely

Raw Win32 programs often pass a boxed state pointer through `CreateWindowExW`, receive it during `WM_NCCREATE`, store it with window-associated user data, and recover it in later messages. That adapter must guarantee:

- the allocation outlives every message that can reference it;
- it is initialized once and detached during final destruction;
- callbacks never create simultaneous mutable Rust references;
- re-entrant sent messages cannot observe a half-mutated state;
- a panic never unwinds across the foreign callback ABI;
- worker threads communicate by messages/channels rather than directly mutating UI state.

A safe wrapper can encode those invariants in one owner type. Keep the rest of the application on the `Message → update → render` architecture shown earlier.

Win32 references: [Microsoft's first-window walkthrough](#), `WM_PAINT`, [high-DPI APIs](#), and the current `windows-rs` [Win32 bindings](#).

Current references: `winit` [event loop](#), `wgpu` [architecture](#), `egui` [immediate-mode GUI](#), `iced` [typed Elm-inspired GUI](#), [Slint's Rust API](#), [GTK4 Rust book](#), and [Ratatui application model](#).



Web Execution Boundaries: Actix Web, WebAssembly, and the DOM



Library lens: Actix Web and `wasm-bindgen` expose opposite host boundaries: server request/response execution and browser-owned objects reached through WebAssembly bindings.

Actix Web and `wasm-bindgen` sit on opposite sides of the web:

```
browser JavaScript
  ↓ wasm-bindgen glue
Rust compiled to WebAssembly
  ↓ HTTP / structured messages
Actix Web service
```

Both are boundary-design problems. Bytes and dynamic values enter Rust, become typed data, pass through application logic, and are converted back into an external representation.

1. Boundary Comparison

Concern	Actix Web boundary	<code>wasm-bindgen</code> boundary
External peer	HTTP client	JavaScript host
Input conversion	Extractors (<code>Path</code> , <code>Query</code> , <code>Json</code>)	Generated JS/Wasm bindings
Error form	HTTP status + response body	<code>Result<T, JsValue></code> / JS exception
Shared state	<code>web::Data<T></code>	Exported Rust object or Wasm linear memory
Trust boundary	Network request	Host calls and JS values
Main resource risk	Connections, bodies, tasks, backend calls	Copies, memory growth, handles, JS GC

2. Actix Web Application Shape

Current Actix Web 4 applications construct an `App` inside an `HttpServer` factory:

 **Naming distinction:** `actix-web` is the HTTP framework; `actix` is the actor framework described in [Actors: Owned State Behind a Typed Mailbox](#). Actix Web applications do not require you to model handlers as Actix actors.

```
use actix_web::{get, web, App, HttpServer, Responder};

#[get("/hello/{name}")]
async fn hello(name: web::Path<String>) -> impl Responder {
    let name = name.into_inner();
    format!("Hello, {name}!")
}

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| App::new().service(hello))
        .bind(("127.0.0.1", 8080))?
        .run()
        .await
}
```

The server factory may run once per worker. Create intentionally shared state outside the closure and clone its `web::Data` handle into each app.

3. Extractors Turn Requests into Typed Inputs

Extractor	Source in the HTTP request
<code>web::Path<T></code>	Variables captured by the route
<code>web::Query<T></code>	URL query string
<code>web::Json<T></code>	JSON request body
<code>web::Data<T></code>	Application state
<code>HttpRequest</code>	Headers, peer data, extensions, method, URI
<code>web::Bytes</code>	Buffered raw body

```
use actix_web::{web, HttpResponse, Responder};
use serde::{Deserialize, Serialize};
use std::sync::atomic::{AtomicU64, Ordering};

struct AppState {
    next_job: AtomicU64,
}

#[derive(Debug, Deserialize, Serialize)]
struct CreateJob {
    source: String,
}

#[derive(Debug, Serialize)]
struct JobAccepted {
    id: u64,
    source_bytes: usize,
}

async fn create_job(
    state: web::Data<AppState>,
    request: web::Json<CreateJob>,
) -> impl Responder {
    let id = state.next_job.fetch_add(1, Ordering::Relaxed);
    web::Json(JobAccepted {
        id,
        source_bytes: request.source.len(),
    })
}

fn configure(config: &mut web::ServiceConfig) {
    config.service(
        web::resource("/jobs")
            .route(web::post().to(create_job)),
    );
}
```



Extraction is parsing: a handler should receive values that already satisfy transport-level syntax, then perform application validation such as source-size, authorization, and allowed-language checks.

4. Bound Request Resources

```
use actix_web::web;

fn json_config() -> web::JsonConfig {
    web::JsonConfig::default()
        .limit(32 * 1024)
}
```

Resource	Boundary to define
Request body	Maximum compressed and decoded size
JSON nesting	Parser/format limits when configurable
Path/query values	Length and allowed character policy
Connections	Worker, backlog, keep-alive, and timeout policy
Backend calls	Timeout and retry budget
Spawned tasks	Concurrency limit and cancellation behavior
Response	Maximum buffered or streamed output

Do not hold a synchronous mutex guard across `.await`. Prefer an atomic for small independent counters, a short critical section for local compound state, or an async-aware/resource-specific client pool for I/O.

5. Typed HTTP Errors

```

use actix_web::{http::StatusCode, HttpResponse, ResponseError};
use std::fmt;

#[derive(Debug)]
enum ApiError {
    InvalidSource,
    Overloaded,
}

impl fmt::Display for ApiError {
    fn fmt(&self, formatter: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            Self::InvalidSource => write!(formatter, "invalid source"),
            Self::Overloaded => write!(formatter, "service overloaded"),
        }
    }
}

impl ResponseError for ApiError {
    fn status_code(&self) -> StatusCode {
        match self {
            Self::InvalidSource => StatusCode::BAD_REQUEST,
            Self::Overloaded => StatusCode::SERVICE_UNAVAILABLE,
        }
    }

    fn error_response(&self) -> HttpResponse {
        HttpResponse::build(self.status_code())
            .body(self.to_string())
    }
}

```

Error layer	Suitable information
Internal error	Source chain, operation, IDs, private diagnostics
Log/trace event	Correlation ID, sanitized context, latency
Client response	Stable code, safe message, appropriate HTTP status

Avoid exposing filesystem paths, database errors, backtraces, secrets, or parser internals to an untrusted client.

6. Middleware and Handler Responsibilities

Concern	Best location
Correlation/request ID	Middleware
Access logging and timing	Middleware
Authentication context	Middleware/extractor
Route-specific authorization	Handler or domain service
Input extraction	Typed extractor
Business rules	Domain function independent of Actix

HTTP response mapping	Handler / <code>ResponseError</code>
-----------------------	--------------------------------------

Body-consuming extractors need special care in middleware because the body is a stream. A middleware that consumes it must deliberately restore or replace what downstream handlers receive.

7. Test the Service Without Opening a Port

```
use actix_web::{http::StatusCode, test, web, App};
use std::sync::atomic::AtomicU64;

#[actix_web::test]
async fn accepts_a_job() {
    let state = web::Data::new(AppState {
        next_job: AtomicU64::new(1),
    });

    let app = test::init_service(
        App::new()
            .app_data(state)
            .app_data(json_config())
            .configure(configure),
    )
    .await;

    let request = test::TestRequest::post()
        .uri("/jobs")
        .set_json(&CreateJob {
            source: "1 + 2".to_owned(),
        })
        .to_request();

    let response = test::call_service(&app, request).await;
    assert_eq!(response.status(), StatusCode::OK);
}
```

Test routing and HTTP conversion at the Actix layer, but keep most language parsing, type checking, and compilation tests independent of the web framework.

8. What `wasm-bindgen` Generates

WebAssembly itself has a deliberately small boundary based on numeric values, linear memory, functions, tables, imports, and exports. `wasm-bindgen` generates glue that maps more ergonomic Rust and JavaScript values across that lower-level interface.

```
Rust source + #[wasm_bindgen]
↓ rustc --target wasm32-unknown-unknown
.wasm module
↓ wasm-bindgen CLI/tooling
processed Wasm + JavaScript/TypeScript bindings
```

Layer	Responsibility
Rust compiler	Produce the Wasm module

<code>wasm-bindgen</code> macro	Describe imported/exported boundary items
Binding generator	Produce JS glue and metadata
JavaScript loader	Instantiate the module and satisfy imports
Wasm linear memory	Store Rust stacks, heaps, strings, and buffers

9. Export Rust Functions and Types

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn read_u32_be(bytes: &[u8]) -> Result<u32, JsValue> {
    let bytes: [u8; 4] = bytes
        .get(..4)
        .ok_or_else(|| JsValue::from_str("expected four bytes"))?
        .try_into()
        .map_err(|_| JsValue::from_str("invalid byte slice"))?;

    Ok(u32::from_be_bytes(bytes))
}

#[wasm_bindgen]
pub struct Counter {
    value: u32,
}

#[wasm_bindgen]
impl Counter {
    #[wasm_bindgen(constructor)]
    pub fn new() -> Self {
        Self { value: 0 }
    }

    pub fn increment(&mut self) -> u32 {
        self.value = self.value.saturating_add(1);
        self.value
    }
}
```

An exported Rust struct appears to JavaScript through a generated wrapper/handle. Its fields remain private unless explicitly exported through methods or accessors.

10. Import JavaScript Functions

```

use wasm_bindgen::prelude::*;

#[wasm_bindgen]
extern "C" {
    #[wasm_bindgen(js_namespace = console)]
    fn log(message: &str);
}

#[wasm_bindgen]
pub fn announce(message: &str) {
    log(message);
}

```

An imported JavaScript function is an environmental dependency just like an OS system call or host function in a custom VM. Keep imports narrow so the module’s capabilities are easy to audit and replace in tests.

11. Value and Ownership Boundaries

Rust-side value	Boundary behavior to remember
Numbers/booleans	Converted directly when supported
<code>String</code> / <code>&str</code>	Encoded/decoded through Wasm memory
<code>Vec<u8></code> / <code>&[u8]</code>	Typically copied to/from typed-array-compatible data
<code>JsValue</code>	Dynamic escape hatch; loses Rust type precision
Exported Rust struct	JavaScript wrapper holds a Rust-side allocation
Imported JS object	Rust holds a generated handle to a host object
<code>Option<T></code>	Often maps to a nullable/optional host value

⚠ Memory-view hazard: Wasm memory can grow. JavaScript typed-array views into old memory may become stale after growth, so reacquire views according to the generated API instead of retaining them indefinitely.

12. Error Design Across JavaScript

Use `Result<T, JsValue>` when a Rust export can fail for a JavaScript caller:

```

use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn checked_frame_size(length: u32) -> Result<u32, JsValue> {
    const MAX_FRAME: u32 = 64 * 1024;
    if length > MAX_FRAME {
        return Err(JsValue::from_str("frame exceeds limit"));
    }
    Ok(length)
}

```

Boundary choice	Trade-off

<code>JsValue</code> error	Natural JS interop, weak Rust-side structure
Numeric error code	Compact ABI, requires an external error table
Serialized error	Stable schema, conversion overhead
Exported error type	Rich API, more bindings and lifetime management

Do not expose Rust panics as normal validation. Convert expected failure into `Result` and install deliberate panic diagnostics only for unexpected bugs.

13. Using Wasm as a Language Target

For your own language, WebAssembly can be:

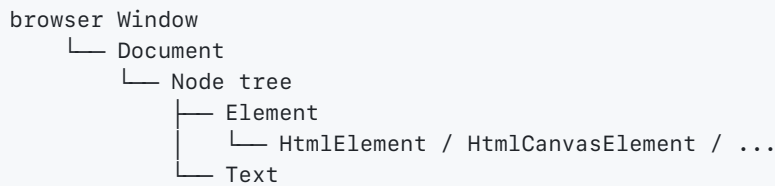
Role	Architecture
Compilation target	Source → typed IR → Wasm
Sandboxable plugin format	Host supplies a small import capability set
Browser runtime	Language VM compiled to Wasm, UI supplied by JS
Portable bytecode	Server/CLI embeds a Wasm engine

Wasm language-target contract	Decision to define
Imports	Module and function names
Representation	Value and memory layouts
Allocation	Ownership across host/module calls
Strings/errors	Encoding and failure representation
Memory growth	Whether growth is permitted and view invalidation
Resource limits	Fuel, wall time, stack, and memory
Capabilities	Deterministic/nondeterministic host functions

This is the same design work as an FFI ABI and a network protocol: **representation, ownership, versioning, and trust** must all be explicit.

14. The DOM Is a Host-Owned Object Graph

As the `wasm-bindgen` [DOM examples](#) demonstrate, the browser's Document Object Model is **not stored inside Wasm linear memory**. `web-sys` exposes typed Rust handles to JavaScript-managed objects.



Rust/Wasm holds generated JS handles → objects above
Rust values, stack, heap, strings → Wasm linear memory

Type	Role
Window	browser global, timers, location, event target
Document	element creation and document-wide lookup
Node	common tree operations such as <code>append_child</code>
Element	attributes, selectors, classes, element content
HTMLElement	HTML-specific style, focus, and common properties
EventTarget	add/remove event listener boundary
JsValue	dynamic JavaScript value and error carrier

`web-sys` APIs are feature-gated. Enable only the browser types and methods your crate uses; a compile error about a missing method may mean a missing Cargo feature rather than a missing browser API.

15. Create and Attach DOM Nodes

```

use wasm_bindgen::prelude::*;

#[wasm_bindgen(start)]
pub fn start() -> Result<(), JsValue> {
    let window = web_sys::window()
        .ok_or_else(|| JsValue::from_str("window is unavailable"))?;
    let document = window
        .document()
        .ok_or_else(|| JsValue::from_str("document is unavailable"))?;
    let body = document
        .body()
        .ok_or_else(|| JsValue::from_str("document has no body"))?;

    let status = document.create_element("p")?;
    status.set_id("compiler-status");
    status.set_text_content(Some("Compiler ready 🦀"));
    body.append_child(&status)?;
    Ok(())
}

```

Prefer `set_text_content` when displaying untrusted text. `set_inner_html` parses HTML, so using it with untrusted source, diagnostics, names, or network data can create an injection vulnerability.

Lookup/creation API	Result shape	Design response

<code>web_sys::window()</code>	<code>Option<Window></code>	host environment may differ
<code>window.document()</code>	<code>Option<Document></code>	worker has no <code>Document</code>
<code>get_element_by_id</code>	<code>Option<Element></code>	missing markup is expected
<code>query_selector</code>	<code>Result<Option<Element>, _></code>	selector may be invalid/missing
<code>create_element</code>	<code>Result<Element, JsValue></code>	browser operation may throw
<code>append_child</code>	<code>Result<Node, JsValue></code>	tree mutation may fail

16. Cast Only at a Checked Boundary

A CSS selector returns a general `Element`. Use `JsCast::dyn_into` when an operation requires a more specific browser type.

```
use wasm_bindgen::{JsCast, JsValue};
use web_sys::HtmlInputElement;

fn source_text(document: &web_sys::Document) -> Result<String, JsValue> {
    let element = document
        .get_element_by_id("source")
        .ok_or_else(|| JsValue::from_str("missing #source"))?;
    let input = element
        .dyn_into::<HtmlInputElement>()
        .map_err(|_| JsValue::from_str("#source is not an input element"))?;
    Ok(input.value())
}
```

Cast style	Use
<code>dyn_ref::<T>()</code>	checked borrowed view; keep original handle
<code>dyn_into::<T>()</code>	checked consuming conversion
<code>unchecked_ref::<T>()</code>	glue boundary only when type is already proven
<code>unchecked_into::<T>()</code>	rare; caller assumes the complete type contract

Unchecked casting is analogous to `unsafe`: keep it local and document what establishes the runtime JavaScript type.

17. DOM Event Listeners Need Lifetime Design

Browser callbacks are JavaScript functions, while Rust closures live in Wasm-managed state. Dropping a `Closure` while JavaScript still holds its callback is invalid. Calling `forget()` keeps it alive forever, which is acceptable only for a truly page-lifetime listener.

```

use wasm_bindgen::{closure::Closure, JsCast, JsValue};
use web_sys::{EventTarget, MouseEvent};

struct ClickListener {
    // Retaining the target lets `Drop` unregister from the same object.
    target: EventTarget,
    // JavaScript may call this later, so the Rust owner must keep it alive.
    callback: Closure<dyn FnMut(MouseEvent)>,
}

impl ClickListener {
    fn install(
        target: EventTarget,
        mut on_click: impl FnMut(MouseEvent) + 'static,
    ) -> Result<Self, JsValue> {
        // Move the user callback into a Wasm closure with a JavaScript-callable shim.
        let callback = Closure::new(move |event: MouseEvent| on_click(event));

        // This unchecked view is narrow: `Closure` guarantees that its exported
        // JavaScript value is a function suitable for EventTarget.
        target.add_event_listener_with_callback(
            "click",
            callback.as_ref().unchecked_ref(),
        )?;

        // Return both owners together so registration and lifetime cannot drift apart.
        Ok(Self { target, callback })
    }
}

impl Drop for ClickListener {
    fn drop(&mut self) {
        // Removal is best-effort in `Drop`; destructors cannot return an error.
        let _ = self.target.remove_event_listener_with_callback(
            "click",
            self.callback.as_ref().unchecked_ref(),
        );
    }
}

```

The application must retain the `ClickListener` for as long as the UI is active. This RAII shape pairs registration with removal and works well inside a Rust application struct exported through `wasm-bindgen`.

Callback strategy	Lifetime/result
local <code>Closure</code> , dropped	listener becomes invalid
<code>closure.forget()</code>	page-lifetime leak by design
store in app state	listener lives with component/application
RAII listener wrapper	automatically unregisters when owner is dropped

18. UI State Crosses Two Memory Managers

```

use std::{cell::Cell, rc::Rc};

let clicks = Rc::new(Cell::new(0_u32));
let callback_state = Rc::clone(&clicks);

let listener = ClickListener::install(button.into(), move |_event| {
    callback_state.set(callback_state.get().saturating_add(1));
})?;

```

Ownership edge	What keeps it alive
Rust state captured by closure	Wasm closure environment
<code>Closure</code> callback wrapper	Rust owner or deliberate <code>forget()</code>
DOM node	browser DOM tree and JS references
Rust handle to DOM node	generated JS reference table/glue
typed-array view of Wasm memory	JS value, but backing memory may grow

`Rc<Cell<T>>` is appropriate for small single-threaded browser state. Use `Rc<RefCell<T>>` for more complex mutable state, but keep borrows short and never call unknown JavaScript while a mutable `RefCell` borrow is held: JavaScript can re-enter Rust through another callback.

19. DOM Work Is Also Performance Work

Expensive pattern	Better direction
alternate layout reads and writes	group reads, compute, then group writes
rebuild a large subtree for one text edit	mutate the smallest stable node
copy huge buffers for every event	batch, reuse, or expose bounded views carefully
compile on every keystroke immediately	debounce/cancel stale work
block the main thread with language work	use a Web Worker or incremental execution

For a browser-hosted language:

```

DOM input event
  ↓ capture bounded source text
debounced compile request
  ↓ lexer → parser → type checker → Wasm/backend
structured diagnostics
  ↓ escape as text, map byte spans to source view
small batched DOM update

```

Keep the compiler core independent of `web-sys`. A browser adapter should translate DOM events into plain Rust inputs and translate structured compiler results back into UI updates. That preserves CLI, test, server, and browser front ends over one language implementation.

➡ **Next:** SDL exposes native windows, input, timing, audio, and drawing without the browser's document and JavaScript host layers.

🎮 Event Loops, Input, Audio, and Rendering Through SDL3

 **Library lens:** SDL3 exposes the portable platform layer beneath games and media: window ownership, event polling, clocks, audio buffers, input state, and presentation.

The [SDL3 API by category](#) is best read as a map of platform services. SDL does not impose a game architecture; it gives a portable boundary over windows, input, graphics, audio, timing, files, threads, and devices.

1. Read the API by Responsibility

API family	What it gives the program	Boundary to design carefully
initialization	subsystem startup/shutdown	partial failure and cleanup
video	windows, displays, surfaces	OS-owned handles
events	keyboard, mouse, touch, window/device events	queue draining and latency
2D render	draw state, textures, primitives, present	frame lifetime and backend state
GPU	explicit modern graphics work	synchronization and resource lifetime
audio	devices, streams, formats	real-time callback constraints
gamepad/joystick	controllers, axes, buttons, hotplug	unstable device identities
time	ticks, delay, high-resolution timing	units, overflow, frame pacing
threads/synchronizing	worker threads, mutexes, conditions	ownership and shutdown
I/O/storage	streams, paths, filesystem helpers	untrusted bytes and platform paths
properties/logging	extensible metadata and diagnostics	type agreement and observability

 **SDL3 migration note:** do not paste SDL2 error checks blindly. Many SDL3 operations—including `SDL_Init` and `SDL_CreateWindowAndRenderer`—return `bool`, with `true` meaning success.

2. Resource Lifetime Is the First Invariant

```
SDL_Init
  ↓
create window + renderer
  ↓
poll events → update state → render → present
  ↓
destroy renderer → destroy window
  ↓
SDL_Quit
```


Every successful acquisition must have one cleanup path. Destroy resources in an order compatible with their dependencies.

3. Minimal SDL3 Window and Render Loop

```
#include <SDL3/SDL.h>
#include <stdbool.h>

int main(void) {
    SDL_Window *window = NULL;
    SDL_Renderer *renderer = NULL;

    if (!SDL_Init(SDL_INIT_VIDEO)) {
        SDL_Log("SDL_Init failed: %s", SDL_GetError());
        return 1;
    }

    if (!SDL_CreateWindowAndRenderer(
        "Language Runtime", 800, 600, 0, &window, &renderer)) {
        SDL_Log("window/renderer creation failed: %s", SDL_GetError());
        SDL_Quit();
        return 1;
    }

    bool running = true;
    while (running) {
        SDL_Event event;
        while (SDL_PollEvent(&event)) {
            if (event.type == SDL_EVENT_QUIT) {
                running = false;
            }
        }

        SDL_SetRenderDrawColor(renderer, 18, 22, 30, 255);
        SDL_RenderClear(renderer);

        SDL_FRect box = {120.0f, 90.0f, 240.0f, 120.0f};
        SDL_SetRenderDrawColor(renderer, 88, 166, 255, 255);
        SDL_RenderFillRect(renderer, &box);
        SDL_RenderPresent(renderer);
    }

    SDL_DestroyRenderer(renderer);
    SDL_DestroyWindow(window);
    SDL_Quit();
    return 0;
}
```

Check the return value of fallible drawing operations in production code. A concise tutorial may omit checks to keep the lifecycle visible; an engine should surface errors with operation, resource, and frame context.

4. Separate Platform Events from Simulation State

```

OS/SDL events
  ↓ normalize
application commands
  ↓ update
deterministic world state
  ↓ interpolate/view
draw commands
  ↓ renderer/GPU
presented frame

```

Layer	Example data
raw event	key code, device ID, window event
normalized command	<code>MoveLeft(true)</code> , <code>Resize { width, height }</code>
simulation state	position, velocity, editor selection
presentation snapshot	visible sprites, boxes, glyphs

Do not let gameplay or language semantics depend directly on platform key codes. An input-mapping layer makes tests deterministic and lets keyboard, gamepad, touch, and replay files produce the same commands.

5. Variable Rendering, Fixed Simulation

```

elapsed = now - previous
accumulator += clamp(elapsed)

while accumulator >= fixed_step:
    update(fixed_step)
    accumulator -= fixed_step

render(interpolation = accumulator / fixed_step)

```

Timing mistake	Result
simulation speed tied to frame count	behavior changes across machines
unbounded catch-up after a pause	spiral of death
mixed milliseconds/nanoseconds	enormous or tiny time steps
nondeterministic input inside update	replays/tests diverge

SDL provides timing primitives such as `SDL_GetTicksNS`; convert once into a well-named duration type. Clamp unusually large elapsed intervals after breakpoints, suspend/resume, or window dragging.

6. Rendering APIs Form a State Machine

The renderer remembers draw color, blend mode, target, viewport, clip rectangle, and texture state. Make state transitions explicit rather than relying on whatever the previous draw left behind.

```
begin frame
  set target/viewport/clip
  clear
  draw ordered opaque and transparent work
  reset temporary state
present
```

Representation	Best use
surface	CPU-side pixels, loading/manipulation
texture	renderer-owned image used for drawing
2D renderer	portable sprites, rectangles, simple UI
GPU API	explicit pipelines, buffers, shaders

7. Audio and Device Events Need Special Discipline

Audio work runs under tight real-time constraints. Avoid blocking I/O, long locks, allocation spikes, and logging in the time-critical path. Feed an audio stream from bounded buffers and define underrun behavior.

Controllers can appear and disappear while the program runs:

```
device-added → open/identify → map controls → sample
device-removed → stop using handle → release → notify application
```

Use a logical player/device handle rather than treating a temporary enumeration index as permanent identity.

8. Wrap SDL Handles at a Safe Language Boundary

```
struct WindowHandle {
    raw: *mut sdl_sys::SDL_Window,
}

impl Drop for WindowHandle {
    fn drop(&mut self) {
        if !self.raw.is_null() {
            unsafe { sdl_sys::SDL_DestroyWindow(self.raw) };
        }
    }
}
```

The full wrapper must also define:

Contract	Question
nullability	Can creation or lookup return null?
thread affinity	Which thread may call this operation?
aliasing	Can two safe wrappers believe they uniquely own one handle?
callbacks	How long do captured values live, and can calls re-enter?

destruction order	Which child resources must be dropped first?
strings/buffers	Who owns bytes, and for how long are pointers valid?

Keep raw pointers private. A custom language runtime should expose stable integer handles or capability objects, validate every lookup, and never let untrusted scripts forge native addresses.

9. SDL as a Backend for a Small Language

```
script/DSL
  ↓ parse + type/check capabilities
runtime commands: CreateWindow, DrawRect, PlayTone, PollInput
  ↓ validated host adapter
SDL resources and event queue
  ↓
OS window, devices, audio, graphics
```

The host adapter should enforce limits on windows, textures, audio buffers, file paths, and per-frame commands. This turns SDL from an unrestricted FFI surface into a capability-based standard library.

 **Next:** Bevy builds a data-oriented scheduler and application framework over the same event/update/render lifecycle.

Application Architecture Through Bevy, Diesel, and Tauri

 **Library lens:** Bevy, Diesel, and Tauri serve as case studies for ECS/data-oriented design, relational algebra and transactions, and capability-checked native/web process boundaries.

These frameworks solve different problems, but they all reward the same habits: explicit data models, narrow boundaries, scheduled work, bounded resources, and testable domain logic.

Framework	Primary problem	Architectural lesson
Bevy	data-driven games/visual applications	behavior as scheduled systems
Diesel	typed relational database access	schema/query contract in Rust types
Tauri 2	cross-platform webview applications	least-privilege frontend/backend IPC

1. Bevy: Data-Oriented Application Structure

The [Bevy quick start](#) introduces a modular, data-focused Rust engine based on the Entity Component System (ECS) paradigm.

```
Entity    = identity only
Component = typed data attached to an entity
System    = function that queries/transforms data
Resource  = unique world-wide data
Schedule  = when and in what constraints systems run
Plugin    = reusable bundle of systems/resources/configuration
```

--	--

ECS concept	Compiler/runtime analogy
entity	stable arena ID or object handle
component	fact attached to an ID
query	select IDs possessing a required fact combination
system	compiler pass over selected data
resource	target configuration, interner, diagnostic sink
schedule	pass pipeline and dependency ordering
plugin	optional language feature/backend package

A Small Scheduled Model

```

use bevy::prelude::*;

#[derive(Component)]
struct Velocity(Vec2);

#[derive(Resource, Default)]
struct SimulationStats {
    moved_entities: u64,
}

fn spawn(mut commands: Commands) {
    commands.spawn((
        Transform::default(),
        Velocity(Vec2::new(3.0, -1.0)),
    ));
}

fn integrate(
    time: Res<Time>,
    mut stats: ResMut<SimulationStats>,
    mut movers: Query<(&Velocity, &mut Transform)>,
) {
    for (velocity, mut transform) in &mut movers {
        transform.translation +=
            velocity.0.extend(0.0) * time.delta_secs();
        stats.moved_entities += 1;
    }
}

fn main() {
    App::new()
        .add_plugins(DefaultPlugins)
        .init_resource::()
        .add_systems(Startup, spawn)
        .add_systems(Update, integrate)
        .run();
}

```

Bevy can parallelize systems when their parameter access is compatible. A system that reads `Res<T>` can overlap with other readers; a `ResMut<T>` or mutable component query declares exclusive access to that data.

ECS Design Questions

Question	Better direction
component or resource?	per-entity fact vs one world-wide value
component or enum field?	queried independently vs always part of one state
event/message or component?	transient notification vs persistent state
one huge system?	separate transformations with explicit ordering
every entity has everything?	compose only the data each behavior requires

Do not use ECS merely to avoid ordinary structs. It is strongest when large collections need different combinations of data-oriented behavior.

2. Diesel: Typed SQL Boundary

The [Diesel guides](#) separate setup, selects, updates, inserts, relations, schema generation, application composition, and extension. Diesel turns many schema/query mismatches into compile-time type errors, but runtime data, connectivity, transactions, constraints, and authorization still require deliberate handling.

```
migration files
  ↓ create/update database schema
generated Rust schema (`table!`)
  ↓ constrains typed query expressions
Selectable / Queryable / Insertable models
  ↓
domain conversion and application rules
```

Separate Row, Insert, and Domain Types

```

use diesel::prelude::*;

#[derive(Debug, Queryable, Selectable)]
#[diesel(table_name = crate::schema::compile_jobs)]
struct JobRow {
    id: i32,
    source: String,
    status: String,
}

#[derive(Insertable)]
#[diesel(table_name = crate::schema::compile_jobs)]
struct NewJob<'a> {
    source: &'a str,
    status: &'a str,
}

fn pending_jobs(
    connection: &mut SqliteConnection,
) -> QueryResult<Vec<JobRow>> {
    use crate::schema::compile_jobs::dsl;

    dsl::compile_jobs
        .filter(dsl::status.eq("pending"))
        .order(dsl::id.asc())
        .select(JobRow::as_select())
        .load(connection)
}

```

Type	Owns which contract
generated schema	SQL table/column types
<code>JobRow</code>	selected database row shape
<code>NewJob<'a></code>	allowed insert fields, borrowing inputs
domain <code>CompileJob</code>	validated application invariants
API response	intentionally exposed external fields

Do not use one struct for every layer merely because the fields currently match.

Transactions Protect Multi-Step Invariants

```

fn claim_job(
    connection: &mut SqlConnection,
    job_id: i32,
) -> QueryResult<JobRow> {
    // Keep the conditional state transition and the confirming read atomic.
    connection.transaction(|connection| {
        use crate::schema::compile_jobs::dsl;

        // The status predicate prevents overwriting a job claimed by another worker.
        // Production code must verify that exactly one row was changed.
        diesel::update(dsl::compile_jobs.find(job_id))
            .filter(dsl::status.eq("pending"))
            .set(dsl::status.eq("running"))
            .execute(connection)?;

        // Reload inside the same transaction so the returned row reflects this claim.
        dsl::compile_jobs
            .find(job_id)
            .select(JobRow::as_select())
            .first(connection)
    })
}

```

The sketch still needs an affected-row check or database constraint to distinguish “claimed” from “already unavailable.” Transactions provide atomicity; they do not invent the business invariant.

Diesel Boundary Checklist

Area	Decision
migrations	versioned, reviewed, reversible strategy
connection	pool size, timeout, health, transaction lifetime
async application	avoid blocking executor threads; use appropriate adapter/pool
query cardinality	<code>first</code> , <code>optional</code> , <code>one</code> , <code>many</code> , or <code>paginated</code>
updates	distinguish absent field from explicit SQL <code>NULL</code>
errors	map not-found, constraint, transient, and internal failures
observability	record sanitized query context and latency

3. Tauri 2: A Desktop Capability Boundary

[Tauri 2](#) combines a web frontend with native/mobile application code. JavaScript calls registered Rust commands across IPC; this makes every command a trust and serialization boundary.


```
untrusted or semi-trusted webview input
  ↓ deserialize command arguments
Tauri command
  ↓ validate + authorize
domain service
  ↓ narrow OS/database/network capability
structured response/error
  ↓ serialize to frontend
```

Register a Narrow Command Surface

```
use serde::Serialize;
use std::sync::atomic::{AtomicU64, Ordering};
use tauri::State;

struct AppState {
    next_request: AtomicU64,
}

#[derive(Serialize)]
#[serde(rename_all = "camelCase")]
struct CompileAccepted {
    request_id: u64,
    source_bytes: usize,
}

#[tauri::command]
fn compile_source(
    source: String,
    state: State<'_, AppState>,
) -> Result<CompileAccepted, String> {
    const MAX_SOURCE_BYTES: usize = 256 * 1024;

    if source.len() > MAX_SOURCE_BYTES {
        return Err("source exceeds configured limit".to_owned());
    }

    let request_id =
        state.next_request.fetch_add(1, Ordering::Relaxed);

    Ok(CompileAccepted {
        request_id,
        source_bytes: source.len(),
    })
}

pub fn run() {
    tauri::Builder::default()
        .manage(AppState {
            next_request: AtomicU64::new(1),
        })
        .invoke_handler(tauri::generate_handler![compile_source])
        .run(tauri::generate_context!())
        .expect("Tauri runtime failed");
}
```

The outer `expect` handles an unrecoverable application-startup failure. Input-dependent command failures remain structured `Result` values.

Capabilities and Permissions

Boundary	Least-privilege question
command registration	Does this command need to be callable at all?
capability configuration	Which windows/webviews receive which permissions?
filesystem	Which paths and operations are allowed?
shell/process	Can fixed programs replace arbitrary commands?
network	Which destinations and protocols are necessary?
plugin	Which plugin commands are exposed?
updater/deep links	How are authenticity and untrusted arguments checked?

Never turn a frontend string into an unrestricted shell command, path, SQL fragment, or URL fetch. Parse it into a typed request and authorize the resulting operation.

Events, Channels, and Commands

Mechanism	Best fit
command	bounded request/response
event	broadcast-style notification
channel	ordered streaming progress/data
managed state	shared native service/configuration handle

For long compilation, download, or indexing jobs, return a request ID promptly and send bounded progress events. Define cancellation and terminal states explicitly.

4. Zero to Production: Service Engineering

The [Zero to Production repository](#) is the evolving reference application for an opinionated Rust backend book. Its newsletter service connects Actix Web, PostgreSQL/SQLx, Redis-backed sessions, configuration, authentication, email delivery, telemetry, migrations, and API tests.



Production mental model: an HTTP handler is only the visible edge. A service is the complete system that can be configured, observed, migrated, tested, deployed, restarted, and operated without guessing.

Grow the System in Dependency Order

Layer	Question to settle
health endpoint	Can orchestration tell whether the process serves?
configuration	How do local, test, and production values differ?
database migration	How does persistent state evolve with the program?
request extraction	Which bytes become trusted domain inputs?

domain/use-case logic	Which invariant must hold independent of HTTP?
outbound dependency	How are email/cache calls bounded and retried?
authentication/session	Who is acting, and how is that identity established?
telemetry	Can an operator reconstruct a failed request?
deployment	Can the artifact start reproducibly in a fresh setting?

Build a thin vertical slice first:

```

TCP listener
  ↓ HTTP request
router → handler → validated command
                ↓
                domain service
                ↓
                repository/outbound port
                ↓
                structured response + trace

```

Each added dependency should arrive behind a narrow interface and with a test strategy.

Bind the Listener Outside the Application Factory

A testable server accepts a listener chosen by its caller. Tests can bind port `0` and ask the OS for an unused port instead of racing over a hard-coded one.

```

use actix_web::{web, App, HttpResponse, HttpServer};
use std::net::TcpListener;

async fn health_check() -> HttpResponse {
    // Liveness is intentionally cheap and independent of downstream services.
    HttpResponse::Ok().finish()
}

fn run(listener: TcpListener) -> std::io::Result<actix_web::dev::Server> {
    // The factory runs once per worker, so each worker receives its own `App`.
    let server = HttpServer::new(|| {
        App::new().route("/health_check", web::get().to(health_check))
    })
    // Accept a pre-bound listener so tests can safely ask the OS for port zero.
    .listen(listener)?
    .run();

    // Return the future-like server handle; the caller chooses where to spawn/await it.
    Ok(server)
}

```

Design choice	Testing/operations benefit
caller supplies <code>TcpListener</code>	ephemeral ports and pre-bound sockets
construction returns <code>Server</code>	test can spawn and control lifetime

startup is separate from routes	configuration errors fail before serving
health check avoids dependencies	distinguishes process health from deep readiness

Define **liveness** and **readiness** deliberately. A liveness probe should not restart a healthy process just because a downstream database briefly fails; readiness can stop new traffic when the service cannot fulfill requests.

Configuration Is Parsed Input

```
use serde::Deserialize;

#[derive(Debug, Deserialize)]
struct Settings {
    application: ApplicationSettings,
    database: DatabaseSettings,
}

#[derive(Debug, Deserialize)]
struct ApplicationSettings {
    host: String,
    port: u16,
    base_url: String,
}

#[derive(Debug, Deserialize)]
struct DatabaseSettings {
    host: String,
    port: u16,
    database_name: String,
    username: String,
}
```

Treat configuration like source code:

Stage	Configuration equivalent
lex/parse	deserialize file/environment values
type-check	ports are numbers; URLs and enums are valid
link	referenced files, secrets, and hosts exist
execute	initialize dependencies and start listeners
diagnostic	name the field, source, bad value class, and fix

Secrets should use a wrapper that avoids accidental `Debug` /log output. Validate all non-secret settings at startup so failures appear before the service accepts traffic.

One Application Object Owns Startup

```

struct Application {
    port: u16,
    server: actix_web::dev::Server,
}

impl Application {
    fn port(&self) -> u16 {
        self.port
    }

    async fn run_until_stopped(self) -> std::io::Result<()> {
        self.server.await
    }
}

```

The construction path should assemble configuration, database pool, outbound clients, secrets, middleware, and routes. This creates one place to reason about dependency lifetimes and startup failures.

Integration Tests Should Use Real Boundaries Selectively

```

#[tokio::test]
async fn health_check_works() {
    let app = spawn_test_app().await;

    let response = request::get(format!(
        "http://127.0.0.1:{}", app.port
    ))
    .await
    .expect("request failed");

    assert!(response.status().is_success());
    assert_eq!(response.content_length(), Some(0));
}

```

Test type	Real component	Replace/fake
unit	domain function	time, IDs, ports as values
handler	extraction/response mapping	use-case service
API integration	TCP/HTTP router and database	external email/payment service
migration	clean database with migration set	production data
end-to-end smoke	deployed artifact and critical route	destructive or expensive actions

Give each integration test isolated database state. Parallel tests sharing tables create false failures and conceal ordering dependencies.

Transactions Preserve a Use-Case Invariant

```

begin transaction
  validate current persistent state
  write all related rows
  record an outbox event if external work must follow
commit

```

Do not hold a database transaction open while calling an email server or unrelated network API. The database cannot atomically roll back the remote side effect. An **outbox** records the intent in the same transaction; a worker delivers it with an idempotency key and retry policy.

Failure question	Design tool
client retries the same request	idempotency key/unique constraint
process dies after commit	transactional outbox
remote service is temporarily unavailable	bounded retry with backoff
request is canceled while work continues	explicit job ownership/cancellation
two workers claim one item	atomic update/locking with state check

Structured Telemetry Carries Causality

```

use tracing::{info, instrument};

#[instrument(
  name = "register subscriber",
  skip(repository),
  fields(request_id = %request_id)
)]
async fn register(
  request_id: &str,
  repository: &Repository,
) -> Result<(), RegisterError> {
  repository.insert_pending().await?;
  info!("subscriber stored");
  Ok(())
}

```

Record	Avoid
request/trace ID	passwords, tokens, raw session cookies
route name and status class	full sensitive request bodies
sanitized error chain	silently discarded causes
dependency latency and outcome	high-cardinality secrets as field names
deployment/version metadata	relying only on free-form strings

Propagate tracing context into blocking tasks and async work. Logs without causality become a pile of unrelated sentences under concurrency.

Production Boundary Checklist

Concern	Minimum deliberate decision
timeouts	connect, request, database, and shutdown deadlines
retries	which errors, how many, with what backoff/jitter
limits	body size, concurrency, pool size, queue depth
errors	stable public response; detailed internal cause
auth/session	secret rotation, cookie attributes, expiry, revocation
migrations	forward/backward compatibility during rolling deployment
shutdown	stop new work, drain bounded work, close resources
observability	metrics, traces, sanitized structured logs
supply chain	locked dependencies, audits, reproducible build artifact

The service is ready for production when failure paths are designed and observable—not merely when the happy-path request returns `200`.

5. One Architecture Using All Four

Tauri webview

↕ typed commands/events

Rust application core

├ Diesel repository → project/job metadata

├ compiler/runtime → parse, type, execute

├ Bevy view/plugin → visualization or interactive simulation

└ Actix service → remote jobs, health, telemetry

Layer	Keep independent of
domain model	Tauri command types, Diesel row types, Bevy ECS
repository trait	webview and rendering
compiler core	database connection and DOM
adapters	one another unless orchestration requires it

This separation lets a CLI, server, Tauri app, and Bevy visualization reuse the same compiler core without making the core depend on every framework.

 **Next:** practical reversing reads framework and runtime boundaries backward: begin with behavior and artifacts, then infer states, layouts, IPC, queries, and calls.

This section draws from [0xZ0F's Reverse Engineering Course](#), which progresses from binary fundamentals through x64 assembly, tools, basic reversing, DLLs, and Windows internals. The course is Windows-focused, but the analysis habits transfer to other architectures and operating systems.

Use these techniques only on binaries you own, intentionally vulnerable exercises, open-source programs, or targets you are explicitly authorized to analyze. Run unknown samples only in an isolated environment designed for that purpose.

 **Reversing habit:** separate **observation**, **hypothesis**, and **validation**. Decompiler output is evidence, not ground truth.

1. Reverse Engineering Is Model Building

The goal is not to translate every instruction into English. The goal is to recover enough structure to explain relevant behavior.

```
raw bytes
  ↓ executable-format metadata
instructions + data
  ↓ control-flow and calling-convention analysis
functions + variables + structures
  ↓ behavior and state analysis
high-level model
```

A productive model answers:

Model dimension	Question
Inputs	What enters the program?
Transformation	Which functions change it?
State	What memory, files, or globals are read/written?
Decisions	Which branches control behavior?
Dependencies	Which OS or library services are called?
Effects	What output or side effect is produced?

Reverse engineering is iterative. Names and types begin as guesses and become more precise as evidence accumulates.

2. Start with Binary Triage

Before debugging, collect facts without executing the target:

Question	Evidence to inspect
What format is it?	PE headers, magic bytes, architecture
32-bit or 64-bit?	Machine type and optional-header format
Native or managed?	Imports, metadata directories, runtime dependencies

What does it import?	DLL names and imported symbols
What does it export?	Export table and symbol names
Are symbols/debug data present?	PDB path, symbol table, debug directory
Does it contain useful text?	UTF-8/ASCII and UTF-16 strings
Is it signed or versioned?	Signature and version resources
Does layout look ordinary?	Sections, permissions, sizes, entropy

Record a cryptographic hash before analysis so notes always refer to one exact file.

PE Mental Model

```

DOS header / stub
  ↓ points to
PE signature + COFF header
  ↓
optional header
  ├── entry point
  ├── image base
  ├── alignment
  └── data directories
section table
  ├── code
  ├── read-only data
  ├── writable data
  ├── imports/exports
  └── resources/relocations

```

The term **optional header** is historical; executable images rely on it.

PE coordinate/size	Meaning
Raw/file offset	Location in the file on disk
RVA	Offset from the loaded image base
VA	Runtime address, normally <code>image_base + RVA</code>
Raw size	Bytes stored in the file for a section
Virtual size	Bytes occupied by the section after loading

Do not paste a runtime address into a file-offset field without converting it through the appropriate section mapping.

3. x64 Register Roles

Register names partly encode width:

64-bit	Low 32 bits	Low 16 bits	Low 8 bits
RAX	EAX	AX	AL

RBX	EBX	BX	BL
RCX	ECX	CX	CL
RDX	EDX	DX	DL

Register(s)	Common Windows x64 role
RIP	Address of the next instruction
RSP	Stack pointer
RBP	Optional frame/base pointer
RAX	Integer return register and accumulator
RCX , RDX , R8 , R9	First integer/pointer argument positions
XMM0 – XMM3	First floating/vector argument positions
XMM0	Floating-point return value

Writing a 32-bit general register such as `EAX` clears the upper 32 bits of the corresponding 64-bit register. Writing only `AX` or `AL` does not.

Volatile and Nonvolatile Registers on Windows x64

Category	Registers	Meaning at a call
Volatile	<code>RAX</code> , <code>RCX</code> , <code>RDX</code> , <code>R8</code> – <code>R11</code>	Caller must assume they are overwritten
Nonvolatile	<code>RBX</code> , <code>RBP</code> , <code>RFI</code> , <code>RSI</code> , <code>RSP</code> , <code>R12</code> – <code>R15</code>	Callee must restore them if used

This classification is evidence. A function that stores a value in `RBX` and restores `RBX` before returning may be keeping that value alive across several calls.

4. Flags, Comparisons, and Branches

`cmp left, right` behaves like a subtraction used only to update flags:

```
left - right → flags
```

Flag	Interpretation
ZF	Result was zero
CF	Unsigned carry or borrow
SF	Result sign bit
OF	Signed overflow
PF	Parity of the low byte

Signed and unsigned conditional jumps interpret flags differently:

Meaning	Signed jump	Unsigned jump
Equal	<code>je / jz</code>	<code>je / jz</code>
Not equal	<code>jne / jnz</code>	<code>jne / jnz</code>
Greater than	<code>jg</code>	<code>ja</code>
Greater or equal	<code>jge</code>	<code>jae</code>
Less than	<code>jl</code>	<code>jb</code>
Less or equal	<code>jle</code>	<code>jbe</code>

Choosing `jg` versus `ja` is a clue about whether the compiler treated a value as signed or unsigned.

5. Read Instructions by Effect

Group instruction variants by the effect that matters to the current question:

Family	Mental effect
<code>mov</code> , <code>movzx</code> , <code>movsx</code>	Copy or extend a value
<code>lea</code>	Compute an address-like expression
<code>add</code> , <code>sub</code> , <code>inc</code> , <code>dec</code>	Arithmetic/state update
<code>and</code> , <code>or</code> , <code>xor</code> , <code>test</code>	Bit manipulation or flag test
<code>cmp</code> + conditional jump	Decision
<code>push</code> , <code>pop</code>	Stack update
<code>call</code> , <code>ret</code>	Function control flow
<code>shl</code> , <code>shr</code> , <code>sar</code>	Shift or scale
<code>imul</code> , <code>idiv</code>	Signed multiply/divide

Do not assume `lea` means “load a pointer.” Compilers also use it for arithmetic such as:

```
lea eax, [rcx + rcx*4] ; eax = rcx * 5
```

Likewise, `xor eax, eax` is commonly a compact way to set `EAX` to zero.

6. Windows x64 Calling Convention

Calling-convention concern	Windows x64 rule
Integer args 1–4	<code>RCX</code> , <code>RDX</code> , <code>R8</code> , <code>R9</code>
Float/vector args 1–4	Corresponding <code>XMM0</code> – <code>XMM3</code> positions
Later arguments	Passed on the stack

Shadow/home space	Caller reserves 32 bytes
Stack alignment	Follows the ABI's 16-byte call-boundary rule
Integer return	Commonly <code>RAX</code>
Floating return	Commonly <code>XMM0</code>
C++ <code>this</code>	Usually occupies the first integer argument position

Conceptual call:

```
result = transform(context, 10, buffer, length, flags);
```

Possible setup:

```
mov rcx, context      ; argument 1
mov edx, 10           ; argument 2
mov r8, buffer        ; argument 3
mov r9d, length       ; argument 4
mov [rsp+20h], flags  ; argument 5, after shadow space
call transform
; integer-like result now in RAX
```

Registers may be prepared in any order. Follow definitions reaching the `call`, not the visual order of setup instructions.

Inferring a Prototype

At each call site, record:

1. which argument registers are assigned;
2. which stack slots are written;
3. whether XMM registers are used;
4. what values flow into each position;
5. how the return register is used afterward;
6. whether the same function is called elsewhere with different evidence.

Then propose the narrowest type that explains all observed calls.

7. Function Boundaries, Prologues, and Epilogues

A traditional function may:

```
save nonvolatile registers
allocate stack space
initialize local state
perform work and calls
restore stack/registers
return
```

But optimized functions may omit a frame pointer, reuse stack slots, inline calls, split into hot/cold regions, or end with a tail call. Do not rely on one prologue byte pattern.

Boundary evidence	Why it helps
Direct call target	Names a likely entry point
Unwind metadata	Describes a recoverable function range
Exported symbol	Exposes a callable entry
Cross-references	Show other code treating the address as a function
Stack setup/restoration	Suggests a coherent frame
Return/tail-call region	Closes a connected control-flow graph

8. Static and Dynamic Analysis Work Together

Static analysis	Dynamic analysis
Does not execute the target	Observes one concrete execution
Shows broad possible control flow	Shows the path actually taken
Good for strings, imports, cross-references	Good for runtime values and indirect calls
Can inspect unreachable or rare paths	Can reveal decoded or generated data
May struggle with packing/indirection	Can be timing- or environment-dependent

A productive cycle:

```
static hypothesis
  ↓ choose breakpoint and input
dynamic observation
  ↓ rename/retype/comment
better static model
  ↓
repeat
```

9. Debugger Controls and Their Meaning

Debugger control	Effect
Breakpoint	Stop before a chosen instruction
Step into	Execute and follow a call target
Step over	Let a call finish and stop after it
Step out	Run until the current function returns
Run to cursor	Continue to a selected location
Data breakpoint/watchpoint	Stop when a memory range is accessed or modified

Conditional breakpoint	Stop only when a value/count matches a condition
-------------------------------	--

Before stepping, write down what you expect to change. After stepping, compare the registers, flags, stack, and relevant memory to that prediction.

High-Signal Breakpoint Locations

Location	Question it can answer
Known imported function	Which external service is being used?
Comparison before a branch	Which value decides the path?
State-variable write	Who changes the state and when?
Function entry	Which arguments arrive?
Function return	How does the caller use the result?
Parser/trust boundary	How do raw bytes become structured values?

Breakpoints are most useful when tied to a question.

10. Recovering Function Calls

Suppose disassembly prepares a format string and values before calling a printing function. Work backward from the call:

```
call known_print
  ↑ argument 4 came from R9
  ↑ argument 3 came from R8
  ↑ argument 2 came from RDX
  ↑ argument 1 came from RCX
```

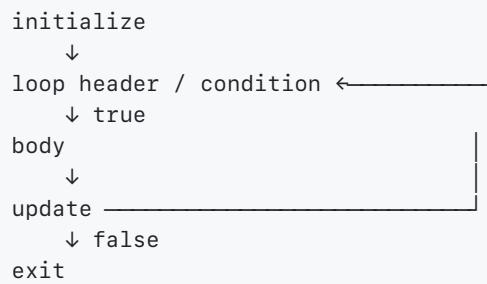
Then inspect:

Call-site evidence	What it reveals
String at a pointer	Format, path, command, or semantic landmark
Register write width	Likely integer width and zero/sign extension
Stack arguments	Parameters beyond the first four
XMM arguments	Floating-point/vector parameters
Variadic behavior	Extra arguments interpreted through a format/contract
Shadow/alignment setup	ABI mechanics rather than source variables

This often reconstructs a recognizable call without understanding the whole function.

11. Recovering Loops

Source keywords disappear, so identify the control-flow shape:



Loop clue	Likely meaning
Backward branch	Control-flow cycle
Preheader initialization	Initial counter or pointer
Increment/decrement	Induction-variable update
Comparison with a bound	Loop termination condition
Fixed-stride pointer move	Array/record traversal
Repeated call/memory access	Loop body operation

`for`, `while`, and `do-while` loops may compile to nearly identical control-flow graphs. Recover behavior first; choose high-level syntax later.

Example:

```

xor  ebx, ebx      ; i = 0
loop_start:
mov  edx, ebx      ; use i
call process_item
inc  ebx           ; i += 1
cmp  ebx, 10
jle  loop_start    ; signed i < 10

```

Possible reconstruction:

```

for (int i = 0; i < 10; i++) {
    process_item(i);
}

```

12. Recovering Structures from Offset Patterns

Repeated accesses through one base register often describe fields:

```

mov  eax, [rcx+08h]
add  eax, [rcx+0Ch]
mov  [rcx+10h], eax

```

Working hypothesis:

```

struct Unknown {
    unsigned char unknown_00[8];
    int field_08;
    int field_0c;
    int field_10;
};

```

Structure evidence	Inference it supports
Access width	Field size
Signed/unsigned branch	Possible numeric interpretation
Pointer dereference	Pointer-like field
Constructor-like initialization	Initial layout and defaults
Repeated offsets	Stable field positions
Type-specific imported API	Likely semantic type
Array stride	Element size

Rename `field_08` only when evidence supports a semantic name. Until then, a stable offset-based name is more honest than a confident guess.

13. Arrays and Indexing

An address expression such as:

```

mov eax, [rcx + rdx*4]

```

suggests:

```

base = RCX
index = RDX
element size = 4 bytes

```

Possible source:

```

value = array[index];

```

Scale factors 1, 2, 4, and 8 directly fit x86 addressing modes. Other structure sizes may use several instructions or pointer increments.

14. DLLs, Imports, and Exports

A Windows DLL is a PE image intended to provide code or data to other modules.

DLL concept	Meaning
Export table	Names/ordinals made available to callers

Import table	External symbols required from other DLLs
Import address table (IAT)	Runtime slots containing resolved function addresses
Module base	Address where the image is mapped
RVA	Offset relative to the module base
Loader lock	Synchronization context affecting DLL initialization

For a normal imported call, code may call indirectly through an IAT slot:

```
call qword ptr [imported_function_slot]
```

The instruction references a slot in the current module; the loader places the actual library function address into that slot.

When reconstructing an undocumented exported function:

1. start from its export entry;
2. inspect all known call sites;
3. infer argument positions from the ABI;
4. observe return-value use;
5. identify referenced structures and globals;
6. build a small prototype only after the evidence agrees.

15. Tool-Agnostic Analysis Views

Different tools use different names, but the useful views are consistent:

View	Question it answers
Hex/bytes	What is physically encoded here?
Disassembly	Which instructions will execute?
Decompiler	What high-level model does the tool infer?
Control-flow graph	Where can execution branch or loop?
Cross-references	Who uses this function, string, or address?
Imports/exports	Which module boundaries are visible?
Memory map	Which runtime regions exist and with what permissions?
Registers/flags	What is the current CPU state?
Stack	Who called whom, and what values are nearby?
Breakpoints/trace	What happened during this run?

Treat a decompiler as an editable hypothesis generator. Correct its types, function boundaries, names, and signatures as evidence improves.

16. A Repeatable Authorized Lab

Use a tiny program you compile yourself:

```
#include <stdio.h>

static int sum_positive(const int *items, int count) {
    int total = 0;
    for (int i = 0; i < count; i++) {
        if (items[i] > 0) {
            total += items[i];
        }
    }
    return total;
}

int main(void) {
    int values[] = {3, -2, 7, 0};
    printf("%d\n", sum_positive(values, 4));
    return 0;
}
```

Practice:

1. compile debug and optimized builds;
2. identify the executable format and architecture;
3. locate the output string or imported print function;
4. find the call to `sum_positive`;
5. map the arguments using the platform ABI;
6. identify the loop's backward edge;
7. infer the 4-byte element stride;
8. identify the signed `> 0` test;
9. compare debug and optimized control flow;
10. annotate the binary until another reader can follow it.

17. CTF Challenge Patterns as Safe Practice

The public [crackmes.one RE CTF 2026 repository](#) contains handouts and, for some challenges, source and official writeups. Use only the published challenge artifacts inside an isolated lab; do not transfer techniques to systems or binaries you are not authorized to analyze.

Challenge pattern in the repository	Skill to practice
custom virtual machine/compiler	recover instruction format and VM state
nested executable/library layers	identify loaders, formats, and handoff points
custom encoding/validation	derive transforms from observed code
time-derived input	distinguish entropy from a bounded search space
simulated routers/switches	reconstruct packet and distributed state models
Windows screensaver/registry state	follow OS-specific configuration and execution

small server artifact

combine static triage with controlled behavior

The Custom-VM Workflow

FlipVM is especially relevant to creating and reversing languages because the repository includes a compiler, virtual machine, bytecode-like files, and a syntax highlighter.

```
virtual source
  ↓ compiler
custom instruction file
  ↓ VM fetch/decode/execute
virtual registers + memory + syscalls
  ↓
observable validation behavior
```

Builder question	Reverser's mirror question
How are opcodes encoded?	Which bytes select each operation?
What is the operand format?	Where are registers, immediates, and offsets?
How is control flow represented?	Which fields change the virtual instruction pointer?
What state does the VM own?	Which memory region/struct stores virtual state?
Which operations cross to the host?	Where are virtual syscalls dispatched?
Is code transformed or randomized?	Which representation remains invariant?

Start by documenting the VM format and state transition; do not begin by renaming every host instruction.

Layered Challenge Triage

1. hash and identify the handout
2. read the challenge's supported OS/architecture notes
3. inspect format, sections, imports, strings, and entropy
4. run only in the appropriate isolated snapshot
5. capture one baseline behavior
6. change one input
7. locate the first comparison or transformation affected
8. reconstruct the smallest relevant state machine
9. validate on a fresh input
10. compare with published source/writeup only after your attempt

Evidence	Record
artifact	cryptographic hash and repository path
environment	OS, architecture, loader, arguments, snapshot
static observation	address/file offset, bytes, xrefs, inferred type
dynamic observation	breakpoint, input, register/memory change
hypothesis	predicted transformation or state transition

validation

new case that could have disproved the hypothesis

The simulated-network challenge is a useful reminder that an IP-looking value may be part of a **custom model**, not a real protocol implementation. Recover the program's actual frame layout and handlers rather than assuming standards from familiar names.



Lab rule: keep challenge networking disconnected or routed only to controlled local services. Treat bundled executables as untrusted even when they are educational.

18. Nightmare: Memory-Corruption Labs & Mitigations

Nightmare organizes binary-exploitation practice around assembly, reversing, Ghidra, GDB, Python tooling, stack bugs, and common mitigations. The most durable lesson is not a particular payload: it is learning to connect a source-level memory bug to machine state and then explain what each mitigation does—and does **not**—prevent.



Authorized-lab boundary: use deliberately vulnerable CTF programs, binaries you compiled yourself, or targets with explicit written authorization. Keep them in an isolated environment. The goal here is root-cause analysis and defense.

Build the Model Before Studying Exploitability

```
untrusted bytes
  ↓ parse/copy/index
stack or heap object
  ↓ bug violates bounds/lifetime/type invariant
adjacent state may change
  ↓
crash, corrupted output, or altered control/data flow
```

Evidence layer	Question
source	Which check or ownership rule is missing?
compiler output	Where are objects, bounds, and branches represented?
ABI/frame	Which registers and stack slots have calling-convention roles?
memory map	Which pages are readable, writable, or executable?
debugger state	Which instruction first observes the invalid state?
mitigation metadata	Which hardening features are present in this build?

Do not equate “program crashed” with “control flow is exploitable.” Exploitability also depends on reachable data, overwrite precision, memory layout, mitigations, allocator behavior, concurrency, and environmental constraints.

Common Memory-Bug Families

Bug class	Broken invariant	Defensive direction
out-of-bounds	index/range lies within allocated object	checked indexing, slices, fuzzing, ASan

write		
out-of-bounds read	every read refers to initialized accessible bytes	bounds checks, length-aware parsing, MSan
use-after-free	object outlives every reference	ownership, RAII, temporal sanitizers
double free	exactly one owner performs destruction	single ownership and invalidation
integer overflow	size arithmetic represents intended allocation	checked math before allocation/copy
format-string misuse	data is not interpreted as a format program	constant format strings, typed formatting
uninitialized data	value is written before it is observed	initialization, compiler warnings, MSan

```
#include <stdbool.h>
#include <stddef.h>
#include <string.h>

bool copy_name(char out[32], const char *src, size_t len) {
    if (src == NULL || len >= 32) {
        return false;
    }

    memcpy(out, src, len);
    out[len] = '\0';
    return true;
}
```

The length check is part of the API contract. In real code, prefer a destination slice or a function that receives the actual destination capacity instead of relying on an array-looking parameter that decays to a pointer.

Mitigations Form Layers, Not a Substitute for a Fix

Mitigation	What it changes	What remains true
NX/DEP	writable data pages are normally non-executable	data corruption and code reuse may remain
ASLR	runtime locations vary	information disclosure can weaken it
PIE	main executable can be relocated	bug still exists
stack canary	detects some overwrites before returning	not every corruption touches the canary
RELRO	hardens dynamic-link relocation metadata	other writable data remains
CFI	restricts allowed indirect control-flow targets	data-only attacks and logic bugs remain
sanitizers	expose violations during testing	production builds may not use full checks

safe Rust	prevents broad classes in safe code	<code>unsafe</code> , FFI, logic, and resource bugs remain
-----------	-------------------------------------	--

Return-oriented programming is a **code-reuse** concept motivated partly by non-executable data. At a high level, an attacker attempts to compose short existing instruction sequences while satisfying the ABI, stack alignment, and control-flow constraints. For defensive study, recognize why NX alone is insufficient and why ASLR, CFI, shadow stacks, reduced gadget surfaces, and eliminating the original write primitive matter.

Controlled Crash-to-Fix Workflow

1. record source/binary hash, compiler, flags, architecture, and mitigations
 2. reproduce with one minimal local input
 3. capture the first invalid read/write—not merely the final crash
 4. map the instruction back to an object, length, and ownership invariant
 5. write a regression test that fails safely
 6. fix the source-level invariant
 7. rerun warnings, sanitizers, tests, and fuzz cases
 8. compare hardened and unhardened builds to understand mitigation evidence

Notebook claim	Strong evidence
"overflow begins here"	sanitizer report plus instruction/memory access
"this field controls the branch"	data-flow trace and changed-input experiment
"ASLR is active"	build metadata and changing runtime mappings
"the fix works"	regression case plus broader randomized testing
"the issue is unreachable remotely"	proven input path and deployment configuration

19. Authorized Memory Inspection & Tooling Labs

The following sources cover dual-use tools and ideas:

- [Black Hat Rust](#) uses offensive-security projects to teach async I/O, trait-based modules, crawling, fuzzing, `no_std` , cross-compilation, and security thinking.
- [GameHackingCode](#) contains chapter-oriented Windows examples for memory, pointers, debugging, scanning, state machines, control flow, and graphics.
- The [Cheat Engine wiki](#) documents value scans, pointers, structures, debugging, assembly, and Lua automation.

Use these ideas on **your own toy process, an included tutorial target, an offline open-source sample, or an explicitly authorized assessment**. Do not bypass anti-cheat/DRM, alter multiplayer software, collect another user’s data, conceal activity, or deploy persistence.

Reframe Offensive Projects as General Systems Lessons

Source topic	General-purpose lesson	Safe exercise
concurrent reconnaissance	bounded work queues, timeouts, backpressure	inventory services on a loopback lab

async scanner	many I/O waits without one thread per socket	probe ports on containers you started
trait-object modules	heterogeneous plugins behind one contract	pluggable file-format inspectors
crawler	URL normalization, deduplication, rate limits	crawl a local documentation server
fuzzing	generate inputs that violate parser assumptions	fuzz your own language lexer/parser
<code>no_std</code> /cross-compilation	runtime dependencies, targets, object layout	tiny freestanding diagnostic program
memory scanner	typed representation and search-space refinement	search a byte buffer or tutorial process
state-machine recognition	infer state from transitions and observable fields	reverse a toy offline game you compiled
graphics/control-flow sample	API boundaries, callbacks, render-loop structure	instrument your own SDL sample

Some source chapters go further into phishing, implants, worms, injection, and evasion. Those are **not implementation exercises for these notes**. Study them as threat-model categories: identify trust boundaries, telemetry signals, least privilege, egress controls, code-signing policy, and incident-response evidence.

Bounded Concurrency for an Authorized Inventory Tool

```

use std::{future::Future, sync::Arc};
use tokio::sync::Semaphore;

async fn run_bounded<I, F, Fut>(items: I, limit: usize, inspect: F)
where
    I: IntoIterator<Item = String>,
    F: Fn(String) -> Fut + Clone + Send + 'static,
    Fut: Future<Output = ()> + Send + 'static,
{
    // `max(1)` prevents a caller-supplied zero from deadlocking all work.
    let permits = Arc::new(Semaphore::new(limit.max(1)));
    let mut tasks = Vec::new();

    for item in items {
        // Acquire before spawning: pending inputs wait here without becoming tasks.
        let permit = Arc::clone(&permits)
            .acquire_owned()
            .await
            .expect("semaphore closed");
        let inspect = inspect.clone();

        tasks.push(tokio::spawn(async move {
            // The owned permit travels with the task and limits active inspections.
            inspect(item).await;
            // Explicit for teaching purposes; scope exit would release it too.
            drop(permit);
        })));
    }

    // Join all tasks so the function does not return while work is still running.
    for task in tasks {
        // A production version should record or propagate JoinError explicitly.
        let _ = task.await;
    }
}

```

This is an architectural sketch, not a complete network scanner. A real authorized inventory tool also needs explicit target allowlists, per-operation deadlines, global rate limits, cancellation, sanitized logs, bounded results, and a record of who approved the scope.

Concurrency control	Failure it prevents
semaphore	unbounded in-flight work
timeout	a silent peer holding work forever
rate limiter	overwhelming the target or network
bounded channel	producers exhausting memory
cancellation token	abandoned work continuing after scope ends
target allowlist	accidental expansion outside authorization

Memory Scanning Is Iterative Set Filtering

Cheat Engine's core scan model is general:


```

candidate addresses = every readable, in-scope location
  ↓ filter by type/alignment/value or pattern
smaller candidate set
  ↓ change the toy program's state
filter by changed / unchanged / increased / decreased / new exact value
  ↓ repeat
one or a few hypotheses to validate in the debugger/source

```

Scan choice	Meaning
exact value	bytes decode to the specified typed value
unknown initial value	begin broadly, then filter by later relationships
changed/unchanged	compare snapshots without knowing the semantic value
increased/decreased	use an observed state transition
array of bytes	find a byte signature with explicit wildcard positions
region filter	search only mappings consistent with the hypothesis

Value type is not decoration. The same bytes can represent an integer, float, pointer, text fragment, bitfield, or instruction. Track **width**, **endianness**, **alignment**, **signedness**, and **encoding**.

Safe Scanner Example: Search an Owned Byte Buffer

```

fn find_u32_le(haystack: &[u8], wanted: u32) -> Vec<usize> {
    // Convert the typed value to the byte order used by the searched format.
    let needle = wanted.to_le_bytes();

    haystack
        // `windows(4)` visits every complete candidate without out-of-bounds reads.
        .windows(needle.len())
        .enumerate()
        // Return byte offsets; these are candidates, not proof of semantic meaning.
        .filter_map(|(offset, bytes)| (bytes == needle).then_some(offset))
        .collect()
}

fn retain_changed(
    old: &[u8],
    new: &[u8],
    candidates: &mut Vec<usize>,
    width: usize,
) {
    candidates.retain(|&offset| {
        // Validate the same half-open range against both snapshots.
        let end = match offset.checked_add(width) {
            Some(end) if end <= old.len() && end <= new.len() => end,
            _ => return false,
        };

        // Keep only locations whose selected bytes changed between observations.
        old[offset..end] != new[offset..end]
    });
}

```

This teaches the algorithm without opening another process. Extend it with typed decoders, alignment filters, masked patterns, and snapshot labels. Keep every arithmetic operation checked because scanners operate near buffer boundaries.

Stable Addressing: Modules, Offsets, and Pointer Paths

A runtime address can change between launches because allocations and module bases move. Record addresses symbolically:

```
module base + relative virtual address
heap object reached through: stable root → field offset → field offset
```

Observation	Interpretation to test
same module-relative offset	likely code/static object within one build
new heap address every launch	allocation is dynamic
pointer chain survives restart	path may reflect stable object relationships
chain fails after update	layout/build changed; never assume permanence
many matching pointers	candidates, not proof of semantic ownership

A pointer scan searches paths through stored addresses and offsets, then rescans after the target moves to eliminate paths that no longer resolve. This is graph search:

```
node = address-sized value/location
edge = dereference plus bounded field offset
goal = currently observed target object
```

Limit depth, offset range, regions, and alignment. Validate across multiple clean runs of the owned target. A surviving path is evidence, not a stable public API.

From Value to Structure

Once two or more fields appear near one another, propose a layout without naming it too early:

```
object candidate
+0x00 unknown pointer-sized field
+0x08 observed f32, changes with x movement
+0x0C observed f32, changes with y movement
+0x10 observed u32, bounded small range
```

Validation experiment	What it can reveal
watch instruction reads field	access width and consumer
compare several object instances	repeated stride and shared layout
change one controlled input	candidate field/state relationship
inspect constructor/init writes	defaults, ownership, and object extent

restart with symbols/debug build

compare inference to ground truth afterward

Use offset-based names such as `field_0c_f32?` until evidence supports a semantic name. This is the same discipline used when recovering an undocumented bytecode VM.

Debugger, Scanner, and Static Tool Answer Different Questions

Tool/view	Best question
value scanner	Where might this changing representation be stored?
memory map	Which regions could legitimately contain it?
"who accesses" watch	Which instructions consume this address?
breakpoint/trace	What state led to this instruction?
disassembler	What operations and branches exist?
decompiler	What higher-level hypothesis fits those instructions?
symbols/source	What was the intended contract?

Correlate at least two evidence types before committing to a conclusion. A changing number may be a display cache, previous-frame copy, network serialization buffer, or derived value rather than authoritative state.

Array-of-Bytes Signatures Are Version-Specific Hypotheses

exact opcode bytes + wildcard relocation/immediate bytes + surrounding context

Weak signature	Stronger direction
too few common bytes	add stable neighboring instruction structure
includes absolute addresses	wildcard relocation-dependent fields
matches many functions	include unique semantic context
assumes one compiler optimization	test debug/release and multiple builds
silently selects first match	require exactly one validated match

Signatures are useful for diagnostics and regression tooling on your own builds, but they are brittle. Prefer exported APIs, symbols, debug information, or explicit instrumentation when you control the program.

Cheat Tables and Lua Are Executable Content

The Cheat Engine wiki documents a broad Lua API for scanning, processes, memory, debugging, forms, and automation. A table can contain scripts, so treat an untrusted table like an untrusted executable:

Before opening/running a table	Defensive action
unknown source	do not execute; inspect in an isolated lab

embedded Lua/assembler	review every enabled script and callback
bundled binary/data	hash and inventory it
process attachment	confirm the exact tutorial/owned target
file/network operations	deny or isolate unless explicitly required
kernel/DBVM feature	avoid for ordinary learning; system-wide risk

Prefer plain notebook observations over downloading opaque tables. Automation should make a reproducible experiment safer, not hide what it changes.

Language-Builder Connection

Memory tools reveal why a language runtime needs explicit representation contracts:

Runtime choice	Reversing/tooling consequence
tagged value format	type bits distinguish integer/object/immediate
moving garbage collector	raw addresses are unstable across collections
handle table	stable IDs map to movable/native objects
bytecode dispatch loop	repeated fetch/decode state exposes VM structure
debug metadata	source spans and names make diagnostics trustworthy
capability-based FFI	scripts cannot forge arbitrary native pointers
deterministic serialization	snapshots and replay traces become comparable

When building your own language, add a **read-only debug protocol** that exposes object IDs, types, safe field summaries, stacks, and source spans. This provides legitimate introspection without requiring arbitrary process-memory access.

20. Reverse-Engineering Notebook Template

```
Target hash:
Architecture / ABI:
Image base:
Tool versions:

Question:
Observation:
Hypothesis:
Evidence for:
Evidence against:
Confidence:
Next experiment:

Address/RVA:
Proposed name:
Proposed prototype/type:
Callers:
Callees:
Important offsets:
Side effects:
```

Separating observation from inference prevents early guesses from becoming invisible assumptions.

21. Common Beginner Traps

Trap	Better habit
Read assembly top-to-bottom	Follow control flow and data dependencies
Treat stack adjustments as locals	Separate ABI mechanics from source values
Forget the calling convention	Annotate argument, return, and preserved registers
Treat every hex value as a pointer	Classify it from use and valid address ranges
Mix RVA/file offset/VA	Write down the coordinate system
Trust decompiler names/types	Verify against instructions and call sites
Rename unknown fields early	Keep stable offset-based names until evidence improves
Ignore width/signedness	Track each access width and branch interpretation
Step into every library call	Stay centered on the analysis question
Inspect one call site	Compare multiple callers and inputs
Expect source-shaped optimized code	Recover behavior before syntax
Save screenshots without context	Record addresses, inputs, hashes, and conclusions

This section is for software you own, purpose-built challenge binaries, and explicitly authorized laboratories. It explains mechanics so you can build debuggers, validate your own runtime, and recognize abuse. It does **not** provide remote-process injection, credential interception, anti-cheat/DRM bypass, persistence, or security-evasion code.

1. Begin with an Authorization and Evidence Contract

Before touching the target	Record
authority	owner, written scope, allowed machines/accounts
artifact	hashes, architecture, format, version, source
isolation	disposable VM/device, network policy, snapshot
data policy	whether secrets or personal data may appear
stopping rule	what behavior must not be triggered
evidence	timestamps, tool versions, observations vs hypotheses

The safest reverse-engineering input is an offline copy. Dynamic behavior belongs in a disposable lab with synthetic accounts and no valuable credentials.

2. Function Pointers Turn Data into Control Flow

A function pointer is a value interpreted as a callable code address under an ABI. Indirect calls appear in several higher-level forms:

Form	Representation clue
callback	function pointer plus optional context pointer
C function table	struct/array of function pointers
C++ virtual dispatch	object points to vtable; slot selects method
Rust trait object	data pointer plus vtable metadata
import table	loader-populated pointer to an external function
JIT dispatch	runtime-generated code pointer or entry table
bytecode VM	opcode indexes a handler table

To recover an indirect call, determine:

1. where the pointer came from;
2. which calling convention and signature the call site implies;
3. which concrete targets can populate that slot;
4. whether the pointer belongs to an expected executable mapping;
5. how the object/table lifetime is protected.

Calling an address with the wrong ABI or signature is undefined behavior even when the address points into valid executable memory.

3. Entity Structures Emerge from Correlated Accesses

Suppose authorized traces repeatedly show:

```
entity_base + index * 0x40 + 0x10  read as three floats
entity_base + index * 0x40 + 0x20  read as a 32-bit integer
entity_base + index * 0x40 + 0x28  followed as a pointer
```

This supports hypotheses—perhaps position, state, and component/owner pointer—but does not prove names. Validate each field independently:

Experiment	Observation
move only one entity	which 12-byte groups change?
toggle one state	which bit/word changes and when?
destroy/recreate	does a generation counter change?
reorder/spawn	is storage dense, sparse, or pointer-based?
cross a scene boundary	which base pointer or pool changes?

Draw a typed structure with `unknown_XX` fields and confidence labels. Preserve alignment and padding; never compress unknown bytes merely to make the sketch pretty.

4. A Safe Call Logger Wraps Your Own Interface

The following records calls to a callback the program already owns. It demonstrates instrumentation without patching code or another process:

```

#[derive(Debug, Clone, PartialEq, Eq)]
struct CallRecord {
    operation: &'static str,
    input: i64,
    output: i64,
}

struct LoggedCallback<F> {
    name: &'static str,
    inner: F,
    records: Vec<CallRecord>,
}

impl<F> LoggedCallback<F>
where
    F: FnMut(i64) -> i64,
{
    fn call(&mut self, input: i64) -> i64 {
        // Invoke the original behavior exactly once.
        let output = (self.inner)(input);
        // Log only explicitly approved, non-secret fields.
        self.records.push(CallRecord {
            operation: self.name,
            input,
            output,
        });
        output
    }
}

fn main() {
    let mut doubled = LoggedCallback {
        name: "double",
        inner: |value| value * 2,
        records: Vec::new(),
    };
    assert_eq!(doubled.call(21), 42);
    assert_eq!(doubled.records.len(), 1);
}

```

Production tracing needs bounded storage, sampling, redaction, recursion protection, thread identity, monotonic timestamps, and an explicit failure policy. Logging raw arguments can leak passwords, tokens, personal data, or cryptographic keys.

5. Pattern Scanning Is Bounded Byte Matching

A pattern scanner searches an **owned byte slice** for known bytes and optional wildcards. It is useful for file-format tests, compiler snapshots, firmware you built, and disposable challenge fixtures.


```

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum PatternByte {
    Exact(u8),
    Any,
}

fn find_pattern(haystack: &[u8], pattern: &[PatternByte]) -> Vec<usize> {
    // Define empty-pattern behavior explicitly instead of relying on `windows(0)`.
    if pattern.is_empty() || pattern.len() > haystack.len() {
        return Vec::new();
    }

    haystack
        .windows(pattern.len())
        .enumerate()
        .filter_map(|(offset, window)| {
            let matches = window.iter().zip(pattern).all(|(byte, expected)| {
                matches!(expected, PatternByte::Any)
                || matches!(expected, PatternByte::Exact(value) if byte == value)
            });
            matches.then_some(offset)
        })
        .collect()
}

#[test]
fn wildcard_pattern_finds_owned_fixture() {
    let bytes = [0x48, 0x8B, 0x01, 0x90, 0x48, 0x8B, 0xFF];
    let pattern = [
        PatternByte::Exact(0x48),
        PatternByte::Exact(0x8B),
        PatternByte::Any,
    ];
    assert_eq!(find_pattern(&bytes, &pattern), vec![0, 4]);
}

```

Patterns are brittle across compiler versions, link order, relocations, and optimization. Prefer symbols, debug metadata, exported IDs, structured headers, or a versioned introspection API when you control the software.

6. A Memory Scanner Is a Query Over Snapshots

Separate a scanner into:

```

authorized acquisition → immutable snapshot → typed decoding → predicate
→ candidate set → repeated experiment → report

```

Scan	Predicate over candidate values
exact	value equals a controlled input
range	value is within valid domain
changed/unchanged	current differs from / equals previous snapshot
increased/decreased	ordered comparison between snapshots
pointer-like	aligned value falls inside a known mapped region

structure

several offsets satisfy independent invariants

The existing [authorized memory-tooling labs](#) show safe snapshots and parsing. Acquisition is platform-sensitive and privileged; keep it outside the pure query engine and refuse targets outside the documented lab scope.

7. Call Traces Need Semantics, Not Just Addresses

A call trace becomes useful when normalized into:

Field	Why it matters
module plus relative offset	stable across address randomization
thread/task	separates interleaved control flow
timestamp/duration	reveals blocking and hot paths
call/return pairing	reconstructs nesting
typed arguments	only after ABI/signature recovery
correlation ID	connects file, network, and application events

Stack unwinding may use frame pointers, platform unwind metadata, debug information, or heuristics. Tail calls, optimized/inlined functions, exceptions, signal frames, and JIT code make “one source call equals one stack frame” unreliable.

8. DLL Loading and Injection: Mechanics and Defensive Signals

A DLL is a mapped executable image with imports, exports, relocations, sections, and initialization behavior. Normal loading already crosses sensitive boundaries:

```
resolve requested module → choose path → map image → relocate → resolve imports  
→ set page protections → run permitted initialization → expose exports
```

Unauthorized injection techniques try to cause a process to map or execute code it did not select. Defenders can look for combinations of:

- unexpected module paths, hashes, signers, or parent-child relationships;
- private executable pages not backed by an expected image;
- writable-to-executable protection transitions;
- thread starts or indirect targets outside known executable modules;
- import/function-table pointers leaving expected ownership ranges;
- cross-process handles with memory/thread rights;
- loader events that disagree with memory mappings;
- abnormal callbacks, scheduled work, or image metadata.

No single signal proves injection: JITs, profilers, debuggers, overlays, accessibility tools, and security products may legitimately produce similar evidence. Correlate process, memory, loader, signing, and behavioral telemetry.

9. Import Tables and Vtables Are Integrity Boundaries

The loader populates an import address table with resolved function addresses. A vtable or callback table similarly routes indirect calls. Altering a pointer can redirect control without modifying the call instruction itself.

Defensive validation asks:

- Is the table stored where the image/object model predicts?
- Does each target fall in an executable mapping owned by an allowed module?
- Does the in-memory table match the loaded image plus legitimate relocations?
- Did a writable transition or unexpected writer precede the change?
- Is control-flow integrity or pointer authentication available?

[ired.team's IAT-hooking walkthrough](#) can be read as an observable map: understand which table changes so defenders can measure ownership and integrity. Do not reproduce the mutation on third-party processes.

10. Obfuscation Adds Transformations, Not New Semantics

Common transformations include:

Transformation	Analyst response
renamed/stripped symbols	recover roles from data flow and call sites
encoded strings	locate decode boundary and record input/output
packed/compressed sections	identify format and observe authorized unpacking
opaque predicates	simplify using known invariants
control-flow flattening	reconstruct dispatcher state transitions
indirect calls	resolve pointer provenance and possible targets
virtualized bytecode	recover VM state, opcode format, and handlers

A safe deobfuscation workflow is:

```
identify representation → find transformation boundary → capture a small owned sample  
→ model inverse transform → validate on held-out samples → document uncertainty
```

Never treat “looks random” as proof of encryption. Measure entropy, locate headers, test compression/container formats, inspect key/nonce-sized data flow, and distinguish encoding, packing, and authenticated ciphertext.

11. Signature-Based and Heuristic Scanning Complement Each Other

Scanner	Matches	Strength	Weakness
exact hash	whole known artifact	precise	any byte change breaks match
		fast and	

byte/string signature	local stable sequence	explainable	version/obfuscation brittle
structural rule	import/section/format relationships	generalizes across builds	parser complexity
heuristic	weighted suspicious properties	finds novel variants	false positives
behavioral	observed actions/sequences	closer to intent	expensive and environment-dependent
reputation	signer/source/prevalence history	useful context	not proof of safety

Good rules bind several independent, explainable features and include negative conditions for known benign cases. Test against clean and malicious corpora, measure precision/recall, version rules, and preserve analyst override.

12. Hypervisor Introspection Changes the Observer

An in-guest tool relies on the guest kernel's view. Hypervisor-based introspection can observe guest-physical memory and selected execution events from outside that view. That can help validate a compromised guest, but creates a **semantic gap**:

```
physical bytes → guest page tables → virtual address spaces → kernel objects
→ process/module/runtime meaning
```

Every arrow depends on guest OS version, layout, and synchronization. Pause/snapshot policy affects consistency; encrypted guest memory and device state may limit visibility. Use symbol/type metadata and versioned profiles instead of hard-coded offsets where possible.

13. Research Catalogs as Defensive Indexes

Source	Best defensive use
secret.club	deep systems research, hypervisors, browsers, runtimes
HackTricks	platform assessment checklist and mitigation prompts
Cloud HackTricks	cloud identity/configuration review checklist
ired.team	Windows/identity technique-to-telemetry mapping
Game Hacking Academy	isolated game/CTF-style reversing concepts
CTF 101	intentionally vulnerable learning categories

These catalogs include offensive material. Keep only the conceptual and defensive parts relevant to an owned lab: preconditions, affected boundary, observable artifacts, mitigations, and validation tests. Current product behavior, laws, and platform protections must be checked against primary documentation.

Developer Environments: Shells, Git, Build Systems, and SSH

[The Missing Semester of Your CS Education](#) treats tool proficiency as a core computing skill: shells, development environments, debugging, profiling, version control, packaging, automation, and code quality.

 **Tooling principle:** if a workflow is repeated, error-prone, or difficult to explain, make its inputs, command, output, and failure status reproducible.

1. Terminal, Shell, and Program

Term	Meaning
Terminal	User interface that displays text and sends input
Shell	Language/runtime that parses commands and launches programs
Program	Executable selected by a path or <code>\$PATH</code> lookup
Process	Running program with memory, descriptors, environment, state
Job	Shell-managed pipeline/process group

The shell is itself a small programming language. It has parsing, quoting, expansion, variables, control flow, exit status, and composition—many of the same concerns as a language you build yourself.

2. Paths and Command Resolution

Path form	Interpretation
<code>/a/b</code>	Absolute path from filesystem root
<code>a/b</code>	Relative to the current working directory
<code>.</code>	Current directory
<code>..</code>	Parent directory
<code>~</code>	Shell expansion for a user's home directory
command name	Searched in the ordered directories of <code>\$PATH</code>
<code>./tool</code>	Exact path; bypasses <code>\$PATH</code> lookup

Useful inspection:

```
pwd
command -v rustc
type cargo
printf '%s\n' "$PATH"
```

Quote variable expansions unless intentional splitting/globbing is required:

```
input_file="My Program.tl"
cargo run -- "$input_file"
```

The `--` convention tells many programs that later values are operands rather than options. This matters when an untrusted filename begins with `-`.

3. Standard Streams and Composition

Descriptor	Stream	Conventional role
0	stdin	Program input
1	stdout	Primary machine/user output
2	stderr	Diagnostics and errors

```
producer stdout —pipe→ consumer stdin
producer stderr —————> terminal/log
```

```
compiler program.tl 2>diagnostics.log \
| disassembler \
| sort \
| uniq -c
```

Operator	Effect
<code>a b</code>	Connect <code>a</code> stdout to <code>b</code> stdin
<code>> file</code>	Replace file with stdout
<code>>> file</code>	Append stdout
<code>2> file</code>	Redirect stderr
<code>< file</code>	Read stdin from file
<code>tee file</code>	Copy stdin to both stdout and a file

Design CLI tools so normal output is composable on stdout and diagnostics go to stderr. A quiet machine-readable mode makes automation more reliable than scraping decorative terminal output.

4. Exit Status Is Part of the Interface

By convention, exit status `0` means success and a nonzero status means some form of failure.

```
if cargo test; then
  echo "tests passed"
else
  echo "tests failed" >&2
  exit 1
fi
```

CLI outcome	Suggested behavior
Successful result	Print result; exit <code>0</code>

User/input error	Explain on stderr; exit nonzero
Internal invariant failure	Preserve diagnostics; exit nonzero
Partial results	Define explicitly; do not silently claim success
Broken pipe	Handle according to CLI convention

5. Safer Shell Scripts

```
#!/usr/bin/env bash
set -euo pipefail

work_dir="$(mktemp -d)"
cleanup() {
    rm -rf -- "$work_dir"
}
trap cleanup EXIT

input="${1:?usage: check-program INPUT}"
output="$work_dir/program.out"

cargo run -- "$input" >"$output"
wc -c -- "$output"
```

Strictness feature	Effect
<code>set -e</code>	Exit after many unhandled command failures
<code>set -u</code>	Treat undefined variables as errors
<code>set -o pipefail</code>	Fail a pipeline when any component fails
<code>trap ... EXIT</code>	Run cleanup on shell exit
<code>"\$variable"</code>	Preserve one argument and suppress accidental globbing
<code>--</code>	End option parsing when supported

These flags reduce mistakes but do not make Bash a safe general-purpose language. Complex data structures, rich error recovery, or large scripts are signs that a Rust, Python, or other purpose-built program may be clearer.

6. Search and Data Wrangling

Task	Tool shape
Search text recursively	<code>rg PATTERN PATH</code>
List candidate files	<code>rg --files</code> OR <code>find</code>
Select columns/records	<code>awk</code>
Transform text	<code>sed</code>
Sort and count	<code>sort</code> , <code>uniq -c</code>

Inspect beginning/end	<code>head</code> , <code>tail</code>
Parse structured data	Format-aware tool rather than fragile regex

Example: count opcode names in a textual bytecode dump:

```
bytecode-dump program.bc \
| awk '{print $2}' \
| sort \
| uniq -c \
| sort -nr
```

Each pipeline stage should have one understandable contract. Save intermediate output when a pipeline becomes hard to debug.

7. Environment and Reproducibility

A process inherits an environment from its parent. Environment variables are strings, not typed configuration.

Configuration source	Strength	Risk
CLI argument	Explicit and visible in invocation	May appear in process listings
Environment variable	Convenient for deployment	Untyped; easy to inherit silently
Config file	Structured, reviewable, persistent	Needs discovery/version rules
Secret store	Access-controlled sensitive values	External dependency
Compiled default	Simple fallback	Requires rebuild to change

Precedence should be documented:

```
explicit CLI > environment > project config > user config > built-in default
```

Never print all environment variables into shared logs; they often contain credentials and tokens.

8. Debugging as Hypothesis Testing

```
reproduce
  ↓ minimize
observe one boundary
  ↓ form hypothesis
predict a new result
  ↓ test
fix cause
  ↓ add regression test
```

Evidence source	Question answered
Logs/traces	Which events and state transitions occurred?
Debugger	What are registers, variables, frames, and memory now?

System-call trace	Which kernel services were requested?
Network capture	Which bytes crossed the network boundary?
Compiler intermediate	Which pipeline stage first became wrong?
Core dump/crash report	What state existed at failure?

Do not add random logging everywhere. Put observations at representation boundaries: tokens, AST, typed IR, bytecode, FFI, file format, network frame, or system call.

9. Profiling: Measure the Correct Resource

Profile target	Useful measurement
CPU time	Samples, call stacks, instruction hotspots
Wall-clock latency	End-to-end timing and blocking waits
Allocation	Count, size, lifetime, and call site
I/O	Bytes, operations, queueing, and wait time
Cache behavior	Misses, branch behavior, locality
Concurrency	Lock contention, task wait, scheduler delay

Benchmark optimized builds and realistic inputs. Debug builds can distort compiler, parser, and allocator performance dramatically.

```
use std::time::Instant;

let start = Instant::now();
let result = compile_program(source)?;
let elapsed = start.elapsed();
eprintln!(
    "compiled {} bytes into {} bytes in {elapsed:?}",
    source.len(),
    result.len()
);
```

One timing is a clue, not a benchmark. Warmup, input distribution, system load, and variance all matter.

10. Git as a Content Graph

Git is easier to reason about as immutable objects and references:

```
working tree
  ↓ git add
index / staging area
  ↓ git commit
commit object → tree → blobs
  ↑
branch reference
```

Git concept	Mental model
Commit	Snapshot plus parent(s) and metadata
Branch	Movable name pointing to a commit
Tag	Usually a stable name for a commit
Index	Proposed next snapshot
Merge	Commit with multiple parents
Rebase	Recreate changes on a different parent chain
Detached HEAD	<code>HEAD</code> points directly to a commit

Use small, coherent commits. A good commit explains one change, includes its tests, and can be reviewed or reverted without pulling in unrelated work.

11. High-Signal Git Tools

Goal	Tool/approach
Inspect exact change	<code>git diff</code> , <code>git diff --staged</code>
Understand history	<code>git log --graph --decorate --oneline</code>
Find introduction of a line	<code>git blame</code> , then inspect the commit
Find first bad commit	<code>git bisect</code> with a repeatable test command
Temporarily save local changes	<code>git stash</code> with care
Work on another branch nearby	<code>git worktree</code>
Recover a lost reference	<code>git reflog</code>

Do not rewrite shared history casually. Branch names are references; commits remain reachable only while some reference or reflog keeps them alive.

12. Build Systems and Metaprogramming

A build system is a dependency graph plus commands:

```

source + grammar + dependencies
  ↓ compiler/build rule
generated parser + object files
  ↓ linker/package rule
artifact

```

Build-system property	Requirement
Inputs	Complete and explicit
Outputs	Predictable paths and formats

Dependencies	Correct edges between generated and consumed files
Incrementality	Rebuild only invalidated nodes
Reproducibility	Same declared inputs produce equivalent outputs
Failure	Stop and preserve the command/error that failed

Generated code should have one source of truth, deterministic formatting, and a clear regeneration command. Check it into version control only when repository policy and tool availability justify it.

13. Packaging and Shipping

Artifact question	Decision to record
Version	Semantic/compatibility policy
Target	OS, architecture, ABI, feature set
Dependencies	Resolved versions, licenses, provenance
Configuration	Build-time versus runtime values
Integrity	Hash/signature and verification path
Debuggability	Symbols, source maps, build IDs, crash metadata
Upgrade	Migration and rollback behavior

For a language implementation, version the source language, bytecode, package manifest, runtime ABI, and wire protocols separately when they can evolve independently.

14. CI Is an Executable Contract

```
format → lint → unit tests → integration tests → artifact checks
      → security/license checks → package
```

CI property	Good practice
Reproducible locally	Every CI step has an equivalent local command
Fast feedback	Cheap deterministic checks run first
Clear failure	Log the command, relevant versions, and failing test
Controlled secrets	Expose only to trusted jobs and avoid printing them
Artifact provenance	Record source revision and toolchain
Caching	Cache performance data, not correctness assumptions

15. Tooling for Your Own Language

Tool	Minimum useful capability

Formatter	Deterministic source output
Parser/AST dumper	Inspect syntax without executing
Type checker mode	Validate without code generation
IR/bytecode dumper	Stable machine-readable and human-readable forms
Disassembler	Decode every instruction with offsets
Test runner	Filters, stable exit status, structured results
Package inspector	Manifest, dependencies, hashes, compatibility
Debugger	Break, step, inspect state, explain source mapping
Profiler hooks	Stable function/pass IDs and event timing

The tooling interface is part of the language. Stable exit codes, diagnostics, source locations, and machine-readable output make editors, CI systems, and external tools possible.

16. Job Control Connects the Shell to OS Process State

The shell tracks a **job**—often a process group attached to a terminal—separately from an individual process ID.

Terminal action	Typical effect
<code>Ctrl-C</code>	terminal asks foreground job to interrupt (<code>SIGINT</code>)
<code>Ctrl-Z</code>	terminal asks foreground job to stop (<code>SIGTSTP</code>)
<code>jobs</code>	list jobs known to the current shell
<code>bg %1</code>	continue stopped job 1 in the background
<code>fg %1</code>	continue/attach job 1 in the foreground
<code>kill -TERM PID</code>	request graceful termination

```
terminal
  ↓ foreground process group
shell — launches pipeline → process group
  └─ job table: running/stopped/done
     └─ transfers terminal foreground ownership
```

Signals express requests or events; most do not guarantee immediate termination. `SIGKILL` cannot be handled for cleanup, so use it as a last resort after confirming the exact target.

Backgrounding with `&` does not automatically make a job durable:

Mechanism	What it solves	Remaining issue
<code>command &</code>	returns the prompt	still tied to shell/terminal output
<code>nohup</code>	ignores hangup and redirects by default	weak interactive management

disown	removes job from the shell's job table	no reattachment interface
tmux / screen	persistent detachable terminal sessions	not a service supervisor
service manager	restart, logs, dependencies, boot integration	needs explicit service definition

Use a service manager for production daemons; use a terminal multiplexer for interactive work that must survive a network disconnect.

17. Terminal Multiplexers, SSH, and Dotfiles Form a Workflow Layer

tmux organizes persistent work hierarchically:

```
server
├─ session: project
│   └─ window: editor
│       ├── pane: source
│       └─ pane: tests
│   └─ window: debugger
```

Object	Meaning
session	detachable workspace
window	tab-like full terminal view
pane	split region inside a window

Named sessions are easier to recover than anonymous ones:

```
tmux new -s compiler
tmux ls
tmux attach -t compiler
```

For remote work:

```
local terminal → SSH transport → remote shell → tmux session → editor/test/debug jobs
```

SSH/dotfile practice	Reason
use host aliases in config	centralize host, user, port, and key selection
verify host keys	detect an unexpected remote identity
use least-privilege keys	limit the effect of credential compromise
avoid agent forwarding by default	forwarded authority can be abused remotely
keep secrets out of dotfiles	version-controlled configuration spreads widely
make setup idempotent	repeated installation converges safely
document shell startup files	login and interactive shells load different files

A good dotfiles repository stores small, portable configuration modules and an explicit installer. Machine-specific paths, credentials, tokens, and private keys belong outside the repository.



Language-tooling connection: a compiler used over SSH should tolerate a lost terminal, send diagnostics to stderr, flush important progress, handle termination, and leave artifacts in a known recoverable state.



Automation as Reliable Systems Programming in Rust

[Automate the Boring Stuff with Python, 3rd Edition](#) progresses from Python basics and debugging into files, command-line programs, web scraping, spreadsheets, databases, documents, structured data, scheduling, notifications, images, OCR, GUI control, and speech. This chapter keeps that useful problem sequence but implements its examples in **Rust**, using ownership and typed errors to make each boundary visible.



Automation mental model: a script is a tiny data pipeline with side effects. Make inputs explicit, validate the transformation, preview the output, then commit the external change.



Example crates: later snippets use `regex`, `csv`, `serde` with `derive`, `serde_json`, `rusqlite`, `request` with `blocking / json`, and `image`. Select current compatible releases and only the features your tool needs.

1. Choose Automation by Boundary

Repetitive work	Prefer
rename/sort local files	<code>std::path</code> , metadata, explicit destination plan
edit structured text	CSV/JSON/XML parser, not ad hoc string splitting
collect web data	documented API first; respectful HTTP otherwise
update spreadsheet values	workbook library with formula/style awareness
query durable local records	SQLite with parameters and transactions
extract document text	format-specific PDF/Word library
schedule a recurring job	OS scheduler plus idempotent script
notify a person/system	email/message API with preview and rate limit
manipulate images	image library with original preserved
drive a GUI	last resort; screenshots, focus, and timing vary

Use the **highest-level stable interface** available:

```
library/API > documented CLI > file format > browser automation > mouse coordinates
```

2. Every Useful Script Has the Same Skeleton

```

use std::collections::HashSet;
use std::fs;
use std::io;
use std::path::{Path, PathBuf};

#[derive(Debug, Clone, PartialEq, Eq)]
struct Rename {
    source: PathBuf,
    destination: PathBuf,
}

fn plan_renames(root: &Path) -> io::Result<Vec<Rename>> {
    let mut sources = Vec::new();

    for entry in fs::read_dir(root)? {
        let path = entry?.path();
        // Select only ordinary `.txt` entries inside the explicit root.
        if path.is_file() && path.extension().is_some_and(|ext| ext == "txt") {
            sources.push(path);
        }
    }

    // Stable ordering makes previews and tests deterministic.
    sources.sort();
    let plans = sources
        .into_iter()
        .filter_map(|source| {
            let stem = source.file_stem()?.to_str()?;
            let destination = source.with_file_name(format!("{}.txt",
stem.trim().to_lowercase()));
            (source != destination).then_some(Rename { source, destination })
        })
        .collect();
    Ok(plans)
}

fn validate(plans: &[Rename]) -> io::Result<()> {
    let unique: HashSet<&Path> = plans.iter().map(|plan|
plan.destination.as_path()).collect();

    // Reject many-to-one normalization before performing any mutation.
    if unique.len() != plans.len() {
        return Err(io::Error::new(io::ErrorKind::AlreadyExists, "destination collision"));
    }
    if plans.iter().any(|plan| plan.destination.exists()) {
        return Err(io::Error::new(io::ErrorKind::AlreadyExists, "destination exists"));
    }
    Ok(())
}

fn apply(plans: &[Rename], dry_run: bool) -> io::Result<()> {
    for plan in plans {
        // Preview and real execution consume the exact same validated plan.
        println!("{}", plan.source.display(), plan.destination.display());
        if !dry_run {
            fs::rename(&plan.source, &plan.destination)?;
        }
    }
    Ok(())
}

```

Phase	Invariant
discover	only intended inputs are selected
parse	bytes/strings become typed values
plan	intended changes are represented without performing them
validate	no collision, escape, invalid state, or missing resource
preview	a human or test can inspect the plan
apply	each external mutation is checked
verify	final state matches the plan
report	failures identify the item and operation

3. Filesystem Automation Needs a Blast-Radius Limit

```

use std::io;
use std::path::{Path, PathBuf};

fn resolved_child(root: &Path, user_name: &str) -> io::Result<PathBuf> {
    // Canonicalization resolves `..` and symbolic links before the containment check.
    let trusted_root = root.canonicalize()?;
    let candidate = trusted_root.join(user_name).canonicalize()?;

    if !candidate.starts_with(&trusted_root) {
        return Err(io::Error::new(
            io::ErrorKind::PermissionDenied,
            "path escapes the allowed root",
        ));
    }
    Ok(candidate)
}

```

Risk	Safer design
broad recursive glob	explicit root, extension, and file-count cap
overwrite destination	fail unless an explicit overwrite mode exists
partial multi-file operation	plan, journal, and resumable/idempotent steps
extension mistaken for format	inspect magic/header and parse defensively
symbolic-link escape	resolve and re-check containment
lost original	backup/version control or recoverable move

Start with a temporary directory containing synthetic files. A script that is safe only when all inputs are perfect is not ready for valuable data.

4. Text and Regular Expressions

Regular expressions are small pattern languages. Use them when the input is genuinely textual and local—not as a substitute for an HTML, JSON, CSV, or programming-language parser.

```
use regex::Regex;

#[derive(Debug, PartialEq, Eq)]
struct LogLine<'a> {
    level: &'a str,
    request_id: &'a str,
    message: &'a str,
}

fn parse_log_line<'a>(pattern: &Regex, line: &'a str) -> Option<LogLine<'a>> {
    // Anchors in the compiled pattern require the complete trimmed line to match.
    let captures = pattern.captures(line.trim_end_matches(&['\r', '\n'][..]))?;
    Some(LogLine {
        level: captures.name("level")?.as_str(),
        request_id: captures.name("request_id")?.as_str(),
        message: captures.name("message")?.as_str(),
    })
}

fn log_pattern() -> Result<Regex, regex::Error> {
    // Compile once during program setup and reuse for every input line.
    Regex::new(
        r"^(?<level>INFO|WARN|ERROR)\s+request_id=(?<request_id>[A-Za-z0-9_-]{1,64})\s+message=(?<message>.*)$",
    )
}
```

Regex habit	Reason
raw string literal	reduces double escaping
named groups	documents field meaning
<code>fullmatch</code> when parsing	rejects trailing/leading surprises
bounded repetitions	limits pathological work and absurd fields
test near-misses	proves the pattern rejects almost-valid input
compile once	centralizes the pattern and avoids repeated parsing

For your own language, use regex for simple tokens only when it keeps precedence and source spans clear. Balanced nesting and contextual syntax belong in a parser.

5. Structured Files Preserve Types and Context

```

use serde::{Deserialize, Serialize};
use std::error::Error;
use std::fs::{self, File};
use std::io::BufWriter;
use std::path::Path;

#[derive(Debug, Deserialize)]
struct CsvRow {
    name: String,
    score: String,
}

#[derive(Debug, Serialize)]
struct Score {
    name: String,
    score: i64,
}

fn csv_to_json(source: &Path, destination: &Path) -> Result<(), Box<dyn Error>> {
    let mut reader = csv::Reader::from_path(source)?;
    let mut normalized = Vec::new();

    for row in reader.deserialize::<CsvRow>() {
        let row = row?;
        // Convert strings at the boundary; invalid numbers stop before output commit.
        normalized.push(Score {
            name: row.name.trim().to_owned(),
            score: row.score.parse()?,
        });
    }

    // A same-directory temporary file can be atomically renamed on common filesystems.
    let temporary = destination.with_extension("json.tmp");
    let output = BufWriter::new(File::create(&temporary)?);
    serde_json::to_writer_pretty(output, &normalized)?;
    fs::rename(temporary, destination)?;
    Ok(())
}

```

Format	Important edge
CSV	dialect, quoting, newline handling, column names
JSON	number precision, missing vs <code>null</code> , encoding
XML	namespaces, entity/security settings, mixed text
YAML	implicit types and unsafe object construction

Parse into a validated domain type before acting. Preserve unknown fields only when the round-trip contract requires them.

6. SQLite Turns a Script into a Small Durable System

```

use rusqlite::{params, Connection, Result};
use std::path::Path;

fn record_run(database: &Path, input_name: &str, output_hash: &str) -> Result<()> {
    let mut connection = Connection::open(database)?;
    // A transaction rolls back on early return unless `commit` succeeds.
    let transaction = connection.transaction()?;
    transaction.execute(
        "CREATE TABLE IF NOT EXISTS runs (
            input_name TEXT PRIMARY KEY,
            output_hash TEXT NOT NULL
        )",
        [],
    )?;
    transaction.execute(
        "INSERT INTO runs(input_name, output_hash)
        VALUES (?1, ?2)
        ON CONFLICT(input_name)
        DO UPDATE SET output_hash = excluded.output_hash",
        // Parameters keep untrusted values separate from SQL syntax.
        params![input_name, output_hash],
    )?;
    transaction.commit()
}

```

Placeholders keep data separate from SQL syntax. The context manager commits on success and rolls back on an exception.

Need	SQLite feature
avoid duplicate work	unique key plus upsert
multi-step invariant	transaction
evolve stored records	numbered schema migrations
inspect/debug state	ordinary queries and a schema
concurrent automation	short transactions and explicit busy handling

SQLite is not merely a bigger dictionary. Define a schema, constraints, migration strategy, backup, and failure behavior.

7. Web Automation Starts with Permission and Protocols

```

use request::blocking::Client;
use request::header::CONTENT_TYPE;
use serde_json::Value;
use std::error::Error;
use std::io;
use std::time::Duration;

fn fetch_json(url: &str) -> Result<Value, Box<dyn Error>> {
    let client = Client::builder()
        .user_agent("personal-automation/1.0")
        .connect_timeout(Duration::from_secs(3))
        .timeout(Duration::from_secs(15))
        .build()?;

    // Non-success status codes become explicit errors before body parsing.
    let response = client.get(url).send()?.error_for_status()?;
    let content_type = response
        .headers()
        .get(CONTENT_TYPE)
        .and_then(|value| value.to_str().ok())
        .unwrap_or("");
    if !content_type.starts_with("application/json") {
        return Err(io::Error::new(io::ErrorKind::InvalidData, "unexpected content
type").into());
    }
    Ok(response.json())
}

```

Before scraping:

Check	Why
documented API/feed	more stable and less ambiguous
terms and authorization	access does not imply permission to republish
<code>robots.txt</code> /site policy	communicates automated-access expectations
authentication boundary	never bypass login, paywall, or access control
request rate and caching	avoid unnecessary load
pagination/retry limits	prevent infinite or explosive work
data minimization	do not collect personal/sensitive fields casually

HTML selectors are assumptions about a changing document. Save a small permitted fixture for parser tests, and fail visibly when required elements disappear.

8. Spreadsheets, Documents, and Images Have Hidden Structure

Artifact	Preserve/check
spreadsheet	formulas vs cached values, types, merged cells, styles
PDF	pages, coordinates, text extraction order, OCR limits
Word file	paragraphs, runs, tables, styles, relationships

image	dimensions, color mode, orientation metadata, format
OCR output	confidence, reading order, and manual review

Never assume that a visually empty spreadsheet cell is absent, that PDF text extraction matches reading order, or that OCR output is exact. Keep the original artifact and render the generated result for visual verification when layout matters.

```
use image::codecs::jpeg::JpegEncoder;
use std::error::Error;
use std::fs::File;
use std::path::Path;

fn make_thumbnail(source: &Path, destination: &Path) -> Result<(), Box<dyn Error>> {
    // Decode into a typed pixel buffer; keep the original file untouched.
    let image = image::open(source)?;
    // `thumbnail` preserves aspect ratio inside the 512-by-512 bounding box.
    let thumbnail = image.thumbnail(512, 512).into_rgb8();
    let output = File::create(destination)?;
    // JPEG is lossy, so make the quality choice explicit at the output boundary.
    JpegEncoder::new_with_quality(output, 90).encode_image(&thumbnail)?;
    Ok(())
}
```

9. Scheduling Requires Idempotency

```
scheduler triggers job
  ↓ acquire single-run lock
load checkpoint
  ↓ discover only pending work
perform bounded/idempotent steps
  ↓ atomically store checkpoint
emit summary and exit status
```

Scheduled-job failure	Design response
runs twice	idempotency key or uniqueness constraint
previous run still active	lock with stale-owner policy
machine sleeps/offline	explicit catch-up or skip policy
network fails halfway	checkpoint and bounded retry
credentials expire	actionable alert without leaking the credential
output grows forever	retention and quota policy

The scheduler should invoke a normal command-line program. Keep scheduling policy out of the transformation core so the same job can be tested manually.

10. Notifications Are External Mutations

Separate rendering from sending:

```
#[derive(Debug, Clone, PartialEq, Eq)]
struct Message {
    recipient: String,
    subject: String,
    body: String,
}

fn build_summary(recipient: impl Into<String>, changed: usize) -> Message {
    // Construct a value only; a separate adapter performs the external send.
    Message {
        recipient: recipient.into(),
        subject: "Automation summary".to_owned(),
        body: format!("Changed items: {changed}\n"),
    }
}
```

Test the `Message` value, preview it, then let a narrow adapter send it. Use recipient allowlists in test/staging, deduplication keys, rate limits, and a clear distinction between retryable and permanent delivery failures. Never log credentials or full sensitive message bodies.

11. GUI Automation Is a Fragile Last Resort

Mouse/keyboard automation depends on focus, coordinates, scale, timing, window state, and accessibility. Prefer a programmatic API. If GUI control is unavoidable:

Guardrail	Purpose
dedicated test profile	protects personal data and settings
visible countdown	gives the operator time to cancel
fail-safe gesture/key	stops runaway input
screenshot assertion	confirms the expected screen before action
bounded repetitions	prevents infinite clicking/typing
no irreversible action	require a human confirmation at the final commit

Automate navigation and data entry; pause before sending, purchasing, deleting, or publishing unless the workflow has explicit authorization and a verified preview.

12. CLI Design Makes Automation Reusable

```
my-tool INPUT --output OUTPUT --dry-run --format json
```

CLI contract	Good behavior
arguments	explicit paths/modes; helpful validation errors
stdout	requested data, optionally machine-readable
stderr	diagnostics and progress
exit status	stable success/failure meaning

dry run	shows planned mutations without applying them
configuration	documented precedence and no secret echo
interruption	cleans temporary state or leaves resumable journal

Keep `main()` thin:

```
parse CLI → load config → call pure planner → validate → preview/apply adapter → report
```

The same architecture works for a compiler driver, asset pipeline, database maintenance job, or personal file organizer.

13. Automation Review Card

Before the first real run	After the run
synthetic test inputs	verify counts, hashes, or database constraints
narrow input and destination	inspect a sample of outputs
dry-run plan	retain an audit summary
backup/recovery path	confirm originals are recoverable
timeout and item-count limit	record failures for retry
redacted logging	remove temporary secrets/files
explicit authorization	confirm no scope expansion



Progression: automate one deterministic transformation first. Then add a CLI, dry-run mode, structured logs, durable checkpoints, scheduling, and notifications in that order.



A Repeatable Low-Level Learning and Debugging Workflow



Repeatable loop: define → isolate → observe → test → explain → automate.

When Building

1. State the invariant.
2. Define the representation.
3. Validate every boundary.
4. Keep unsafe or platform-specific code isolated.
5. Create a tiny executable example.
6. Inspect intermediate output.
7. Add failure-path and boundary tests.
8. Measure before optimizing.

When Reversing or Debugging

1. Confirm authorization and isolate the target.
2. Record architecture, operating system, file format, and hashes.
3. Establish one controlled input and one observable output.
4. Locate a landmark: string, symbol, file offset, packet field, or instruction.
5. Trace data flow before guessing intent.
6. Change one variable and repeat.
7. Separate observations from hypotheses.
8. Predict a new result, then validate it.

Questions That Transfer Across Every Layer

Dimension	Transferable question
Representation	What representation am I looking at?
Ownership	Who owns this value or resource?
Lifetime	How long is it valid?
Validation	Which invariants have already been checked?
Encoding	What are byte order, width, alignment, and signedness?
Numeric domain	Address, offset, index, ID, or count?
Trust	Where does trust change?
Partial effects	Can the operation partially succeed?
State machine	What state must come before and after it?
Falsifiability	What evidence would disprove the current model?

Learning Progression and Projects

The four stages of building a language from scratch:

 **Progression principle:** grow the language in vertical slices so every stage remains runnable, testable, and explainable.

Feature	Calculator	Firstlang	Secondlang	Thirdlang
Grammar size	~18 lines	~70 lines	~77 lines	~140 lines
Type System	None	Dynamic	Static	Static + Classes
Variables	No	Yes	Yes	Yes
Functions	No	Yes	Yes	Yes + Methods
Classes	No	No	No	Yes
Memory	Stack	Stack	Stack	Stack + Heap

Execution	Interpreter / VM / JIT	Interpreter	LLVM JIT	LLVM JIT
-----------	------------------------	-------------	----------	----------

Each stage adds one layer of abstraction:

1. **Calculator** — Learn the basics: parsing, AST, evaluation.
2. **Firstlang** — Add programming: variables, functions, control flow.
3. **Secondlang** — Add types: static checking, LLVM compilation.
4. **Thirdlang** — Add OOP: classes, objects, heap memory management.

Grammar growth, side by side — each stage layers new pest rules onto the last without disturbing the expression rules underneath:

```
# Firstlang - statements, functions, control flow
Stmt = { Function | Return | Assignment | Expr }
Function = { "def" ~ Identifier ~ "(" ~ Params? ~ ")" ~ Block }
Conditional = { "if" ~ "(" ~ Expr ~ ")" ~ Block ~ "else" ~ Block }
WhileLoop = { "while" ~ "(" ~ Expr ~ ")" ~ Block }
```

```
# Secondlang - same shape, with type annotations bolted on
Type = { IntType | BoolType }
TypedParam = { Identifier ~ ":" ~ Type }
ReturnType = { "->" ~ Type }
Function = { "def" ~ Identifier ~ "(" ~ TypedParams? ~ ")" ~ ReturnType? ~ Block }
```

```
# Thirdlang - classes and object operations
ClassDef = { "class" ~ Identifier ~ "{" ~ ClassBody ~ "}" }
FieldDef = { Identifier ~ ":" ~ Type }
MethodDef = { "def" ~ Identifier ~ "(" ~ SelfParam ~ "," ~ Params? ~ ")" ~ ReturnType? ~ Block }
NewExpr = { "new" ~ Identifier ~ "(" ~ Args? ~ ")" }
Delete = { "delete" ~ Expr }
```

The expression rules (`Expr` , `Comparison` , `Additive` , etc.) barely change across all four stages — growth comes from *statements* and *declarations*, not from how arithmetic is parsed.

What to Explore Next

Direction	Topics
Language Features	Inheritance (vtables, dynamic dispatch), interfaces/traits, generics (<code>List<T></code>), closures, pattern matching, algebraic data types
Type System	Nullability (<code>Point?</code>), reference vs owned types, Hindley-Milner full inference
Memory Management	Garbage collection (mark-and-sweep, generational), reference counting, Rust-style ownership
Execution Models	AOT compilation to executables, bytecode VMs, transpilation to JS/C/WASM
Optimization	

s	Inlining, escape analysis (stack-allocate short-lived objects), devirtualization
Tooling	Debuggers, formatters, Language Server Protocol (LSP) for IDE support

Sketching a few of these directly on Thirdlang:

```
# Inheritance — needs vtables + dynamic dispatch
class Animal { def speak(self) -> int { return 0 } }
class Dog extends Animal { def speak(self) -> int { return 1 } } # override

# Interfaces/traits — polymorphism without inheritance
trait Printable { def print(self) -> int }
impl Printable for Point { def print(self) -> int { return self.x } }

# Closures — capture the enclosing scope
def make_adder(n: int) -> (int) -> int {
  return def(x: int) -> int { return x + n }
}
add5 = make_adder(5)
add5(10) # 15

# Pattern matching + algebraic data types
enum Option<T> { Some(T), None }
match point {
  Point { x: 0, y } => "on y-axis",
  Point { x, y: 0 } => "on x-axis",
  _ => "elsewhere",
}
```

Generics require either **monomorphization** (generate specialized code per concrete type — Rust’s approach) or **type erasure** (use runtime type info instead — Java’s approach).

A **minimal debugger** works the same way as our interpreter, just paused: it steps through the AST/bytecode one node at a time and lets you inspect the environment at each `break` :

```
(debug) break main.tl:10
(debug) run
Breakpoint hit at main.tl:10
(debug) print x
x = 42
(debug) step
```

Real-world Rust language projects worth reading, roughly in order of complexity: start with smaller, approachable codebases like [Koto](#) or [Rhai](#), then graduate to more advanced implementations like [Gleam](#) or [Boa](#) (a JS engine). They use the same techniques covered here — grammars, ASTs, type systems — at production scale.

The concepts you’ve learned — grammars, ASTs, type systems, code generation — appear everywhere: SQL, GraphQL, YAML, regex, CSS, template engines. You now have the foundation to understand, modify, or create any of them.

All paths lead to the same truth: computing is structured transformation — from text, to trees, to bytes, to machine code, to electrons.

Cross-Linked Computer Science and Low-Level Glossary

Use this glossary to decode a term quickly, then follow **Learn more** for the full mental model, examples, invariants, and failure modes.

A–D

Term	Low-level meaning	Learn more
3D fundamentals	Coordinates, vectors, transforms, clipping, rasterization, depth, and shading turn geometry into pixels.	3D transform pipeline
accessibility tree	A semantic parallel to painted UI exposes roles, names, values, focus, and actions to assistive technology.	GUI accessibility
allocator	A policy over finite storage chooses aligned free ranges while preserving non-overlap, ownership, and reuse invariants.	Kernel allocator models
API	An application programming interface is a source/behavior contract; it may sit above a different binary ABI or wire protocol.	ABI versus API
asymmetric cryptography	A public/private key relationship supports key establishment or signatures without sharing the private key.	Symmetric and asymmetric cryptography
async / asynchronous	Work is represented so the caller can make progress before that work completes.	Async, futures, and promises
audio	Timed frames of per-channel samples flow through buffers, devices, resamplers, mixers, and codecs.	Digital audio
authentication	Evidence is checked for a claimed identity before a session is established.	Authentication and authorization
authorization	Policy decides whether an authenticated identity may perform an action on a particular resource.	Authentication and authorization
backend	Server-side policy, persistence, computation, and integrations behind a client-facing boundary.	Platform and application layers
bitmap	One bit per member compactly represents allocation or membership in a known finite set.	Filesystem allocation maps
bootloader	Early privileged code initializes enough hardware to choose, verify, load, and enter a kernel.	Firmware-to-application chain
cache	A nearer copy trades space and possible staleness for lower access time and origin work.	Cache layers
call logger	Instrumentation records approved metadata around calls so behavior and timing can be reconstructed.	Safe call logger

canvas fingerprinting	Rendered pixel differences contribute a probabilistic browser/device correlation signal.	Fingerprinting and canvas
CDN	A distributed reverse-proxy edge serves cached content nearer users and forwards misses to an origin.	CDNs
certificate / digital certificate	A signed structure binds a public key to names or attributes under an issuer and policy.	Certificates
client-server	The client initiates a service interaction; the server accepts and answers under a protocol contract.	Network roles
command-line environment	A terminal, shell, environment, streams, paths, and process model compose text-oriented tools.	Terminal, shell, and program
compositor	A system combines application surfaces/layers into the final display image under clipping, transform, and z-order rules.	GUI layers
compression	A transform reduces redundant representation; it may be lossless or deliberately lossy.	Encoding, compression, and encryption
consensus algorithm	Replicas agree on a value or ordered log despite delays and a defined set of failures.	Consensus
consistency: eventual	Replicas are allowed to differ but should converge after updates stop and communication succeeds.	Consistency models
consistency: strong	An operation observes a tightly ordered, current view such as linearizable behavior.	Consistency models
consistency: weak	The system gives few freshness or ordering promises for the operation.	Consistency models
container / containerization	A packaged process receives restricted resource views while normally sharing the host kernel.	Containers
cookie	Browser-managed state is stored and attached to matching requests under security/scope attributes.	Sessions, cookies, and tokens
CORS	HTTP response headers let a server opt selected web origins into browser-readable cross-origin responses.	CORS, CSRF, and XSS
cryptographic signing	A private key signs exact context-bound bytes; a public key verifies integrity and signer-key possession.	Signing
crash consistency	Persistent write ordering constrains which metadata and data states may remain after power loss or a crash.	Filesystem recovery
CSRF	A browser may attach ambient credentials to an unwanted cross-site request.	CORS, CSRF, and XSS

database	A storage engine provides a data model, access paths, concurrency rules, and durability/failure semantics.	Database families
decoding	Bytes in an encoding are parsed back into another representation under explicit validity rules.	Encoding distinctions
deserialization	A byte/text representation is parsed into typed values and must then pass semantic validation.	Serialization boundary
DLL injection	A process is induced to map or execute code it did not select; defenders correlate loader, memory, thread, and handle evidence.	DLL loading and injection signals
DMA / DMA card	A device reads or writes mapped memory without CPU copying; an IOMMU can translate and restrict those accesses.	DMA and IOMMU
DNS	A delegated, cached database maps names to typed resource records through resolvers and authoritative servers.	DNS
Docker	A container platform that builds images and runs isolated processes sharing the host kernel by default.	Containers
document database	Records naturally store nested aggregate-shaped documents and may duplicate related data for access locality.	Database families
driver	Concurrent privileged code translates an OS resource contract into device registers, queues, interrupts, and DMA.	Driver model

E–M

Term	Low-level meaning	Learn more
emulator	Software reproduces another ISA, machine, or device closely enough to run target software.	Execution environments
encoding	A syntax maps information into representable bytes or characters without providing secrecy.	Encoding distinctions
encryption	A keyed transformation provides confidentiality; authenticated encryption also detects modification.	Cryptography families
entity structure	Identity and component storage form data relationships inferred from strides, offsets, pointers, and controlled changes.	Recovering entity structures
event loop	A platform/runtime dispatches input, lifecycle, timers, tasks, and redraw work while owning thread-sensitive UI state.	GUI event loop
event-driven development	Commands/events are dispatched to handlers, decoupling producers while introducing ordering and backpressure concerns.	Event-driven and stream processing
fault tolerance	The design continues safely or recovers under an explicit crash, delay, loss, corruption, or partition model.	Fault tolerance

file storage	Filesystems or object stores retain named byte sequences plus metadata under durability and atomicity rules.	Database and storage families
filesystem	Metadata layers turn numbered storage blocks into allocated objects, directories, paths, byte ranges, and recovery rules.	Filesystem model
fingerprinting	Multiple observable attributes are combined into a noisy correlation signal, not cryptographic identity.	Fingerprinting and canvas
firmware	Software stored with a device/platform initializes or controls hardware below ordinary application boundaries.	Firmware-to-application chain
floating point	Significand/exponent values approximate real numbers with finite precision and special non-finite states.	Floating-point contract
forward proxy	A relay acts on behalf of clients when reaching destinations.	Network roles
frontend	User-facing presentation and interaction code consuming host/backend capabilities.	Platform and application layers
FTP / FTPS / SFTP	Distinct file-transfer protocols: legacy FTP, TLS-protected FTP, and SSH's file subsystem.	File-transfer protocols
function pointer	A value selects callable code under a specific ABI and signature.	Function pointers
future	A pollable value represents computation that may produce a result later.	Async, futures, and promises
graphics API	A command/resource interface submits buffers, textures, shaders, and synchronization to a GPU or renderer.	GUI and graphics libraries
GPU	An asynchronous throughput processor consumes queued shader/compute/transfer work over explicit memory contracts.	GPU pipeline
GraphQL	A typed API query language/runtime that resolves requested fields over arbitrary backing systems; it is not a database.	GraphQL
GUI	A graphical user interface maps input events and application state through layout/paint into window-system surfaces.	GUI architecture
headers, HTTP	Metadata fields select representations, credentials, caching, routing context, and body interpretation.	HTTP messages and headers
heartbeat	Periodic liveness evidence supports suspicion and leases but cannot distinguish every cause of silence.	Heartbeats
heuristic scanning	Weighted structural or behavioral signals generalize beyond exact signatures but require false-positive management.	Scanner comparison
HTML canvas	A scriptable pixel surface used for graphics whose outputs can also become a fingerprinting signal.	Fingerprinting and canvas
HTTP	An application protocol defines request methods, status	HTTP messages

	responses, fields, representations, and caching semantics.	
HTTP caching	Freshness directives and validators coordinate reuse of stored responses across browsers and intermediaries.	HTTP caching
HWND	An opaque Win32 handle identifies one live window under OS lifetime rules; it is not a Rust reference or ownership proof.	Native Window API basics
HWID	A hardware-derived identifier is mutable, privacy-sensitive, and unsuitable as the sole proof of identity.	Platform identity
Hyper-V	Microsoft's hypervisor architecture organizes a root partition and child guest partitions with synthetic device channels.	Hypervisors and Hyper-V
hypervisor	Privileged software virtualizes CPU, memory, interrupts, and devices for guest machines.	Hypervisors and Hyper-V
immediate-mode UI	The application describes the desired controls each frame/pass while the toolkit retains interaction state by identity.	UI models
inode	A stable filesystem object record stores metadata and block references separately from the directory names that reach it.	Inodes and names
JTAG / SWD	Hardware debug transports can expose scan, halt, registers, memory, trace, and programming capabilities.	JTAG and SWD
JWT	A standardized claims container that may be signed/encrypted; it is a format, not an authentication design.	Sessions, cookies, and tokens
kernel	The privileged resource manager controls processes, virtual memory, scheduling, devices, files, and isolation.	Operating systems
layout engine	Constraints, intrinsic sizes, policy, and scale become rectangles, clips, and transforms for a UI tree.	GUI layout
lifetime	A type-level relationship bounds how long borrowed access may remain valid; an annotation does not extend storage.	Lifetimes as relationships
load balancing	A policy selects a healthy destination while accounting for capacity, affinity, draining, and retries.	Load balancing
memory scanner	A bounded query compares typed values across authorized immutable memory snapshots.	Memory scanners
middleware	Reusable translation/policy behavior sits between application logic and a platform, protocol, or data service.	Platform and application layers

N–Z

Term	Low-level meaning	Learn more
NoSQL	An umbrella for non-relational models such as document, key-value, wide-column, and graph—not one consistency policy.	Database families

NULL / nullability	Absence/unknown is a distinct state whose logic and wire/update meaning must be modeled explicitly.	Null, missing, and empty
obfuscation	Transformations raise analysis cost but do not provide cryptographic trust or authorization.	Obfuscation analysis
ownership	A static resource protocol assigns responsibility for a value and its eventual release, transfer, or controlled sharing.	Ownership model
password salting	A unique non-secret salt ensures equal passwords do not produce equal stored verifiers.	Password storage
password storage	A slow memory-hard password-hashing verifier replaces plaintext, reversible encryption, or fast hashes.	Password storage
pattern scanner	A bounded matcher locates exact/wildcard byte sequences in an owned file or snapshot.	Pattern scanning
permissions	ACLs, ownership, capabilities, page protections, and handles restrict specific operations on resources.	Sandbox authority
plist	Apple's property-list model serializes structured configuration in XML or binary form.	Platform storage
port	A 16-bit transport endpoint selector used with addresses and protocol state.	Ports
port observation / scanning	Bounded connection attempts classify endpoint evidence as connected, refused, timed out, or another OS error.	Bounded TCP port observation
promise	In some async models, the completion/writable counterpart to a future.	Async, futures, and promises
protocol	A shared syntax, state machine, timing model, and error contract between participants.	Layered networking
proxying	A relay terminates one participant role and initiates another, changing trust and observation points.	Network roles
RAII	Resource acquisition is tied to an owning value whose destructor performs structural cleanup at scope exit.	Ownership beyond memory
random number	A value comes from a generator selected for either reproducibility/performance or cryptographic unpredictability.	Random-number domains
Redis	A networked typed data-structure server often used for caching, coordination, queues, or fast derived state.	Cache layers and Redis
Registry	Windows' typed hierarchical configuration database has hives, views, permissions, and platform-specific semantics.	Platform storage
relational database	Tables, keys, constraints, indexes, joins, and transactions model and protect relationships.	Relational design
reliability	The probability a system satisfies its behavior contract over	Fault tolerance

	time, distinct from availability and durability.	and reliability
retained-mode UI	A toolkit keeps a persistent widget/object tree and invalidates or redraws it when properties change.	UI models
reverse proxy	A relay appears to clients as the service and forwards selected requests to origins.	Network roles
sandbox	Several mechanisms reduce ambient authority and isolate a component's address, syscall, resource, or runtime view.	Sandboxing
serialization	Typed state is encoded into a versioned byte/text representation with framing and compatibility rules.	Serialization boundary
session management	A service creates, rotates, expires, and revokes continuity of authenticated state.	Sessions, cookies, and tokens
Shadowsocks	A secure split proxy protects traffic between local and remote relay components; it is not automatically a full VPN.	VPNs and Shadowsocks
signature-based scanning	Exact hashes or stable byte/structural signatures recognize known artifacts efficiently.	Scanner comparison
simulator	An abstract behavioral model reproduces selected system properties without necessarily running the exact target machine.	Execution environments
SSH	A secure protocol provides remote login, authenticated host/user channels, forwarding, and subsystems such as SFTP.	TLS and SSH
stream processing	Operators transform an unbounded event sequence using ordering, time, windows, state, and recovery policy.	Event-driven and stream processing
superblock	A filesystem's root format record declares its version, geometry, counts, and the locations needed to decode other regions.	On-disk layout
symmetric cryptography	Participants share a secret for fast encryption or message authentication.	Cryptography families
sync / synchronous	Control flow waits for an operation to complete before proceeding.	Async, futures, and promises
test-driven development (TDD)	A failing behavioral example precedes the smallest implementation change, followed by refactoring.	TDD and reliability tests
terminal emulator	A GUI program interprets terminal control sequences and renders a character-cell model into pixels.	How a TUI reaches the screen
threat model	A structured statement identifies controlled inputs, trust boundaries, assets, attacker capabilities, and acceptable failures.	Threat modeling

TLS	A handshake and record protocol authenticates endpoints and protects application data in transit.	TLS and SSH
token	A credential or typed symbol represents authority or syntax; security tokens must be scoped and protected as secrets.	Sessions, cookies, and tokens
TUI	A terminal user interface lays out widgets in cells and emits only the changed terminal state through a backend.	TUI rendering
UI toolkit	A library owns widgets, input routing, layout, styling, accessibility, and rendering integration at some chosen level.	GUI library selection
UUID	A 128-bit versioned identifier supports decentralized or ordered/name-derived generation; it is not proof of identity.	UUIDs
video	Timestamped frames are compressed into codec streams and multiplexed with metadata/tracks in containers.	Video
virtual machine (VM)	Virtualized hardware runs a guest OS with its own kernel under a hypervisor.	Execution environments
VPN	A tunnel/routing policy carries selected IP traffic through a gateway; it changes paths, not endpoint trust.	VPNs and Shadowsocks
widget	A logical UI element contributes semantics, layout, event handling, and paint/cell output.	GUI architecture
window API / Win32	A native windowing API registers window behavior, creates opaque handles, dispatches events/messages, and schedules paint.	Native Window API basics
window procedure	A Win32 system-ABI callback receives typed-by-message parameters and must delegate unhandled behavior to the OS default.	Window procedure and pump
XSS	Untrusted content executes as script in a trusted origin unless contextual encoding and safe DOM APIs prevent injection.	CORS, CSRF, and XSS
XML	A structured text format with elements, attributes, namespaces, mixed content, and security-sensitive entity policy.	Serialization boundary



Glossary habit: if two terms sound interchangeable, follow both links and compare their boundaries. Common costly confusions include encoding versus encryption, container versus VM, GraphQL versus database, SFTP versus FTPS, JWT versus session, and CORS versus CSRF.

How These Domains Feed Your Own Language

Domain	When building a language	When reversing a language/runtime
numeric/ media	define integer overflow, floating point, SIMD/vector types, units, and foreign buffers	identify numeric width, matrix convention, buffer layout, and hardware calls
	define ownership, allocation, page	recover object layout, lifetimes, mappings,

memory/ OS	permission, thread, and file/socket abstractions	syscalls, and synchronization
bytecode /native code	define instruction encoding, call frames, ABI, imports, and debug metadata	reconstruct opcodes, dispatch, indirect calls, symbols, and control flow
serializati on/protoc ols	define stable schemas, framing, versioning, and bounded parsing	infer message boundaries, state machines, checksums, and field meaning
async/ev ents	define task, future, cancellation, channel, and backpressure semantics	recognize compiler-generated state machines, event loops, queues, and callbacks
database/ cache	define persistence APIs, null/missing states, transactions, and deterministic encoding	distinguish source of truth from cache, index, journal, or derived state
cryptogra phy/identi ty	expose safe library-backed types and keep keys/nonces unforgeable in ordinary code	locate trust decisions, canonical bytes, certificate use, and failure paths
sandbox/ VM	restrict imports, syscalls, memory, time, and execution fuel	identify host/guest boundaries and the authority of each escape edge
GUI/TUI	define components, stable identity, events, layout, tasks, and backend capabilities	recover widget trees, dispatch, layout boxes, display lists, and terminal output
observabi lity	emit source maps, spans, structured events, and stable IDs	correlate addresses with modules, calls, data flow, and external effects

 **Design principle:** give a programmer safe, typed access to the concept—not raw authority by default. A `Socket`, `File`, `Future`, `SecretKey`, or `GpuBuffer` should carry ownership, lifetime, state, and permission rules that the compiler/runtime can enforce.
